

Unit 01 Homework – Moneyball

KAGGLE name: *NikhilAgarwal*

Nikhil Agarwal
Northwestern University
PREDICT 411, Section 55

I am requesting a total of **30** bingo bonus points for the unit 1 homework. Please see my justification below.

Points Requested	Category	Justification
5	Decision Tree	I used R to construct a decision tree. The results for one variable were implemented into the submitted SAS code
5	Macro	I used a small amount of macro coding in my SAS code
20	R code	I constructed quite a bit of this homework in R. Please see separate R code (NikhilAgarwal_Unit1HW_Rcode.txt)

Table of Contents

INTRODUCTION	4
RESULTS	4
DATA EXPLORATION	4
DATA PREPARATION	7
IMPUTATION	7
OUTLIERS	8
DATA TRANSFORMATIONS	9
MODEL DEVELOPMENT	10
MODEL 1	10
MODEL 2	11
MODEL 3	13
MODEL SELECTION	15
CONCLUSION	16
BIBLIOGRAPHY	17

INTRODUCTION

The intent of this assignment is to ultimately develop an ordinary least squares (OLS) function that can be used to predict wins for a baseball team given a certain set of variables. Information from over 2000 teams, spanning from 1871 through 2006, was used to help construct a model. Prior to developing a single model, multiple regression models were explored (single and multivariate) using primarily the stepwise function. Various selection parameters (e.g., SBC, AIC, Adjusted R-Squared, etc.) were used to determine the best model.

RESULTS

Data Exploration

The original dataset contains 15 distinct variables (outlined in Table 1). These variables can be considered potential predictors. Note that the response variable will be TARGET_WINS. Of particular note is the “Theoretical Effect” column. This column succinctly describes whether the potential predictor variable has a positive or negative impact on wins (i.e., the response variable TARGET_WINS).

Table 1: Brief description of default variables

VARIABLE NAME	DEFINITION	THEORETICAL EFFECT
TEAM_BATTING_H	Base Hits by batters (1B,2B,3B,HR)	Positive Impact on Wins
TEAM_BATTING_2B	Doubles by batters (2B)	Positive Impact on Wins
TEAM_BATTING_3B	Triples by batters (3B)	Positive Impact on Wins
TEAM_BATTING_HR	Homeruns by batters (4B)	Positive Impact on Wins
TEAM_BATTING_BB	Walks by batters	Positive Impact on Wins
TEAM_BATTING_HBP	Batters hit by pitch (get a free base)	Positive Impact on Wins
TEAM_BATTING_SO	Strikeouts by batters	Negative Impact on Wins
TEAM_BASERUN_SB	Stolen bases	Positive Impact on Wins
TEAM_BASERUN_CS	Caught stealing	Negative Impact on Wins
TEAM_FIELDING_E	Errors	Negative Impact on Wins
TEAM_FIELDING_DP	Double Plays	Positive Impact on Wins
TEAM_PITCHING_BB	Walks allowed	Negative Impact on Wins
TEAM_PITCHING_H	Hits allowed	Negative Impact on Wins
TEAM_PITCHING_HR	Homeruns allowed	Negative Impact on Wins
TEAM_PITCHING_SO	Strikeouts by pitchers	Positive Impact on Wins

Continuing on the data exploration journey, it is also wise to identify if any of the variables are missing. Figure 1 illustrates some basic statistics on the dataset.

Variable	N	N Miss	Minimum	Maximum	Median	Mean	Std Dev
INDEX	2276	0	1.0000000	2535.00	1270.50	1268.46	736.3490405
TARGET_WINS	2276	0	0	146.0000000	82.0000000	80.7908612	15.7521525
TEAM_BATTING_H	2276	0	891.0000000	2554.00	1454.00	1469.27	144.5911954
TEAM_BATTING_2B	2276	0	69.0000000	458.0000000	238.0000000	241.2469244	46.8014146
TEAM_BATTING_3B	2276	0	0	223.0000000	47.0000000	55.2500000	27.9385570
TEAM_BATTING_HR	2276	0	0	264.0000000	102.0000000	99.6120387	60.5468720
TEAM_BATTING_BB	2276	0	0	878.0000000	512.0000000	501.5588752	122.6708615
TEAM_BATTING_SO	2174	102	0	1399.00	750.0000000	735.6053358	248.5264177
TEAM_BASERUN_SB	2145	131	0	697.0000000	101.0000000	124.7617716	87.7911660
TEAM_BASERUN_CS	1504	772	0	201.0000000	49.0000000	52.8038564	22.9563376
TEAM_BATTING_HBP	191	2085	29.0000000	95.0000000	58.0000000	59.3560209	12.9671225
TEAM_PITCHING_H	2276	0	1137.00	30132.00	1518.00	1779.21	1406.84
TEAM_PITCHING_HR	2276	0	0	343.0000000	107.0000000	105.6985940	61.2987469
TEAM_PITCHING_BB	2276	0	0	3645.00	536.5000000	553.0079086	166.3573617
TEAM_PITCHING_SO	2174	102	0	19278.00	813.5000000	817.7304508	553.0850315
TEAM_FIELDING_E	2276	0	65.0000000	1898.00	159.0000000	246.4806678	227.7709724
TEAM_FIELDING_DP	1990	286	52.0000000	228.0000000	149.0000000	146.3879397	26.2263853

Figure 1: Descriptive Statistics on default variables

Note how six of the potential predictor variables have missing values. On a side note, the variable INDEX is simply a unique identifier that will not be used for any modeling purpose. The variable TEAM_BATTING_HBP is missing over 2000 values. Recall that this dataset only contains 2276 observations. Therefore, the variable TEAM_BATTING_HBP will be dropped from the analysis as there are too many missing values and imputing would most likely lead to false assumptions and conclusions.

Table 2 highlights the correlation of each predictor variable to the response variable of TARGET_WINS. The closer the value is to +1 or -1, the stronger the linear relationship. Unsurprisingly, no single predictor variable is highly correlated with TARGET_WINS. Recall from Table 1 that certain predictor variables have a negative impact on the overall wins. However, in Table 2, it is evident that the variable TEAM_BASERUN_CS has a very slight positive relationship with TARGET_WINS. A similar surprise can be found with the variables TEAM_PITCHING_BB and TEAM_PITCHING_HR – which both have a positive correlation, yet they have a negative impact on wins. In other words, the higher the values for these variables, the ‘higher’ the wins.

Table 2 : Correlation table of default variables to TARGET_WINS

Variable	Correlation
TEAM_BATTING_H	0.38877
TEAM_BATTING_2B	0.2891
TEAM_BATTING_3B	0.14261
TEAM_BATTING_HR	0.17615
TEAM_BATTING_BB	0.23256
TEAM_BATTING_HBP	0.0735
TEAM_BATTING_SO	-0.03175
TEAM_BASERUN_SB	0.13514
TEAM_BASERUN_CS	0.0224
TEAM_FIELDING_E	-0.17648
TEAM_FIELDING_DP	-0.03485
TEAM_PITCHING_BB	0.12417
TEAM_PITCHING_H	-0.10994
TEAM_PITCHING_HR	0.18901
TEAM_PITCHING_SO	-0.07844

Another key check is to determine if there are outliers. All 15 predictor variables (along with the response variable, TARGET_WINS) were checked for outliers. Figure 2 is an example of a histogram and boxplot for the variable TEAM_BATTING_H. Note the many circles that are outside the whiskers in the boxplot. This is a strong indicator that outliers may be present.

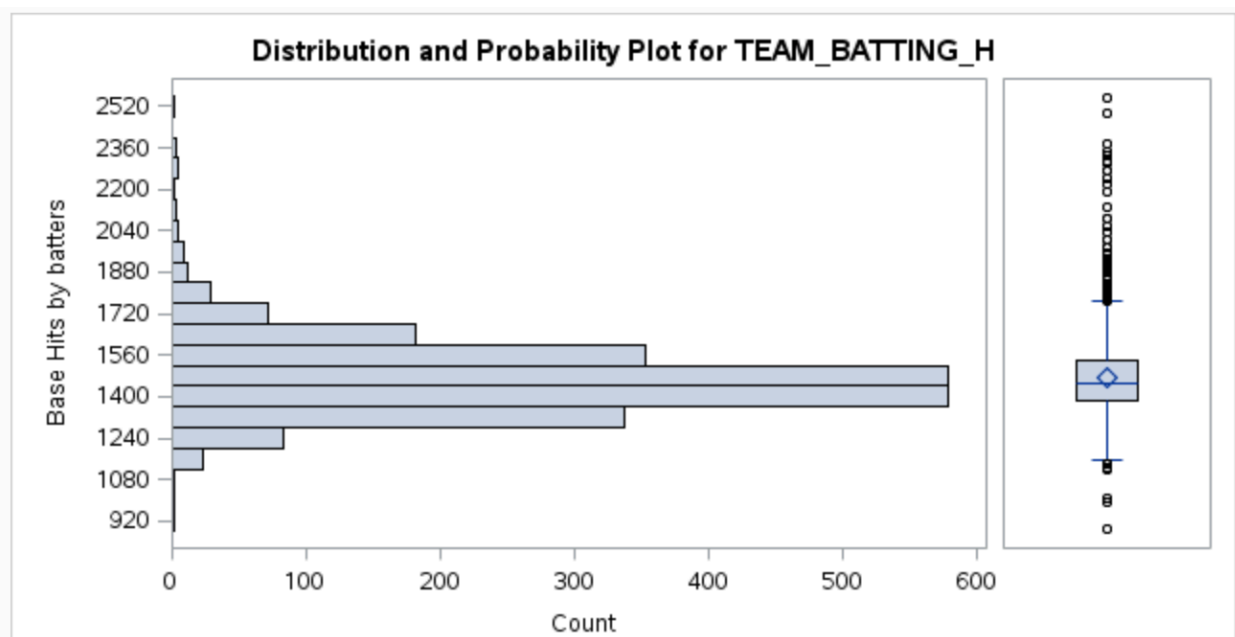


Figure 2: Example histogram & boxplot for variable TEAM_BATTING_H

It was found that almost all of the variables had outliers. Therefore, value caps will be deployed on each variable to ensure that outliers do not skew the model unnecessarily. The caps are explained in greater detail in the section, “Data Preparation”.

Data Preparation

Imputation

Figure 1 shows that there are six predictor variables with missing values. Recall that the variable TEAM_BATTING_HBP will be dropped as it is missing a significant amount of information. Therefore, five of variables require imputation. Table 3 summarizes the imputation method used for each of the four variables. All five of these variables were removed from the modelling process and the imputed values were stored in different variables starting with the prefix “imp_”. If there was a non-null value for any of the variables in Table 3, then that value was used in lieu of an imputed value. Furthermore, a flag variable was created (with the prefix “m_”) to indicate if an imputed value was entered (indicated with a 1 meaning true) or if the original value was used (indicated with a 0 meaning false).

Table 3: Imputation Methods for Variables with Missing Data

Variable	Method of Imputation	Imputed Variable Name
TEAM_PITCHING_SO	Decision Tree	imp_TEAM_PITCHING_SO
TEAM_FIELDING_DP	Mean	imp_TEAM_FIELDING_DP
TEAM_BASERUN_CS	Median	imp_TEAM_BASERUN_CS
TEAM_BASERUN_SB	Median	imp_TEAM_BASERUN_SB
TEAM_BATTING_SO	Median	imp_TEAM_BATTING_SO

Using R, a simple decision tree was constructed for the variable TEAM_PITCHING_SO (see Figure 3).

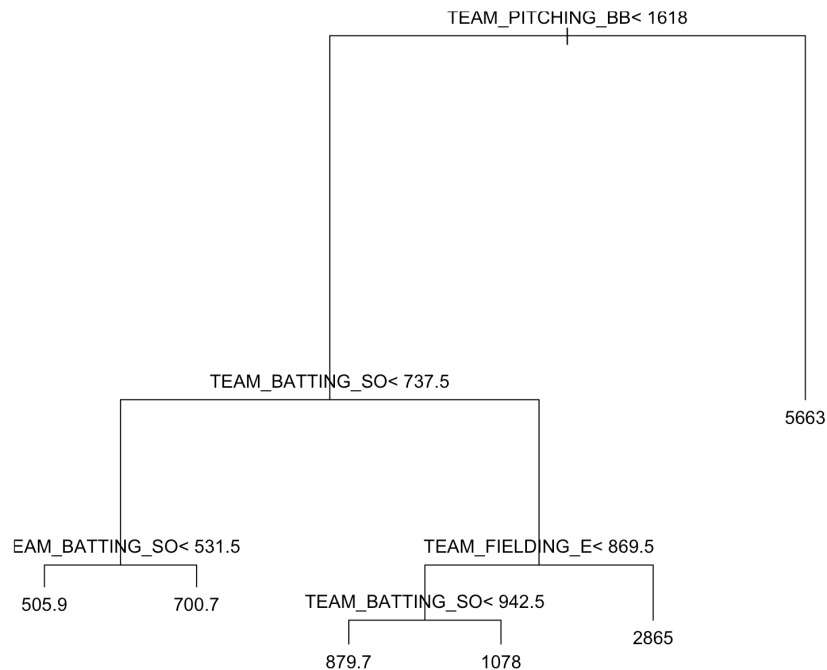


Figure 3: Decision tree for variable TEAM_PITCHING_SO

The following logic briefly describes (converting logic syntax to English) the decision tree for the variable TEAM_PITCHING_SO:

If TEAM_PITCHING_BB is greater than 1618 then TEAM_PITCHING_SO will be 5663. If that is not true, then if TEAM_BATTING_SO is less than 737.5 then the imputed value of TEAM_PITCHING_SO will be 609. If that is also not true, then if TEAM_FIELDING_E is less than 869 then the imputed TEAM_PITCHING_SO value is 970. Otherwise, the imputed TEAM_PITCHING_SO value will be 2865.

The mean value found amongst non-null data for TEAM_FIELDING_DP was used as the imputed value if there was a missing value. For this variable, it was discovered that the maximum value was not significantly higher than the 99th percentile value (228 vs. 204, respectively). Therefore, the average was deemed appropriate. However, the same cannot necessarily be said for the other three variables (TEAM_BASERUN_CS, TEAM_BASERUN_SB, and TEAM_BATTING_SO). Since the imputation process is taking place prior to the reduction of outliers, it is appropriate to use the 50th percentile data point (median) as the default value for any missing value for these variables.

Outliers

In order to reduce the effects of outliers, all of the 14 predictor variables' 1 and 99th percentiles were identified as the value thresholds (i.e., lower and upper limits). The intent is not to necessarily eliminate all outliers, but to reduce their effect on the overall model. These percentiles still allow for some 'peak' and 'valley' points that illustrate the teams' abilities to be

below or above expectations (in terms of wins). Table 4 highlights the lower (1 percentile) and the upper (99th percentile) thresholds used for each variable.

Table 4: Lower and Upper Thresholds for variables

Variable	Lower Threshold	Upper Threshold
TEAM_BATTING_H	1188	1950
TEAM_BATTING_2B	141	352
TEAM_BATTING_3B	17	134
TEAM_BATTING_HR	4	235
TEAM_BATTING_BB	79	755
imp_TEAM_BATTING_SO	67	1200
imp_TEAM_BASERUN_SB	23	439
imp_TEAM_BASERUN_CS	16	143
TEAM_PITCHING_H	1244	7093
TEAM_PITCHING_HR	8	244
TEAM_PITCHING_BB	237	924
imp_TEAM_PITCHING_SO	205	1474
TEAM_FIELDING_E	86	1237
imp_TEAM_FIELDING_DP	79	204

For the actual simulated results (part of the scoring process), the wins were also capped using the 1 percentile and 99th percentile values. For simplicity's sake, the minimum number of wins in the scoring process was limited to 40 wins and the maximum wins was limited to 114.

Data transformations

The original dataset values have been adjusted to match the performance of 162 games. In the early years of baseball, 162 games were not necessarily played by each team in each season. Therefore, each of the variables outlined in table YY were divided by 162 to obtain a per-game assessment. These values were stored in new variable names using a similar nomenclature found in table YY followed by “_pg”.

In an effort to minimize potential variation and skewness in the data, both log and square root transformations were applied and new variables were created. All new variables that used log transformations are accompanied with the prefix “log_” and new variables that used square root transformations are accompanied with the prefix “sqrt_”.

Another new variable constructed is known as SB_PCT. This variable simply describes the stolen base percentage as a ratio of stolen bases over the total of stolen bases and caught stealing. It was calculated using the following equation:

Equation 1: SB_PCT Calculation

$$SB_PCT = \frac{imp_TEAM_BASE_RUN_SB}{(imp_TEAM_BASE_RUN_SB + imp_TEAM_BASERUN_CS)}$$

Model Development

Eight different regression models were constructed, but for the sake of succinctness, only three of them will be discussed in this report. Furthermore, all of the models were constructed using the stepwise selection criteria. This approach ensures that all of the predictor variables are statistically significant and has the ability to remove variables once they enter the model and fail to be statistically significant after newer variables are introduced.

As a primer, all of the models constructed have the following equation format:

$$TARGET_WINS = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \varepsilon$$

where β_0 is the intercept, β_n is the parameter estimate of the variable number n , X_n is the observed value in the dataset for the variable number n , and ε is the standard error term.

Model 1

This first model was designed to explore 19 predictor variables (including the imputation flag variables) with the response variable being TARGET_WINS. Table 5 illustrates a summary of the variables chosen, their coefficients, and the corresponding VIF values. Note how this model only uses 12 of the 19 predictor variables.

Table 5: Model 1 Parameter Estimates & VIF

Variable	Variable Code	Coefficient Notation	Coefficient Value	VIF
Intercept		β_0	21.05224	0
TEAM_BATTING_H_pg	X1	β_1	6.10805	2.56182
TEAM_BATTING_3B_pg	X2	β_2	18.34527	3.10942
TEAM_BATTING_HR_pg	X3	β_3	12.14337	4.95189
TEAM_BATTING_BB_pg	X4	β_4	6.78694	18.9289
TEAM_PITCHING_BB_pg	X5	β_5	-4.18658	11.94574
TEAM_FIELDING_E_pg	X6	β_6	-4.15511	5.40092
TEAM_PITCHING_H_pg	X7	β_7	0.48377	6.53825
m_TEAM_PITCHING_SO	X8	β_8	8.17901	1.9066
imp_TEAM_BATTING_SO_pg	X9	β_9	-3.73472	23.05112
imp_TEAM_BASERUN_SB_pg	X10	β_{10}	5.0427	1.88297
imp_TEAM_PITCHING_SO_pg	X11	β_{11}	2.66629	14.97049
imp_TEAM_FIELDING_DP_pg	X12	β_{12}	-16.86646	1.51045

This model appears to have several variables with high VIF values. Generally, a VIF value of 5 or higher tends to suggest multicollinearity and VIF values at ten or higher strongly suggest that multicollinearity is present. On another note, it is curious that the coefficient value for the variable `imp_TEAM_FIELDING_DP_pg` (recall that this variable refers to the double plays made by the fielding team) is negative. This negative coefficient value suggests that this variable has a negative impact on wins. Intuitively, we would expect that as more double plays are made, then the fielding team may end up winning more games. However, this could also be interpreted that just because a fielding team does well (conducting more double plays), this does not necessarily mean that its batting performance is good.

Table 6 highlights some of the goodness of fit diagnostics of this model.

Table 6: Model 1 Goodness of Fit Diagnostics

Characteristic	Value
R-Square	0.29888
Adjusted R-Square	0.29516
RMSE	13.2247
AIC	11766.60
BIC	11768.77
SBC	11841.10

It is important to note that although the adjusted R-square value is quite low, this does not necessarily mean the model is incorrect or insufficient. Furthermore, for this analysis, the only key metric that will be used to determine the most appropriate model will be SBC. This is explained further in section: **Model Selection**.

Finally, this model was adjusted slightly through the removal of the variables: `TEAM_PITCHING_BB_pg` and `imp_TEAM_BATTING_SO_pg`. When the model was reran, the goodness of fit diagnostics were similar to what was found in Table 6.

Model 2

For this model, the log transformed predictor variables were used (along with the imputation flags). As seen in Model 1, this model also consists of 19 variables with 1 response variable (`TARGET_WINS`). Table 7 illustrates a summary of the variables chosen, their coefficients, and the corresponding VIF values. Note how this model also only uses 12 of the 19 predictor variables.

Table 7: Model 2 Parameter Estimates & VIF

Variable	Variable Code	Coefficient Notation	Coefficient Value	VIF
Intercept		β_0	-71.17207	0
m_TEAM_PITCHING_SO	X1	β_1	3.79362	1.45456
log_TEAM_BATTING_H_pg	X2	β_2	71.79477	3.5304
log_TEAM_BATTING_2B_pg	X3	β_3	-4.10007	2.58181
log_TEAM_BATTING_3B_pg	X4	β_4	6.10749	2.67269
log_TEAM_BATTING_HR_pg	X5	β_5	-8.94744	51.72002
log_TEAM_BATTING_BB_pg	X6	β_6	5.31283	2.80443
log_TEAM_PITCHING_HR_pg	X7	β_7	12.14965	41.76897
log_TEAM_FIELDING_E_pg	X8	β_8	-11.03641	5.75686
log_imp_TEAM_BASERUN_SB_pg	X9	β_9	4.68466	1.95589
log_imp_TEAM_BASERUN_CS_pg	X10	β_{10}	-2.91443	1.39726
log_imp_TEAM_PITCHING_SO_pg	X11	β_{11}	-1.96853	2.39799
log_imp_TEAM_FIELDING_DP_pg	X12	β_{12}	-15.71114	1.64452

Similar to Model 1, this model appears to have two variables with very high VIF values. Intuitively, it is somewhat clear to see the relationship between log_TEAM_BATTING_HR and log_TEAM_PITCHING_HR. Of particular interest is the coefficient of log_TEAM_BATTING_2B. Since this value is negative, it could be interpreted as saying “for a unit increase in X3, it could be expected that TARGET_WINS would decrease by approximately 4.1%. This is somewhat odd as the more double hits a batting team has, one would think that the team would make more runs. However, from a different perspective, this also could be construed as increasing the probability of the fielding team of tagging a player out.

Table 8 highlights some of the goodness of fit diagnostics of this model. Note the R-Square value of 0.29682. This can be interpreted as roughly 29.6% of the variation found in wins can be explained by the chosen predictor variables.

Table 8: Model 2 Goodness of Fit Diagnostics

Characteristic	Value
R-Square	0.29682
Adjusted R-Square	0.29310
RMSE	13.2440
AIC	11773.27
BIC	11775.41
SBC	11847.76

This model was slightly modified with the removal of the variable log_TEAM_BATTING_HR_pg. This resulted in slightly different goodness of fit diagnostics (see Table 9).

Table 9: Model 2 (revised) Goodness of Fit Diagnostics

Characteristic	Value
R-Square	0.29594
Adjusted R-Square	0.29252
RMSE	13.2495
AIC	11774.14
BIC	11776.27
SBC	11842.90

Note how the SBC value is lower, but both the AIC and BIC values have slightly worsened (as compared to the values seen in Table 8). Furthermore, notice how the R-square value did not change significantly.

Model 3

The objective of any of the models is to project the overall wins. In that spirit, 48 variables (including the imputation flags) were entered into the model development. This consisted of all the “_pg”, “log_”, and “sqrt_” variables. After running the regression model with the stepwise selection, 26 variables were retained. Table 10 illustrates a summary of the variables chosen, their coefficients, and the corresponding VIF values.

Table 10: Model 3 Parameter Estimates & VIF

Variable	Variable Code	Coefficient Notation	Coefficient Value	VIF
Intercept		β_0	4116.5805	0
TEAM_BATTING_H_pg	X1	β_1	5.38229	8.05094
TEAM_BATTING_2B_pg	X2	β_2	1184.45396	42542
TEAM_BATTING_3B_pg	X3	β_3	23.86704	3.47063
TEAM_BATTING_BB_pg	X4	β_4	177.71041	2714.76142
TEAM_PITCHING_BB_pg	X5	β_5	-150.1981	18019
TEAM_FIELDING_E_pg	X6	β_6	-32.50384	1208.32683
TEAM_PITCHING_H_pg	X7	β_7	-18.28425	4719.54227
m_TEAM_PITCHING_SO	X8	β_8	3.45968	2.76542
imp_TEAM_BATTING_SO_pg	X9	β_9	-45.36556	1119.30827
imp_TEAM_BASERUN_SB_pg	X10	β_{10}	7.99394	2.67856
imp_TEAM_PITCHING_SO_pg	X11	β_{11}	-1.74724	32.14662
log_TEAM_BATTING_2B_pg	X12	β_{12}	1783.33331	41027
log_TEAM_BATTING_HR_pg	X13	β_{13}	-5.86404	43.92345
log_TEAM_BATTING_BB_pg	X14	β_{14}	299.27984	2189.74418
log_TEAM_PITCHING_BB_pg	X15	β_{15}	-382.89843	17887
log_TEAM_FIELDING_E_pg	X16	β_{16}	-69.01397	1012.53768
log_TEAM_PITCHING_H_pg	X17	β_{17}	-292.20308	4859.0128
log_imp_TEAM_BATTING_SO_pg	X18	β_{18}	-112.73409	719.89872
log_imp_TEAM_FIELDING_DP_pg	X19	β_{19}	-12.60626	1.78729
sqrt_TEAM_BATTING_2B_pg	X20	β_{20}	-5838.73625	166133
sqrt_TEAM_BATTING_BB_pg	X21	β_{21}	-957.78231	9401.39843
sqrt_TEAM_PITCHING_HR_pg	X22	β_{22}	30.34083	36.99459
sqrt_TEAM_PITCHING_BB_pg	X23	β_{23}	971.79692	71382
sqrt_TEAM_FIELDING_E_pg	X24	β_{24}	170.69434	4162.83487
sqrt_TEAM_PITCHING_H_pg	X25	β_{25}	306.85225	18557
sqrt_imp_TEAM_BATTING_SO_pg	X26	β_{26}	301.42524	3256.29868

Prior to discussing this model in greater detail, Table 11 highlights some of the goodness of fit diagnostics of this model.

Table 11: Model 3 Goodness of Fit Diagnostics

Characteristic	Value
R-Square	0.41201
Adjusted R-Square	0.40521
RMSE	12.1485
AIC	11394.11
BIC	11396.54
SBC	11548.82

This model is of absolute amazement! There are enormous amounts of multicollinearity and the coefficient values for some of the variables are absolutely large. What is also very surprising is the fact this model has a much higher adjusted R-square value and a significantly lower SBC value. It is important to note that the R-squared value simply describes the amount of variation in the response variable explained by the model. Also interesting about this model is the fact that many of the coefficient values for the parameters make sense in terms of the sign (positive or negative). However, two of the variables chosen (sqrt_TEAM_BATTING_2B_pg and sqrt_TEAM_BATTING_BB_pg) have negative values which would seem contrary. Intuitively, both of these variables should help yield more wins, but in this model they lead to a reduction in wins.

Model Selection

In this analysis, three of the eight different models constructed were discussed. Recall that all of the models constructed used a stepwise selection criterion. The key metric that was used for model selection is the SBC (Schwarz Bayesian Criteria). In a paper by Dennis Neal, the metrics AIC, BIC, and SBC were compared for ten independent variables used in a given model with varying sample sizes. The conclusion from the paper: “SBC performed best by correctly selecting the true model the most consistently after enumerating all possible true models from the simulated data. SBC consistently performed best for both sample sizes $n = 1000$ and $n = 100$ ” (Beal, 2007). Although the models discussed in this paper use more than ten variables, the consistency of SBC found by Dennis Neal is used as a guiding light to create a simple ‘go/no-go gauge’ for model usage.

Table 12 summarizes the SBC values found for each of the three models discussed.

Table 12: SBC Summary for all 3 Models

Model	SBC
Model 1	11841.10
Model 2	11847.76
Model 3	11548.82

Based on the criterion of SBC alone, Model 3 is the 'best' model to deploy. Although eight models were constructed, Model 3 had the lowest SBC value of any of the models constructed. There are a couple caveats that should be noted. First, Model 3 has a significant amount of multicollinearity. This may result in parameter coefficient values that are not necessarily meaningful. However, given Model 3's performance (in terms of SBC), it is worthwhile overlooking this shortcoming for the **initial** deployment. Second, a couple of the variables in this model have a sign issue that is not easy to explain. Since the intent of the model is to project overall wins, for this first iteration, it is acceptable to use this model due to its lower SBC value (compared to all the other models constructed). At this point, it may not be as essential to be concerned with individual parameters as continuing evaluation of the model is necessitated anyways.

Prior to deployment, it would be prudent to consult a subject matter expert to review each parameter's sign. For instance, if a parameter is negative, does that make sense? Furthermore, it would also be wise to see if additional data sources or data points could be retrieved and potentially implemented into the model. Nevertheless, as this is the first model, implementing into production will enable the team to understand its performance. It is suggested that this model be monitored for a period of three months (at a minimum) to better understand its performance as new data are consumed.

Conclusion

Eight different models were developed (of which only three have been discussed in this analysis) to predict the number of wins spanning from 1871 through 2006. The original dataset only contained 15 parameters and like any sport, baseball has dozens (if not hundreds) more parameters that could be used. Of note is the fact that many of the data points have been corrected to match the performance of today's 162 game season. This may have inevitably led to many values being extreme outliers or having outlandish results. Although a model has been chosen, future work is required to better understand some of the non-intuitive sign issues as well as the extreme VIF values. This will require further investigation which is outside the scope of this analysis. Finally, it would be prudent to monitor this chosen model's performance for the next three months as new data are fed into it. Next steps may include (but not limited to) refining the model and focusing the reduction of multicollinearity issues.

Bibliography

Beal, D. J. (2007). *Information Criteria Methods in SAS for Multiple Linear Regression Models*.

Retrieved October 02, 2016, from Institute for Advanced Analytics - North Carolina State University: <http://analytics.ncsu.edu/sesug/2007/SA05.pdf>

Unit 02 Homework – Insurance

KAGGLE Name: *NikhilAgarwal*

Nikhil Agarwal
Northwestern University
PREDICT 411, Section 55

I am requesting a total of 80 bingo bonus points for the unit 2 homework. Please see my justification below.

Points Requested	Category	Justification
20	Decision Tree	I have used JMP to develop decision trees for almost all of the variables that had missing values
10	Macros	I have extensively used macros
20	R Code	Much of the homework has been rewritten in R. Please see the attached homework (NikhilAgarwal_HW2_RCode.r)
5	PROBIT Model	The chosen model was also reran using PROBIT
5	CLOGCLOG Model	The chosen model was also reran using CLOGCLOG
20	KS Statistic	I used the PROC NPAR1WAY to calculate the KS statistic – specifically looking for the D value

Table of Contents

TABLE OF CONTENTS	3
INTRODUCTION	4
RESULTS	4
DATA EXPLORATION	4
DATA PREPARATION	7
IMPUTATION	7
OUTLIERS	9
NEGATIVE VALUE HANDLING	11
NEW VARIABLES	11
MODEL DEVELOPMENT	11
MODEL 1	12
MODEL 2	14
MODEL 3	16
MODEL SELECTION	19
CONCLUSION	20
BINGO BONUS: PROBIT MODEL	21
BINGO BONUS: CLOGCLOG MODEL	24

INTRODUCTION

The intent of this assignment is to develop a logistic function that can be used to predict the likelihood of an individual becoming involved in a vehicular accident. Information on over 8000 customers was used to help construct the model. Prior to developing a single model, multiple logistic models were explored using primarily the stepwise and backward selection methods. Various model diagnostic parameters (e.g., AIC, SBC, and ROC curves) were used to determine the best model.

RESULTS

Data Exploration

The original dataset contains 23 distinct variables (outlined in Table 1¹). These variables can be considered potential predictors. Note that the response variable will be TARGET_FLAG. The TARGET_FLAG is technically either a 0 or 1 (0 meaning customer not involved in accident and 1 meaning the customer involved in accident) in the data dictionary. However, the predicted response variable will simply indicate the likelihood (between 0 and 1) of the customer being involved in a vehicular accident.

Table 1: Brief description of default variables

Variable Name	Variable Type	Definition	Theoretical Effect
AGE	Continuous	Age of Driver	Very young people tend to be risky. Maybe very old people also.
BLUEBOOK	Continuous	Value of Vehicle	Unknown effect on probability of collision, but probably effect the payout if there is a crash
CAR_AGE	Continuous	Vehicle Age	Unknown effect on probability of collision, but probably effect the payout if there is a crash
CAR_TYPE	Categorical	Type of Car	Unknown effect on probability of collision, but probably effect the payout if there is a crash
CAR_USE	Categorical	Vehicle Use	Commercial vehicles are driven more, so might increase probability of collision
CLM_FREQ	Continuous	#Claims(Past 5 Years)	The more claims you filed in the past, the more you are likely to file in the future
EDUCATION	Categorical	Max Education Level	Unknown effect, but in theory more educated people tend to drive more safely
HOMEKIDS	Continuous	#Children @Home	Unknown effect
HOME_VAL	Continuous	Home Value	In theory, home owners tend to drive more responsibly
INCOME	Continuous	Income	In theory, rich people tend to get into fewer crashes
JOB	Categorical	Job Category	In theory, white collar jobs tend to be safer

¹ The information in this table were provided for this assignment

KIDSDRIV	Continuous	#Driving Children	When teenagers drive your car, you are more likely to get into crashes
MSTATUS	Categorical	Marital Status	In theory, married people drive more safely
MVR_PTS	Continuous	Motor Vehicle Record Points	If you get lots of traffic tickets, you tend to get into more crashes
OLDCLAIM	Continuous	Total Claims(Past 5 Years)	If your total payout over the past five years was high, this suggests future payouts will be high
PARENT1	Categorical	Single Parent	Unknown effect
RED_CAR	Categorical	A Red Car	Urban legend says that red cars (especially red sports cars) are more risky. Is that true?
REVOKED	Categorical	License Revoked (Past 7 Years)	If your license was revoked in the past 7 years, you probably are a more risky driver.
SEX	Categorical	Gender	Urban legend says that women have less crashes than men. Is that true?
TIF	Continuous	Time in Force	People who have been customers for a long time are usually more safe.
TRAVTIME	Continuous	Distance to Work	Long drives to work usually suggest greater risk
URBANICITY	Categorical	Home/Work Area	Unknown
YOJ	Continuous	Years on Job	People who stay at a job for a long time are usually more safe

As the data are explored, it is wise to identify if any of the variables are missing. Figure 1 illustrates some basic statistics on the dataset for the continuous variables.

Variable	Label	N	N Miss	Mean	Median	Minimum	Maximum	1st Pctl	99th Pctl
INDEX		8161	0	5151.87	5133.00	1.0000000	10302.00	103.0000000	10197.00
TARGET_FLAG		8161	0	0.2638157	0	0	1.0000000	0	1.0000000
TARGET_AMT		8161	0	1504.32	0	0	107586.14	0	19866.59
KIDSDRIV	#Driving Children	8161	0	0.1710575	0	0	4.0000000	0	2.0000000
AGE	Age	8155	6	44.7903127	45.0000000	16.0000000	81.0000000	25.0000000	64.0000000
HOMEKIDS	#Children @Home	8161	0	0.7212351	0	0	5.0000000	0	4.0000000
YOJ	Years on Job	7707	454	10.4992864	11.0000000	0	23.0000000	0	17.0000000
INCOME	Income	7716	445	61898.10	54028.17	0	367030.26	0	215536.28
HOME_VAL	Home Value	7697	464	154867.29	161159.53	0	885282.34	0	500309.15
TRAVTIME	Distance to Work	8161	0	33.4887972	32.8709696	5.0000000	142.1206304	5.0000000	75.1443301
BLUEBOOK	Value of Vehicle	8161	0	15709.90	14440.00	1500.00	69740.00	1500.00	39090.00
TIF	Time in Force	8161	0	5.3513050	4.0000000	1.0000000	25.0000000	1.0000000	17.0000000
OLDCLAIM	Total Claims(Past 5 Years)	8161	0	4037.08	0	0	57037.00	0	42820.00
CLM_FREQ	#Claims(Past 5 Years)	8161	0	0.7985541	0	0	5.0000000	0	4.0000000
MVR_PTS	Motor Vehicle Record Points	8161	0	1.6955030	1.0000000	0	13.0000000	0	8.0000000
CAR_AGE	Vehicle Age	7651	510	8.3283231	8.0000000	-3.0000000	28.0000000	1.0000000	21.0000000

Figure 1: Basic statistics on numerical variables

Note how five of the potential predictor variables have missing values. On a side note, the variable INDEX is simply a unique identifier that will not be used for any modelling purpose. Furthermore, the variable TARGET_FLAG is the key response variable for this assignment. TARGET_AMT is not documented nor modelled in this assignment.

Recall that there are ten categorical variables. Each one was explored in detail and it was discovered that the variable JOB had over 500 observations with missing values (see Figure 2).

For all variables (continuous and categorical) that have missing values, imputed values will be used in order to create a healthier model.

Job Category				
JOB	Frequency	Percent	Cumulative Frequency	Cumulative Percent
	526	6.45	526	6.45
Clerical	1271	15.57	1797	22.02
Doctor	246	3.01	2043	25.03
Home Maker	641	7.85	2684	32.89
Lawyer	835	10.23	3519	43.12
Manager	988	12.11	4507	55.23
Professional	1117	13.69	5624	68.91
Student	712	8.72	6336	77.64
z_Blue Collar	1825	22.36	8161	100.00

Figure 2: PROC FREQ output on JOB

Table 2 highlights the correlation of each continuous predictor variable to the response variable of TARGET_FLAG. The closer the value is to +1 or -1, the stronger the linear relationship. Unsurprisingly, no single predictor variable is highly correlated with TARGET_FLAG. Recall from Table 1 that certain continuous variables (such as TRAVTIME) could increase the likelihood of having an accident. In this case, if a variable (such as TRAVTIME) has a positive correlation, then it could be construed that it has a tendency to increase the TARGET_FLAG.

Table 2: PROC CORR output of TARGET_FLAG vs. numerical variables

Variable	Correlation
KIDSDRIV	0.10367
AGE	-0.10322
HOMEKIDS	0.11562
YOJ	-0.07051
INCOME	-0.14201
HOME_VAL	-0.18374
TRAVTIME	0.04815
BLUEBOOK	-0.10338
TIF	-0.08237
OLDCLAIM	0.13808
CLM_FREQ	0.2162
MVR_PTS	0.2192
CAR_AGE	-0.10065

Another key check is to determine if there are outliers. All 13 continuous predictor variables were checked for outliers. Figure 3 is an example of a histogram and boxplot for the variable INCOME. Note the many circles that are outside the whiskers in the boxplot. This is a strong indicator that outliers may be present.

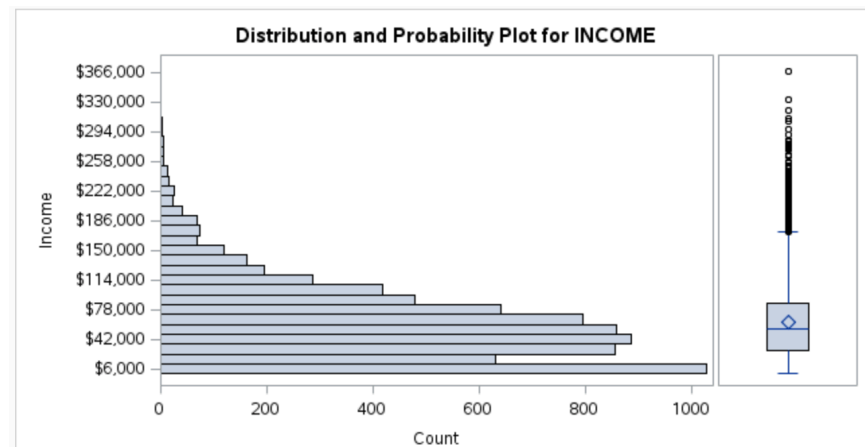


Figure 3: Histogram & Boxplot for INCOME

It was found that many of the continuous variables had outliers. Therefore, value caps will be deployed on each variable to ensure that outliers do not skew the model unnecessarily. The caps are explained in greater detail in the section, “Data Preparation”.

Data Preparation

Imputation

Recall from the previous section, “Data Exploration”, that there are six total variables (five continuous and one categorical) that have missing values. Table 3 summarizes the imputation method used for each variable. All six of these variables were removed from the modelling process and the imputed values were stored in different variables starting with the prefix “imp_”. If there was a non-null value for any of the variables in Table 3, then that value was used in lieu of an imputed value. Furthermore, a flag variable was created (with the prefix “m_”) to indicate if an imputed value was entered (indicated with a 1 meaning true) or if the original value was used (indicated with a 0 meaning false).

Table 3: Summary of imputation methods

Variable	Method of Imputation	Imputed Variable Name
AGE	Mean	imp_age
YOJ	Decision Tree	imp_yoj
INCOME	Decision Tree	imp_income
HOME_VAL	Decision Tree	imp_home_val
CAR_AGE	Decision Tree	imp_carage
JOB	Comparison of predictability	imp_job

Using JMP, decision trees were constructed for the appropriate variables (as listed in Table 3). An example decision tree for the variable YOJ is shown in Figure 4.

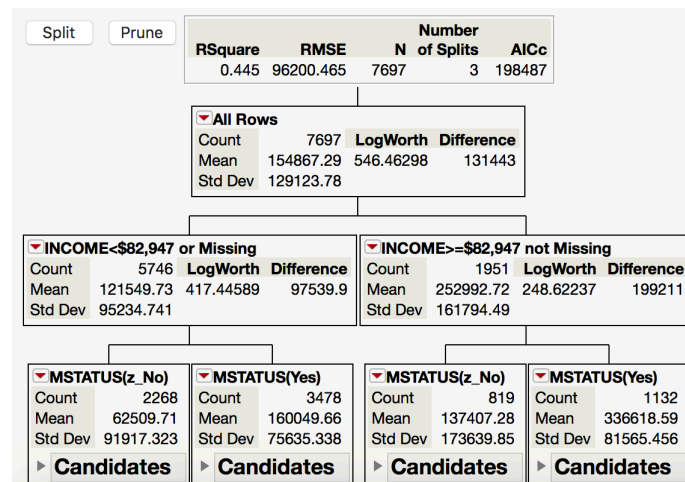


Figure 4: Example Decision Tree diagram from JMP

The following logic briefly describes (converting logic syntax to English) the decision tree for the variable HOME_VAL:

If INCOME is less than 82947 and then if MSTATUS is equal to NO, the imputed value for HOME_VAL should be 62509.71 otherwise it should be 160049.66. If INCOME is greater than or equal to 82947 and MSTATUS is NO then the imputed value for HOME_VAL is 137407.28 otherwise it should be 336618.59.

For the variable AGE, it was found that the distribution was fairly normal (see Figure 5) and it was decided to simply impute the missing values of AGE with the mean. Note that there are some outliers and they will be identified. Recall from Figure 1 that the mean value of AGE is approximately 45.

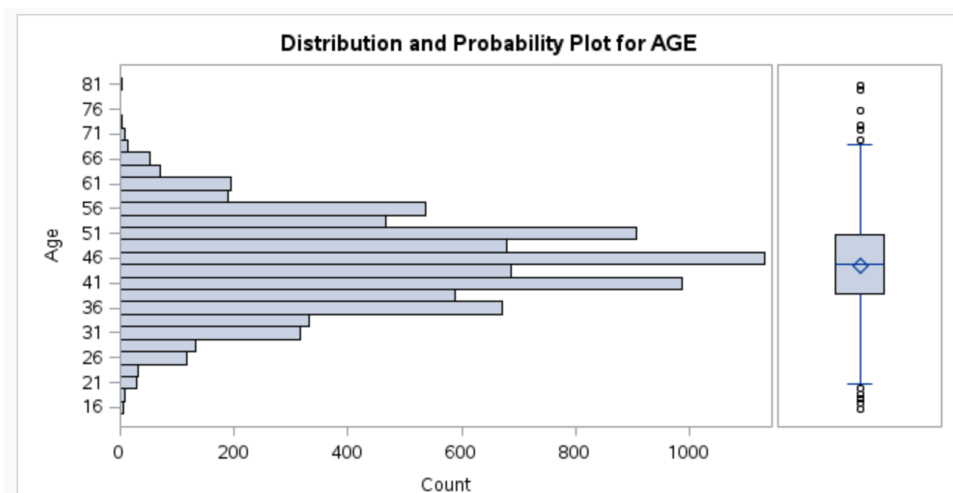


Figure 5: Histogram and Boxplot for AGE

For the variable JOB, a different approach was taken. Once all the other variables' missing values were imputed, a PROC MEANS statement comparing the imputed income to JOB was executed. The results are in Figure 6. The intent is to utilize the median income by job category

to construct an imputed value if the JOB value was left blank and then use the imputed income value to determine the job.

Analysis Variable : imp_income						
Job Category	N Obs	N	N Miss	Minimum	Maximum	Median
Clerical	1271	1271	0	2954.14	113845.14	32200.31
Doctor	246	246	0	9619.42	215000.00	126677.71
Home Maker	641	641	0	17.9823292	185747.34	11000.00
Lawyer	835	835	0	1404.91	215000.00	82659.88
Manager	988	988	0	1021.68	215000.00	77825.66
Professional	1117	1117	0	8458.21	215000.00	70293.04
Student	712	712	0	5.2859290	93066.98	6000.00
z_Blue Collar	1825	1825	0	5014.93	198319.70	55506.87

Figure 6: PROC MEANS for *imp_income* Vs. *JOB*

Using the median values in Figure 6, the thresholds for each job were created. Table 4 summarizes the chosen values (which are slightly different from the values in Figure 6).

Table 4: Summary of minimum income threshold by JOB

Job	Minimum Income for Job
Doctor	128000
Lawyer	90000
Manager	80000
Professional	75000
z_Blue Collar	57000
Clerical	33000
Home Maker	11000
Student	Less than 11000

Outliers

In order to reduce the effects of outliers, an upper threshold value was employed for seven of the continuous variables based on the 99th percentile. The intent is not to necessarily eliminate all outliers, but to reduce their effect on the overall model. These upper limits still allow for some 'peak' and 'valley' points. Table 5 highlights the upper thresholds used for the seven continuous variables. For each of these values, an outlier flag (with the syntax "out_") was used to indicate if the original value was an outlier.

Table 5: Outlier threshold summary

Variable	Upper Threshold
AGE	64
INCOME	215000
HOME_VAL	500000
TRAV_TIME	75
BLUEBOOK	40000
OLDCLAIM	43000
TIF	16

The minimum values were left alone for almost all of the variables. However, for the variable INCOME, many customers had reported a value of 0. This is somewhat peculiar, so a minimum threshold was identified for the variable INCOME. However, this minimum was designed to be dependent on the imputed job and was essentially the new median for each job (as seen in Figure 6).

Analysis Variable : imp_income						
imp_job	N Obs	N	N Miss	Minimum	Maximum	Median
Clerical	1325	1325	0	2954.14	113845.14	33055.77
Doctor	449	449	0	9619.42	215428.49	148595.01
Home Maker	655	655	0	17.9823292	185747.34	11000.00
Lawyer	975	975	0	1404.91	215000.00	90939.16
Manager	1021	1021	0	1021.68	215000.00	78582.21
Professional	1136	1136	0	8458.21	215000.00	70736.14
Student	713	713	0	5.2859290	93066.98	6000.00
z_Blue Collar	1887	1887	0	5014.93	198319.70	56848.52

Figure 7: PROC MEANS for imputed income with minimum threshold vs. imp_job

Table 6 summarizes the values coded into the algorithm for the minimum value of INCOME based on occupation. Recall that these values were only applied if and only if the original income reported by the customer was 0.

Table 6: Minimum Income Limit for Occupation

Occupation	Minimum Income Limit
Doctor	128000
Lawyer	90000
Manager	80000
Professional	75000
z_Blue Collar	57000
Clerical	33000
Home Maker	11000
Student	6000

Negative Value Handling

During the exploratory data analysis, it was discovered that the variable CAR_AGE had a negative value. None of the predictor variables should have a negative value. Therefore, all continuous variables were initially imputed with the absolute value of the original value. This will ensure that all values used in the model are correct in terms of having the wrong sign.

New Variables

Four new variables were created in order to better understand a customer's behavior. Table 7 illustrates the variables created and their intent.

Table 7: Summary of new variables constructed

Variable	Condition	Intent
f_renter	If HOME_VAL = 0 then 1 else 0	To understand if the customer is renting or owning a home
f_claimchk	If OLD_CLM > 0 then 1 else 0	To understand if the customer has had a claim in the past 5 years
f_speeder	If MVR_PTS > 0 then 1 else 0	To understand if the customer is a safe driver. Any points incurred may suggest he/she is not a safe driver
f_newjobchk	If YOJ > 0 then 1 else 0	If a customer has a new job, then it is possible he/she may be a more careful driver. 1 indicates that the customer does NOT have a new job
payout_per_year	OLDCLAIM/CLM_FREQ	To understand the average payout per instance if a payout occurred in the past 5 years

The intent of these variables is to understand a customer's 'behavior' at a high level with binary-like values (i.e., 0 meaning false and 1 meaning true) as well as understand the 'average' payout per instance if a payout has occurred.

Model Development

Over nine different models were constructed, but for the sake of but for the sake of succinctness, only three of them will be discussed in this report. Some of the models were constructed using either a stepwise selection method or a backward selection method. This approach enables a greater understanding of the statistical significance of a variable. For all models, the Log Odds and the corresponding probability calculation (i.e., the response variable, P_TARGET_FLAG) are all designed to derive the probability of a customer being involved in a vehicular accident. This was accomplished using the PROC LOGIT function in SAS with the reference set to 0.

As a primer, all of the models constructed have the following equation format:

$$\text{Log Odds} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

Using the derived log odds value, it is necessary to then convert this value to a probability using the following syntax:

$$P_Target_Flag = \frac{e^{LogOdds}}{1 + e^{LogOdds}}$$

This conversion is absolutely necessary in order to derive the probability of a customer being involved in a vehicular accident.

Model 1

The first model was designed to explore 41 total predictor variables including all imputation and outlier flags with the response variable being Log Odds. Table 8 illustrates a summary of the variables chosen and their respective coefficients.

Table 8: Model 1 Parameter Estimates

Variable	Variable Sub Element	Variable Code	Coefficient Notation	Coefficient Value
Intercept			β_0	-0.9323
CAR_TYPE	Minivan	X1	β_1	-0.6975
CAR_TYPE	Panel Truck	X2	β_2	-0.1037
CAR_TYPE	Pickup	X3	β_3	-0.1488
CAR_TYPE	Sports Car	X4	β_4	0.2565
CAR_TYPE	Van	X5	β_5	-0.0488
CAR_USE	Commercial	X6	β_6	0.7574
EDUCATION	<High School	X7	β_7	-0.0158
EDUCATION	Bachelors	X8	β_8	-0.408
EDUCATION	Masters	X9	β_9	-0.3716
EDUCATION	PhD	X10	β_{10}	-0.3664
MSTATUS	Yes	X11	β_{11}	-0.4437
PARENT1	No	X12	β_{12}	-0.458
REVOKED	No	X13	β_{13}	-0.9612
URBANICITY	Highly Urban/ Urban	X14	β_{14}	2.3613
imp_job	Clerical	X15	β_{15}	0.0555
imp_job	Doctor	X16	β_{16}	-0.4323
imp_job	Home Maker	X17	β_{17}	-0.3054
imp_job	Lawyer	X18	β_{18}	-0.1344
imp_job	Manager	X19	β_{19}	-0.8154
imp_job	Professional	X20	β_{20}	-0.1388
imp_job	Student	X21	β_{21}	-0.4019
KIDSDRIV		X22	β_{22}	0.4174

MVR_PTS		X23	β_{23}	0.0959
f_claimchk		X24	β_{24}	0.6371
f_newjobchk		X25	β_{25}	-0.5206
f_renter		X26	β_{26}	0.3353
imp_bluebook		X27	β_{27}	-0.00003
imp_income		X28	β_{28}	-5.69E-06
imp_oldclaim		X29	β_{29}	-0.00002
imp_tif		X30	β_{30}	-0.0562
imp_travtime		X31	β_{31}	0.0151
m_yoj		X32	β_{32}	-0.4195
out_bluebook		X33	β_{33}	0.6551
out_income		X34	β_{34}	0.7914
out_oldclaim		X35	β_{35}	8.8267

This model utilized a stepwise variable selection method and resulted in a total of 22 predictor variables (the table above shows many more variables, but note how the categorical variables are listed multiple times).

Figure 8 illustrates three key model fit statistics. Of key importance is to note that the values for these statistics is reduced when exploring both the intercept and covariates. Note that the lower the value of these statistics, the stronger the indicator that the model may be a better model.

Model Fit Statistics		
Criterion	Intercept Only	Intercept and Covariates
AIC	9419.962	7328.283
SC	9426.969	7580.539
-2 Log L	9417.962	7256.283

Figure 8: Model 1 Fit Statistics

Figure 9 illustrates the ROC curve for this model. The ROC curve simply describes the amount true positive rate versus the false positive rate. Essentially, the closer the curve is to the left hand side and towards the top, the truer positive the rate. However, this would most likely be an indicator of an over-fitted model. In this case, the goal is to balance the curve without overfitting the model. For this model, the metric of the ROC curve (i.e., area under the curve) is 0.8163 (a perfect curve would be 1 – a potential indicator of an over-fitted model).

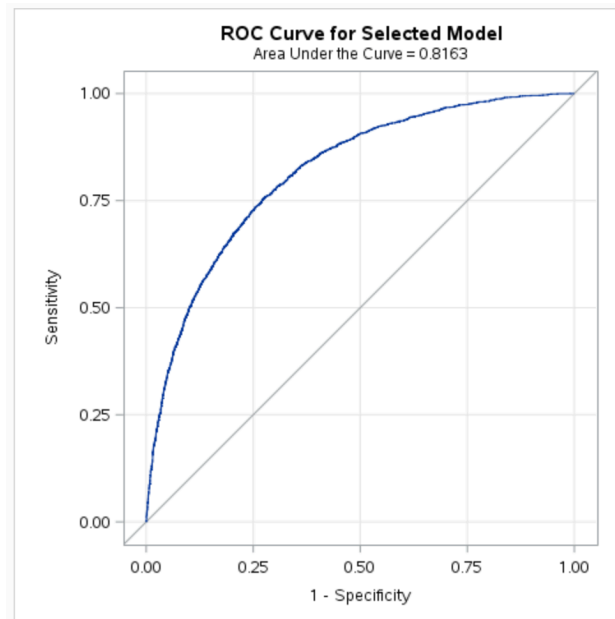


Figure 9: Model 1 ROC Curve

Model 2

For this model, a slightly different tactic was taken. Each variable's predictability was compared to the response variable of TARGET_FLAG. This enabled a greater understanding if a variable was predictable and useful to the overall modelling process. For instance, in this model, the variable RED_CAR or SEX were not provided as available variables for the model. It was found, during the EDA process, that these variables did not necessarily add predictability to the response variable. This model utilizes a backward variable selection method. Table 9 provides an overview of the variables chosen and their respective coefficients.

Table 9: Model 2 Parameter Estimates

Variable	Variable Subelement	Variable Code	Coefficient Notation	Coefficient Value
Intercept			β_0	-0.8466
f_renter		X1	β_1	0.3419
f_claimchk		X2	β_2	0.6536
f_newjobchk		X3	β_3	-0.2973
imp_income		X4	β_4	-6.54E-06
imp_oldclaim		X5	β_5	-0.00002
out_income		X6	β_6	0.9073
out_oldclaim		X7	β_7	9.0288
KIDSDRIV		X8	β_8	0.4073
TIF		X9	β_9	-0.0543
MVR_PTS		X10	β_{10}	0.0987

EDUCATION	<High School	X11	β_{11}	-0.00851
EDUCATION	Bachelors	X12	β_{12}	-0.3985
EDUCATION	Masters	X13	β_{13}	-0.3339
EDUCATION	PhD	X14	β_{14}	-0.3684
imp_job	Clerical	X15	β_{15}	0.0424
imp_job	Doctor	X16	β_{16}	-0.4455
imp_job	Home Maker	X17	β_{17}	-0.2172
imp_job	Lawyer	X18	β_{18}	-0.2051
imp_job	Manager	X19	β_{19}	-0.8784
imp_job	Professional	X20	β_{20}	-0.1607
imp_job	Student	X21	β_{21}	-0.2978
URBANICITY	Highly Urban/ Urban	X22	β_{22}	2.2258
PARENT1	No	X23	β_{23}	-0.4243
REVOKED	No	X24	β_{24}	-0.9586
CAR_TYPE	Minivan	X25	β_{25}	-0.7643
CAR_TYPE	Panel Truck	X26	β_{26}	-0.4419
CAR_TYPE	Pickup	X27	β_{27}	-0.1611
CAR_TYPE	Sports Car	X28	β_{28}	0.2635
CAR_TYPE	Van	X29	β_{29}	-0.2436
MSTATUS	Yes	X30	β_{30}	-0.4397
CAR_USE	Commercial	X31	β_{31}	0.7375

This model utilizes 27 predictor variables (in the table above, the categorical variables are listed multiple times) resulting in a chosen number of 18 predictor variables. This is in contrast to Model 1, which had 25 unique variables in the model. Looking at the various coefficients, there are some interesting conclusions that could be drawn by simply looking at the signs of each coefficient. For instance, a Minivan has a negative coefficient. This could be construed as Minivan is less likely to be involved in a vehicular accident since the driver may be transporting children and is driving safer. This is in contrast to the Sports Car which has a positive coefficient, suggesting that is more likely to be involved in a vehicular accident. An interesting peculiarity is the fact that a commercial car use significantly increases the probability of an accident, but the use of a panel truck has a significant reduction in the likelihood of an accident. A further point of investigation may be if a customer is using a vehicle for both personal and commercial use, but at the time of claim, the customer may have claimed the use on a commercial policy.

Figure 10 illustrates three key model fit statistics. Of key importance is to note that the values for these statistics is reduced when exploring both the intercept and covariates. Note that the lower the value of these statistics, the stronger the indicator that the model may be a better model.

Model Fit Statistics		
Criterion	Intercept Only	Intercept and Covariates
AIC	9419.962	7418.277
SC	9426.969	7642.505
-2 Log L	9417.962	7354.277

Figure 10: Model 2 Fit Statistics

Figure 11 illustrates the ROC curve for this model. The ROC curve simply describes the amount true positive rate versus the false positive rate. Essentially, the closer the curve is to the left hand side and towards the top, the truer positive the rate. However, this would most likely be a true indicator of an over-fitted model. In this case, the goal is to balance the curve without overfitting the model. For this model, the metric of the ROC curve is 0.8101 (a perfect curve would be 1 – a potential indicator of an over-fitted model).

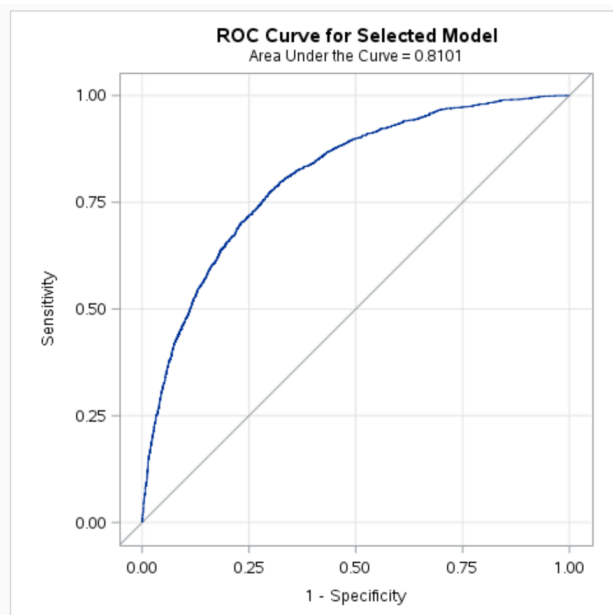


Figure 11: Model 2 ROC Curve

Model 3

For this model, all of the default variables were used without any imputation, outlier detection, missing value detection, etc. This model is most definitely wrong since it contains variables that have not been corrected for errors. As in the other models, the response variable continues to TARGET_FLAG. Table 10 summarizes the parameter estimates for this model.

Table 10: Model 3 Parameter Estimates

Variable	Variable Subelement	Variable Code	Coefficient Notation	Coefficient Value
Intercept			β_0	-1.1937

EDUCATION	<High School	X1	β_1	0.1709
EDUCATION	Bachelors	X2	β_2	-0.2306
EDUCATION	Masters	X3	β_3	-0.3229
EDUCATION	PhD	X4	β_4	0.2179
JOB	Clerical	X5	β_5	0.4064
JOB	Doctor	X6	β_6	-0.5026
JOB	Home Maker	X7	β_7	0.1165
JOB	Lawyer	X8	β_8	0.2448
JOB	Manager	X9	β_9	-0.6817
JOB	Professional	X10	β_{10}	0.1125
JOB	Student	X11	β_{11}	0.0933
URBANICITY	Highly Urban/ Urban	X12	β_{12}	1.1533
PARENT1	No	X13	β_{13}	-0.2363
REVOKED	No	X14	β_{14}	-0.4272
CAR_TYPE	Minivan	X15	β_{15}	-0.6216
CAR_TYPE	Panel Truck	X16	β_{16}	0.118
CAR_TYPE	Pickup	X17	β_{17}	-0.0705
CAR_TYPE	Sports Car	X18	β_{18}	0.4339
CAR_TYPE	Van	X19	β_{19}	-0.0212
MSTATUS	Yes	X20	β_{20}	-0.2077
CAR_USE	Commercial	X21	β_{21}	0.4148
KIDSDRIV		X22	β_{22}	0.3292
INCOME		X23	β_{23}	-3.50E-06
HOME_VAL		X24	β_{24}	-1.44E-06
TRAVTIME		X25	β_{25}	0.0156
BLUEBOOK		X26	β_{26}	-0.00002
TIF		X27	β_{27}	-0.0524
OLDCLAIM		X28	β_{28}	-0.00001
CLM_FREQ		X29	β_{29}	0.1995
MVR_PTS		X30	β_{30}	0.1165

This model also utilizes 30 predictor variables, but note how not all values were used to create this model (see Figure 12).

Number of Observations Read	8161
Number of Observations Used	6045

Figure 12: Model 3 observations summary

Note how some of the coefficients are in contrast to what was seen earlier. In this model, the student has a higher likelihood of being involved in a vehicular accident as compared to what was seen in Model 2. The same is true for the occupation of Professional. It is possible that this could be attributed to the fact that an individual with a higher paying job may be more likely to purchase a more expensive vehicle or drive at a higher speed (thus increasing the likelihood of being involved in an accident).

In this model, over 2000 of the observations were excluded by SAS during the model creation. This is virtually unacceptable as it eliminates the model's ability to accurately predict. However, this model is included to highlight how its AIC value is far below that of the other values. Recall that the AIC calculation relies on the number of observations. As the number of observations are reduced, it is inherent that the overall AIC value will also be lower (see Figure 13 and compare to Figure 10).

Model Fit Statistics		
Criterion	Intercept Only	Intercept and Covariates
AIC	6992.858	5432.223
SC	6999.565	5640.140
-2 Log L	6990.858	5370.223

Figure 13: Model 3 Fit Statistics

Figure 14 illustrates the ROC curve for this model. The ROC curve simply describes the amount true positive rate versus the false positive rate. Essentially, the closer the curve is to the left hand side and towards the top, the truer positive the rate. However, this would most likely be a true indicator of an over-fitted model. In this case, the goal is to balance the curve without overfitting the model. For this model, the metric of the ROC curve is 0.8180 (a perfect curve would be 1 – a potential indicator of an over-fitted model).

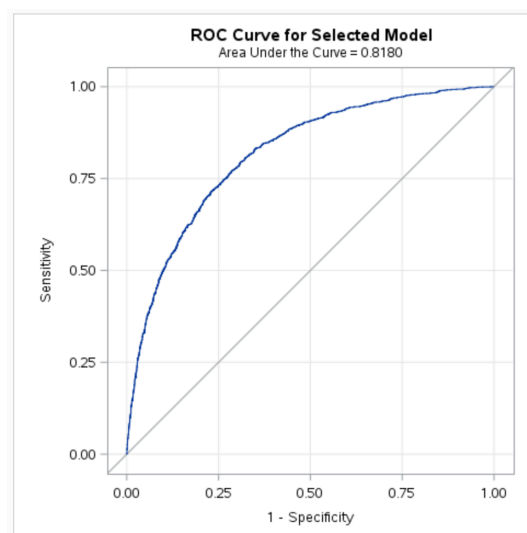


Figure 14: Model 3 ROC Curve

Model Selection

For the model selection process, three key metrics will be looked at: AIC, AUR (area under ROC curve), and the KS statistic. The KS statistic was determined using the PROC NPAR1WAY SAS procedure in order to develop the values. This value is not provided using PROC LOGISTIC. In the table below (Table 11), the D value is shown as it is the correct representation of the KS statistic. The AIC metric utilizes the intercept and covariate values. The D value is actually the two sample K-S test and analyzes the difference in both location and shape of the cumulative distribution functions of the two samples. The lower this number, the better the model (potentially). With the KS Statistic, the closer to 1, the more explanatory the model may be.

Table 11: Summary of fit statistics by model

Model	AIC	AUR	D Statistic (part of KS)	KS Statistic
Model 1	7328.283	0.8163	0.481161	0.212048
Model 2	7418.277	0.8101	0.474232	0.208994
Model 3	5432.223	0.8180	0.484227	0.213930

Based on these metrics alone, it would appear that Model 3 is the best model, followed by Model 1, and then by Model 2. However, at this point, it is important to point out that these metrics do not necessarily take into account the inherent cost of the model. By far, the parsimonious model is Model 2 as it only uses variables that have predictability against the response variable of TARGET_FLAG. Furthermore, as the D statistic is the maximum deviation between the two random samples, the minimal value would suggest that the model has better predictability. It is for this reason that Model 2 is the suggested model to use. This model is by no means the ‘perfect’ model as it does have some peculiarities. For instance, it is surprising to note that having an occupation of “clerical” tends to increase the likelihood of an individual being involved in a vehicular accident when compared to the other occupations. Another interesting case is how a commercial usage increases the likelihood of an accident, but a panel truck – which tends to be intuitively related to commercial usage – tends to have reduced likelihood of being involved in an accident.

Prior to deployment, it would be prudent to consult a subject matter expert to review each parameter’s sign. For instance, if a parameter is negative, does that make sense? Furthermore, it would also be wise to see if additional data sources or data points could be retrieved and potentially implemented into the model. Nevertheless, as this is the first model, implementing into production will enable the team to understand its performance. It is suggested that this model be monitored for a period of three months (at a minimum) to better understand its performance as new data are consumed.

CONCLUSION

Ten different models were developed (of which only three have been discussed in this analysis) to predict the likelihood of an individual being involved in a vehicular accident. The original dataset contained 23 distinct variables (a mixture of numeric and categorical types) with over 8000 observations. In the insurance world, there are much more data available that could potentially improve the overall model. One legal concern is the use of income to potentially penalize customers which may be considered illegal. For this analysis, many of the data points have been adjusted for being outliers or imputed if missing. Although a model has been chosen, future work is required to better understand some of the non-intuitive sign issues. This will require further investigation which is outside the scope of this analysis. Finally, it would be prudent to monitor this chosen model's performance for the next three months as new data are fed into it. Next steps may include (but not limited to) refining the model and focusing on improved data imputation.

BINGO BONUS: PROBIT MODEL

For the bingo bonus, it was asked to construct a model using the PROBIT option.

The chosen model for this assignment was reran using the PROBIT option. Note how the ROC curve (see Figure 15) has an AUC value of 0.81. This is roughly the same as what was seen in Model 2.

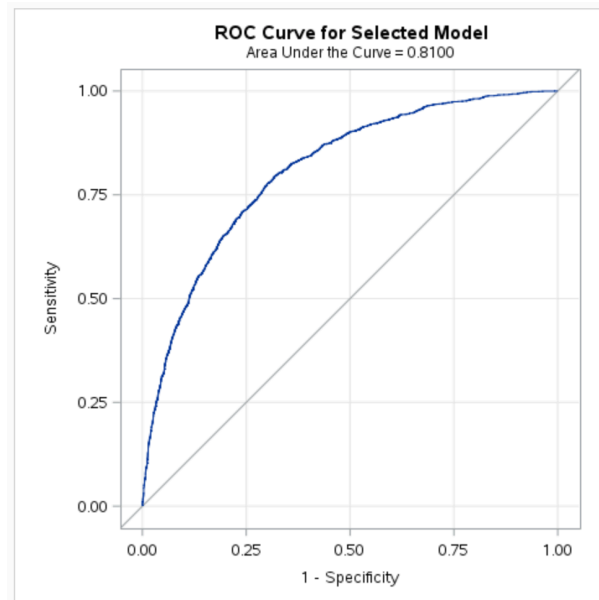


Figure 15: ROC Curve - PROBIT model

Note the AIC value of 7428.019 in Figure 16. This is a slightly higher value than what was seen for Model 2.

Model Fit Statistics		
Criterion	Intercept Only	Intercept and Covariates
AIC	9419.962	7428.019
SC	9426.969	7652.247
-2 Log L	9417.962	7364.019

Figure 16: Fit Statistics - PROBIT model

Table 12 highlights the parameter estimates for this model. Note how this model also has 31 parameter estimates – similar to Model 2.

Table 12: Parameter estimates - PROBIT model

Variable	Variable Subelement	Variable Code	Coefficient Notation	Coefficient Value
Intercept			β_0	-0.4387
f_renter		X1	β_1	0.1981
f_claimchk		X2	β_2	0.3885
f_newjobchk		X3	β_3	-0.1704
imp_income		X4	β_4	-3.65E-06
imp_oldclaim		X5	β_5	-0.00001
out_income		X6	β_6	0.4913
out_oldclaim		X7	β_7	5.248
KIDSDRIV		X8	β_8	0.2336
TIF		X9	β_9	-0.0321
MVR_PTS		X10	β_{10}	0.0581
EDUCATION	<High School	X11	β_{11}	-0.012
EDUCATION	Bachelors	X12	β_{12}	-0.2352
EDUCATION	Masters	X13	β_{13}	-0.19
EDUCATION	PhD	X14	β_{14}	-0.1992
imp_job	Clerical	X15	β_{15}	0.024
imp_job	Doctor	X16	β_{16}	-0.2863
imp_job	Home Maker	X17	β_{17}	-0.1362
imp_job	Lawyer	X18	β_{18}	-0.134
imp_job	Manager	X19	β_{19}	-0.4991
imp_job	Professional	X20	β_{20}	-0.0985
imp_job	Student	X21	β_{21}	-0.1588
URBANICITY	Highly Urban/ Urban	X22	β_{22}	1.2191
PARENT1	No	X23	β_{23}	-0.2427
REVOKED	No	X24	β_{24}	-0.5559
CAR_TYPE	Minivan	X25	β_{25}	-0.4315
CAR_TYPE	Panel Truck	X26	β_{26}	-0.259
CAR_TYPE	Pickup	X27	β_{27}	-0.0992
CAR_TYPE	Sports Car	X28	β_{28}	0.1506
CAR_TYPE	Van	X29	β_{29}	-0.1403
MSTATUS	Yes	X30	β_{30}	-0.2555
CAR_USE	Commercial	X31	β_{31}	0.4151

Finally, Figure 17 shows the KS statistic. Note how this D value (0.484227) is higher than the D value for Model 2 (0.474232). This is not a large difference, but it is noteworthy.

Kolmogorov-Smirnov Two-Sample Test (Asymptotic)			
KS	0.213930	D	0.484227
KSa	17.665778	Pr > KSa	<.0001

Figure 17: KS Statistic output

BINGO BONUS: CLOGCLOG MODEL

As an additional bingo bonus opportunity, Model 2 was reran using the CLOGCLOG option. Note how the area under the curve in Figure 18 is 0.8092. This is similar to what Model 2 had and similar to the ROC curve using the PROBIT option (see Figure 15).

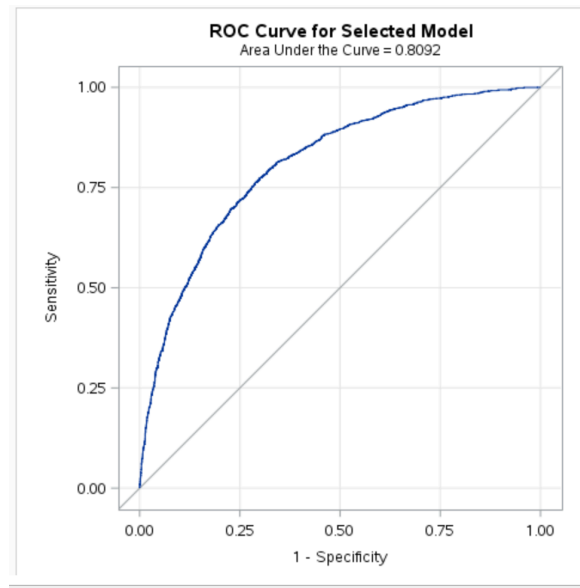


Figure 18: ROC curve - CLOGCLOG model

Note how the AIC value in Figure 19 is a bit higher than that of the PROBIT option (see Figure 16).

Model Fit Statistics		
Criterion	Intercept Only	Intercept and Covariates
AIC	9419.962	7446.808
SC	9426.969	7671.036
-2 Log L	9417.962	7382.808

Figure 19: Fit Statistics - CLOGCLOG model

Table 1 highlights the parameter estimates for this model. Note how this model also has 31 parameter estimates – similar to Model 2 and the PROBIT option. Recall that this equation simply produces the Log Odds value. This value must then be converted to a probability using the following equation (which is different from the PROBIT & LOGIT options):

$$P_TARGET_FLAG = e^{-1 * e^{LogOdds}}$$

Table 13: Parameter Estimates - CLOGCLOG model

Variable	Variable Subelement	Variable Code	Coefficient Notation	Coefficient Value
Intercept			β_0	-1.0896
f_claimchk		X1	β_1	0.4946
f_newjobchk		X2	β_2	-0.1912
imp_income		X3	β_3	-3.75E-06
imp_homeval		X4	β_4	-1.23E-06
imp_oldclaim		X5	β_5	-0.00002
out_income		X6	β_6	0.7288
out_oldclaim		X7	β_7	6.8258
KIDSDRIV		X8	β_8	0.2629
TIF		X9	β_9	-0.041
MVR_PTS		X10	β_{10}	0.0741
EDUCATION	<High School	X11	β_{11}	-0.0267
EDUCATION	Bachelors	X12	β_{12}	-0.3067
EDUCATION	Masters	X13	β_{13}	-0.2621
EDUCATION	PhD	X14	β_{14}	-0.3112
imp_job	Clerical	X15	β_{15}	-0.00189
imp_job	Doctor	X16	β_{16}	-0.3998
imp_job	Home Maker	X17	β_{17}	-0.176
imp_job	Lawyer	X18	β_{18}	-0.177
imp_job	Manager	X19	β_{19}	-0.7535
imp_job	Professional	X20	β_{20}	-0.155
imp_job	Student	X21	β_{21}	-0.2159
URBANICITY	Highly Urban/ Urban	X22	β_{22}	1.8619
PARENT1	No	X23	β_{23}	-0.2698
REVOKED	No	X24	β_{24}	-0.7512
CAR_TYPE	Minivan	X25	β_{25}	-0.5906
CAR_TYPE	Panel Truck	X26	β_{26}	-0.2705
CAR_TYPE	Pickup	X27	β_{27}	-0.0767
CAR_TYPE	Sports Car	X28	β_{28}	0.2109
CAR_TYPE	Van	X29	β_{29}	-0.1557
MSTATUS	Yes	X30	β_{30}	-0.3387
CAR_USE	Commercial	X31	β_{31}	0.5373

Finally, Figure 20 shows the KS statistic. Note how this D value (0.474107) is lower than the D for the PROBIT model (0.484227) and very similar to the D value for Model 2 (0.474232).

Kolmogorov-Smirnov Two-Sample Test (Asymptotic)			
KS	0.208939	D	0.474107
KSa	18.875194	Pr > KSa	<.0001

Figure 20: KS Statistic - CLOGCLOG model

Unit 03 Homework – Wine

KAGGLE Name: *NikhilAgarwal*

Nikhil Agarwal
Northwestern University
PREDICT 411, Section 55

I am requesting a total **90** bingo bonus points for the unit 3 homework. Please see my justification below.

Points Requested	Category	Justification
20	R Code	Much of the homework has been rewritten in R. Please see the attached homework (NikhilAgarwal_HW3_RCode.r)
10	Macros	Extensive use of macros
20	Decision Tree	Decision trees were investigated and not used. Explanation is provided on page 10-11.
20	Logistic Hurdle Model	I created & discussed a Logistic Hurdle model (see page 19).
20	Decision Tree used to predict Target	I used JMP to create a decision tree and predict TARGET. See page 22.

Table of Contents

TABLE OF CONTENTS	3
INTRODUCTION	4
RESULTS	4
DATA EXPLORATION	4
DATA PREPARATION	8
CONVERSION TO NON-NEGATIVE VALUES	8
IMPUTATION	10
MODEL DEVELOPMENT	12
MODEL 1 – REGRESSION	12
MODEL 2 – POISSON	14
MODEL 3 – NEGATIVE BINOMIAL	15
MODEL 4 – ZERO INFLATED POISSON (ZIP)	16
MODEL 5 – ZERO INFLATED NEGATIVE BINOMIAL (ZINB)	18
MODEL 6 – LOGISTIC HURDLE	19
MODEL SELECTION	20
CONCLUSION	21
BINGO BONUS – USING A DECISION TREE TO PREDICT TARGET CASES	22

Introduction

The intent of this assignment is to develop a model that can predict the number of sample cases of wine purchased by distributors. Over 12,000 data points were used to construct five types of models: linear regression, Poisson, negative binomial, zero inflation Poisson, and zero inflation negative binomial. Many of the data points provided offer an insight into the chemical properties of each type of wine. Various model diagnostic parameters (e.g., AIC, SBC, RMSE) were used to determine the best model.

Results

Data Exploration

The original dataset contains over 12,000 data points consisting of 14 potential predictor variables with one response variable (see Table 1).

Table 1: Variable Overview

Variable	Description
AcidIndex	Proprietary method of testing total acidity of wine by using a weighted average
Alcohol	Alcohol Content
Chlorides	Chloride content of wine
CitricAcid	Citric Acid Content
Density	Density of Wine
FixedAcidity	Fixed Acidity of Wine
FreeSulfurDioxide	Sulfur Dioxide content of wine
LabelAppeal	Marketing Score indicating the appeal of label design for consumers. High numbers suggest customers like the label design. Negative numbers suggest customers don't like the design.
ResidualSugar	Residual Sugar of wine
STARS	Wine rating by a team of experts. 4 Stars = Excellent, 1 Star = Poor
Sulphates	Sulfate content of wine
TotalSulfurDioxide	Total Sulfur Dioxide of Wine
VolatileAcidity	Volatile Acid content of wine
pH	pH of wine

Two of the variables, LabelAppeal and STARS, are interesting in the sense that as both values increase, the generally accepted thought is that net sales are also higher. As for the chemical properties, the analysis posited assumes that no prior knowledge is available. The response variable TARGET (not listed in Table 1) is essentially the number of cases of wine samples sold.

Table 2 illustrates the descriptive statistics (PROC MEANS) of the variables contained within the original dataset.

Table 2: Descriptive Statistics on Original Variables

Variable	N	N Miss	Minimum	Maximum	Median	Mean	1st Pctl	99th Pctl
TARGET	12795	0	0	8	3	3.0290739	0	7
FixedAcidity	12795	0	-18.1	34.4	6.9	7.0757171	-10.9	24.4
VolatileAcidity	12795	0	-2.79	3.68	0.28	0.3241039	-1.865	2.59
CitricAcid	12795	0	-3.24	3.86	0.31	0.3084127	-2.18	2.66
ResidualSugar	12179	616	-127.8	141.15	3.9	5.4187331	-91	99.2
Chlorides	12157	638	-1.171	1.351	0.046	0.0548225	-0.859	0.957
FreeSulfurDioxide	12148	647	-555	623	30	30.8455713	-388	469
TotalSulfurDioxide	12113	682	-823	1057	123	120.7142326	-531	767
Density	12795	0	0.88809	1.09924	0.99449	0.9942027	0.9168	1.06981
pH	12400	395	0.48	6.13	3.2	3.2076282	1.32	5.125
Sulphates	11585	1210	-3.13	4.24	0.5	0.5271118	-2.13	3.16
Alcohol	12142	653	-4.7	26.5	10.4	10.4892363	0.1	20.3
LabelAppeal	12795	0	-2	2	0	-0.009066	-2	2
AcidIndex	12795	0	4	17	8	7.7727237	6	13
STARS	9436	3359	1	4	2	2.041755	1	4

This dataset contains exactly 12,795 observations, but from Table 2, it is clear to see that eight (ResidualSugar, Chlorides, FreeSulfurDioxide, TotalSulfurDioxide, pH, Sulphates, Alcohol, and STARS) of the 14 variables have missing values and will require imputation. Another peculiar observation has to do with the minimum values for several of the chemical properties. General intuition dictates that many of the chemical properties cannot have negative values as this would be against basic laws of matter, chemistry, and physics. For instance, it is not possible to have negative values for alcohol content. The same is true for all of the other chemical properties listed. Therefore, all of the chemical properties, at a minimum, will be imputed to the absolute value of the provided negative value. If the value is missing, it will be imputed using either the mean or the median (explained in greater detail in the section “Data Imputation”)

The variable LabelAppeal can have a negative value as well as a positive value. A negative value would simply imply that the appearance of the label has a detrimental effect on the net sales of the particular wine. It is not clearly understood of the impact of the variable AcidIndex. The variable STARS implies that the higher number of stars that a wine may have, the higher the likelihood of the wine selling in larger quantities.

Table 3 is the correlation matrix of the variable TARGET versus the 18 different predictor variables. The intent here is to understand which set of variables have the strongest correlation to the response variable TARGET. Furthermore, this table can also be used to understand the variable’s impact on the response variable. For instance, if it has a negative sign, then it has a detrimental impact.

Table 3: Correlation Matrix of Target vs. Predictor Variables

	TARGET
TARGET	1
FixedAcidity	-0.04901
VolatileAcidity	-0.08879
CitricAcid	0.00868
ResidualSugar	0.01649
Chlorides	-0.03826
FreeSulfurDioxide	0.04382
TotalSulfurDioxide	0.05148
Density	-0.03552
pH	-0.00944
Sulphates	-0.03885
Alcohol	0.06206
LabelAppeal	0.3565
AcidIndex	-0.24605
STARS	0.55879

No variable has an impressively strong correlation (positive or negative) to the response variable TARGET. Of particular note is the variable STARS. It is clear to see that it has a moderately strong correlation of about 0.56. This is also unsurprising since it is generally accepted that the more stars a wine has, the more likely it will sell in larger quantities. Similarly, the variable LabelAppeal also has a moderate positive impact on TARGET.

Note how AcidIndex has a moderate negative impact on TARGET. This suggests that the higher a wine's AcidIndex is, the more likely it is that it will sell in fewer quantities. Surprisingly, the variables CitricAcid and pH do not seem to have any correlation (very little) to the TARGET. Similarly, it is also surprising to note that the variable Alcohol also has a weak correlation to the response variable.

Several histograms and boxplots were constructed to understand the distribution of each of the predictor variables. An example of the histogram and boxplot are represented in Figure 1. Note how the distribution is not necessarily normal and has some skewness. The boxplot clearly shows that there are many outliers. It is important to note that at this exploration point, no values have been corrected for negativity (as explained earlier) nor have any imputations taken place.

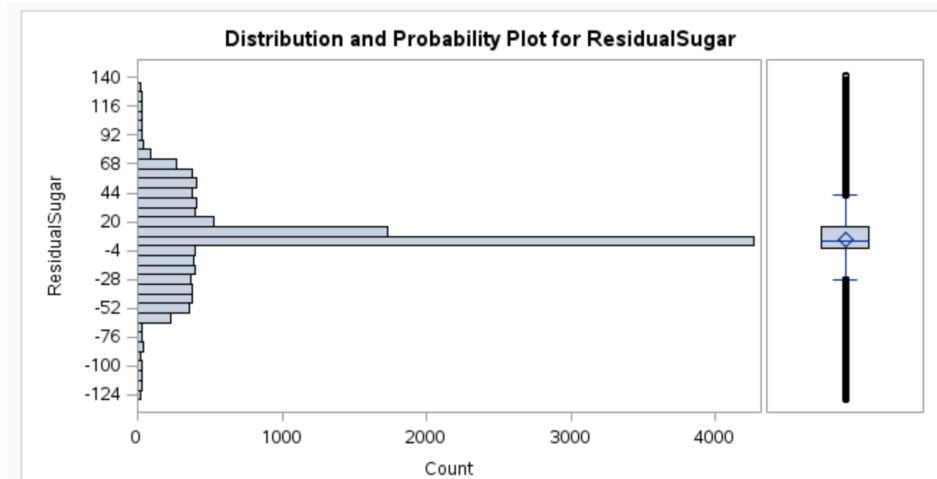


Figure 1: Histogram and Boxplot for Variable ResidualSugar

For the sake of brevity in terms of graphical outputs, Table 4 highlights the skewness¹ and kurtosis² for each of the predictor variables. Recall that if a variable has a skewness of 1 and a kurtosis value of 3, then it can be construed as normally distributed.

Table 4: Skewness, Kurtosis, and Variance for Predictor Variables

Variable	Skewness	Kurtosis	Variance
TARGET	-0.3263776	-0.8767876	3.7108945
FixedAcidity	-0.0225913	1.6768536	39.9126188
VolatileAcidity	0.0203847	1.8341516	0.6146783
CitricAcid	-0.0503188	1.8398842	0.7431816
ResidualSugar	-0.053136	1.886761	1139.02
Chlorides	0.0304347	1.7906222	0.1014214
FreeSulfurDioxide	0.0063946	1.8385435	22116.02
TotalSulfurDioxide	-0.0071811	1.6766257	53783.74
Density	-0.0186981	1.9019373	0.000704247
pH	0.0442987	1.6481659	0.4619745
Sulphates	0.0059134	1.7546612	0.868865
Alcohol	-0.0307234	1.5413715	13.8966348
LabelAppeal	0.0084314	-0.2614968	0.79404
AcidIndex	1.6488825	5.1938712	1.752781
STARS	0.4473775	-0.6917759	0.8145785

It's clear to see that all of the variables are not perfectly normally distributed. Not the large variance for the variable TotalSulfurDioxide.

¹ Skewness measures symmetry of a distribution with 0 meaning equal distribution on both sides.

² Kurtosis describes the 'shape' of a distribution in terms of heavy tailed or light tailed relative to a normal distribution.

Data Preparation

Conversion to Non-Negative Values

The first step in the data preparation process was to convert all variable (except for LabelAppeal) values to the absolute values. This would ensure that no non-negative values would exist. For this process, two new types of variables were created. A variable starting with “n_” was created to act as a negative flag indicator with a default value of 0. Essentially, if the variable value was negative and then converted to an absolute value, the flag value was changed from 0 to 1. The negative value could be predictive to the overall model. Note that a negative flag indicator was not created for the variable LabelAppeal as this variable was deemed to predictive by itself. The next variable created started with the syntax “imp_”. This new variable held the absolute value of the original value. This enables the analysis to maintain data integrity for the original values. Finally, the variable names following the aforementioned prefixes were also shortened for brevity and simplicity during the coding process (see Table 5).

Table 5: Summary of Variable Names

Original Variable	New Variable Name	Missing Flag Indicator Variable Name	Negative Flag Indicator Variable Name
FixedAcidity	imp_fa	m_fa	n_fa
VolatileAcidity	imp_va	m_va	n_va
CitricAcid	imp_ca	m_ca	n_ca
ResidualSugar	imp_rs	m_rs	n_rs
Chlorides	imp_chlo	m_chlo	n_chlo
FreeSulfurDioxide	imp_fsd	m_fsd	n_fsd
TotalSulfurDioxide	imp_tsd	m_tsd	n_tsd
Density	imp_density	m_density	n_density
pH	imp_ph	m_ph	n_ph
Sulphates	imp_sulf	m_sulf	n_sulf
Alcohol	imp_alcohol	m_alcohol	n_alcohol
LabelAppeal	imp_la	m_la	NOT CREATED
AcidIndex	imp_ai	m_ai	n_ai
STARS	imp_stars	m_stars	n_stars

Table 6 highlights the new descriptive statistics using the new non-negative values. Note that at this point, no missing data have been imputed. Of particular interest is the “Delta Mean” column. This column describes the difference between the Mean in Table 6 and Table 2 (descriptive statistics of the original values).

Table 6: Descriptive Statistics for New Predictor Variables

Variable	Minimum	Maximum	Median	Mean	1st Pctl	99th Pctl	Delta Mean
imp_fa	0	34.4	7	8.0632513	0.2	24.4	0.9875342
imp_va	0	3.68	0.41	0.6410856	0.03	2.65	0.3169817
imp_ca	0	3.86	0.44	0.686315	0.01	2.86	0.3779023
imp_rs	0	141.15	12.9	23.3678093	0.9	112.7	17.9490762
imp_chlo	0	1.351	0.098	0.2225586	0.011	1.065	0.1677361
imp_fsd	0	623	56	106.6790418	3	503	75.8334705
imp_tsd	0	1057	154	204.31912	9	771	83.6048874
imp_density	0.88809	1.09924	0.99449	0.9942027	0.9168	1.06981	0
imp_ph	0.48	6.13	3.2	3.2076282	1.32	5.125	0
imp_sulf	0	4.24	0.59	0.8466681	0.03	3.16	0.3195563
imp_alcohol	0	26.5	10.4	10.5237775	1.6	20.3	0.0345412
imp_la	-2	2	0	-0.009066	-2	2	0
imp_ai	4	17	8	7.7727237	6	13	0
imp_stars	1	4	2	2.041755	1	4	0

Note the substantial difference in means for the variables imp_fsd (related to FreeSulfurDioxide) and imp_tsd (related to TotalSulfurDioxide). This implies that the change in using the absolute value has had a major impact on the overall understanding of the variables. Table 7 highlights the skewness and kurtosis of the new variables using the absolute values rather than negative values.

Table 7: Skewness, Kurtosis, and Variance for New Variables

Variable	Skewness	Kurtosis	Variance	Delta skewness	Delta kurtosis	Delta variance
imp_fa	1.174556	1.968638	24.9612014	1.1971473	0.2917844	-14.9514174
imp_va	1.6533658	3.0859297	0.3087071	1.6329811	1.2517781	-0.3059712
imp_ca	1.6431954	2.9499946	0.3672423	1.6935142	1.1101104	-0.3759393
imp_rs	1.4691618	2.2353592	622.2863083	1.5222978	0.3485982	-516.7336917
imp_chlo	1.4811473	2.1772043	0.0548908	1.4507126	0.3865821	-0.0465306
imp_fsd	1.5301341	2.445134	11686.19	1.5237395	0.6065905	-10429.83
imp_tsd	1.6112749	3.0384944	26607.12	1.618456	1.3618687	-27176.62
imp_density	-0.0186981	1.9019373	0.000704247	0	0	0
imp_ph	0.0442987	1.6481659	0.4619745	0	0	0
imp_sulf	1.6918105	3.2105619	0.429827	1.6858971	1.4559007	-0.439038
imp_alcohol	0.1825913	1.0607644	13.170759	0.2133147	-0.4806071	-0.7258758
imp_la	0.0084314	-0.2614968	0.79404	0	0	0
imp_ai	1.6488825	5.1938712	1.752781	0	0	0
imp_stars	0.4473775	-0.6917759	0.8145785	0	0	0

The last three columns in Table 7 describe the change in skewness, kurtosis, and variance from the original variable values (compared to the values in Table 4). Through the use of the absolute values, skewness and kurtosis for almost all of the variables has changed. As an example, the kurtosis for alcohol has been reduced. In terms of variance, the change in values has been somewhat impressive. The variance for imp_rs (ResidualSugar) has been greatly reduced as well as for both imp_fsd (FreeSulfurDioxide) and imp_tsd (TotalSulfurDioxide). Nevertheless, the variance for both imp_fsd and imp_tsd is quite high compared to the other variables.

Imputation

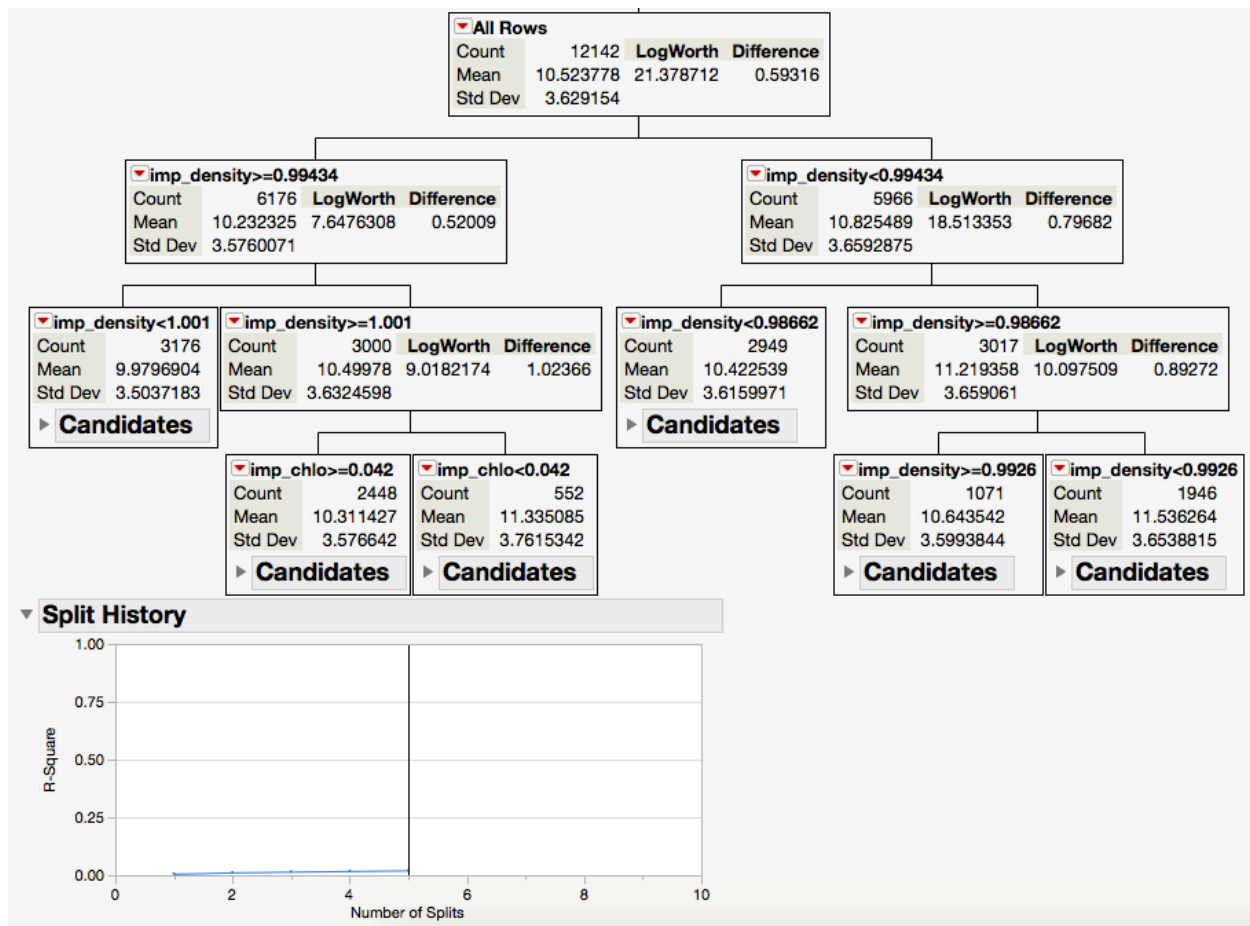
Recall from Table 2 that there are eight predictor variables that have missing values. For the imputation process, the means of each the variables (except for `imp_stars`) after converting to non-negative values was used. A decision tree was investigated, but it was discovered that the decision trees for each of the eight different variables did not produce satisfying results.

JMP was used to construct decision trees for seven of the eight predictor variables. Table 8 highlights the R-square value obtained after five splits were constructed for each of the seven predictor variables.

Table 8: R-Square Value of Decision Tree Splits for Variables Needing Imputation

Variable	R ² Value
<code>imp_alcohol</code>	0.021
<code>imp_chlo</code>	0.006
<code>imp_fsd</code>	0.005
<code>imp_tsd</code>	0.011
<code>imp_ph</code>	0.008
<code>imp_sulf</code>	0.006
<code>imp_rs</code>	0.004

The obtained R² values for these seven variables is unacceptably low. Furthermore, the obtained trees are needlessly complex and suspected of doing providing little value to the overall imputation process. Figure 2 highlights the decision tree for the variable `imp_alcohol`.

Figure 2: Example Decision Tree Split for *imp_alcohol*

In this tree, the imputed values for *imp_alcohol* tend to be between 10 and 11 (approximately). Recall from Table 6, that the mean for *imp_alcohol* is 10.5. Therefore, it is much simpler and cost effective to utilize the mean of the variables rather than a decision tree. Table 9 provides an overview of the imputation method and the imputed value for all of the eight predictor variables with missing values.

Table 9: Imputation Method & Value Summary

Variable	Method of Imputation	Imputed Value
<i>imp_alcohol</i>	mean	10.524
<i>imp_chlo</i>	mean	0.222
<i>imp_fsd</i>	mean	106.679
<i>imp_tsd</i>	mean	204.319
<i>imp_ph</i>	mean	3.207
<i>imp_sulf</i>	mean	0.847
<i>imp_rs</i>	mean	23.368
<i>imp_stars</i>	defaulting to 0	0

For the variable `imp_stars`, a value of 0 was used to impute missing values. The thought behind this step is that the lack of stars may be predictive and therefore indicative of the net sales. In an effort to reduce errors when using new data points after a model has been constructed, a safety check was built into the coding. This safety check imputes the mean for all variables (except `imp_stars`). As an example, it was found that the variable `imp_density` did not have any missing values. However, the mean of `imp_density` (according to Table 6) is approximately 0.994. This value has been used as the imputed value if and only if a value is missing for `imp_density`. In terms of the training dataset, this variable did not have any missing values. However, future data may have missing values for this variable.

Finally, a missing flag variable was created (annotated with the prefix “m_”) with a default value of 0. If there was a missing value, then the flag would change to 1 indicating true. The thought here is that the missing value in itself may be predictive.

Model Development

Six types of models were constructed: Regression, Poisson, Negative Binomial, Zero Inflation Poisson, Zero Inflation Negative Binomial, and Logistic Hurdle. For each type, multiple models were constructed utilizing a variety of parameters. Ultimately, a routine was written to calculate both the Average Error and the Root Mean Square Error (RMSE). The chosen model was based primarily on the derived RMSE in combination with parameter value intuition. For the sake of brevity, only one model for each type will be discussed in this analysis.

Model 1 – Regression

For this model, a multiple linear regression model was constructed. This model utilized the stepwise variable selection method. 41 various predictor variables were fed into the model. These included 14 missing flag indicators (one for each predictor variable, with the prefix “m_”), 13 negative flag indicators (one for each predictor variable excluding `LabelAppeal`, with the prefix “n_”), and 14 imputed variables (imputations of the original variables; containing the prefix “imp_”). The regression equation for this model can be constructed as:

$$TARGET = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

β_0 is the intercept, β_n is the parameter estimate of the variable number n , X_n is the observed value in the dataset for the variable number n , and ε is the standard error term.

When executing the regression modeling with stepwise variable selection, only 18 of the 41 predictor variables were selected. Table 10 provides the appropriate parameter values for this model.

Table 10: Parameter Estimate for Model 1 (Regression)

Parameter	Notation	Value
Intercept	β_0	4.40897
m_fsd	β_1	0.09741
m_ph	β_2	-0.09652
m_stars	β_3	-0.68713
n_va	β_4	0.07151
n_fsd	β_5	-0.05997
n_alcohol	β_6	0.07468
imp_va	β_7	-0.11876
imp_ca	β_8	0.03167
imp_chlo	β_9	-0.08509
imp_fsd	β_{10}	0.00024562
imp_tsd	β_{11}	0.00025049
imp_density	β_{12}	-0.8292
imp_ph	β_{13}	-0.03095
imp_sulf	β_{14}	-0.03383
imp_alcohol	β_{15}	0.01398
imp_la	β_{16}	0.46529
imp_ai	β_{17}	-0.20255
imp_stars	β_{18}	0.78051

Several key observations can be made from this output. For instance, notice how the coefficient for imp_fsd (FreeSulfurDioxide) and imp_tsd (TotalSulfurDioxide) is almost 0. It also seems that as the density of the wine increases, the more of a negative impact on overall cases sold. This is also the case with the variable imp_ai (AcidIndex). This suggests that as the wine's AcidIndex increases, the relationship to cases sold is inverse. Unsurprisingly, the more stars (imp_stars) a wine has, the more of a positive impact to the cases sold. This implies that the higher a wine is rated, the more likely it will be sold in larger quantities. Perhaps a surprising discovery is the positive value associated with the negative flag variable n_alcohol. Could it be that a negative alcohol reported value has an impact on the sale of wine cases? The variable imp_alcohol suggests that there is a very slight positive impact to the overall wine sample case sales. Finally, the pH (imp_ph) coefficient of -0.031 suggests that as a wine becomes more basic, the lower the overall sales. Recall that the pH scale goes from 0 to 14 with a pH value of 7 being neutral, greater than 7 being basic, and a value of less than 7 being acidic. This seems to be in contrast to the AcidIndex (imp_ai) variable. Further clarification needs to be sought for the metric AcidIndex.

The variables chosen in this model (outlined in Table 10) will serve as the basis for all the subsequent models in this analysis. The stepwise variable selection method identified statistically significant variables and this is an acceptable starting point for the other types of models.

Model 2 – Poisson

The next model constructed was a Poisson regression model. Recall from prior discussion that the goal of this analysis is to predict how many sample cases of wine a customer may purchase. Figure 3 illustrates the histogram (generated using R) of the response variable, TARGET.

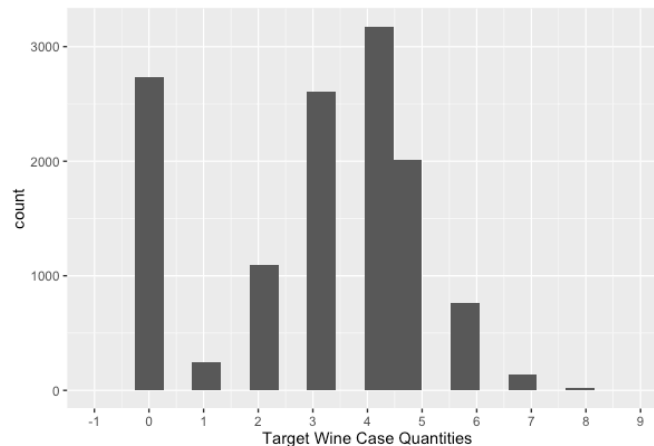


Figure 3: Histogram of TARGET

Note how there's a major spike at a value of 0. This clearly suggests that a zero-inflated model may be more appropriate. However, for illustrative purposes, a Poisson model is continued with. Furthermore, note how the distribution is approximately normally distributed (if the spike at 0 is excluded).

One of the key characteristics of a Poisson model is the need to predict a target variable that is a positive integer. The response variable, in this analysis, has a minimum value of 0 and can be any positive integer value. Furthermore, in a Poisson distribution, the mean and variance are also the same (not necessarily the same, but approximately close). The mean for the TARGET variable is approximately 3.03 and the variance is approximately 3.71. The general equation for a Poisson regression is like the following:

$$\ln(Y) = \beta_0 + \beta_1 X_1 + \cdots \beta_n X_n + \varepsilon$$

$$Y = e^{\ln(Y)}$$

The first step essentially a natural log of the count and the second step will convert the natural log to a meaningful value. β_0 is the intercept, β_n is the parameter estimate of the variable number n , X_n is the observed value in the dataset for the variable number n , and ε is the standard error term.

Recall from the Regression Model that only 18 predictor variables were selected. Since there was not an easy way to do automatic variable selection with Poisson regression, only these 18 predictor variables were used to model the response variable TARGET. Table 11 provides the appropriate parameter values for this model.

Table 11: Parameter Estimates for Model 2 (Poisson)

Parameter	Notation	Value
Intercept	β_0	1.7904
m_fsd	β_1	0.0313
m_ph	β_2	-0.038
m_stars	β_3	-0.6478
n_va	β_4	0.0232
n_fsd	β_5	-0.0216
n_alcohol	β_6	0.0215
imp_va	β_7	-0.0412
imp_ca	β_8	0.0094
imp_chlo	β_9	-0.0301
imp_fsd	β_{10}	0.0001
imp_tsd	β_{11}	0.0001
imp_density	β_{12}	-0.2889
imp_ph	β_{13}	-0.013
imp_sulf	β_{14}	-0.0129
imp_alcohol	β_{15}	0.004
imp_la	β_{16}	0.1587
imp_ai	β_{17}	-0.0813
imp_stars	β_{18}	0.1883

Similar conclusions to the ones made for the Regression model can be drawn upon again. Of interest is the continued predictability drawn from the variable m_stars. Recalls that this variable has a binary (has a value of 0 meaning false or 1 meaning true) value indicating if the original data point is missing or not. Missing stars for a wine continue to have a negative impact on the overall unit sales. Note how the density of a wine (imp_density) continues to have a negative impact. A data point that would assist this analysis is the type of wine. From the original dataset, it is not clear if a wine is red, white, port, Riesling, etc. This would enable a clearer understanding of what kind of wine has what density.

Chlorides (imp_chlo) continue to have a negative impact on the overall sales, but this is where a subject matter expert could assist the findings to ensure that the negative impact makes sense. Surprisingly, the sulfur dioxide variables (imp_fsd and imp_tsd) seem to have negligible impact on the overall sales. Another surprising note is the minimal impact of the alcohol (imp_alcohol). This is another aspect that could be explored further with a subject matter expert.

Model 3 – Negative Binomial

Recall that the mean and variance for the response variable, TARGET, are approximately 3.03 and 3.71 respectively. Since the variance is larger, a negative binomial model may be more appropriate³. Conceptually, the Poisson regression is a special scenario of the negative binomial, thus, the underlying regression equation structure remains the same as the Poisson regression.

³ Recall from Figure 3 that there is a large spike at a value of 0, suggesting that a zero-inflation model may be more appropriate.

Using the 18 predictor variables selected from the Regression Model, a negative binomial model was constructed. However, the results obtained were exactly the same as found in Table 11. In order to create some contrast, only the following variables were kept in the revised negative binomial model: `m_stars`, `imp_va`, `imp_alcohol`, `imp_la`, `imp_ai`, and `imp_stars`. This list was obtained by removing any variable from the Poisson regression whose 95% Wald Confidence Intervals contained the value of 0. The reasoning is that if the parameter estimate could be 0, then that variable could be potentially be meaningless in the overall model. Table 12 provides the appropriate parameter values for this model.

Table 12: Parameter Estimates for Model 3 (Negative Binomial)

Parameter	Notation	Value
Intercept	β_0	1.4831
<code>m_stars</code>	β_1	-0.6511
<code>imp_va</code>	β_2	-0.0406
<code>imp_alcohol</code>	β_3	0.0037
<code>imp_la</code>	β_4	0.1587
<code>imp_ai</code>	β_5	-0.0815
<code>imp_stars</code>	β_6	0.1884

This significantly reduced model does not contain significant surprises. It continues to highlight the fact that missing stars have a negative impact whereas stars in general improve the likelihood of wine sales. As seen in earlier models, the variable `imp_alcohol` continues to have negligible impact.

Model 4 – Zero Inflated Poisson (ZIP)

The ZIP (Zero Inflated Poisson) model is quite similar in its assumptions as the Poisson model. The major difference is that this model assumes a spike at a value of 0. Note how Figure 3 is a clear example of where a ZIP model makes more sense than a Poisson or Negative Binomial model. When developing ZIP model, two different scenarios are created: a model with non-zero cases and a model with only zero cases (i.e., a logistic model). These two models are then used to put the overall model into a production-ready model.

First, a similar approach to what was done in the Poisson Model (Model 3) is used:

$$\ln(Y1) = \beta_0 + \beta_1 X_1 + \cdots \beta_n X_n + \varepsilon$$

$$P_Target_All = e^{\ln(Y1)}$$

Second, a logistic model approach is executed for the likelihood of obtaining a 0:

$$Log\ Odds = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

$$P_Target_0 = \frac{e^{LogOdds}}{1 + e^{LogOdds}}$$

With the two probabilities obtained, the expected value is calculated using the following equation:

$$P_Score_ZIP = P_Target_All * (1 - P_Target_0)$$

This approach provides a comprehensive derivation of the ZIP model. Utilizing this methodology, Table 13 provides the appropriate parameter values for this model with both components.

Table 13: Parameter Estimates for Model 4 (Zero Inflation Poisson)

Non-Zero			Zero		
Parameter	Notation	Value	Parameter	Notation	Value
Intercept	β_0	1.4436	Intercept	β_0	-4.1789
m_fsd	β_1	0.0132	imp_va	β_1	0.2205
m_ph	β_2	-0.0097	imp_ca	β_2	-0.1047
m_stars	β_3	0.0369	imp_chlo	β_3	0.0614
n_va	β_4	0.0119	imp_fsd	β_4	-0.0007
n_fsd	β_5	-0.0091	imp_tsd	β_5	-0.0012
n_alcohol	β_6	0.0096	imp_density	β_6	0.275
imp_va	β_7	-0.0147	imp_ph	β_7	0.2087
imp_ca	β_8	-0.0013	imp_sulf	β_8	0.196
imp_chlo	β_9	-0.0172	imp_alcohol	β_9	0.0266
imp_fsd	β_{10}	0	imp_la	β_{10}	0.7226
imp_tsd	β_{11}	0	imp_ai	β_{11}	0.4404
imp_density	β_{12}	-0.2903	imp_stars	β_{12}	-2.3945
imp_ph	β_{13}	0.0052			
imp_sulf	β_{14}	0.0058			
imp_alcohol	β_{15}	0.0072			
imp_la	β_{16}	0.2323			
imp_ai	β_{17}	-0.0189			
imp_stars	β_{18}	0.107			

This model is quite different from the outputs of the other models. Immediately note how the variables `imp_fsd` and `imp_tsd` have a coefficient value of 0 in the Non-Zero component. Similarly, in the Zero component, these two variables have a negligible impact. However, there is a telling story when exploring the variable `imp_density`. It appears that as the density increases, the likelihood of no wine sales increases (see the Zero component). This contrasts with the `imp_density` parameter value in the Non-Zero component (-0.2903). In this one, as the density increases, the lower the likelihood of selling wine cases. Similarly, the variable `imp_stars` implies that if a wine as no stars, there is a larger likelihood that no sales will occur – a stark contrast to if a wine has stars. Another interesting note is how the variable `imp_ph` has a negligible impact on the Non-Zero component, but a large positive impact on the Zero component (0.0052 vs. 0.2087 respectively). From this, it could be concluded that the pH of a wine plays a stronger role in having 0 cases sold versus any non-zero cases sold. Finally, it is interesting to note that the variable `m_stars` has a positive value. This suggests that if a star is

missing, it may contribute in a positive way to improving sales. Intuitively, this could be indicative that certain customers are willing to try out new wines that have not yet been rated to assess their viability for their customers.

Perhaps the most interesting aspect is how the Zero component did not have any parameters for the different flags (negative and missing flags). This may be explored further, but is outside the scope of this analysis.

Model 5 – Zero Inflated Negative Binomial (ZINB)

The final model constructed was the Zero Inflated Negative Binomial (ZINB) model. This model is very similar to the ZIP model (see section Model 4). As seen in Model 4, the ZINB model will also output two different components: a Zero component and a Non-Zero component. All of the derivations are similar to Model 4. Table 14 provides the appropriate parameter values for this model with both components.

Table 14: Parameter Estimates for Model 5 (Zero Inflation Negative Binomial)

Non-Zero			Zero		
Parameter	Notation	Value	Parameter	Notation	Value
Intercept	β_0	1.4492	Intercept	β_0	-4.01
m_fsd	β_1	0.0126	imp_va	β_1	0.2105
m_ph	β_2	-0.0095	imp_ca	β_2	-0.1033
m_stars	β_3	0.0313	imp_chlo	β_3	0.0639
n_va	β_4	0.0118	imp_fsd	β_4	-0.0006
n_fsd	β_5	-0.0088	imp_tsd	β_5	-0.0011
n_alcohol	β_6	0.0093	imp_density	β_6	0.2318
imp_va	β_7	-0.0145	imp_ph	β_7	0.2012
imp_ca	β_8	-0.0014	imp_sulf	β_8	0.1883
imp_chlo	β_9	-0.0173	imp_alcohol	β_9	0.0258
imp_fsd	β_{10}	0	imp_la	β_{10}	0.6926
imp_tsd	β_{11}	0	imp_ai	β_{11}	0.4248
imp_density	β_{12}	-0.2931	imp_stars	β_{12}	-2.2366
imp_ph	β_{13}	0.0053			
imp_sulf	β_{14}	0.0057			
imp_alcohol	β_{15}	0.0072			
imp_la	β_{16}	0.2319			
imp_ai	β_{17}	-0.0186			
imp_stars	β_{18}	0.1053			

As previously mentioned, the ZINB model is quite similar to the ZIP model. Note how the ZINB model also has a parameter estimate of 0 for the variables imp_fsd and imp_tsd. There are times where a ZINB model may converge to the same value as a ZIP model, but such is not the case. The parameter estimates are quite close to each other (when comparing ZIP and ZINB), but they are not exact. Many of the observations made for Model 4 can be applied to Model 5 as well.

Model 6 – Logistic Hurdle

An optional model that was explored is the Logistic Hurdle model. This model is somewhat similar to the ZIP and ZINB models. The Logistic Hurdle model also has two components: a component to predict if the count is zero (or non-zero) and a component to predict the count (assuming the count is non-zero). The end result is obtained by multiplying the probability of non-zero and the count to obtain the expected count.

In order to develop this model, two variables were created. The first variable TARGET_FLAG was created to indicate a 1 if the response variable TARGET was greater than 0. Otherwise, TARGET_FLAG would be set to 0. The second variable TARGET_AMT was shifted down by one unit. The intent here was to eliminate the need for modeling sales of 0 units. TARGET_AMT was adjusted to null if the variable TARGET_FLAG was zero. This would ensure that the zero sales counts would be negated. In the final scoring process, the shift of one unit was reversed (1 was added back into the overall model) to ensure that the dataset was not shifted needlessly.

Table 15 highlights the parameter estimates for the probability of obtaining a non-zero count (Non-Zero) and the predicted count (Count).

Table 15: Parameter Estimates for Logistic Hurdle Model

Non-Zero				Count			
Parameter	Sub-Parameter	Notation	Value	Parameter	Sub-Parameter	Notation	Value
Intercept		β_0	20.1126	Intercept		β_0	1.7889
n_va		β_1	0.1578	m_stars		β_1	-0.4372
imp_va		β_2	-0.2021	imp_alcohol		β_2	0.0096
imp_tsd		β_3	0.000949	imp_la	-2	β_3	-1.4595
imp_ph		β_4	-0.1836	imp_la	-1	β_4	-0.8074
imp_sulf		β_5	-0.1568	imp_la	0	β_5	-0.4331
imp_alcohol		β_6	-0.021	imp_la	1	β_6	-0.192
imp_la	-2	β_7	1.8192	imp_la	2	β_7	0
imp_la	-1	β_8	1.3239	imp_ai		β_8	-0.0207
imp_la	0	β_9	0.9137	imp_stars	0	β_9	0
imp_la	1	β_{10}	0.3668	imp_stars	1	β_{10}	-0.3753
imp_ai		β_{11}	-0.3955	imp_stars	2	β_{11}	-0.2342
imp_stars	0	β_{12}	-17.5075	imp_stars	3	β_{12}	-0.1244
imp_stars	1	β_{13}	-15.6792	imp_stars	4	β_{13}	0
imp_stars	2	β_{14}	-13.2538				
imp_stars	3	β_{15}	-0.1485				

For the Predicted Count component, statistically insignificant variables were removed prior to the modeling process. This component made use of the PROC GENMOD function that does not have the ability for automatic variable selection. In contrast, the Non-Zero component used the PROC LOGISTIC function with the stepwise automatic variable selection method. In both instances, the variables imp_la (LabelAppeal) and imp_stars (STARS) were categorized as class variables (i.e., categorical variables).

Note how, in the Non-Zero component, the significant impact that 0, 1, or 2 stars have on the overall predicted probability of non-zero sales. Surprisingly, the label appeal (imp_la) coefficients have a positive value. This is somewhat counter intuitive as one would think that the increased label appeal would have more non-zero sales and negative label appeal would have a detrimental impact on the sales.

Model Selection

For each model, a model validation algorithm was developed. This algorithm provided the average error and root mean square error. Furthermore, for each model, the AIC and BIC values (generated through the appropriate SAS functions) were also recorded. Table 16 summarizes these findings.

Table 16: Model Validation Summary

	AIC	BIC	Avg. Error	RMSE
Model 1 - Regression	6951.07	6953.14	1.56201	1.92732
Model 2 - Poisson	45757.4834	45899.1628	1.03433	1.31595
Model 3 - NB	45762.9136	45815.1113	1.03285	1.31651
Model 4 - ZIP	40899.1174	41137.7353	0.96187	1.27407
Model 5 - ZINB	40965.0954	41211.1701	0.96597	1.27308
Model 6 – Logistic Hurdle	n/a	n/a	0.96339	1.26456

From an AIC or BIC perspective, the best performing model is the regression model (Model 1). However, this model is not the most intuitive as it does a poor job of modeling for the spike in zeros that the response variable has (see Figure 3). For this analysis, the intent of the regression model was to have the model identify which variables are statistically significant using the stepwise automatic variable selection method. In this event, 18 of the 41 predictor variables were identified. These 18 variables were then used for the subsequent models.

The metric of choice here is to use the RMSE metric. This metric will sufficiently enable an understanding of the validity of each model. Note that the lower the RMSE value, the ‘better’ a model may be. Model 3 (negative binomial) was not be selected since it was heavily modified from Model 2. Recall that the original version of Model 3 produced the exact same values as Model 2. Nevertheless, the RMSE value for the modified version of Model 3 is surprisingly similar to Model 2 while using a substantially smaller set of predictor variables.

The ZINB (Zero Inflation Negative Binomial) model has the second lowest RMSE value (1.27308) followed by the ZIP (Zero Inflation Poisson) model (1.27407). The model with the lowest RMSE is the Logistic Hurdle model (Model 6). Although this model performs the best in terms of RMSE, it has not been chosen due to the lack of simplicity. Therefore, the chosen model is the ZINB (Model 5) model. It’s parameter estimates provide an intuitive understanding of the variables’ relationship to the response variable TARGET.

Conclusion

Several models of each type (Regression, Negative Binomial, etc.) were constructed to predict the amount of sample cases a wine distributor may purchase. The original dataset contained 14 predictor variables and over 12,000 observations. Most of the variables captured the chemical composition of a wine (e.g., pH, free sulfur dioxide, alcohol, etc.). A few of the variables offered insight into general characterizations of the wine such as the star rating, acid index, or the label appeal. One of the key challenges in this data was the inability to classify the type of wine investigated. There was no variable that clearly identified each wine as being red, white, etc. Surprisingly, many of the variables are numeric, but could be considered categorical such as stars. For this analysis, all variables were explored as numeric variables and not categorical variables. Many of the data points were imputed if missing or adjusted if negative. A subject matter expert in wine composition should be consulted to understand the applicability of the signs in each parameter estimate as well as support the assumption that no chemical value could be negative. This will require further investigation which is outside the scope of this analysis. Finally, it would be prudent to monitor this chosen model's performance for the next three months as new data are fed into it. Next steps may include (but not limited to) refining the model and focusing on improved data imputation.

Bingo Bonus – Using a Decision Tree to Predict Target Cases

For this bingo bonus, I used JMP to create a decision tree to predict the number of wine cases sold. This process required creating the necessary flags (both missing and negative) as well as data imputation. Note that the steps discussed in this analysis were applied directly to JMP. Approximately four splits were created for an R^2 value of 0.514. Figure 4 illustrates the decision tree that was created and the appropriate steps.

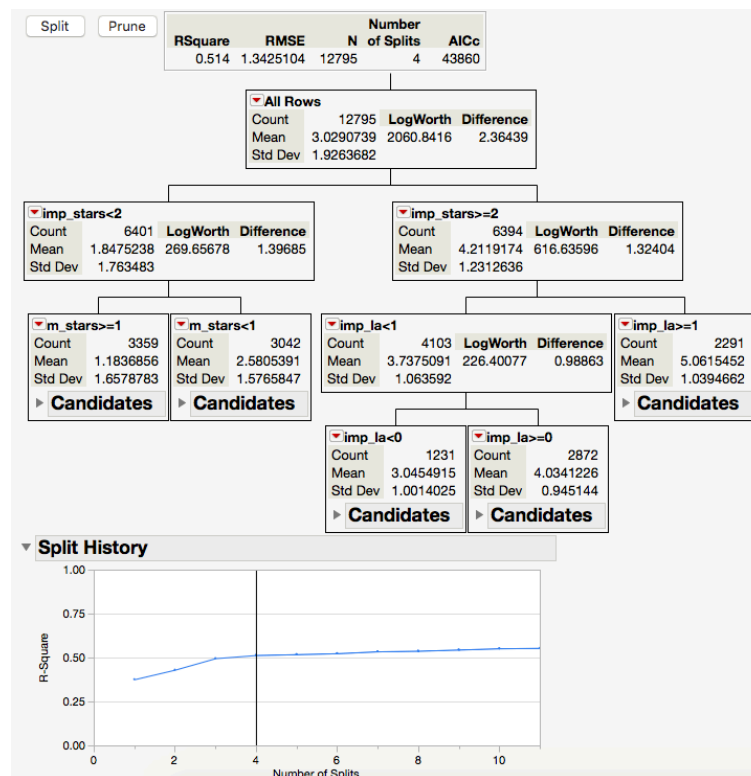


Figure 4: JMP Decision Tree for TARGET

Loosely put, the logic can be described as follows:

If the `imp_stars` is less than 2 and `m_stars` is greater than or equal to 1 then the predicted target value is 1.18. Otherwise it is 2.58. If the `imp_stars` is greater than or equal to 2 and if `imp_la` is less than 1 and if `imp_la` < 0 then the predicted target value is 3.045. If the `imp_la` value is greater than or equal to 0, then the predicted target value is 4.034. If the `imp_la` value is greater than or equal to 1 then the predicted target value is 5.06.

Table 17 summarizes the model validation values (Avg. Error and RMSE) for the previous six models in addition to this new model using a decision tree.

Table 17: Model Validation Summary with Decision Tree Model

	AIC	BIC	Avg. Error	RMSE
Model 1 - Regression	6951.07	6953.14	1.56201	1.92732
Model 2 - Poisson	45757.4834	45899.1628	1.03433	1.31595
Model 3 - NB	45762.9136	45815.1113	1.03285	1.31651
Model 4 - ZIP	40899.1174	41137.7353	0.96187	1.27407
Model 5 - ZINB	40965.0954	41211.1701	0.96597	1.27308
Model 6 – Logistic Hurdle	n/a	n/a	0.96339	1.26456
Model 7 – Decision Tree	n/a	n/a	1.03202	1.34251

The RMSE for this decision tree model (Model 7) is worse than the RMSE for the Poisson model (Model 2). Note how the prior models were using several variables to predict target cases. However, the decision tree relied on only three distinct variables: `imp_stars`, `m_stars`, `imp_la`. It is possible that more splits could have been made to make use of more predictor variables. However, as can be seen in Figure 4, over 10 splits were made and only incremental changes were seen in the R^2 value. In an effort to find a parsimonious model, fewer splits were kept. Needless to say, this model would not have been chosen as compared to the prior models.

MIDTERM

Nikhil Agarwal

PREDICT 413 | Section 57

Kaggle screenname: NikAgarwal

Table of Contents

PROBLEM DESCRIPTION	4
SIGNIFICANCE	4
OVERVIEW OF DATA	4
DATASETS	4
NEW VARIABLE CREATION	5
DESCRIPTIVE STATISTICS	5
VISUAL DATA EXPLORATION	6
OUTLIER MANAGEMENT AND DATA CLEANSING	8
LITERATURE	9
KNN	9
RANDOM FORESTS	9
XGBOOST	9
LINEAR DISCRIMINANT ANALYSIS (LDA)	10
QUADRATIC DISCRIMINANT ANALYSIS (QDA)	10
TYPE OF MODELS	10
EXTREME GRADIENT BOOSTING (XGBOOST)	11
RANDOM FORESTS	11
K-NEAREST NEIGHBOR (KNN)	11
LINEAR (LDA) AND QUADRATICS DISCRIMINANT ANALYSIS (QDA)	11
FORMULATION	12
PERFORMANCE/ACCURACY	12
LIMITATIONS	13
FUTURE WORK	13
TEXT ANALYTICS	14
OUTLIER MANAGEMENT	14
GPS COORDINATE EXPLORATION	14
EXTERNAL DATA SOURCES	14
LEARNING	14
WORKS CITED	16
APPENDIX	17
DATA DICTIONARY	17

VARIABLE MGR_SKILL DEVELOPMENT AND EXPLANATION	17
FREQUENCY OF LISTINGS BY DAY OF WEEK	18
FREQUENCY OF LISTINGS BY HOUR OF DAY	18
FREQUENCY OF LISTINGS BY DAY (DATE)	19
CONFUSION MATRIX & ACCURACY OUTPUT FOR XGBOOST MODEL	20
CONFUSION MATRIX & ACCURACY OUTPUT FOR RANDOM FOREST MODEL	21
CONFUSION MATRIX & ACCURACY OUTPUT FOR KNN MODEL	22
CONFUSION MATRIX & ACCURACY OUTPUT FOR LDA MODEL	22
CONFUSION MATRIX & ACCURACY OUTPUT FOR QDA MODEL	23

Problem Description

The intent of this activity was to predict the interest level of potential customers researching places to live. Renthop is a website that hosts rental listings in many major cities across the United States. For this activity, many of the rental listings were from the New York City boroughs. This dataset was provided by Kaggle.com and featured 14 different potential predictor variables. Several statistical models were constructed for this activity. This report will discuss five of the models constructed using K-Nearest Neighbors (KNN), Extreme Gradient Boosting (XGBoost), Random Forests, Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA). Since the results were posted in Kaggle, a summary of the log-loss scores for each submitted model through the Kaggle competition is provided in the Performance/Accuracy section. Prior to submission, confusion matrices and the accuracy for each model was assessed and is discussed in the Performance/Accuracy section.

Significance

Each year, millions of potential tenants' search for rental properties to move into. Renthop is an innovative service that enables renters to find properties that match their interests and requirements quickly. For the rental companies (and landlords), Renthop is an excellent way to showcase their properties and minimize the amount of time a property remains unoccupied. The significance of this activity is to identify opportunities that will enable Renthop to highlight sub-par information that may result in less interest for a property. According to Kaggle.com, this activity "will help Renthop better handle fraud control, identify potential listing quality issues, and allow owners and agents to better understand renters' needs and preferences" (Kaggle, 2017).

Overview of Data

Datasets

Two key datasets were provided by Kaggle. The first was the training dataset and the second was the test dataset. The key difference between the training and test datasets was the presence of the response variable, *interest_level*, in the training dataset but not the test dataset. Both datasets were provided in JSON format and upon import to R, was converted to a dataframe. Table 5 (in APPENDIX) provides a summary of the imported predictor variables and their descriptions. The training dataset was further split, using a 70%-30% ratio, to create a validation dataset. This ensured that models developed using the new dataset could be validated with the validation dataset prior to operationalizing with the test dataset. Recall that the test dataset does not have any response variable and the goal is to develop the probability of the potential response (for each possible response of high, medium, or low).

New Variable Creation

Text analytics was not used in any of the exploratory or modeling steps. Rather, new variables were constructed to help summarize some of the attributes of the text-only columns (i.e., description, photos, and features). For instance, a new variable was constructed that counted the number of words found in the description. Another variable was constructed that counted the number of characters in the description. A variable was also developed that looked at the total number of features listed for each listing.

Table 1 outlines the derived variables.

Table 1: List of Derived Variables

Derived Variable Name	Description
total_pix	Counts the number of photos for each listing
total_features	Counts the number of features listed for each listing
total_description_length	Counts the number of characters in the description column
total_words	Counts the total number of words in the description column
price_per_room	Rental price divided by the total number of bedrooms and bathrooms
price_per_bedroom	Rental price divided by the total number of bedrooms
real_address_chk	Compares the street address and display address. If they match, this value is 1 otherwise 0
mgr_skill	Metric that calculates a score for manager IDs based on the interest of each of their respective listings
create_hour	The hour listing was created (on 24-hour)
create_day	The day of the week (e.g., Friday, Saturday, etc.)
create_daynumber	The numeric day

A key change for the model was to have the `interest_level` values to be considered as factors. This ensured that all models constructed yielded an answer that was one of the three possible responses¹ (for the response variable). An interesting metric, `mgr_skill`, was developed to help gauge a manager's ability to 'showcase' the property. See APPENDIX for explanation of the development of this metric.

It was discovered that the provided training and test datasets had overlapping time periods. For instance, both datasets covered the months of April through June. Therefore, it was decided to parse the date and time stamp of the `created` variable. With this parsing, three new variables were constructed (as seen in Table 1): `create_hour`, `create_day`, and `create_daynumber`.

Descriptive Statistics

Table 2 provides a brief summary of the numeric variables in the training (after the 70/30 split) dataset. Note the strong presence of outliers in almost each variable.

¹ The response variable, `interest_level`, has three possible responses: low, medium, high.

Table 2: Descriptive Statistics on Numeric Variables

Variable	Min	Median	Mean	Max
bathrooms	0	1	1.215	10
bedrooms	0	1	1.549	8
price	43	3150	3902	4490000
total_pix	0	5	5.62	68
total_features	0	5	5.429	39
total_description_length	0	564	602.5	3367
total_words	0	86	93	567
price_per_room	43	1300	1600	1496667
price_per_bedroom	43	2150	2491	2245000
mgr_skill	0.01327	0.34483	0.40542	1.88235

Table 3 provides a brief overview of the count of interest levels within the training dataset.

Table 3: Frequency of interest level for training dataset

Interest Level	Frequency
high	2665
medium	7881
low	24000

Table 6, Table 7, and Table 8 (in the APPENDIX) provide a frequency summary of the respective variables. Note that each of these variables were classified as factors. The reasoning was that perhaps there's a significance to the day of when listings are created or perhaps an interest level. For instance, some rental properties (in general) allow for rent payments to be made twice a month, instead of the more common, first of the month. Furthermore, the hour of the day was interesting to include since the analysis could then explore the significance of the hour of listing creation.

Visual Data Exploration

Numerous visual charts were created to help explore the data visually. However, for succinctness, only a few have been included and discussed in this paper. Figure 1 breaks down the variable *mgr_skill* by interest level. It's clear to see that low interest level also tends to have a smaller IQR (inter-quartile range) than high or medium interest levels. Note how the median for the low interest level is below that of the medium and high interest levels.

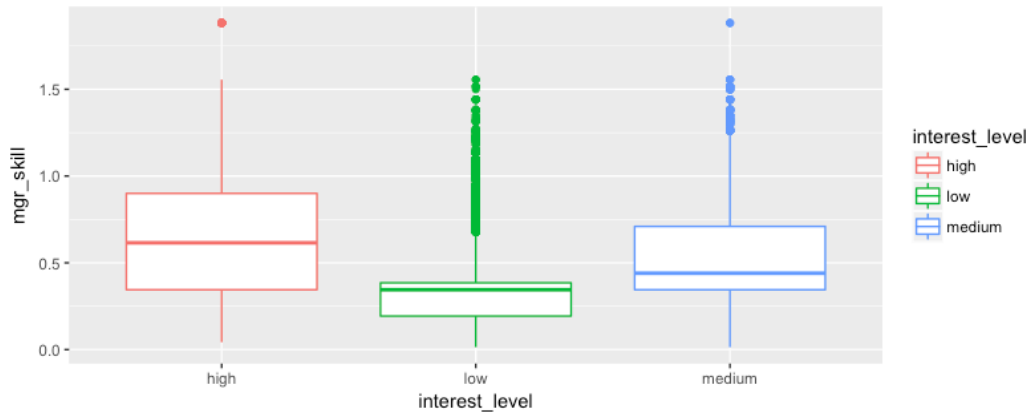


Figure 1: mgr_skill vs. interest_level boxplot

Figure 2 breaks down the median rental price by interest level and day of the week. This chart helps clarify the median listing price by day of week for each interest level and how it varies. For instance, the median price for any day of the week with high interest tends to be \$2,500 or less. In contrast, with low interest levels, the median price is well above \$3,100. Interestingly, with medium interest, the median price tends to be between \$2,750 and \$3,000. Note how the median price decreases (for medium interest level) after Tuesday, plateauing on Friday, and then sharply increasing on Saturday. It is possible that there's stronger interest in properties on Saturday (since more people have more free time to browse Renthop). The same pattern is not necessarily there for the other interest levels.

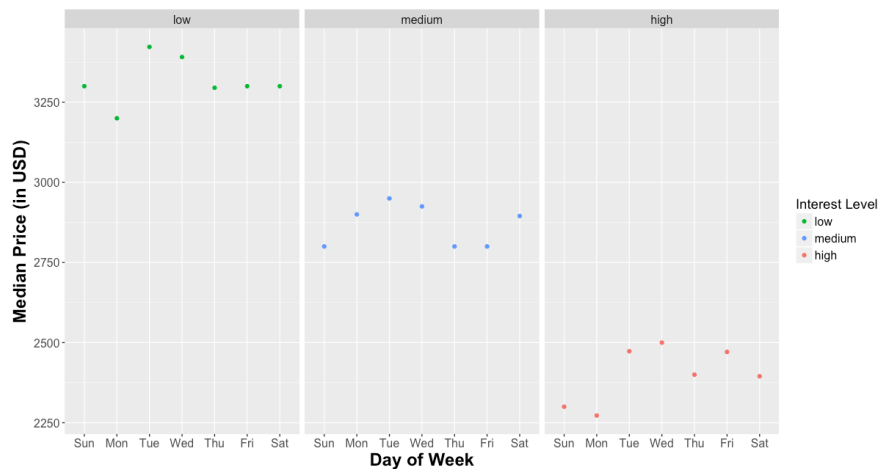


Figure 2: Median rental price by day of week and interest level

Figure 3 is a slight twist on what was seen in Figure 2. In this case, a grouping called Time of Day was created based on the hour of listing creation time. Different hours of the day were classified² as morning, afternoon, evening, or late night. From this, a median rental price was

² Morning hours were 06:00 to less than 12:00, Afternoon hours were 12:00 to less than 18:00, Evening hours were 18:00 to less than 23:00, Late night hours were 23:00 to less than 06:00.

calculated and separated by interest level. Note how if a property has high interest, it's median price is lowest during the afternoon hours. This contrasts with both low and medium interest levels as their median prices were lowest during evening hours.

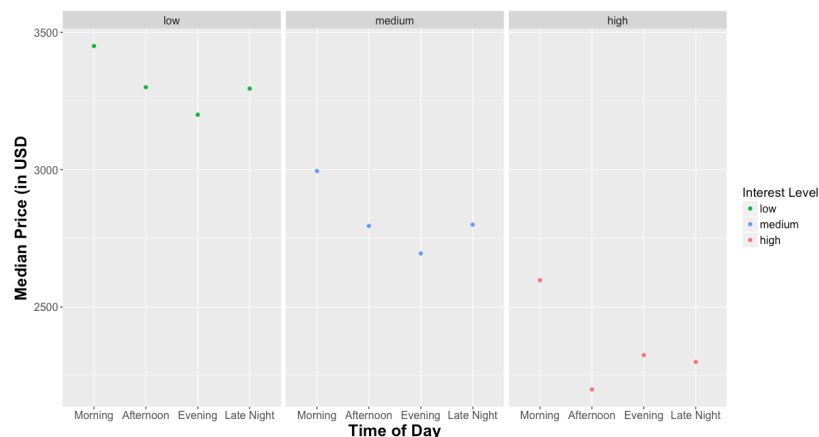


Figure 3: Median rental price by time of day and interest level

Figure 4 illustrates the average number of words in the description by time of day and separated by interest level. In all three interest levels, it appears that the most number of words are occurring in the morning and late night hours. Intuitively, it makes sense as the managers making the listings have more time to type in descriptions in the early and late hours since there may not be as many showings or visitors. It's also clear to see that the high interest properties also tend to have more words on average than the low interest properties.

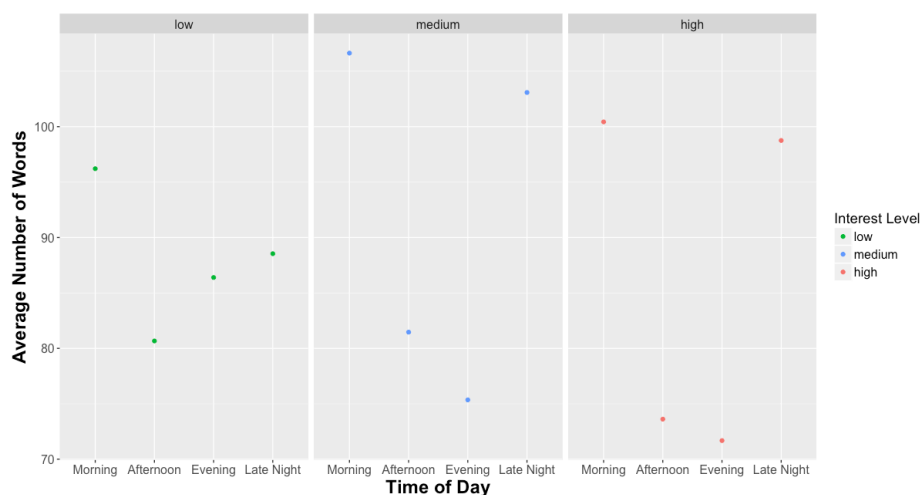


Figure 4: Average number of words by time of day and interest level

Outlier Management and Data Cleansing

For this iteration of the analysis, no outlier management techniques were undertaken. The primary reason for this was to develop models with actual values in place and understand their performance. Outlier management will be reserved for future steps. A check was made for missing data and none were found. Furthermore, observed values in the training dataset were not transformed nor were they imputed with other values to minimize the effects of erroneous or outlier values.

Literature

This analysis is focused on predicting (i.e., forecasting) the interest level for a potential rental listing. Recall from earlier discussions that the interest levels are actually factors and that this analysis becomes a simple classification example. In the previous section (Type of Models), five different models were selected. This section discusses academic (from peer reviewed journals) papers that utilize similar models for classification.

KNN

In the paper, “Large scale biomedical texts classification: a kNN and an ESA-based approaches”, the authors discuss two methods of classifying texts into several categories. Ideally, they want their model to perform with unseen texts with high accuracy. Their “kNN-based approach requires a collection of documents previously annotated for the neighbours’ retrieval. For a given document, the aim is to retrieve its k most similar documents” (Dramé, Mougin, & Diallo, 2016). This methodology is very similar to constructing a training dataset and determining what the current classifications are to help the model learn and then apply to unknown values or test dataset. Ultimately, in their research, the authors found KNN to be a very good model and also enhanced it with random forests.

Random Forests

In the paper, “Crop classification of upland fields using Random forest of time-series Landsat 7 ETM+ data”, the authors discuss using crop classification through the use of random forests. An interesting approach that the authors used was satellite data with a time series focus. Although the analysis in this Kaggle competition did not require a time series approach, the use of external data sources (discussed in FUTURE STEPS) could be a great way to augment the random forest model’s accuracy. One of the key conclusions the authors made is the random forest model’s ability to help detect false classifications and still ensure fairly high accuracy (Tatsumi, Yamashiki, Torres, & Taïpe, 2015).

XGBoost

In the paper, “Bioactive Molecule Prediction Using Extreme Gradient Boosting”, the authors discuss “(Xgboost)...was investigated for the prediction of biological activity based on

quantitative description of the compound's molecular structure" (Mustapha & Saeed, 2016). Interestingly, the authors conduct a direct comparison to other ensemble models and neural network models and distinctively conclude the superior performance of the XGBoost model. From an accuracy standpoint, their research concluded that XGBoost had over 94% accuracy on their datasets and outperformed the Random Forest model.

Linear Discriminant Analysis (LDA)

In the paper, "Classification of rice wine according to different marked ages using a novel artificial olfactory technique based on colorimetric sensor array", the authors describe how they used an LDA model to classify RGB (red, green, blue) components. Their primary focus was comparing PCA (principal component analysis) and LDA. The PCA model that they constructed was actually decent, however, the distinctiveness of the class groupings found by the LDA model was exceptional. Figure 5 (Ouyang, Zhao, Chen, & Lin, 2012) illustrates the groupings between the two models. Note how closely clustered the LDA model found its groupings. In contrast, the PCA model is subject to misclassification.

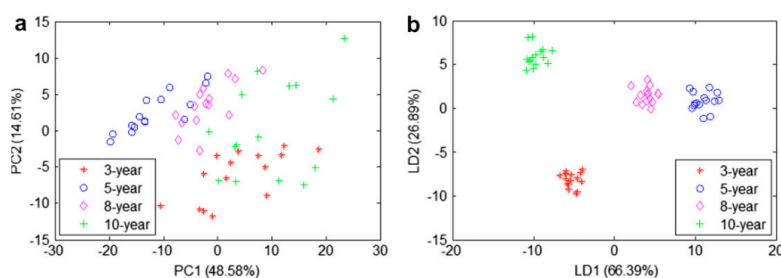


Fig. 4. Score plots of PCA analysis (a) and LDA analysis (b) for rice wine of different marked ages.

Figure 5: LDA vs. QCA (Ouyang, Zhao, Chen, & Lin, 2012)

Quadratic Discriminant Analysis (QDA)

In the paper, "Environmental Noise Classification using LDA, QDA and ANN Methods", the authors attempt to build LDA, QDA, and ANN models to classify environmental noise. The interesting approach in their work is their direct comparison of LDA and QDA (for this brief write-up, ANN models are disregarded). Their research showed that a QDA model did outperform an LDA model. Nevertheless, the underlying message in their research shows that construction of different models is necessary to truly understand the significance of a model.

Type of Models

Several different types of models were constructed, but only five of these models were chosen to be submitted on Kaggle. All five models can be considered as machine learning models. The five models are:

- Extreme Gradient Boosting (XGBoost)
- Random Forests
- K-Nearest Neighbor (KNN)
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)

Extreme Gradient Boosting (XGBoost)

The XGBoost model was chosen due to its veracity and its complex learning abilities. Recall from the VISUAL EXPLORATION section that there appears to be numerous behaviors that affect interest level. Decision trees and boosted models are able to capture nuances in the datasets and produce accurate models.

Random Forests

Random forests is an interesting approach as this machine learning staple creates ‘tiny’ trees that are not very deep (in terms of branches). Furthermore, the algorithms create thousands of trees (each one could virtually use any number of predictors and no tree necessarily has all possible predictors) very quickly. According to the authors of “An Introduction to Statistical Learning with Applications in R”, “random forests [are] an improvement over bagged trees by way of a small tweak that decorrelates the trees” (James, Witten, Hastie, & Tibshirani, 2015). This is a clear reason why bagging models were not considered since bagging models will enable the use of all predictors in a tree.

K-Nearest Neighbor (KNN)

KNN models are somewhat simple and utilize Bayes classifiers. Recall that the variable *interest_level* is a factor with three values (high, medium, low). At first glance, the probability of a listing being one of those three is not known. Distinctively, “KNN applies Bayes rule and classifies the test observation...to the class with the largest probability” (James, Witten, Hastie, & Tibshirani, 2015). Surprisingly, the KNN model can be quite accurate yet easy to explain. Compared to the decision tree models (XGBoost and random forests), KNN models can be computationally inexpensive.

Linear (LDA) and Quadratics Discriminant Analysis (QDA)

LDA and QDA models are quite popular for multiple class classification modelling. Since this analysis is an exemplary example of multiple classification, both LDA and QDA are appropriate. The key difference between LDA and QDA is the fact that the QDA approach assumes that “each class has its own covariance matrix” (James, Witten, Hastie, & Tibshirani, 2015). Both modelling techniques assume that the classifications are from a Gaussian distribution. However, such information is not necessarily known with the provided datasets. Therefore, the author of this analysis is not confident in the performance of these models.

Formulation

All the models were constructed using R³. The following are the key packages used within R:

- Caret
- MASS
- XGBoost

The caret package is quite powerful and enables easier cross-validation for any type of model. One of the control parameters for caret is the trainControl function. With this control parameter, XGBoost, random forest, and the KNN models utilized repeated cross validation, using 10 folds and repeated three times. The cross-validation approach enables a stronger learning capability for the models since a small subset of the training data can be used for validation prior to the validation dataset being used. Since this is done inherently within the caret package, the final model can be considered a decent model with the given parameters.

For the LDA and QDA models, the library MASS was used. For these two models, no cross validation was done.

As mentioned earlier, the original dataset from the Kaggle competition was subdivided into a 70%-30% split. In other words, 70% of the data were used to 'train' the models. After that, the 30% of the data were used as a validation dataset to validate the trained models. Once the model accuracy and confusion matrices were satisfactory, the trained model would then be applied to the test dataset that was provided by Kaggle. The 30% of the data were not reintroduced to the model as that would constitute a potential leakage that could result in false perceptions.

Some of the provided information was also not utilized in the model exploration or development phases. For instance, the images were not explored for their content. However, the number of images was used.

Performance/Accuracy

In order to assess the models, a confusion matrix was created for each model to compare the model's performance on the validation dataset. All models were also assessed for their accuracy (i.e., mean square error). Furthermore, all five models discussed in this analysis were submitted to Kaggle to obtain a log-loss score. See the APPENDIX for the confusion matrices for each model.

³ See the R code to see the exact syntax for each model.

Table 4 summarizes the results of each of the models

Table 4: Model accuracy & Kaggle score summary table

Model	Accuracy	Kaggle Private Score	Kaggle Public Score
Random Forest	0.7379	0.60147	0.60351
KNN	0.7069	0.64217	0.86642
LDA	0.7082	0.71016	0.72011
QDA	0.2616	4.22534	4.23345
XGBoost	0.7471	0.57548	0.57727

The Kaggle Public Score was found using only 10% of the dataset to assess the test model. The Kaggle Private Score was found using all of the data in the test dataset and was only available after the competition closed. In both instances, the lower the score, the better the model.

Based on the models developed and submitted to Kaggle, the XGBoost model outperformed all others. However, the next closest model was random forest. Note how the KNN model, which is simple, was third best. Surprisingly, the QDA model was terrible compared to the other four models.

Limitations

There are two key limitations to the models constructed complexity and optimization. Both random forest and XGBoost are complex models that are not very easy to explain. For instance, both approaches constructed thousands of trees, but there is no distinctive equation that can be documented and easily explained. Rather, the explanation of each tree would result in a convoluted explanation of the logic and the randomness.

Furthermore, both models were computationally expensive. For instance, the XGBoost model utilized 32 cores for a run time of over 75 minutes. One key area to explore would be the optimization of each of these algorithms. For instance, the shrinkage parameter would enable the reduction of predictors and could improve a fit of a model while also reducing the likelihood of overfitting a model. Although it is difficult to overfit a model using boosting methods, it is still possible and the choice of parameters could affect the accuracy.

Future Work

There are four key areas of improvement for the next steps:

- Text analytics
- Outlier management
- GPS coordinate exploration
- External data sources

Text Analytics

For this analysis, a deep dive into the context of the description and listed features was not done. Exploring these two areas in terms of text would enable a greater understanding of the type of features and keywords that generate stronger interest. Another analysis to incorporate would be sentimental analysis to help gauge an 'excitement' factor that may encourage users to click on a listing and potentially drive more interest.

Outlier Management

As previously mentioned, the presence of outliers was quite glaring. The risk with elimination of outliers is that the model may work well with both training and validation datasets, it may not work as well with the test dataset. Furthermore, the outliers themselves need a greater understanding. It may be possible that there are different types of rental postings such as postings catering to wealthier clientele. One of the benefits of machine learning algorithms is their inherent ability to account for outliers. However, if outliers are managed properly, these algorithms may experience improvements in their accuracy.

GPS Coordinate Exploration

Although the GPS coordinate was used as is in this analysis, it may be prudent to explore the GPS coordinates more. One glaring error in the dataset was the presence of 0 in both latitude and longitude. For reference, a latitude of 0 and a longitude of 0 suggests the location is in the middle of the ocean off the coast of Africa. Therefore, it may be prudent to either impute these values or determine the GPS coordinate for these locations based on street address.

External Data Sources

Finally, more data exploration would be prudent to develop other key metrics as necessary. The *building_id* variable, for instance, has been repeated multiple times and it may allude to interest level amongst Renthop visitors. Examples such as this are a great segue way in to conducting deeper analysis of the available information. One of the restrictions in this competition through Kaggle was the inability to bring in external data sources. External data sources such as weather or temperature may also drive other insights that could influence interest level.

Learning

Perhaps the best part about this assignment was the use of JSON files. I have never directly worked with JSON files and being able to parse them for information is astounding. Another key benefit of this assignment was the ability to put my knowledge on machine learning models

(e.g., random forest, boosting, etc.) to the test with real world data. In the machine learning class, we used academic datasets in which the answers were somewhat known. In this assignment, since we did not have the known final answers, it was quite exhilarating to develop models and compete.

Finally, a key takeaway for me is the importance of data exploration. I spent a considerable amount of time reading the different forum posts on Kaggle to better understand how some of the variables worked or what they actually meant. In hindsight, I easily could have spent another eight weeks just exploring the data and developing new variables to help improve the models.

Works Cited

- Dramé, K., Mougin, F., & Diallo, G. (2016). Large scale biomedical texts classification: a kNN and an ESA-based approaches. *Journal of Biomedical Semantics*, 1-12.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2015). *An Introduction to Statistical Learning with Applications in R* (6th Edition ed.). New York: Springer Science + Business Media.
- Kaggle. (2017). *Two Sigma Connect: Rental Listing Inquiries*. Retrieved April 26, 2017, from Kaggle: <https://www.kaggle.com/c/two-sigma-connect-rental-listing-inquiries#description>
- Mustapha, I. B., & Saeed, F. (2016). Bioactive Molecule Prediction Using Extreme Gradient Boosting. *Molecules*, 21, 1-11.
- Ouyang, Q., Zhao, J., Chen, Q., & Lin, H. (2012, December 5). Classification of rice wine according to different marked ages using a novel artificial olfactory technique based on colorimetric sensor array. *Elsevier Food Chemistry*, 1320-1324.
- Tanweer, S., Mobin, A., & Alam, A. (2016, September). Environmental Noise Classification using LDA, QDA and ANN Methods. *Indian Journal of Science and Technology*, 9, 1-8.
- Tatsumi, K., Yamashiki, Y., Torres, M. A., & Taipe, C. L. (2015, June 14). Crop classification of upland fields using Random forest of time-series Landsat 7 ETM+ data. *Elsevier Computers and Electronics in Agriculture*, 171-179.

APPENDIX

Data Dictionary

Table 5: Data dictionary

Column	Description
bathrooms	Number of bathrooms
bedrooms	Number of bedrooms
building_id	Unique identifier for location of property
created	Date listing created
description	Text describing the property (written by the creator of the listing on Renthop)
display_address	Address of property
features	List of features of the apartment
latitude	Latitude of the address
longitude	Longitude of the address
listing_id	Unique identifier for each listing.
manager_id	Unique identifier for each manager
photos	List of photos posted (URLs)
price	Price of rental property (in USD)
street_address	Address of property
interest_level	Target (response variable): high, medium, low

Variable mgr_skill development and explanation

The variable, mgr_skill, was developed as a metric to gauge the effectiveness of a manager tied to a listing. Note that this variable was created using the data found in the training dataset after the 70% split from the provided training data. It was NOT updated afterwards using the 30% of the data in the validation dataset. This was done to ensure that there was no 'leakage' of information in the modelling process.

For each unique manager ID, a count of all their listings along with the total number of listings in each potential interest level (i.e., high, medium, low) was constructed. Then a ratio was calculated (e.g., total number of high interest level properties divided by total number of listings). Two points (the scoring was arbitrary) were multiplied by the ratio of total high interest over total properties and one point was multiplied by the ratio of total medium interest over total properties. The equation below describes the formula:

$$mgr\ skill = 2 * \frac{\frac{total\ number\ of\ high\ interest\ level\ properties}{total\ number\ of\ properties\ listed\ for\ manager} + 1}{\frac{total\ number\ of\ medium\ interest\ level\ properties}{total\ number\ of\ properties\ listed\ for\ manager}}$$

To reduce the likelihood of penalizing managers with few listings, any manager with less than 10 total properties listed will be given the median score (approximately. 0.344) of all the manager skills.

Frequency of listings by Day of Week

Table 6: Frequency of listings by day of week

Day of Week	Frequency
Sunday	3147
Monday	2997
Tuesday	5847
Wednesday	6610
Thursday	5723
Friday	5335
Saturday	4887

Frequency of listings by hour of day

Table 7: Frequency of listings by hour of day

Hour	Frequency	Hour	Frequency
1	4002	13	408
2	7356	14	525
3	5885	15	525
4	3597	16	243
5	5544	17	318
6	3099	18	316
7	735	19	193
8	232	20	135
9	94	21	127
10	205	22	98
11	325	23	60
12	486	0	38

Frequency of Listings by Day (date)

Table 8: Frequency of listings by day (date)

Day Number	Frequency	Day Number	Frequency
1	641	16	1278
2	1238	17	1101
3	1244	18	1037
4	1059	19	988
5	1274	20	1236
6	1298	21	1728
7	1255	22	1180
8	1191	23	758
9	1054	24	1274
10	1112	25	1047
11	1224	26	1030
12	1631	27	1115
13	1204	28	1139
14	1365	29	1153
15	1192	30	496
		31	4

Confusion Matrix & Accuracy Output for XGBoost Model

Confusion Matrix and Statistics						
Reference						
Prediction	high	low	medium			
high	310	52	163			
low	305	9506	1940			
medium	559	726	1245			
Overall Statistics						
Accuracy : 0.7471						
95% CI : (0.74, 0.754)						
No Information Rate : 0.6946						
P-Value [Acc > NIR] : < 2.2e-16						
Kappa : 0.379						
McNemar's Test P-Value : < 2.2e-16						
Statistics by Class:						
	Class: high Class: low Class: medium					
Sensitivity	0.26405	0.9243	0.37186			
Specificity	0.98423	0.5035	0.88785			
Pos Pred Value	0.59048	0.8090	0.49209			
Neg Pred Value	0.93950	0.7453	0.82869			
Prevalence	0.07929	0.6946	0.22612			
Detection Rate	0.02094	0.6420	0.08409			
Detection Prevalence	0.03546	0.7937	0.17088			
Balanced Accuracy	0.62414	0.7139	0.62986			

Figure 6: XGBoost Confusion Matrix (Caret Output)

Confusion Matrix & Accuracy Output for Random Forest Model

Confusion Matrix and Statistics			
Reference			
Prediction	high	low	medium
high	302	57	196
low	366	9499	2028
medium	506	728	1124
Overall Statistics			
Accuracy : 0.7379			
95% CI : (0.7307, 0.7449)			
No Information Rate : 0.6946			
P-Value [Acc > NIR] : < 2.2e-16			
Kappa : 0.3497			
McNemar's Test P-Value : < 2.2e-16			
Statistics by Class:			
	Class: high	Class: low	Class: medium
Sensitivity	0.25724	0.9237	0.33572
Specificity	0.98144	0.4706	0.89230
Pos Pred Value	0.54414	0.7987	0.47668
Neg Pred Value	0.93881	0.7305	0.82134
Prevalence	0.07929	0.6946	0.22612
Detection Rate	0.02040	0.6416	0.07592
Detection Prevalence	0.03748	0.8033	0.15926
Balanced Accuracy	0.61934	0.6971	0.61401

Figure 7: Random Forest Confusion Matrix (Caret Output)

Confusion Matrix & Accuracy Output for KNN Model

Confusion Matrix and Statistics						
Reference						
Prediction	high	low	medium			
high	47	15	33			
low	797	9795	2690			
medium	330	474	625			
Overall Statistics						
Accuracy : 0.7069						
95% CI : (0.6995, 0.7143)						
No Information Rate : 0.6946						
P-Value [Acc > NIR] : 0.000542						
Kappa : 0.1735						
McNemar's Test P-Value : < 2.2e-16						
Statistics by Class:						
Class: high Class: low Class: medium						
Sensitivity	0.040034		0.9525	0.18668		
Specificity	0.996479		0.2289	0.92983		
Pos Pred Value	0.494737		0.7375	0.43737		
Neg Pred Value	0.923391		0.6791	0.79644		
Prevalence	0.079292		0.6946	0.22612		
Detection Rate	0.003174		0.6616	0.04221		
Detection Prevalence	0.006416		0.8971	0.09651		
Balanced Accuracy	0.518256		0.5907	0.55825		

Figure 8: KNN Confusion Matrix (Caret Output)

Confusion Matrix & Accuracy Output for LDA Model

	high	low	medium
high	181	105	170
low	741	9729	2602
medium	252	450	576
> mean(p_lda\$class == v1\$interest_level)			
[1] 0.7082264			

Figure 9: LDA Confusion Matrix

Confusion Matrix & Accuracy Output for QDA Model

	high	low	medium
high	781	3302	1552
low	15	1349	53
medium	378	5633	1743
> mean(p_qda\$class == v1\$interest_level) [1] 0.2615831			

Figure 10: QDA Confusion Matrix

FINAL

Nikhil Agarwal

PREDICT 413 | Section 57

Driven Data screenname: niks

Table of Contents

TABLE OF CONTENTS	2
PROBLEM DESCRIPTION	4
SIGNIFICANCE	4
OVERVIEW OF DATA	4
DATASETS	4
MISSING VALUES AND IMPUTATION	4
DESCRIPTIVE STATISTICS	5
VISUAL DATA EXPLORATION	6
NEW VARIABLE CREATION	8
OUTLIER MANAGEMENT	9
LITERATURE	9
ARIMA/RANDOM FOREST	9
XGBOOST	9
NEURAL NETWORK	9
NEGATIVE BINOMIAL	10
NEGATIVE BINOMIAL	10
TYPE OF MODELS	10
NEGATIVE BINOMIAL	10
EXTREME GRADIENT BOOSTING (XGBOOST)	11
RANDOM FORESTS	11
ARIMA	11
NEURAL NETWORK	11
FORMULATION	11
PERFORMANCE/ACCURACY	12
LIMITATIONS	13
FUTURE WORK	14
NEW DATA SOURCES	14
FEATURE CREATION	14
OUTLIER MANAGEMENT	14
MODEL OPTIMIZATION	14
LEARNING	15
WORKS CITED	16

Problem Description

The intent of this activity was to predict the total number of dengue fever cases in the cities of San Juan, Puerto Rico and Iquitos, Peru. In this dataset, there was five years of information on San Juan and three years of information on Iquitos. This dataset was provided by an organization called Driven Data and consisted of 23 potential predictor variables. Interestingly, the potential predictors were sourced from four different systems: NOAA GHCN, PERSIANN, NOAA NCEP, NOAA CDR Normalized Difference Vegetation Index. Several statistical models were constructed for this activity. This report will discuss five of the models constructed: Negative Binomial, Extreme Gradient Boosting (XGBoost), Random Forests, Arima, and Neural Networks. Since the results were submitted directly to Driven Data, a mean absolute error (MAE) was generated for each submitted model and are available in the Performance/Accuracy section. Prior to submission, the MAE for each model was derived and is discussed in the Performance/Accuracy section.

Significance

Dengue fever is a common tropical disease that affects millions around the world every year. Typically, the virus that causes the Dengue fever is transmitted via mosquitos (Driven Data, 2016). There are no specific treatments for this disease other than rest and the use of pain killers (Gilbert, 2016). According to an article in the New York Times by Jonathan Gilbert, “On rare occasions, the virus can become fatal. There are four types of dengue, which gives the virus the opportunity to re-establish itself in people who have developed immunity against one of the strains.” (Gilbert, 2016). The ability to predict the number of cases that may happen due to a variety of parameters can greatly assist emergency responders and health officials in managing this disease.

Overview of Data

Datasets

Although the different predictor variables are from different data sources, Driven Data (the hosts of this online competition) combined all the information into a single CSV file. The response variable, *total_cases*, was provided in a different file which was then combined in the R script. Finally, the test dataset was also provided, but without the actual number of cases. Table 4 (Appendix) provides an overview of the default predictor variables.

A key note to make is that the data were separated by city: San Jose (*sj*) and Iquitos (*iq*). This enabled the opportunity to construct independent models for each city.

Missing Values and Imputation

Prior to any variable creation, an analysis was done to understand the number of missing values for the default predictor variables. Table 1 summarizes the number of missing values for each of the default predictor variables. Most of them are far below 20% with the exception of the

variable *ndvi_ne* for San Juan. Due to the presence of missing values, imputation will be necessary. Overall, less than 2% of all the observations are missing. Therefore, it was decided to impute the missing information prior to further data exploration.

Table 1: Summary of missing values for each predictor variable

Predictor Variable	Number of Missing Values		Percent Missing by City		Percent Missing Overall
	iq	sj	iq	sj	
<i>year</i>	0	0	0.00%	0.00%	0.00%
<i>weekofyear</i>	0	0	0.00%	0.00%	0.00%
<i>week_start_date</i>	0	0	0.00%	0.00%	0.00%
<i>ndvi_ne</i>	3	191	0.58%	20.41%	13.32%
<i>ndvi_nw</i>	3	49	0.58%	5.24%	3.57%
<i>ndvi_se</i>	3	19	0.58%	2.03%	1.51%
<i>ndvi_sw</i>	3	19	0.58%	2.03%	1.51%
<i>precipitation_amt_mm</i>	4	9	0.77%	0.96%	0.89%
<i>reanalysis_air_temp_k</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_avg_temp_k</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_dew_point_temp_k</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_max_air_temp_k</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_min_air_temp_k</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_precip_amt_kg_per_m2</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_relative_humidity_percent</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_sat_precip_amt_mm</i>	4	9	0.77%	0.96%	0.89%
<i>reanalysis_specific_humidity_g_per_kg</i>	4	6	0.77%	0.64%	0.69%
<i>reanalysis_tdttr_k</i>	4	6	0.77%	0.64%	0.69%
<i>station_avg_temp_c</i>	37	6	7.12%	0.64%	2.95%
<i>station_diur_temp_rng_c</i>	37	6	7.12%	0.64%	2.95%
<i>station_max_temp_c</i>	14	6	2.69%	0.64%	1.37%
<i>station_min_temp_c</i>	8	6	1.54%	0.64%	0.96%
<i>station_precip_mm</i>	16	6	3.08%	0.64%	1.51%

The imputation method chosen was ‘last one carried forward’ (LOCF). This method allows the last known values to be copied into the ‘missing’ value. The LOCF method is fairly simple to understand and it provides acceptable context to variables.

Descriptive Statistics

Table 5 and Table 6 (in the Appendix) provide a brief summary of the default predictors and the response variable, *total_cases*, separated by city. Note that the response variable, *total_cases*, has also been included in these table. Perhaps the most obvious characteristic of the split dataset is the amount of information available for San Juan versus Iquitos (18 years’ worth of information versus 10 years’ worth). In terms of the response variable, *total_cases*, note how the number of cases vary widely amongst the two cities (416 max cases for San Juan vs. 116 max cases for Iquitos).

Many of the predictors’ average and median values are close to each other – suggesting that the outliers do not necessarily have as much of an impact. However, there are predictors whose average is significantly different from the median. For instance, in San Juan, the variable, *precipitation_amt_mm*, has an average of 35.45 mm versus a median of 20.61 mm. Furthermore, in Iquitos, the variable, *station_precip_mm*, has an average of 62.44 mm versus a median of 45.65 mm. This is an early indication that outliers may need to be handled and that skewness may be present. Figure 5 and Figure 6 (both in the Appendix) provide a histogram and

boxplot view for these variables. Note the skewness in both charts as well as the presence of outliers.

Visual Data Exploration

Numerous visual charts were created to help explore the data visually. However, for succinctness, only a few have been included and discussed in this paper. All of the visual data exploration was conducted after the missing values were imputed. Figure 1 illustrates the number of dengue fever cases for both San Juan and Iquitos. This chart also shows the different date ranges for each city. Note how the dengue fever cases in San Juan in the mid to late 1990's were significantly higher. There is also some distinguishable seasonality for both cities. Intuitively, this makes sense as each city experiences different seasons throughout the year.

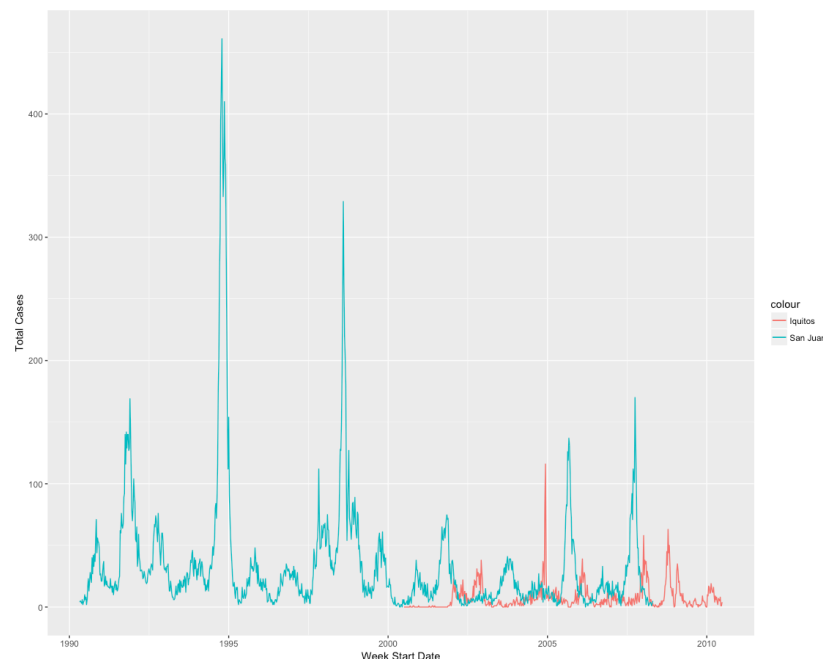


Figure 1: Plot of total_cases over time by city

Figure 2 illustrates the total number of cases for each city by week. Interestingly, week 18 (late April) seems to have the lowest total number of dengue fever cases in San Juan. Accordingly, week 30 (late July) seems to have the lowest total number of dengue fever cases in Iquitos.

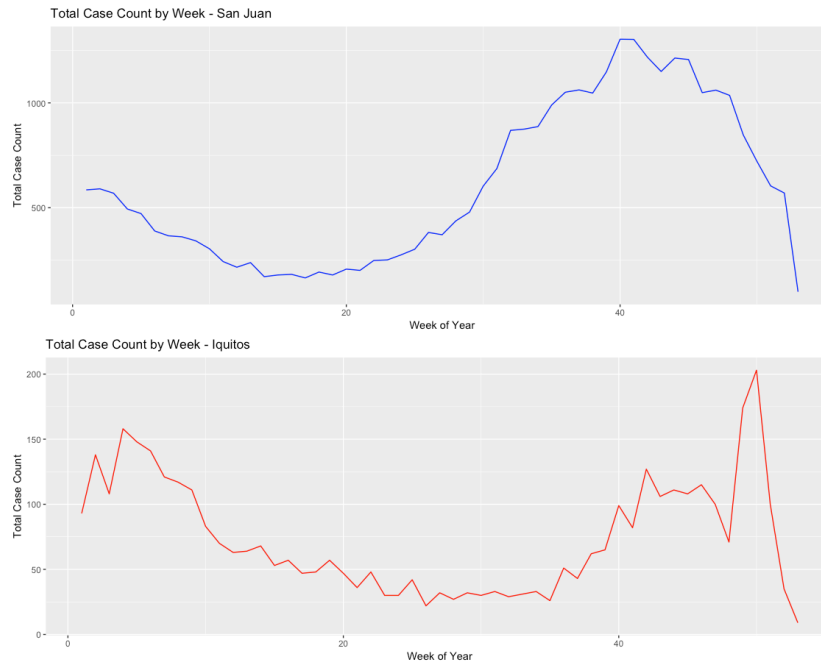


Figure 2: Total case count by week for each city

Figure 3 shows the correlation bar plots for each predictor variable against the response variable, *total_cases*.

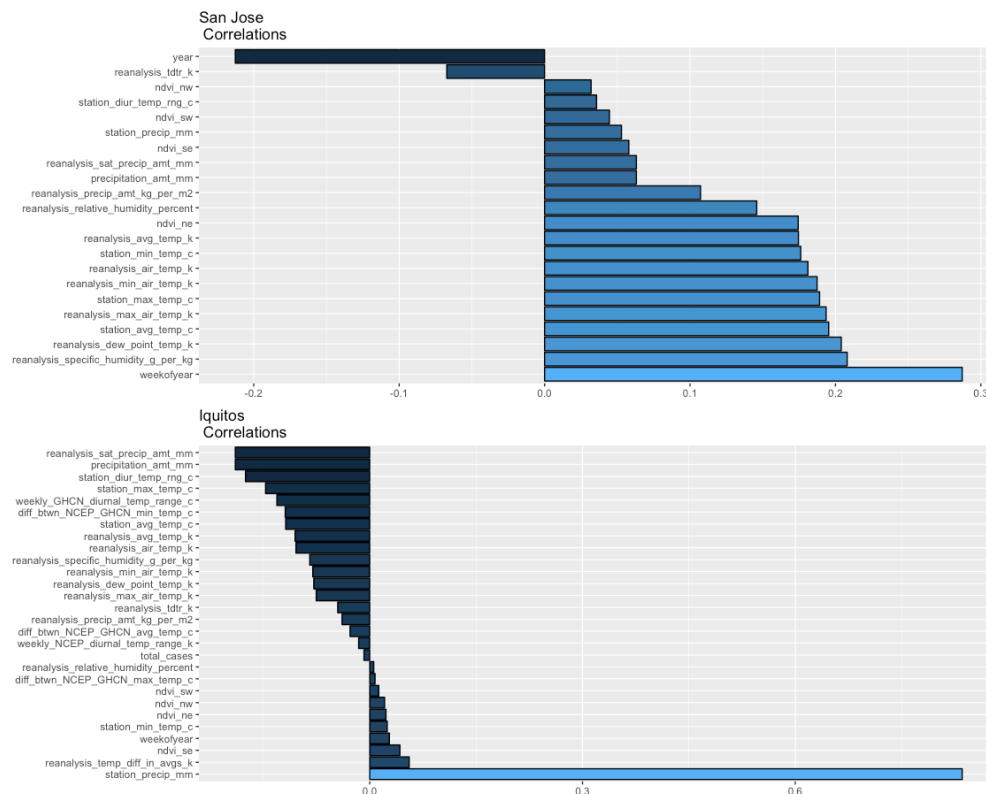


Figure 3: Correlation bar plot by city

Note how strongly the variable, *station_precip_mm*, is correlated to *total_cases* for the city of Iquitos. In contrast, this variable has a fairly weak correlation for the city of San Juan. Interestingly, for the city of San Juan, the variable *year* seems to have a negative correlation to *total_cases*. Additionally, all of the predictors for San Juan do not seem to have strong positive or strong negative correlations – they are below 0.3 or above -0.3.

New Variable Creation

Some of the default predictors tend to have similar names, but they provide information from different sources. Therefore, the new variables constructed attempt to derive the difference between the sources. Table 2 summarizes the new variables that were created.

Table 2: Summary of derived variables

Variable	Description
precipitation_diff	station_precip_mm - precipitation_amt_mm
weekly_GHCN_diurnal_temp_range_c	station_max_temp_c - station_min_temp_c
weekly_NCEP_diurnal_temp_range_k	reanalysis_max_air_temp_k - reanalysis_min_air_temp_k
diff_btwn_NCEP_GHCN_max_temp_c	(reanalysis_max_air_temp_k - 273.15) - station_max_temp_c
diff_btwn_NCEP_GHCN_min_temp_c	reanalysis_min_air_temp_k - 273.15 - station_min_temp_c
diff_btwn_NCEP_GHCN_avg_temp_c	reanalysis_avg_temp_k - 273.15 - station_avg_temp_c
reanalysis_temp_diff_in_avgs_k	(reanalysis_air_temp_k) - (reanalysis_avg_temp_k)

Figure 4 shows the updated correlation bar plot with the new variables.

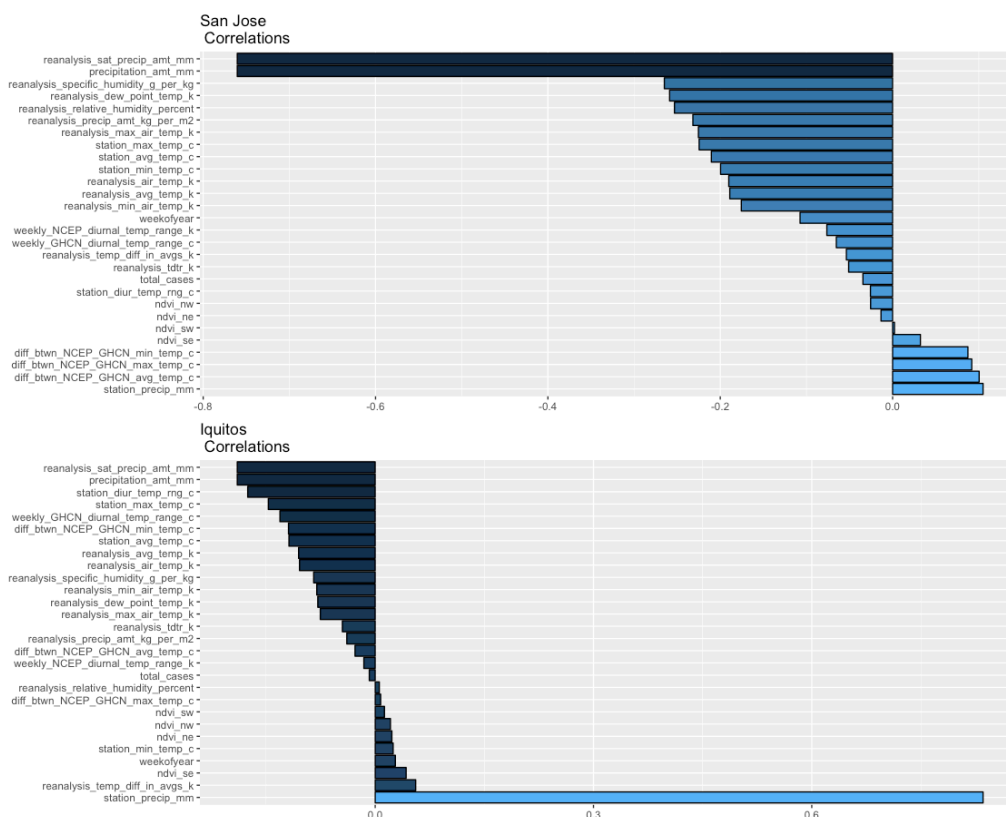


Figure 4: Updated correlation bar plot with derived variables

The most glaring difference, for the city of San Jose, is the strong negative correlation found with the predictor *reanalysis_sat_precip_amt_mm*. This change occurred with the creation of the new variables. For the city of Iquitos, the variable, *station_precip_mm*, continues to be strongly correlated to the response variable, *total_cases*.

Outlier management

For this iteration of the analysis, no outlier management techniques were undertaken. The primary reason for this was to develop models with actual values in place and understand their performance. Outlier management will be reserved for future steps.

Literature

This analysis is focused on predicting (i.e., forecasting) the number of dengue fever cases in the cities of San Juan and Iquitos. In the following section (Type of Models), five different models are discussed. This section discusses academic (from peer reviewed journals) papers that utilize similar models for classification.

ARIMA/Random Forest

In the article, “Comparison of ARIMA and Random Forest time series models for prediction of avian influenza H5N1 outbreaks”, the authors compare an ARIMA strategy to a random forest strategy. The objective of their research is somewhat similar to what is being undertaken with this activity. During their testing, they found that the random forest model outperformed a retrospective ARIMA model. Nevertheless, they did find that both models were not as satisfactory. For instance, the ARIMA model predicted negative cases – which is not possible. (Kane, Price, Scotch, & Rabinowitz, 2014). However, their research into both ARIMA and random forest suggests that random forests may be able to detect patterns at a more complex level than ARIMA.

XGBoost

In the article, “A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring”, the authors discuss using decision trees to predict credit scores. The most interesting aspect of this model is the discussion surrounding hyper-parameters. Hyper-parameters are a powerful way to ‘simulate’ multiple different tuning parameter values to obtain a model that is very good. In their research, the authors found that a Bayesian hyper-parameter optimization is far superior to other approaches, such as a grid search (Xia, Liua, Li, & Liu, 2017).

Neural Network

In the article, “Comparative study among different neural net learning algorithms applied to rainfall time series”, the authors use neural networks to forecast rainfall. Many of the approaches that the authors discuss are quite advanced, but their conclusion was simply, “artificial neural network is a suitable predictive tool for average monsoon rainfall”

(Chattopadhyay & Chattopadhyay, 2008). Neural network models, like decision tree models, are quite complex and very powerful.

Negative Binomial

In the article, “Using the negative binomial distribution to model over dispersion in ecological count data”, the authors describe the use of a negative binomial regression model in bird migration (as an example). Recall from the section Type of Models, the variance of the variable, *total_cases*, is far higher than the mean. Therefore, a negative binomial model would be more appropriate than a Poisson model. In their research, the authors also describe how over dispersion is a common occurrence in nature (Lindén & Mäntyniemi, 2011).

Negative Binomial

In the paper, “Modeling Forest Fire Occurrences Using Count-Data Mixed Models in Qiannan Autonomous Prefecture of Guizhou Province in China”, the authors use a negative binomial model, but with a twist. In their research, the authors use mixed-effects to improve the predictability. They also used hurdle models along with zero-inflated binomial regression models to clarify the effects of introducing mixed-effects. This strategy enables randomness to be introduced to the fixed-events and thereby increasing the ability of the model to perform well. According to the authors, the occurrence of forest fires can be unpredictable due to a variety of factors other than just wind (Xiao, Zhang, & Ji, 2014). This article also serves as inspiration for next steps.

Type of Models

Several different types of models were constructed, but only five of these models were chosen to be submitted on Driven Data. The five models are:

- Negative Binomial
- Extreme Gradient Boosting (XGBoost)
- Random Forests
- ARIMA
- Neural Net

Negative Binomial

Recall that the response variable is essentially counting the number of dengue fever cases in two cities. According to the author of “Generalized Linear Models An Applied Approach”, “When a dependent variable is over dispersed, many statistical modelers prefer to use...a negative binomial regression model” (Hoffman, 2004). From TABLE XX, the average number of cases for San Juan and Iquitos, respectively, are 19 and 5. However, the variances are (for San Juan and Iquitos respectively) approximately 2640 and 115.9. Therefore, a negative binomial model is a great model to consider.

Extreme Gradient Boosting (XGBoost)

The XGBoost model was chosen due to its veracity and its complex learning abilities. Recall from the VISUAL EXPLORATION section that there appears to be numerous behaviors that affect the number of dengue fever cases. Decision trees and boosted models are able to capture nuances in the datasets and produce accurate models.

Random Forests

Random forests is an interesting approach as this machine learning staple creates ‘tiny’ trees that are not very deep (in terms of branches). Furthermore, the algorithms create thousands of trees (each one could virtually use any number of predictors and no tree necessarily has all possible predictors) very quickly. According to the authors of “An Introduction to Statistical Learning with Applications in R”, “random forests [are] an improvement over bagged trees by way of a small tweak that decorrelates the trees” (James, Witten, Hastie, & Tibshirani, 2015). This is a clear reason why bagging models were not considered since bagging models will enable the use of all predictors in a tree.

ARIMA

ARIMA is an interesting concept in which autoregression and moving average strategies are combined (Hyndman & Athanasopoulos, 2013). This approach was used with the assumption that the information in this dataset is a univariate time series. The results of this model will clearly demonstrate some of the limitations of assuming that the total cases of dengue fever of each city is univariate.

Neural Network

Neural networks, like decision trees, are quite informative and powerful. According to the text, “Applied Predictive Modelling”, “the outcome is modeled by an intermediary set of unobserved variables” (Kuhn & Johnson, 2013). In this iteration of the analysis, a neural network model was constructed to help clarify its performance against the other machine learning techniques such as boosting and random forests (both of which are advanced decision trees).

Formulation

All the models were constructed using R. The following are the key packages used within R:

- Caret
- MASS
- XGBoost
- Forecast

The caret package is quite powerful and enables easier cross-validation for any type of model. One of the control parameters for caret is the trainControl function. With this control parameter, XGBoost and random forest models utilized repeated cross validation, using 10 folds and repeated three times. The cross-validation approach enables a stronger learning capability for the models since a small subset of the training data can be used for validation

prior to the validation dataset being used. Since this is done inherently within the caret package, the final model can be considered a decent model with the given parameters.

The forecast package enabled the creation of ARIMA and the neural network model. Both models were created using fundamental principles with no cross-validation.

Different datasets were constructed for use in each model. However, the random forest and xgboost models utilized the same datasets. Recall from earlier that a model was constructed for each city independently. For the machine learning models (xgboost and random forest), the variable *year* and *week_start_date* were removed. No scaling or variable transformation was undertaken. Furthermore, the training and validation split was 80/20 – 80% of the information was used to train the model and 20% of the data were used to validate the model. The 20% of the data (used in the validation) were not reintroduced to the model as that would constitute a potential leakage that could result in false perceptions.

For the negative binomial model, only the variables that had absolute correlation value¹ greater than 0.1. This resulted in a substantially smaller dataset, but the reasoning was that if the variable has a correlation between -0.1 and 0.1, then it may not be as useful. Interaction terms were not considered for the first iteration of this analysis.

For the ARIMA model, all predictors were not considered and the variable *total_cases* was converted into a univariate time series. This is in contrast to the other models where the other variables were used to predict the total number of cases. A grid search was also done to find the optimal *p*, *d*, and *q* values (autoregression, differencing², and moving average order respectively). As the grid search commenced, the mean absolute error (MAE) was calculated and the *p* and *q* values were capped at 10 whereas the *d* value was capped at 2. The MAE, however, was assessed against the validation dataset in the grid search. The thought process was that if the ARIMA values are fitted against the validation dataset, the training dataset would be overfitted, however, the validation dataset would be fitted correctly.

For the neural network model, all of the predictors were used and they were centered and scaled. Recall from TABLE XX that many of the variables have different ranges and the contexts vary greatly. For instance, the temperatures (the ones in Celsius) have a different scale than the precipitation. Therefore, by centering and scaling, the values in each predictor can be close to normal (with a mean of 0).

Performance/Accuracy

In order to assess the models, the mean absolute error (MAE) was calculated for each city. On Driven Data, the submitted models were also assessed by their MAE. The challenge is that the

¹ The absolute value means that -0.2 is equal to 0.2 (for example).

² Differencing is necessary in order to ensure a stationary time series. Figure YY clearly shows that the time series are not stationary for either city.

derived MAE values are not as meaningful since the predicted case counts for each city have to be combined into one file prior to submission on Driven Data.

Table 3 summarizes the results of each model.

Table 3: Summary of MAE for each model

Model	Validation dataset MAE (San Jose, Iquitos)	Driven Data Test MAE
Negative Binomial	21.553, 5.568	35.0288
XGBoost	16.592, 5.833	27.2212
Random Forest	17.122, 5.490	27.2212
ARIMA	10.851, 1.656	71.9663
Neural Network	100.78, 4.392	33.1971

Since the Driven Data MAE is of premier significance, the XGBoost and Random Forest models perform the best. However, based on the validation dataset MAE, it would appear that the ARIMA model performed the best. Surprisingly, the neural network model performed terribly for cases in San Jose, but performed the best of all models for Iquitos.

Limitations

There are two key limitations to the models constructed complexity and optimization. Random forest, XGBoost, and neural networks are complex models that are not very easy to explain. For instance, the random forest and XGBoost approaches constructed thousands of trees, but there is no distinctive equation that can be documented and easily explained. Rather, the explanation of each tree would result in a convoluted explanation of the logic and the randomness. Similarly, the neural network model is also convoluted and is essentially a network of non-linear equations to model the final response.

One key area to explore would be the optimization of each of these algorithms. For instance, the shrinkage parameter would enable the reduction of predictors and could improve a fit of a model while also reducing the likelihood of overfitting a model. Although it is difficult to overfit a model using boosting methods, it is still possible and the choice of parameters could affect the accuracy. Another avenue would be to investigate the type of external regressors used and their optimizations. It is very likely that not all features have been created that could better explain the different number of dengue cases.

Finally, the different data points seem to measure different things. For instance, it is not clear what the diurnal range is. Therefore, it was prudent to create a new diurnal temperature range that looked at the weekly maximum and minimum. On a personal note, this limitation could be overcome by focusing on the source of the data and how they are compiled. The Driven Data website was quite sparse on the background of the information, however, they did link to the NOAA websites – which would prove to be useful.

Future Work

There are four areas of improvement for the next step:

- New data sources
- Feature creation
- Outlier management
- Model Optimization

New Data Sources

The Driven Data competition did not explicitly discourage the use of external data sources. Two interesting data sources would be the GPS coordinates of the reported cases and the location of the different clinics treating the reported cases. This would enable a greater understanding of the impact of transmission and dengue fever localization. Another great resource would be the outcome of the reported case along with healing time. For instance, is it possible that within Iquitos (for example), a particular region is more susceptible for dengue fever cases?

Feature creation

As previously mentioned, the weather data provided are quite convoluted and not easy to understand. Through increased research on the weather information itself, new features could be created. Furthermore, with the ability to bring in external data sources, that could result in even more meaningful information. Another feature would be to understand abnormal weather patterns or other catastrophic events of nature that may have influenced the number of dengue fever cases.

Outlier Management

As previously mentioned, the presence of outliers was quite glaring. The risk with elimination of outliers is that the model may work well with both training and validation datasets, it may not work as well with the test dataset. Furthermore, the outliers themselves need a greater understanding. For instance, one of the weeks in San Juan had over 400 reported cases. Why is that? One of the benefits of machine learning algorithms is their inherent ability to account for outliers. However, if outliers are managed properly, these algorithms may experience improvements in their accuracy.

Model Optimization

Earlier, a note was made about the lack of interaction terms in this first analysis. Interaction terms would be an excellent way to understand how other variables (e.g., ones with low correlation) can be instrumental when combined with other variables. This type of optimization could provide even more insight than the default predictors. Another approach would be to leverage mixed-effects (the introduction of randomness) to help improve the overall model's ability to forecast dengue fever cases. Other approaches include the ARCH-GARCH method and Hurdle models.

Learning

The key takeaway of this activity has been the pleasure of working with weather data. One of my projects at work this summer will be incorporating weather data to understand machine (i.e., combine) behavior. I was actually quite ecstatic to use weather data in this project. However, I am also overwhelmed with the amount of information that exists regarding weather and the fact that there are so many different type of weather data available. I am looking forward to continuing the study of weather data as it will greatly help me at work.

Another important lesson for me was the difference between univariate time series along with using neural network with external regressors. This past quarter in PREDICT 413 has been an incredible learning experience and I have absolutely enjoyed applying my skills on real world data. The incorporation of ARIMA and neural network in this paper was definitely fun and a great learning experience.

Works Cited

- Chattopadhyay, S., & Chattopadhyay, G. (2008, April 18). Comparative study among different neural net learning algorithms applied to rainfall time series. *Royal Meteorological Society*, 15(2), 273-280.
- Driven Data. (2016). *DengAI: Predicting Disease Spread*. Retrieved May 28, 2017, from Driven Data: <https://www.drivendata.org/competitions/44/dengai-predicting-disease-spread/page/80/>
- Gilbert, J. (2016, February 16). *Argentina Battles Major Outbreak of Dengue as Mosquito Population Swells*. Retrieved May 28, 2017, from New York Times: <https://www.nytimes.com/2016/02/18/world/americas/argentina-battles-major-outbreak-of-dengue-as-mosquito-population-swells.html?action=click&contentCollection=Health&module=RelatedCoverage®ion=EndOfArticle&pgtype=article>
- Hoffman, J. P. (2004). *Generalized Linear Models An Applied Approach*. New York, New York: Pearson Education Inc.
- Hyndman, R. J., & Athanasopoulos, G. (2013). *Forecasting: principles and practice*. Retrieved May 27, 2017, from Forecasting: principles and practice: <http://otexts.org/fpp/>
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2015). *An Introduction to Statistical Learning with Applications in R* (6th Edition ed.). New York, New York: Springer Science + Business.
- Kane, M. J., Price, N., Scotch, M., & Rabinowitz, P. (2014, August 13). Comparison of ARIMA and Random Forest time series models for prediction of avian influenza H5N1 outbreaks. *BMC Bioinformatics*, 276.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modelling* (5th Edition ed.). New York, New York: Springer Science+Business Media.
- Lindén, A., & Mäntyniemi, S. (2011). Using the negative binomial distribution to model overdispersion in ecological count data. *Ecological Society of America*, 1414-1421.

- Xia, Y., Liua, C., Li, Y., & Liu, N. (2017, February 9). A boosted decision tree approach using Bayesian hyper-parameter optimization for credit scoring. *Elsevier Expert Systems With Applications*, 225-241.
- Xiao, Y., Zhang, X., & Ji, P. (2014, June 10). Modeling Forest Fire Occurrences Using Count-Data Mixed Models in Qiannan Autonomous Prefecture of Guizhou Province in China. *PLOS One*.

Appendix

Table 4: Data dictionary for default predictor variables

Predictor Variable	Description	Source
year	Year of the date	N/A
weekofyear	Week number of the year	N/A
week_start_date	Date given in yyyy-mm-dd format	N/A
station_max_temp_c	Maximum temperature	NOAA's GHCN
station_min_temp_c	Minimum temperature	NOAA's GHCN
station_avg_temp_c	Average temperature	NOAA's GHCN
station_precip_mm	Total precipitation	NOAA's GHCN
station_diur_temp_rng_c	Diurnal temperature range	NOAA's GHCN
precipitation_amt_mm	Total precipitation	PERSIANN satellite
reanalysis_sat_precip_amt_mm	Total precipitation	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_dew_point_temp_k	Mean dew point temperature	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_air_temp_k	Mean air temperature	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_relative_humidity_percent	Mean relative humidity	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_specific_humidity_g_per_kg	Mean specific humidity	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_precip_amt_kg_per_m2	Total precipitation	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_max_air_temp_k	Maximum air temperature	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_min_air_temp_k	Minimum air temperature	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_avg_temp_k	Average air temperature	NOAA's NCEP Climate Forecast System Reanalysis
reanalysis_tdtr_k	Diurnal temperature range	NOAA's NCEP Climate Forecast System Reanalysis
ndvi_se	Pixel southeast of city centroid	NOAA's CDR Normalized Difference Vegetation Index
ndvi_sw	Pixel southwest of city centroid	NOAA's CDR Normalized Difference Vegetation Index
ndvi_ne	Pixel northeast of city centroid	NOAA's CDR Normalized Difference Vegetation Index
ndvi_nw	Pixel northwest of city centroid	NOAA's CDR Normalized Difference Vegetation Index

Table 5: Descriptive statistics for default predictor variables for San Juan

Variable	Minimum	Maximum	Average	Median
year	1990	2008	1999	1999
weekofyear	1	53	26.5	26.5
week_start_date	1990-04-30	2008-04-22	1999-04-26	1999-04-26
ndvi_ne	-0.40625	0.4934	0.058466	0.05935
ndvi_nw	-0.4561	0.4371	0.06599	0.06595
ndvi_se	-0.01553	0.39313	0.17796	0.17759
ndvi_sw	-0.06346	0.38142	0.16614	0.16737
precipitation_amt_mm	0	390.6	35.45	20.61
reanalysis_air_temp_k	295.9	302.2	299.2	299.2
reanalysis_avg_temp_k	296.1	302.2	299.3	299.4
reanalysis_dew_point_temp_k	289.6	297.8	295.1	295.4
reanalysis_max_air_temp_k	297.8	304.3	301.4	301.5
reanalysis_min_air_temp_k	292.6	299.9	297.3	297.5
reanalysis_precip_amt_kg_per_m2	0	570.5	30.45	21.3
reanalysis_relative_humidity_percent	66.74	87.58	78.57	78.67
reanalysis_sat_precip_amt_mm	0	390.6	35.45	20.61
reanalysis_specific_humidity_g_per_kg	11.72	19.44	16.54	16.83
reanalysis_tdtr_k	1.357	4.429	2.514	2.45
station_avg_temp_c	22.84	30.07	27	27.21
station_diur_temp_rng_c	4.529	9.914	6.753	6.757
station_max_temp_c	26.7	35.6	31.6	31.7
station_min_temp_c	17.8	25.6	22.59	22.8
station_precip_mm	0	305.9	26.8	17.8
total_cases	0	461	34.18	19

Table 6: Descriptive statistics for default predictor variables for Iquitos

Variable	Minimum	Maximum	Average	Median
year	2000	2010	2005	2005
weekofyear	1	53	26.5	26.5
week_start_date	2000-07-01	2010-06-25	2005-06-28	2005-06-28
ndvi_ne	0.06173	0.50836	0.26355	0.26355
ndvi_nw	0.03586	0.45443	0.23844	0.2325
ndvi_se	0.02988	0.53831	0.24991	0.24976
ndvi_sw	0.06418	0.54602	0.26637	0.26187
precipitation_amt_mm	0	210.83	64.19	60.47
reanalysis_air_temp_k	294.6	301.6	297.9	297.8
reanalysis_avg_temp_k	294.9	302.9	299.1	299.1
reanalysis_dew_point_temp_k	290.1	298.4	295.5	295.9
reanalysis_max_air_temp_k	300	314	307.1	307.1
reanalysis_min_air_temp_k	286.9	296	292.9	293.1
reanalysis_precip_amt_kg_per_m2	0	362.03	57.59	46.45
reanalysis_relative_humidity_percent	57.79	98.61	88.65	90.94
reanalysis_sat_precip_amt_mm	0	210.83	64.19	60.47
reanalysis_specific_humidity_g_per_kg	12.11	20.46	17.1	17.43
reanalysis_tdtr_k	3.714	16.029	9.202	8.964
station_avg_temp_c	21.4	30.8	27.52	27.57
station_diur_temp_rng_c	5.2	15.8	10.54	10.55
station_max_temp_c	30.1	42.2	33.99	34
station_min_temp_c	14.7	24.2	21.2	21.35
station_precip_mm	0	543.3	62.44	45.65
total_cases	0	116	7.565	5

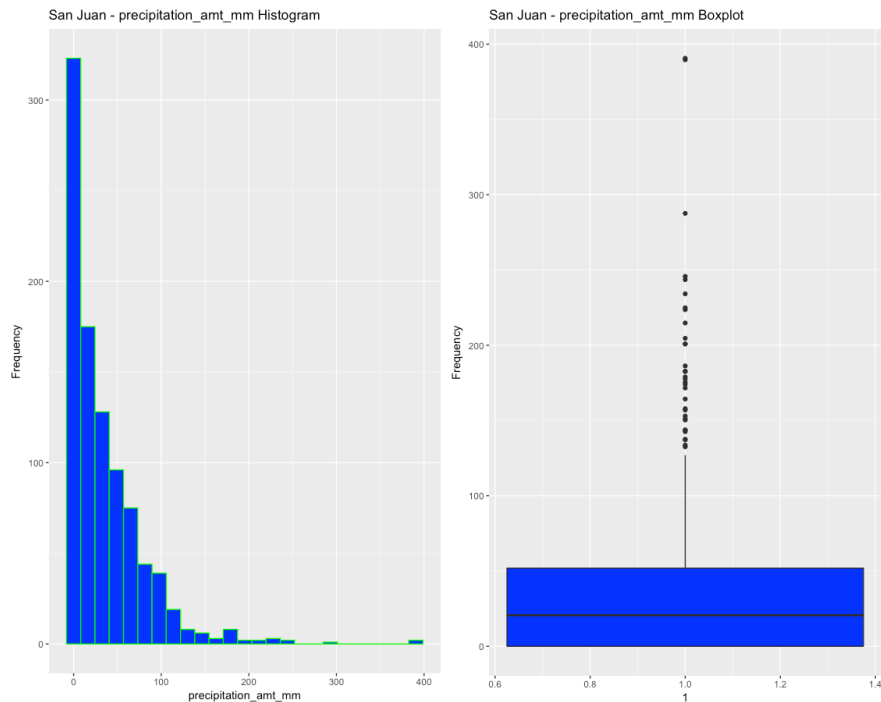


Figure 5: Histogram and boxplot for variable precipitation_amt_mm for San Juan

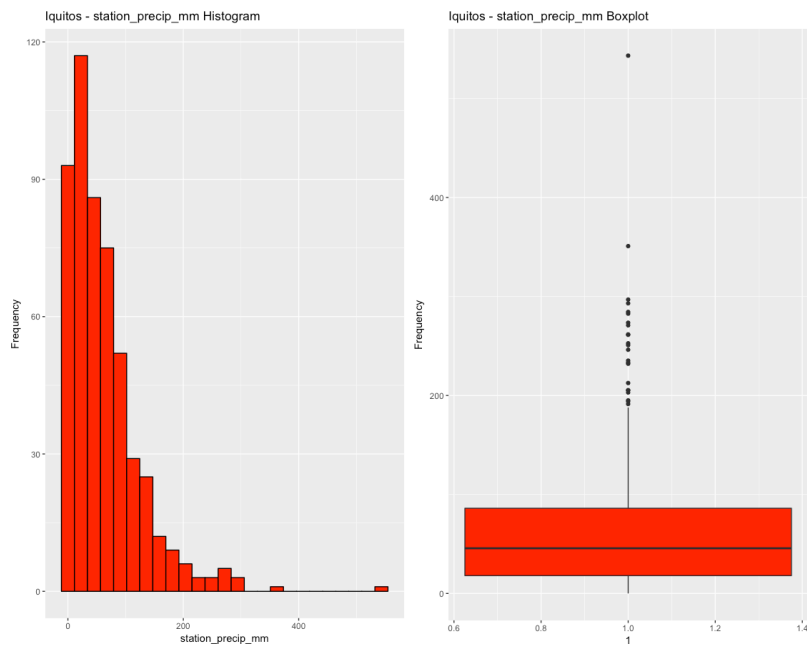


Figure 6: Histogram and boxplot for variable station_precip_mm for Iquitos

PREDICT 422 Charity Project

**Nikhil Agarwal
Northwestern University
PREDICT 422 | Section 58**

Introduction

The intent of this project is to construct a classification and a predictive model that can predict potential donors (DONR) and the expected donation amount (DAMT) respectively. Over 8000 observations have been used to construct the candidate models. Furthermore, two groups of datasets (one using factors and the other not, with and without variable transformations) were used to construct over 12 types of models. The key metrics to determine the best models are maximum profit (for the response variable DONR) and average squared prediction error (for the response variable DAMT). This report provides a succinct overview of the models and results. Next steps are also discussed.

Analysis

Dataset Overview

The primary dataset consists of 8,009 observations with 20 potential predictors and two response variables. Table 1 is a data dictionary that provides a quick summary of the variables. Note that the data types listed are unmodified from the import process and have not been transformed.

Table 1: Data dictionary

Columns	Definition	Data Type
REG1	Region (There are five geographic regions; only 4 are needed for analysis since if a potential donor falls into none of the four he or she must be in the other region.	Binary
REG2	Same as REG1	Binary
REG3	Same as REG1	Binary
REG4	Same as REG1	Binary
HOME	(1 = homeowner, 0 = not a homeowner)	Binary
CHLD	Number of children	Integer
HINC	Household income (7 categories)	Integer
GENF	Gender (0 = Male, 1 = Female)	Binary
WRAT	Wealth Rating (Wealth rating uses median family income and population statistics from each area to index relative wealth within each state. The segments are denoted 0-9, with 9 being the highest wealth group and 0 being the lowest.)	Integer
AVHV	Average Home Value in potential donor's neighborhood in \$ thousands	Numeric
INCM	Median Family Income in potential donor's neighborhood in \$ thousands	Numeric
INCA	Average Family Income in potential donor's neighborhood in \$ thousands	Numeric
PLOW	Percent categorized as "low income" in potential donor's neighborhood	Numeric
NPRO	Lifetime number of promotions received to date	Integer
TGIF	Dollar amount of lifetime gifts to date	Numeric
LGIF	Dollar amount of largest gift to date	Numeric
RGIF	Dollar amount of most recent gift	Numeric
TDON	Number of months since last donation	Integer
TLAG	Number of months between first and second gift	Integer
AGIF	Average dollar amount of gifts to date	Numeric
DONR	Classification Response Variable (1 = Donor, 0 = Non-donor)	Binary
DAMT	Prediction Response Variable (Donation Amount in \$).	Numeric

For the model training process, the dataset had already pre-defined observations that were part of the training, validation, or testing datasets. As such, each of the observations were then split up according to the pre-defined classification. Table 2 provides a break out of each data collection and the number of observations in each one.

Table 2: Breakdown of training, validation, and testing data counts

Data Collection	Number of Observations	Percentage of observations
Training	3984	49.7%
Validation	2018	25.2%
Testing	2007	25.1%

Approximately, 50% of the observations are part of the training dataset. No variable within the dataset had missing values. Therefore, data imputation will not be necessary for this dataset. The next section provides an exploratory data analysis of the training dataset¹. Outliers and multicollinearity analyses were considered out of scope for this report.

EDA of Training Dataset

Table 3 illustrates some basic statistics on the many numeric variables along with the response variable DAMT.

Table 3: Descriptive statistics on numeric variables

Variable	Mean	Maximum	Skewness ²	Kurtosis ³	Standard Deviation	Median
AVHV	185.1842	710	1.54803	7.372442	74.70008	171
INCM	44.2884	287	2.000311	11.10594	25.17556	39
INCA	57.13554	287	1.867308	9.955876	25.27604	52
PLOW	13.73268	87	1.381409	4.935768	13.09388	10
NPRO	61.62927	164	0.2796	2.385898	30.3397	60
TGIF	116.7447	1974	5.053572	73.31685	85.45911	91
LGIF	23.19302	642	7.87338	108.4112	31.27813	15
RGIF	15.54669	173	2.653386	18.7009	12.0772	12
TDON	18.81476	40	1.126973	5.425644	5.582317	18
TLAG	6.301958	34	2.415135	11.45665	3.598049	5
AGIF	11.65924	64.22	1.64415	7.953055	6.411263	10.22
DAMT	7.26004	25	0.102645	1.156039	7.378314	10

Interestingly, there are many variables that are not normally distributed and are heavily skewed. There is also evidence of outliers (when comparing the mean, median, and maximum values) amongst some of the variables. For instance, AVHV has a mean of 185.18, but the median is only 171 and the maximum value is 710.

¹ Note that an EDA is not done on the validation or testing datasets to ensure minimal influence on the training dataset modeling process

² A skewness of 0 indicates normal distribution

³ A kurtosis of 3 indicates normal distribution

Table 4 shows the correlation matrix for all of the predictor variables versus the two response variables. The intent here is to discover which predictor variables tend to have most correlation with the response variables.

Table 4: Correlation matrix of response variables vs. potential predictors

Predictor Variable	donr	damt
reg1	0.056	0.047
reg2	0.247	0.211
reg3	-0.104	-0.084
reg4	-0.126	-0.089
home	0.289	0.288
chld	-0.531	-0.551
hinc	0.028	0.045
genf	-0.017	-0.019
wrat	0.249	0.243
avhv	0.119	0.118
incm	0.158	0.164
inca	0.139	0.142
plow	-0.143	-0.137
npro	0.136	0.141
tgif	0.116	0.127
lgif	0.026	0.082
rgif	0.015	0.085
tdon	-0.096	-0.088
tlag	-0.141	-0.133
agif	0.009	0.079

Surprisingly, there is no one predictor variable that has a strong positive or negative correlation with either DONR or DAMT. Nevertheless, the predictor CHLD has a medium negative correlation with the response variables. Intuitively, this makes sense as individuals with children may be less likely to donate and if they do, may have reduced donations. Another interesting note is the little impact that the variable GENF has on either response variable.

Note how the category region is already mutated into four⁴ binary variables (REG1, REG2, etc.). Figure 1 illustrates the size of each region. Region 1 makes up about 20.5%, Region 2 makes up about 33.6%, Region 3 makes up about 12.3%, Region 4 makes up about 13.5%, and Region 5 makes up about 20.1% of the training dataset.

⁴ If all 4 regions (REG1, REG2, etc.) are 0, then the observation is considered as part of Region 5. Consider these variables as indicator (i.e., dummy) variables

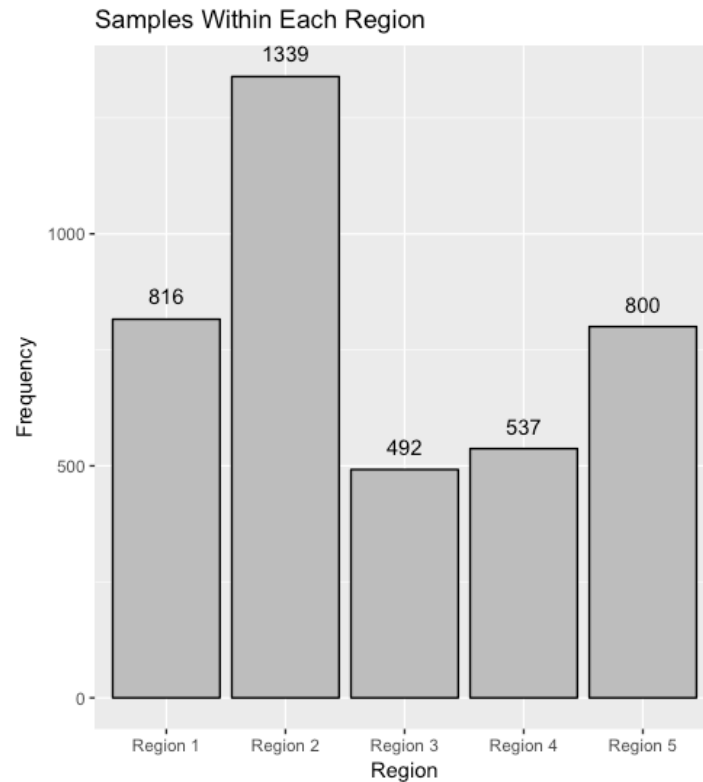


Figure 1: Training data observation counts by Region

Numerous histograms and boxplots were created for each numeric and integer related variables. For brevity not all of the histograms and boxplots are in this report. However, Figure 2 illustrates an example of the boxplot and histogram for the variable AVHV.

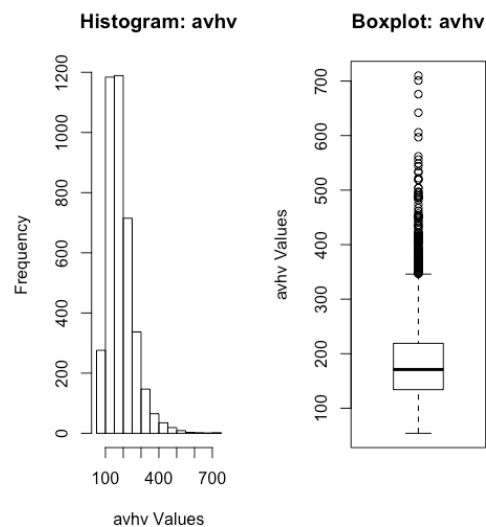


Figure 2: Example histogram and boxplot of variable AVHV

Note the skewness in the histogram as well as the presence of outliers for this predictor variable. Table 3 alluded to this skewness (1.54) earlier. For this project, outlier management was not conducted. For future steps, this may be a necessary step or if a more robust model is required than the one suggested in this exercise.

Variable Transformations

Many of the numeric variables (e.g., AVHV, INCA, etc.) have significant skewness and do not adhere to normality assumption. Figure 2 provided an excellent example of the skewness for variable AVHV. Furthermore, many of the numeric variables were also standardized so that they would have a mean of 0 and a standard deviation of 1. Table 5 summarizes the variable transformations applied to each of the variables, if any. Note that no standardization or transfixion was done for either response variable.

Table 5: Overview of datasets, predictor variable transformations, and data type

	DATASET 1				DATASET 2			
Predictor Variables	Raw	Log Transformation	Standardized	Data Type	Raw	Log Transformation	Standardized	Data Type
REG1	Y	N	N	Binary	Y	N	N	Binary
REG2	Y	N	N	Binary	Y	N	N	Binary
REG3	Y	N	N	Binary	Y	N	N	Binary
REG4	Y	N	N	Binary	Y	N	N	Binary
HOME	Y	N	N	Binary	Y	N	N	Binary
CHLD	Y	N	N	Factor	Y	N	N	Numeric
HINC	Y	N	N	Factor	Y	N	N	Numeric
GENF	Y	N	N	Binary	Y	N	N	Binary
WRAT	Y	N	N	Factor	Y	N	N	Numeric
AVHV	Y	Y	Y	Numeric	Y	Y	Y	Numeric
INCM	Y	Y	Y	Numeric	Y	Y	Y	Numeric
INCA	Y	Y	Y	Numeric	Y	Y	Y	Numeric
PLOW	Y	Y	Y	Numeric	Y	Y	Y	Numeric
NPRO	Y	N	N	Numeric	Y	N	N	Numeric
TGIF	Y	Y	Y	Numeric	Y	Y	Y	Numeric
LGIF	Y	Y	Y	Numeric	Y	Y	Y	Numeric
RGIF	Y	Y	Y	Numeric	Y	Y	Y	Numeric
TDON	Y	N	N	Numeric	Y	N	N	Numeric
TLAG	Y	N	N	Numeric	Y	N	N	Numeric
AGIF	Y	Y	Y	Numeric	Y	Y	Y	Numeric

In Table 5, there is also mention of two different datasets. The first dataset (Dataset 1), considers predictor variables CHLD, HINC, and WRAT as factors. The second dataset (Dataset 2), considers these variables as numeric. The same transformation was applied to the validation datasets as well. The use of these two datasets alludes to an assumption on whether a particular variable (e.g., CHLD, HINC, or WRAT) could be considered as classifications or numeric values. A log transformation was used since it presented more adherence to the normality assumption. Figure 3 illustrates the variable AVHV after it undergoes a log transformation.

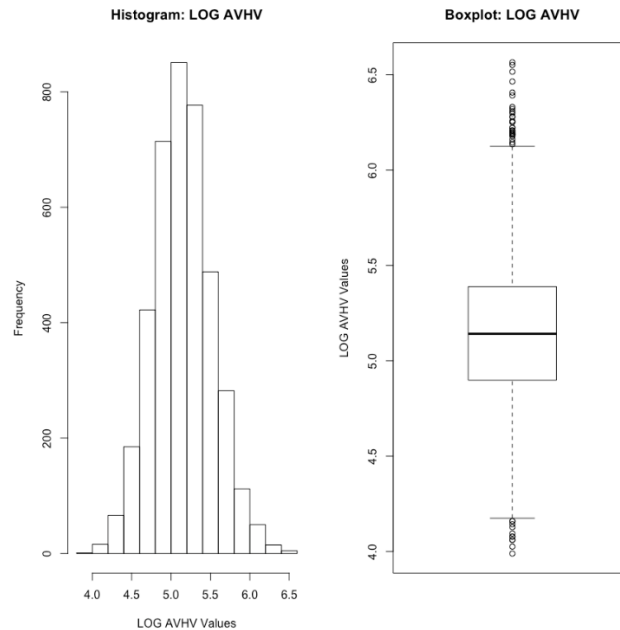


Figure 3: Example histogram and boxplot of log transformation of variable AVHV

After the log transformation of this variable, the skewness and kurtosis values were approximately 0.168 and 3.123 respectively. Another note to make here is that no variable that had undergone a log transformation was standardized (mean of 0 with a standard deviation of 1).

Analysis of the DONR Classification Models

For the classification model, five different models were constructed using the two different datasets (identified in Table 5): logistic, LDA, QDA, Boosting, and Random Forest. Table 6 summarizes all of the results for each type of model. In this section, succinct highlights of each type of model is provided. Prior to operationalizing any classification model, the resulting projections will need to be adjusted for the weighted sampling used in the training and validation datasets. In the **Results of the DONR Classification Models** section, justification is provided for the model chosen for each response variable. For each classification model, a plot like Figure 4 was constructed to help illustrate the idea that as the number of mailings increased, the profit would increase, but then decrease as well. Rather than using a threshold of 0.5 to convert the probabilities into a particular class (i.e., 0 or 1), the maximum profit was determined and then the appropriate probability threshold (that would yield the maximum profit) was used. In every model, the arbitrary value of 0.5 was found to be not as optimal as the calculated threshold.

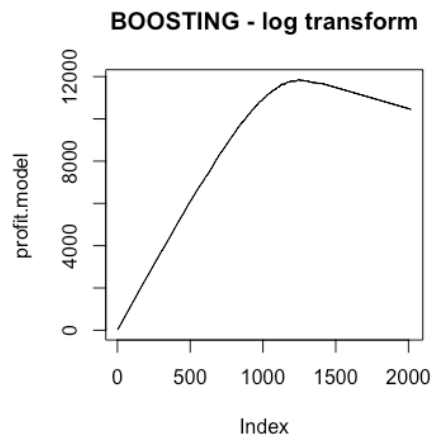


Figure 4: Example plot showing effect of profit vs. number of mailings

Classification Model – Logistic Regression Model

Six logistic models were created: 3 using each dataset described in Table 5. Each model used all predictors initially and then a stepwise automated variable selection (using BIC as the selection criteria) was used to eliminate non-statistically significant predictors. Using Dataset 1, the highest realized profit was about \$11852.50 with an error rate of 13.1%. Using Dataset 2, the highest realized profit was about \$11848.50 with an error rate of 14.3%.

Classification Model – LDA

Similar to the logistic regression model, six linear discriminant analysis (LDA) models were constructed. Neither LDA model within each Dataset was superior to the best performing logistic model for each dataset. Using Dataset 1, the best performing LDA model predicted a maximum profit of \$11843, used log transformations, and had an error rate of 13.6%. Using Dataset 2, the best performing LDA model predicted a maximum profit of \$11368, used log transformations, and had an error rate of 21.8%.

Classification Model – QDA

Similar to the LDA models, six quadratic discriminant analysis (QDA) models were constructed. Neither QDA model within each dataset was superior to the best performing logistic model for each dataset. Using Dataset 1, the best performing QDA model predicted a maximum profit of \$11041.50, used log transformations, and had an error rate of 28.5%. Using Dataset 2, the best performing QDA model predicted a maximum profit of \$11270.50, used log transformations, and had an error rate of 21.3%. It is interesting to note that the error rate for the QDA model using Dataset 2 had a lower error rate than the QDA model using Dataset1 AND the respective LDA model (which had an error rate of 21.8%). Some of the high error can be attributed to the lack of higher polynomial transformations.

Classification Model – Boosting

All six boosting models, using all 20 predictors in both Datasets, outperformed the logistic regression models. For each boosting model, the relative influence plot and statistics were analyzed. Figure 5 illustrates a sample output of the relative influence statistics and Figure 6 illustrates the relative influence plot. These plots help one understand impactful predictors in the boosted model. Using Dataset 1, the best boosting model projected a maximum profit of \$11852.50 with an error rate of 13.1%. Using Dataset 2, the best boosting model projected a maximum profit of \$11848.50 with an error rate of 14.3%. Unlike QDA, the boosting models have a lower error rate and is in line with the LDA and logistic regression models thus far. One key advantage of these boosted models is the 10-fold cross validation that took place during their development.

	var	rel.inf
chld	chld	42.436880945
hinc	hinc	16.801286942
reg2	reg2	10.543684371
home	home	9.638140442
wrat	wrat	8.503021354
tdon	tdon	2.737889724
log_incm	log_incm	2.537273153
tlag	tlag	2.264902929
log_tgif	log_tgif	2.107227485

Figure 5: Example of relative influence statistics

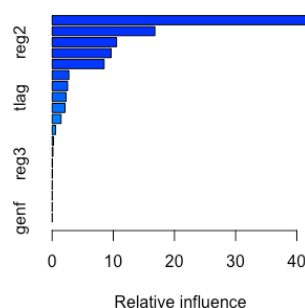


Figure 6: Example relative influence plot

Classification Model – Random Forest

Like the other strategies, six different random forest models were constructed. Unlike the other previous models constructed, the raw data using Dataset 1 (no transformations and no standardization) resulted in a projected profit of \$11783.00 with an error rate 14.1%. Using Dataset 2, the log-transformed variables resulted in a projected profit of \$11793.50 with an error rate of 15.1%. Compared to the boosting models, the random forest models have a similar error rate, but reduced maximum profit. Figure 7 illustrates an example plot that shows the importance of each variable. This plot was closely analyzed to help provide deeper insight into the impact of each predictor.

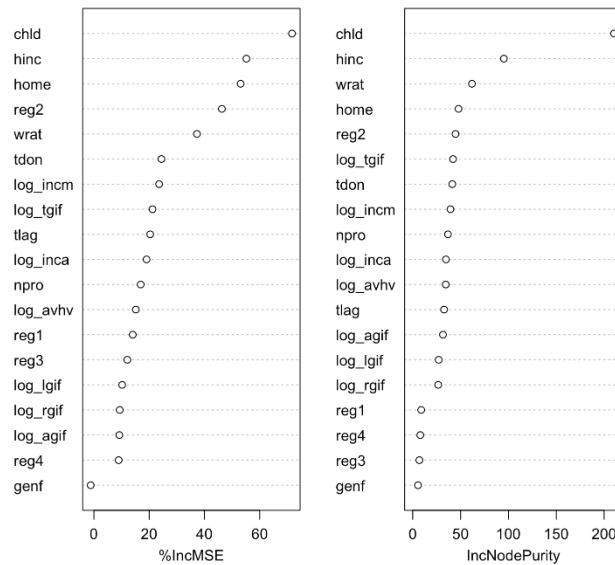


Figure 7: Variable importance plot example

Results of the DONR Classification Models

Recall from the DATA PREPARATION section that there are two different datasets. The key metric that is to be used, per the business requirements of this exercise, is maximum profit. From a pure metric standpoint, the boosting model for Dataset 1 (identifying certain variables as factors⁵) that uses log transformations is the preferred model. However, Dataset 2 considers many of the variables as numeric values (instead of factors) and this results in the model utilizing standardized⁶ variables as the preferred model. Only \$4 separates these two models, suggesting that both models and approaches may be plausible. Comparing the preferred model using Dataset 1 to the preferred model in Dataset 2, it's clear to see that the Dataset 1 model has a higher maximum profit with a slightly lower number of correct observations (1244 vs. 1275).

The chosen model is highlighted in a green cell in Table 6. Adjusting for the oversampling, this model classifies (using the test dataset) 1703 as non-donors and 304 as donors. Therefore, it can be expected to mail 304 donors with the highest posterior probabilities.

Table 6 summarizes the results of all of the models used in the classification step (response variable DONR) for each dataset.

⁵ Recall that CHLD, HINC, and WRAT are considered factors

⁶ Standardized variables have a mean of 0 and a standard deviation of 1

Table 6: Classification model results for response variable DONR

Model	Transformation	Dataset 1 (with factors)			Dataset 2 (no factors)		
		Max Profit	Error Rate	Correct Obs.	Max Profit	Error Rate	Correct Obs.
logistic	raw	11823.5	0.144	1273	11416	0.216	1397
logistic	standardized	11823.5	0.144	1273	11416	0.216	1397
logistic	log-transform	11848.5	0.143	1275	11362.5	0.222	1402
LDA	raw	11826.5	0.127	1228	11367.5	0.246	1472
LDA	standardized	11826.5	0.127	1228	11367.5	0.246	1472
LDA	log-transform	11843	0.136	1256	11368	0.218	1392
QDA	raw	10996.5	0.291	1527	11238	0.229	1399
QDA	standardized	10996.5	0.291	1527	11238	0.229	1399
QDA	log-transform	11041.5	0.285	1519	11270.5	0.213	1361
Boosting	raw	11850.5	0.132	1245	11846.5	0.143	1276
Boosting	standardized	11850.5	0.132	1245	11848.5	0.143	1275
Boosting	log-transform	11852.5	0.131	1244	11834	0.144	1275
Random Forest	raw	11783	0.141	1257	11732	0.138	1239
Random Forest	standardized	11777.5	0.145	1267	11718.5	0.144	1253
Random Forest	log-transform	11771.5	0.157	1299	11793.5	0.151	1288

Analysis of the DAMT Predictive Models

For the predictive models, seven different types of model were constructed using the two different datasets (identified in Table 5): linear regression, ridge regression, lasso regression, boosting, random forest, principal component regression, and partial least square regression. Table 7 summarizes all the results for each of these models. In this section, succinct highlights for each type of model is provided. In the **Results of the DAMT Predictive Models** section, justification is provided for the model chosen for each response variable. Recall that the key metric in choosing the appropriate model is the average squared prediction error using the validation dataset.

Predictive Model – Linear Regression

Six different linear regression models were constructed and each one underwent the stepwise automatic variable selection. BIC was the key metric for evaluation of variables entering or exiting the linear regression models. The intent of these regression models was to construct a baseline in which to compare the other models. As such, many of the required conditions (such as lack of multicollinearity) were not met. Intuitively, it was found that the linear regression utilizing the log transformations in both datasets yielded the best linear regression models (see Table 7).

Predictive Model – Ridge Regression

For this type of model, the information in the dataframe was converted into matrices in order to make use of the package `glmnet`. Furthermore, cross-validation was used to determine the

best lambda value (i.e., the tuning parameter) across a large range of values (from intercept only to least-squares). For both datasets, it was found that the log-transformed data performed the best. However, neither of the best regression models were better than the linear regression models using the same transformed variables. In this type of model, coefficient values are not necessarily 0, but are very close. Hence, all of the predictors were used – regardless of their viability. Figure 8 illustrates a cross-validation plot that helps visualize the optimal tuning parameter, lambda. This value is calculated algorithmically instead of visual estimation. Figure 9 shows an example output of the coefficients. Note how none of the predictor variables are at absolute 0.

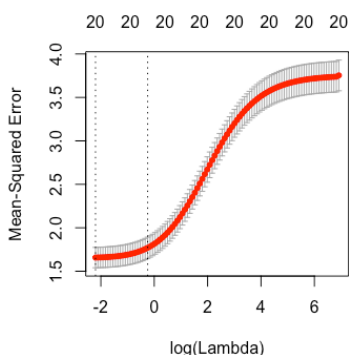


Figure 8: Cross validation plot for lambda

```
> coef(ridge.mod1)[,10]
(Intercept)  reg1      reg2      reg3      reg4      home      chld
1.449824e+01 -1.222527e-10 -5.202331e-10  2.180764e-10  4.244262e-10  4.176523e-11 -1.472004e-09
      hinc      genf      wrat      avhv      incm      inca      plow
8.862990e-10 -3.458405e-11 -1.696087e-10  2.252569e-09  7.875441e-09  4.404000e-09  1.628607e-09
      npro      tgif      lgif      rgif      tdon      tlag      agif
7.909043e-09  3.799403e-08  6.193082e-08  2.992696e-08  1.337884e-09  7.114601e-10  1.572143e-08
```

Figure 9: Example ridge regression coefficient values

Predictive Model – Lasso Regression

Similar to Ridge Regression, information in the dataframe was converted into matrices in order to make use of the package `glmnet`. Furthermore, cross-validation was used to determine the best lambda value (i.e., the tuning parameter) across a large range of values (from intercept only to least-squares). For both datasets, it was found that the log-transformed data performed the best. However, neither of the best regression models were better than the linear regression models using the same transformed variables. The key difference here was that the lasso regression resulted in coefficients being 0 for some of the predictors. A similar cross-validation plot (see Figure 8) to understand the impact of different lambda values was also constructed for each lasso model. Figure 10 illustrates an example coefficient list for a particular lambda value. In contrast to ridge regression, note how some of the lasso regression model (example) coefficients actually have a value of 0. In a way, lasso models are inherently conducting variable selection.

```
(Intercept)  reg1      reg2      reg3      reg4      home      chld
10.1760169827 -0.0633095577 -0.1552841499  0.8763246648  1.7477981780  0.3219807239 -0.4213895130
      hinc      genf      wrat      avhv      incm      inca      plow
0.3485591944 -0.0848137262  0.0000000000 -0.0005732300  0.0117829462  0.0016532741  0.0192131346
      npro      tgif      lgif      rgif      tdon      tlag      agif
0.0043867153  0.0006996292 -0.0016460689  0.0428937901  0.0128428863  0.0060155098  0.1040580173
```

Figure 10: Example coefficient values for lasso model

Predictive Model – Boosting

A similar approach to the prior boosting models (for classification) was used. The lowest average square prediction error using Dataset 1 was 1.27 with either the raw (untransformed) data or the standardized data. Using Dataset 2, however, the lowest was with the standardized data. Nevertheless, this was quite close to the untransformed data as well. This boosting model, for Dataset 2, was the best performing model (in terms of average prediction error) of any type of model. One key advantage of these boosted models is the 10-fold cross validation that took place during their development. As done for the classification model, plots were also constructed to help understand the relative influences. For brevity, the plots for this specific model are not attached in this report.

Predictive Model – Random Forest

A similar approach to the prior boosting models (for classification) was used. The lowest average square prediction error using Dataset 1 was 1.65 with the standardized data. Using Dataset 2, however, the lowest was with the untransformed data. As done for the classification model, the variable importance plot was also constructed. For brevity, this plot is not attached in this report.

Predictive Model – Principal Component Regression

Six different models were constructed, each with cross validation. Furthermore, a validation plot was constructed to help determine the number of principal components to use for the predictive model (when testing against the validation dataset). This approach helped to ensure that the number of principal components used would result in a low MSE (mean square error predicted). Each of the models ended up using a different number of principal components. The log-transformed variables led to the lowest average square prediction error for both datasets (see Table 7). Figure 11 is an example of a validation plot that visually shows the number of components and the MSE. In this example, the greater the number of components, the better. However, such visual analysis is not ideal as it may be misleading. Therefore, a summary output was also done (see Figure 12). Note how 36 components may be sufficient versus 37. In this case, it was decided to go with 37 components.

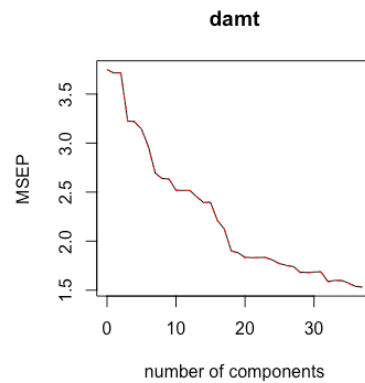


Figure 11: Validation plot for MSEP vs. principal component count

VALIDATION: RMSEP
Cross-validated using 10 random segments.

	(Intercept)	1 comps	2 comps	3 comps	4 comps	5 comps	6 comps	7 comps	8 comps	9 comps
CV	1.937	1.927	1.928	1.796	1.794	1.773	1.722	1.641	1.625	1.622
adjCV	1.937	1.927	1.928	1.795	1.793	1.772	1.719	1.640	1.624	1.622
	10 comps	11 comps	12 comps	13 comps	14 comps	15 comps	16 comps	17 comps	18 comps	19 comps
CV	1.587	1.587	1.587	1.566	1.548	1.548	1.487	1.457	1.380	1.372
adjCV	1.587	1.586	1.586	1.565	1.547	1.547	1.486	1.457	1.378	1.370
	20 comps	21 comps	22 comps	23 comps	24 comps	25 comps	26 comps	27 comps	28 comps	29 comps
CV	1.355	1.353	1.354	1.354	1.344	1.331	1.324	1.320	1.297	1.296
adjCV	1.353	1.352	1.353	1.353	1.343	1.330	1.323	1.319	1.295	1.295
	30 comps	31 comps	32 comps	33 comps	34 comps	35 comps	36 comps	37 comps		
CV	1.298	1.299	1.261	1.265	1.265	1.254	1.241	1.238		
adjCV	1.297	1.299	1.259	1.263	1.263	1.253	1.239	1.237		

Figure 12: Numerical output of cross-validated RMSEP vs. number of components

Predictive Model – Partial Least Squares Regression

Six different models were constructed, each with cross validation. Furthermore, a validation plot was constructed to help determine the number of components to use for the predictive model (when testing against the validation dataset). This approach helped to ensure that the number of components used would result in a low MSEP (mean square error predicted). Each of the models ended up using a different number of components. The log-transformed variables led to the lowest average square prediction error for both datasets (see Table 7). Similar visual plots (see Figure 11 and Figure 12) were constructed for these models as well.

Results of the DAMT Predictive Models

Recall that the key metric to decide the most appropriate model is Mean Square Error Prediction (MSEP). Using this metric, the best model was the linear regression using log transformations on Dataset 1 (see cell highlighted in green on TABLE XX). It is surprising to see that such a simple model outperformed the corresponding boosting models for the same dataset. One of the key advantages of this model is its simpler interpretability compared to the boosting models.

Table 7 summarizes the results of all of the models used in the prediction step (response variable DAMT) for each dataset.

Table 7: Predictive model results for response variable DAMT

Model	Transformation	Dataset with factors	Dataset with numeric
		Prediction Error	Prediction Error
Linear	raw	1.643	1.858
Linear	standardized	1.643	1.858
Linear	log-transform	1.396	1.629
Ridge Regression	raw	1.958	1.958
Ridge Regression	standardized	1.961	1.961
Ridge Regression	log-transform	1.734	1.734
Lasso	raw	1.877	1.877
Lasso	standardized	1.871	1.871
Lasso	log-transform	1.633	1.633185
Boosting	raw	1.527	1.539435
Boosting	standardized	1.527	1.539267
Boosting	log-transform	1.555	1.567842
Random Forest	raw	1.653	1.65574
Random Forest	standardized	1.65	1.668346
Random Forest	log-transform	1.673	1.681108
PCR	raw	1.653	1.866657
PCR	standardized	1.653	1.866657
PCR	log-transform	1.435	1.62172
PLS	raw	1.682	1.866785
PLS	standardized	1.682	1.864312
PLS	log-transform	1.414	1.622855

Conclusion and Next Steps

The boosted model for classifying donors (DONR) outperformed the other models, including the simpler and easier to interpret logistic models. On the other hand, the predictive model for determining the potential donation amount (DAMT) is a simple linear regression model that is simple to understand and outperforms the more complex boosted models. One key takeaway from this exercise is that complex models do not necessarily outperform the simpler models. The models demonstrated here are by no means an exhaustive list. Note how two different datasets (one using factors and another not) resulted in slightly different results. Next steps include (but certainly not limited to) further analysis into interaction terms, outlier management, and other types of variable transformations. This would also provide a more insightful opportunity to recraft the models here for further improvements.

Introduction

In this assignment, a general attitudinal post-hoc segmentation analysis was completed using a sample survey dataset consisting of 1,800 observations. The intent of this analysis is to help App Happy develop a deeper understanding of the market for a new social entertainment app. Since App Happy is not yet in the consumer app business, the hope is to provide a comprehensive understanding of distinctive groups that the organization can leverage using segment specific marketing strategies.

Attitudinal questions around the respondents' level of agreement on technology, personal characteristics, and purchasing behavior were formed as the basis variables. The responses were foundational to the construction of three different clustering models (i.e., schemas): K-Means, PAM (partition around medoids), and Hierarchical. Once the appropriate method was identified, it was used to profile the derived cluster groups to provide greater insight into each segment using demographics or personality characteristics. Furthermore, a brief discussion has been provided to highlight the potential techniques that could be used as a "typing tool" to classify customers for whom no data exists.

Data Overview

The sample survey dataset (provided by the Consumer Spy Corporation) contains 1,800 observations across 89 variables. However, upon exploration of the data dictionary, it appears that there are 16 distinct questions - some of which have 'sub-questions'. The information was provided through an R data file and consisted of two dataframes: a numerically coded response data frame and a data frame with the character values of the survey responses. From the data dictionary provided, it appears that there may have been 57 questions on the survey (compared to the 16 made available). Of the 16 primary questions, three of them are attitudinal and the responses to these three questions make up approximately 41 of the 89 variables. The other questions within the survey dataset provide both demographic and personality characteristics.

A comprehensive exploratory data analysis was completed and a small sample of the histograms generated are provided in the Appendix. An interesting note to make is the skewness of the respondents' age. The majority of observations seem to be for individuals that are 25 and younger. Furthermore, it also appears that there are more female respondents than males.

Market Segmentation

Attitudinal Variables

Questions 24, 25, and 26 are prime candidates for attitudinal segmentation. All three questions use a Likert scale (a general 6-point scale) where 1 means "Strongly Agree" and 6 means "Strongly Disagree". Question 24 appears to focus on technology; question 25 appears to focus on an individual's personal characteristic & point of view; and question 26 appears to focus on an individual's purchasing preferences. These three questions seem best suited to ascertain an individual's perspective or attitude.

Due to the large number of potential attitudinal variables, a principal component analysis (PCA) was initially done to understand the amount of variation explained. It was discovered that in order to explain at least 70% of the variation, 14 principal components would be required (see Table 9.1 in the Appendix). The attitudinal variables were then regrouped to create a more concise grouping in an effort to reduce the number of principal components. Table 9.2 in the Appendix summarizes how the responses were grouped (i.e., consolidated). In other words, the consolidated values were essentially the average of the response values for each group. Another PCA was completed using this new grouping structure and it was discovered

that three principal components were needed to explain at least 70% of the variation (see Table 9.3 in the Appendix). Thus, this new grouping dataset is used as foundation for the different clustering methods.

Clustering Methodology

This analysis focuses on three different clustering methods: K-Means, PAM, and Hierarchical. Two of the methods (K-Means and PAM) are non-hierarchical methods that determine a centroid or mediod, respectively, and effectively try to minimize the Euclidean distance between the centroid/mediod and the observation. The hierarchical method attempts to take each observation as its own cluster and then combine it with neighboring observations or clusters to create a tree-like structure. Eventually, the algorithm ends when all of the nodes are linked (Chapman & Feit, 2015).

K-Means Clustering

K-Means typically requires a pre-defined number of clusters. Two methods were employed to help determine the optimal number of clusters: a scree plot and the average silhouette width. A scree plot essentially summarizes the minimum within-cluster sum of square for each number of clusters. Identifying where the 'bend' takes place on the plot would imply an optimum number of clusters.

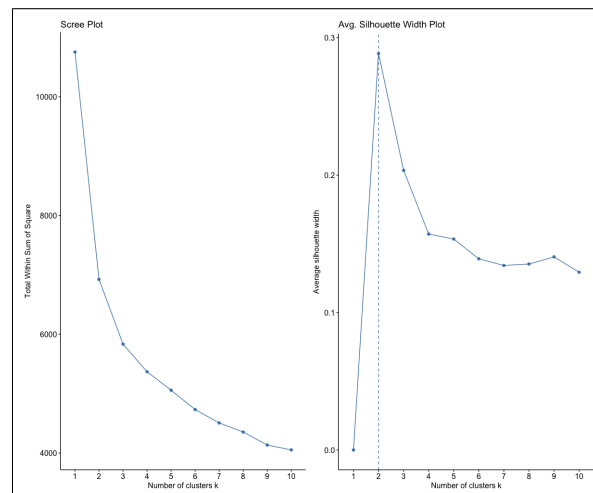


Figure 3.1: Scree Plot & Average Silhouette Width Plot

Figure 3.1 illustrates both the scree plot and the average silhouette widths for multiple clusters. From the scree plot, it appears that the 'bend' occurs at either two or three number of clusters. However, from the average silhouette width plot, it is clear that the optimal number of clusters is two. Note that it is not sufficient to say that two clusters is sufficient without comprehensively profiling the clusters.

Assuming that two clusters are chosen, the following plot (Figure 3.2) illustrates the first two principal components (which explain almost 70% of the variation - see Table 9.3 in the 9). Note how the two clusters are not very well separated. In fact, there are several overlapping points within this plot. This suggests that some of the responses to the survey overlap in terms of attitudinal responses.



Figure 3.2: Two Cluster PCA Plot

PAM Clustering

PAM (Partitioning Around Medoids) clustering is similar to K-Means, however, the two key differences are that the medoids are used instead of centroids and the dissimilarity-based distance is used instead of the squared-Euclidean distance (Izenman, 2013). This enables the algorithm to be a bit more resistant to the influence of outliers. Similar to Figure 3.1, Figure 3.3 illustrates the scree plot and the average silhouette width for this method.

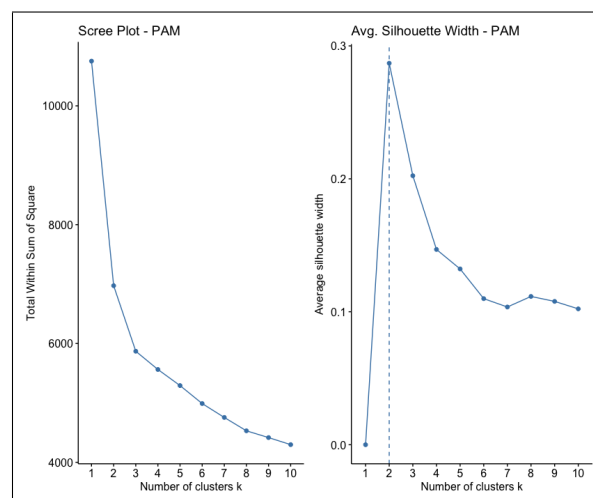


Figure 3.3: Scree Plot & Average Silhouette Width (PAM Clustering)

Note how there is a distinctive bend (when looking at the scree plot) at three clusters. However, the average silhouette width plot continues to show that two clusters is optimal. Figure 3.4 shows the two clusters superimposed on an XY plot of the first two principal components. This figure is very similar to Figure 3.2.

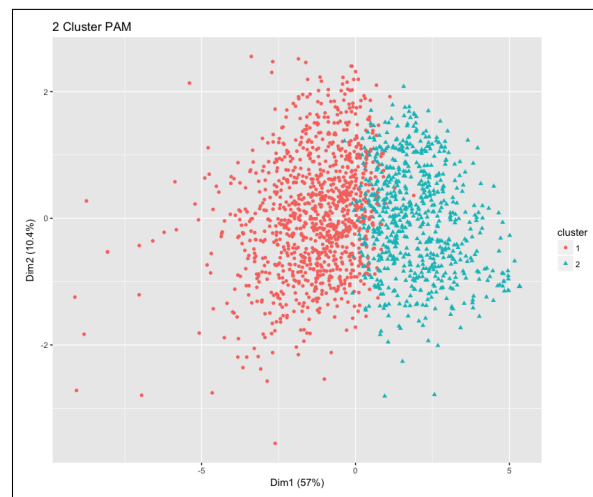


Figure 3.4: PAM Two Cluster PCA Plot

Hierarchical Clustering

Similar to the procedures for the K-Means Clustering and PAM Clustering, the scree and the average silhouette width plots are constructed.

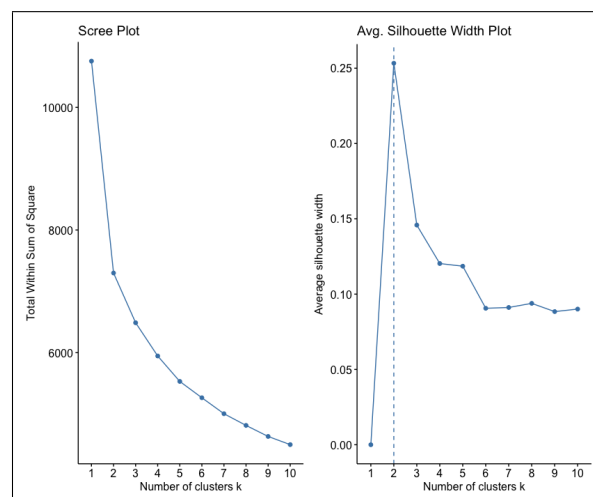


Figure 3.5: Scree Plot & Average Silhouette Width (Hierarchical Clustering)

It is not especially clear where the 'bend' is in the scree plot (Figure 3.5), however, it appears that two clusters would be optimal. This assumption is confirmed in the average silhouette width plot (Figure 3.5). For the hierarchical method, we leveraged the agglomerative technique. Essentially, this technique assumes that each observation is its own cluster and then links them together to form the upper parts of a tree diagram - known as a dendrogram.

The dissimilarity matrix was used with the Euclidean distance method, however, the linkages were investigated to ensure optimum dissimilarity between two clusters of observations (Analytics, n.d.). This approach led to the investigation of multiple linkage methods (e.g., complete, single, average, and Ward). Therefore, the agglomerative coefficient, which is used to determine which measuring method, can determine

the strongest clustering structure. Table 3.1 summarizes the results of this analysis. Note the closer the coefficients are to 1 the stronger the clustering structure.

Table 3.1: Agglomerative Coefficient Analysis

Average	Single	Complete	Ward
0.874	0.714	0.929	0.989

Note how the Ward method was very close to 1. The Ward method attempts to minimize the total within cluster variance (Analytics, n.d.). Hence, the Ward linkage method is used for the dendrogram creation. Figure 3.6 is the dendrogram using the agglomerative approach with the Ward linkage.

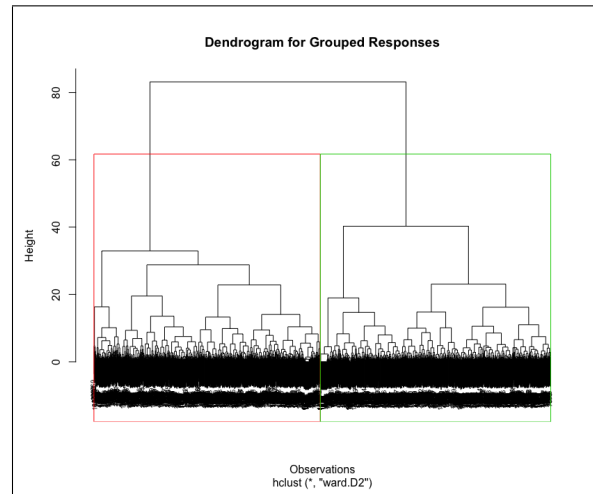


Figure 3.6: Dendrogram

Recall that from the scree and average silhouette width plots (Figure 3.5), two clusters were determined to be optimal. In Figure 3.6, the two clusters are 'encircled' with two rectangles. Figure 3.7 is the XY plot of the first two principal components with two clusters highlighting the observations. Note the similarity between this method and the previous methods.

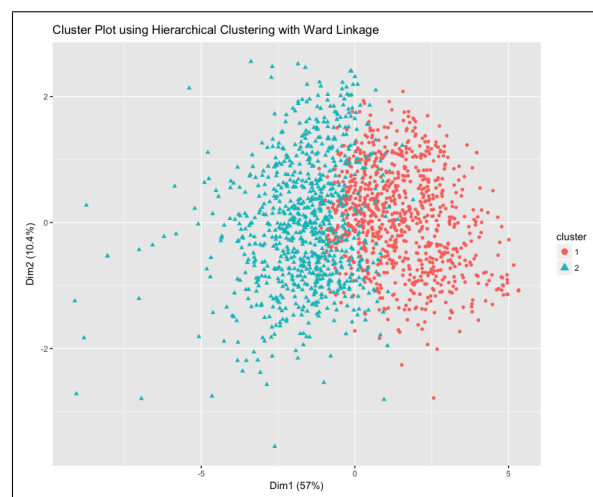


Figure 3.7: Hierarchical Two Cluster PCA Plot

Clustering Method Review & Conclusions

From the three different clustering methods analyzed for this analysis, it is not readily apparent which method may be of most value. Furthermore, the different scree plots and average silhouette width plots for all three methods suggested either two or three number of clusters. Reflecting on the different XY plots constructed, it is clear that there are many points that are overlapping. Ultimately, the profiling of each of the clusters will help determine the contextual value. Therefore, we have selected to proceed forward with two clusters using the K-Means method.

Profiling with K-Means

In order to provide context around the chosen number of clusters, both visual histograms and tabular data of the histograms were leveraged. The contextual profiles have been separated into two features: demographic and preferences. Many of the figures that are referenced in the next parts can be found in the Appendix along with tables that describe the observations proportionally.

Demographics

In terms of age, it appeared that cluster two was made up older consumers (41% of the observations were 40 and older) whereas cluster one was more geared towards a younger (48% were younger than 35) audience (see Figure 9.4 and Table 9.4 in the Appendix). In terms of education, cluster two seemed to consist more of observations (12% for cluster 2 versus 9% for cluster 1) with a post-graduate degree. Cluster two also seems to consist of more separated, widowed, and/or divorced respondents (11% vs 6% - see Table 9.6). However, both clusters have approximately equal number of married and single consumers. Interestingly, cluster two consumers seem to have older children (ages 18+) whereas cluster one seems to be mostly no children or younger children (younger than age 6). Both clusters are predominantly White/Caucasian and do not identify as Latino/Hispanic (Figure 9.8). Both clusters are also predominately female (Figure 9.9). Finally, both clusters seem to have a large portion of observations between \$30,000 and \$80,000 (Figure 9.10). However, cluster two also has a larger number of observations at the higher end of the income level (although proportionally, they are equal).

From a demographics point of view, cluster two seems to be made up of consumers that are slightly older, more likely to possess a post-graduate degree, more likelihood of being separated/divorced/widowed, and have a higher income. On the other hand, cluster one seems to consist of a younger audience that is still educated (college graduate), have younger or no children, single or married, and a higher likelihood of having a more modest income.

Preferences

In both clusters, the prevalent device owned was the iPhone (see Figure 9.11) followed by Android (which is an OS¹, and not a device). In addition, both clusters had substantial tablet ownership, however, it is difficult to determine what kind of tablet users (e.g., brand and OS type). Both clusters were pretty even (see Figure 9.12) when it came to general news apps, gaming, and shopping apps. Cluster 2 had a higher proportion preferring music identification apps (see Table 9.13). However, cluster one seemed to have a higher preference for TV check-in, TV show apps, and entertainment like apps. Cluster two, however, was more inclined to prefer social networking apps.

Both clusters seem to have an even concentration of consumers with 11 to 30 plus apps on their device. However, cluster two seems to also have a larger number of consumers with fewer than ten apps (see Figure

¹Operating System

9.13). Cluster one also had a larger preference for fewer free apps as compared to cluster 2 observations (see Figure 9.14). For many of the specific websites identified in the survey, it seems that consumers in cluster 2 do not visit them (see Figure 9.15). Antithetically, consumers in cluster one appear to visit some of the pre-defined websites more frequently.

From a consumer preferences point of view, cluster one appears to have preference for TV related apps and tend to have numerous apps installed with a strong inclination to have paid apps. Consumers in cluster two seem to prefer more social networking and music identification apps. They also tend to have more free apps and do not visit certain websites at all.

Product Opportunity Recommendations

App Happy is interested in developing a social entertainment app, however, that definition could be broad. One cluster of consumers prefers entertainment like apps while the other cluster of consumers prefers more social media apps. If App Happy were to develop an app that catered to both tastes, then there is a good opportunity for App Happy to realize incredible results. An example app that App Happy may wish to investigate further would be a social media app that revolves around a popular TV show. This would enable App Happy to enter a niche market with ample opportunity to create delightful solutions with potentially strong adoption.

Another avenue that App Happy will have to explore is the platform of their social entertainment app. From the results, it is clear to see that both clusters have a strong preference for either an iOS based device (e.g., iPhone, iPod Touch) or Android. Focusing on these two platforms will encourage disciplined expenditures and increased adoption. However, it is not clear what kind of tablet OS is preferred for either cluster. From a demographics point of view, App Happy may want to consider using a website to drive traffic to the app as well since younger consumers may prefer a website over an app while older consumers may prefer an app to a website.

App Happy may want to consider conducting another survey that is more focused on social media and the type of devices used. In modern times, the distinction between social media networks and entertainment apps is virtually non-existent. For instance, Facebook hosts videos, live feeds, and more. However, Facebook is first and foremost a social networking site, but it is also considered an entertainment portal. A similar argument could be made for YouTube. Another avenue of research would be to determine the functionality of a website and an app. For instance, a website could be used to provide information and act as a landing page. However, the main focus could be on the app - especially since mobile device usage is on the rise. Finally, App Happy should consider more finely defining what the term social entertainment means. In some regards, LinkedIn is not necessarily a social entertainment platform. It is more a professional networking site and is not recommended to be used as a personal platform as compared to Facebook.

Limitations & Assumptions

Recall that the chosen clustering method for this analysis was K-Means. One key assumption that was made in order to leverage this approach was that the response values were continuous for the attitudinal questions. Since K-Means is working with distances between observations and centroids, categorical values are incompatible.

One limitation of the analysis is its focus on a subset of the information on the overall survey. It appears that the survey consisted of 57 questions, however, only a small portion of them were available for analysis. There is potential that other missing questions may have provided deeper insights when conducting the profiling. Another limitation found was around some of the responses in the survey. For instance, in question 2, the respondents could pick from an iPhone, iPod Touch, Android, etc. However, Android is not a type of device. Rather, it's an operating system such as iOS (which powers both iPhones and iPod Touches). This could have resulted in confused responses since some respondents may not have known that (for example) a Samsung smartphone may have been running Android. Thus, they may have marked Android or other

since Samsung wasn't listed as an option. Furthermore, one of the choices for this question was "Tablet" which lumped both iPads (powered by iOS) and multiple Android powered tablets into one. A suggestion here would be to reword the question to separate tablet and mobile users and then further separate them by type of OS. Similarly, question 50 (regarding children of certain age groups) could be expanded to let the respondent indicate how many children in each age group they have. This could also prove to be beneficial in terms of demographic profiling. In other words, certain questions should either be rewritten or expanded upon to help create a more comprehensive insight.

Another limitation was the decision to not remove outliers in responses or potentially false responses. For instance, a respondent may have answered (on the Likert scale) all 1's or all 6's (specifically on questions 24 - 26). These may be accurate responses, but the way the questions are structured and written, some of the responses are contrary and it would be unlikely that the agreement level would be the same. Removing these values may prove to improve the overall clustering schema.

It is also not very clear from the survey the distinction between visiting a social media website versus using the social media website's app. For instance, it is not very clear to distinguish users who use Facebook's app more than visiting the website since much of that will depend on the usage of a mobile device and a personal computer.

Classification Models Discussion

Another requirement from App Happy is the ability to classify consumers that it does not currently have information on. This can be accomplished by leveraging the aforementioned clustering technique. For instance, the samples used all have a cluster number attached to them and now a predictive model could be built on top of that to facilitate this task. Two potential predictive models that could be leveraged are Random Forest and Naive Bayes. For both approaches, the response (or the dependent variable) would be the cluster number and the predictors would be any of the demographic, attitudinal, or consumer preferences that a consumer may supply or be available.

Random Forest

Random Forest is essentially an advanced decision tree method that enables complex relationships to be modeled quite well. What separates Random Forest from other tree-based models is that it can sample both the observations and the predictors (Bruce & Bruce, 2017). Furthermore, this method constructs hundreds of trees and the ability to only sample a subset of the observations and predictors enables the trees to be decorrelated (James, Witten, Hastie, & Tibshirani, 2015). One key benefit of the Random Forest approach is its ability to reduce overfitting. A major disadvantage of this approach is the inability to visually illustrate the different decision trees that are constructed.

Naive Bayes

The Naive Bayes approach is another interesting approach that essentially assumes that the pair of predictors are independent. This enables the method to take into account both discrete and continuous features (Hastie, Tibshirani, & Friedman, 2009). From a classification standpoint, Naive Bayes could perform as well as decision trees and be cheaper to run. Another way of thinking about this approach is that it tries to determine the most probable predictor for each response variable and then uses that probability to estimate the probability of the response variable within the new dataset (Bruce & Bruce, 2017). One disadvantage of this method is that if a new predictor is introduced, the method will assume zero probability and the results could be misleading.

Classification Model Recommendation

Based on the information provided for this analysis, it is recommended that the Random Forest method be used as a starting point. There are several benefits to using a tree-based modeling approach. One of the perks is the ability to take into account complex nuanced relationships that are not readily apparent. Furthermore, tree-based models are not as sensitive to outliers, which may be of value especially with limited information regarding potential consumers. However, once a model is constructed, it should be continually evaluated to ensure that its usefulness continues.

Conclusion

Three key clustering methods were investigated for this analysis: K-Means, PAM, and Hierarchical Clustering. Using the scree plot and the average silhouette width assessment, it was deemed that two clusters would be optimal. The K-Means clustering method was chosen for in-depth consumer profiling using consumer demographic information and consumer preferences. It was found that consumers preferring social media were in one cluster and consumers preferring entertainment apps were in another. Both clusters showed strong preference for iOS mobile devices (specifically iPhone and iPod Touch) as well as Android (as an OS). Finally, a brief overview of two different classification models (Random Forest and Naive Bayes) was provided to cultivate insight into how consumers could be classified with incomplete information sets. Ultimately, App Happy may want to conduct another survey that is more updated with better questions that help segment the consumers better in addition to more clearly defining what is "social entertainment" to them.

References

- Analytics, U. B. (n.d.). *Hierarchical cluster analysis*. http://uc-r.github.io/hc_clustering. (Accessed: 2018-01-20)
- Bruce, P., & Bruce, A. (2017). *Practical statistics for data scientists*. Boston: O'Reilly.
- Chapman, C., & Feit, E. M. (2015). *R for marketing research and analytics*. New York: Springer.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning*. New York: Springer.
- Izenman, A. J. (2013). *Modern multivariate statistical techniques regression, classification, and manifold learning*. New York: Springer.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2015). *An introduction to statistical learning with applications in r*. New York: Springer.

Appendix

EDA

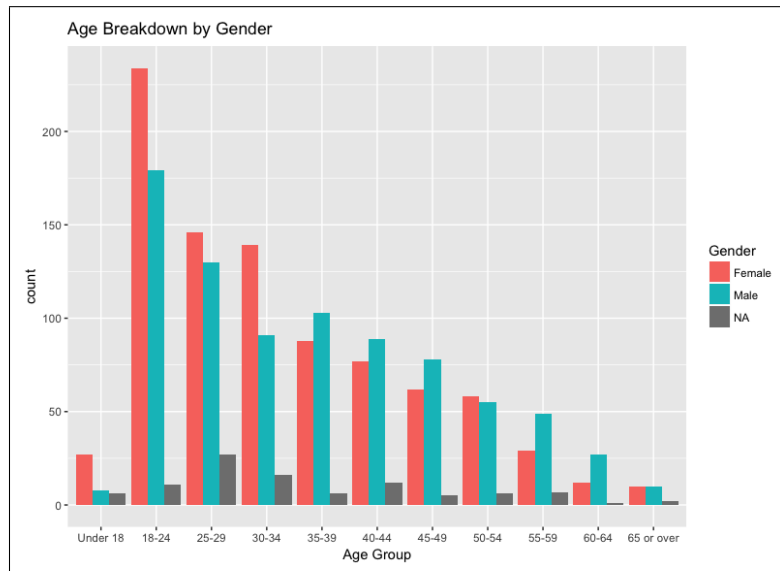


Figure 9.1: Histogram of Age by Gender

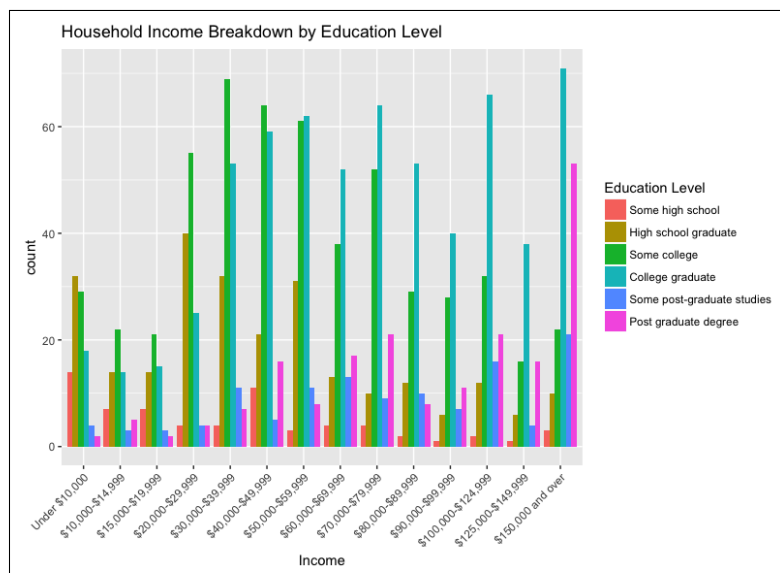


Figure 9.2: Histogram of Income by Education Level

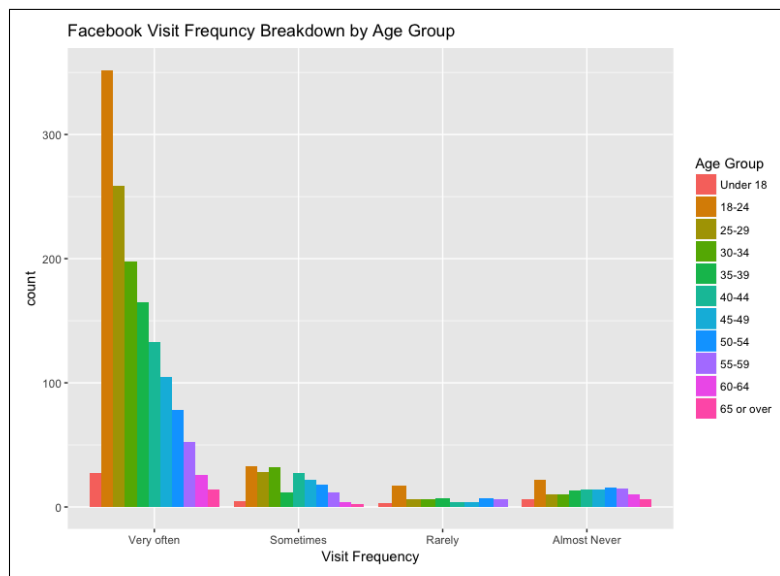


Figure 9.3: Histogram of Visiting Facebook Website by Age

PCA

Table 9.1: Principal Component Analysis Results - Top 15 Principal Components

principal component	std dev	percent	cumulative_pct
1	4.334	0.269	0.269
2	2.543	0.093	0.361
3	1.923	0.053	0.414
4	1.669	0.04	0.454
5	1.603	0.037	0.491
6	1.437	0.03	0.521
7	1.393	0.028	0.548
8	1.373	0.027	0.575
9	1.308	0.024	0.6
10	1.283	0.024	0.623
11	1.263	0.023	0.646
12	1.244	0.022	0.668
13	1.217	0.021	0.69
14	1.137	0.019	0.708
15	1.1	0.017	0.726

Table 9.2: Attitudinal Response Groupings

Grouping	Questions Within Grouping
Trendsetter	q25r2, q26r18, q26r7, q26r8, q26r10
Early Adopter	q24r1, q24r3, q25r5, q25r8, q25r9, q25r10, q26r9
Ease of Life	q24r5, q24r6, q25r12, q26r5
Entertainment	q24r7, q24r8
Late Adopter	q24r4, q24r9, q24r12, q25r11
Leader	q25r1, q25r4, q25r7
Shopper	q26r3, q26r4, q26r6, q26r12, q26r13
Social	q24r2, q24r10, q24r11, q25r3, q25r6, q25r11

Table 9.3: Grouped Responses Principal Components Analysis - Top 5 Principal Components

principle component	std_dev	percent	cumulative_pct
1	2.462	0.411	0.411
2	1.619	0.178	0.589
3	1.141	0.088	0.677
4	1.009	0.069	0.746
5	0.859	0.05	0.796

Profiling

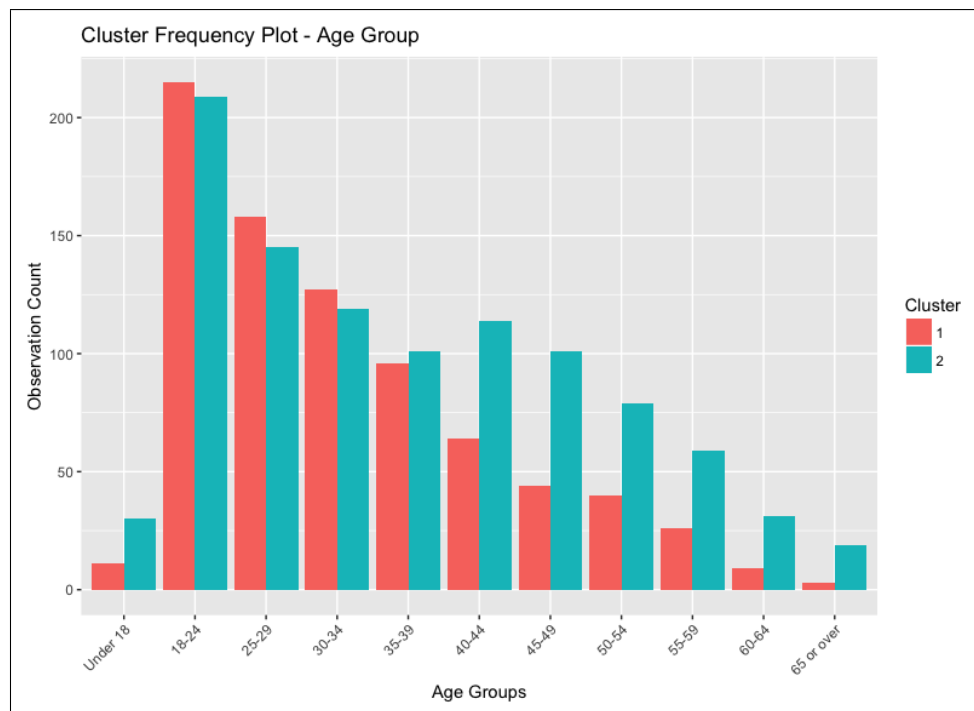


Figure 9.4: Cluster Breakout by Age Group

Table 9.4: Age Group Proportion Table by Cluster

Age Group	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Under 18	11	1%	30	3%
18-24	215	27%	209	21%
25-29	158	20%	145	14%
30-34	127	16%	119	12%
35-39	96	12%	101	10%
40-44	64	8%	114	11%
45-49	44	6%	101	10%
50-54	40	5%	79	8%
55-59	26	3%	59	6%
60-64	9	1%	31	3%
65 or over	3	0%	19	2%

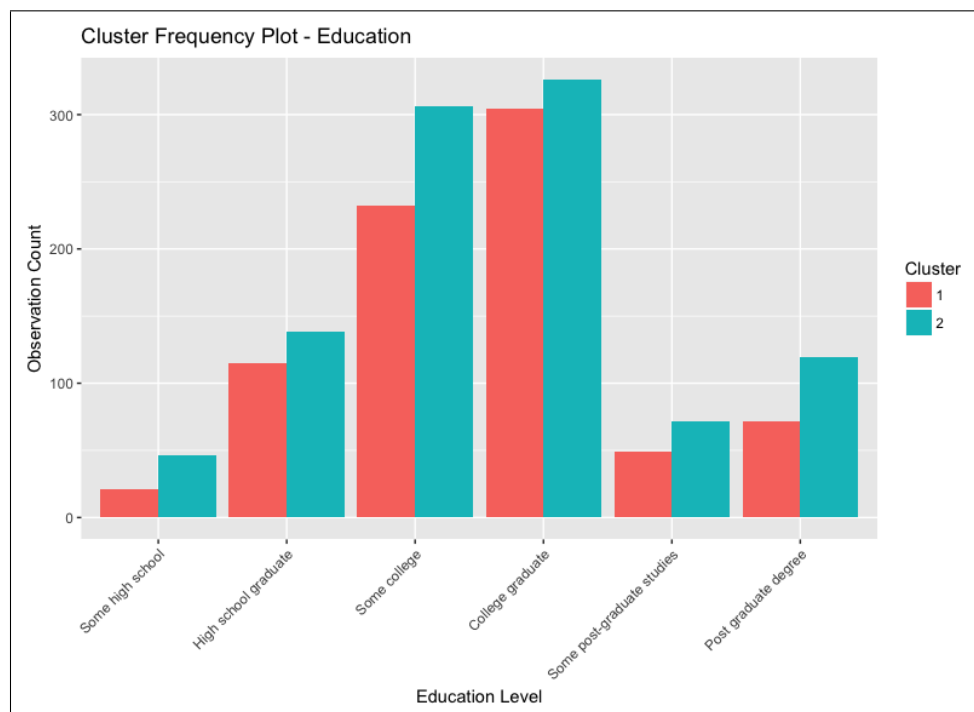


Figure 9.5: Cluster Breakout by Education Level

Table 9.5: Education Level Proportion Table by Cluster

Education Level	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Some high school	21	3%	46	5%
High school graduate	115	15%	138	14%
Some college	232	29%	306	30%
College graduate	304	38%	326	32%
Some post-graduate studies	49	6%	72	7%
Post graduate degree	72	9%	119	12%

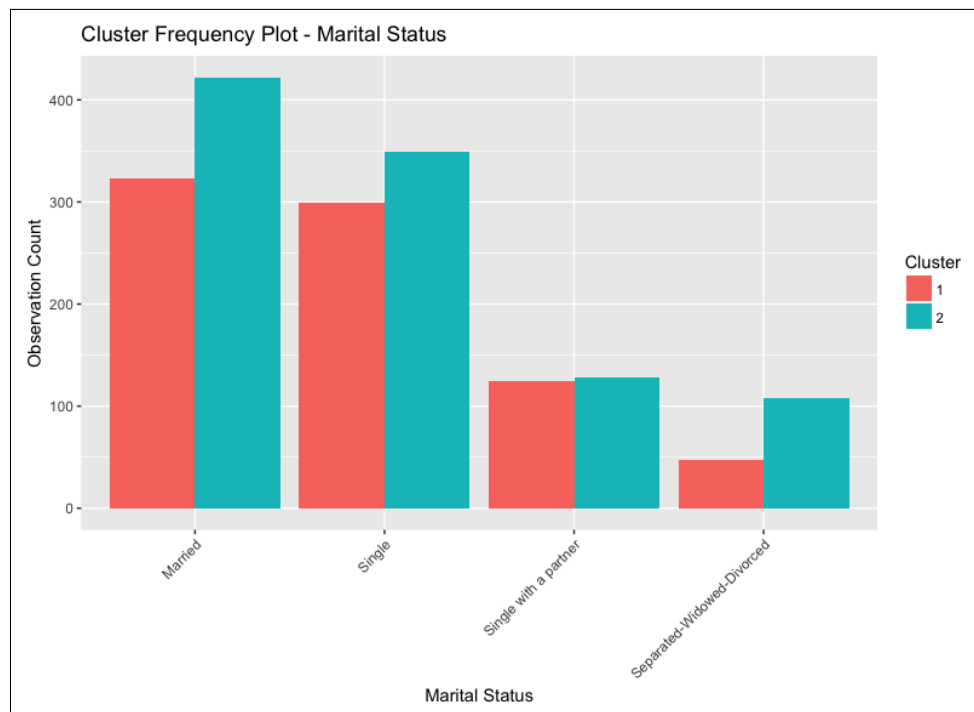


Figure 9.6: Cluster Breakout by Marital Status

Table 9.6: Marital Status Proportion Table by Cluster

Marital Status	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Married	323	41%	422	42%
Single	299	38%	349	35%
Single with a partner	124	16%	128	13%
Separated-Widowed-Divorced	47	6%	108	11%

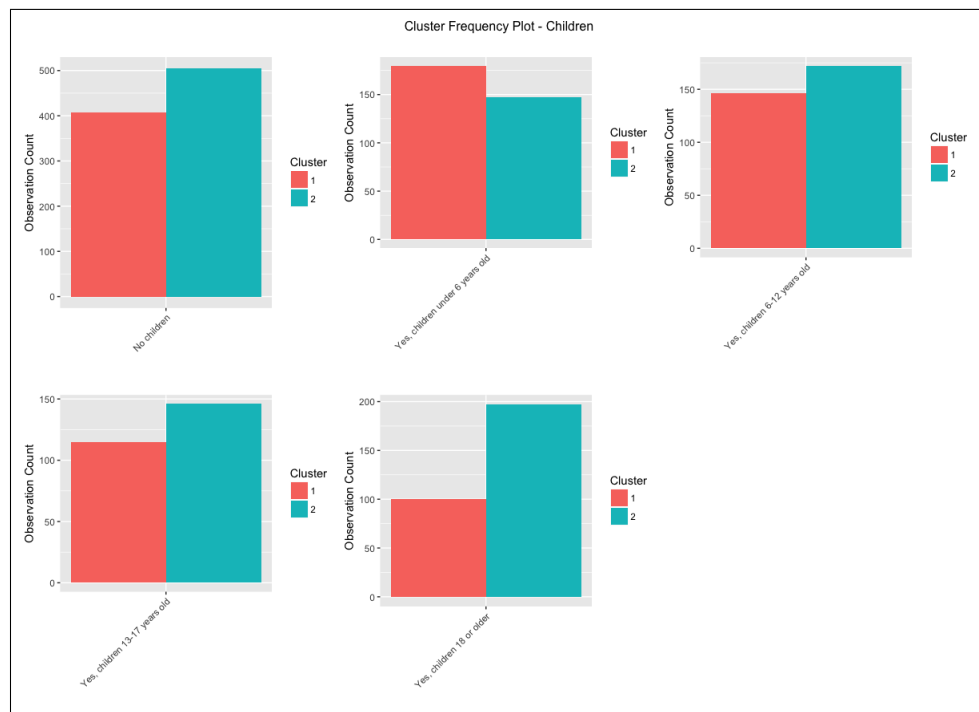


Figure 9.7: Cluster Breakout by Number of Children

Table 9.7: Children Age Group Proportion Table by Cluster

Children	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
No Children	407	43%	505	43%
Yes, children under 6 years old	180	19%	147	13%
Yes, children 6-12 years old	146	15%	172	15%
Yes, children 13-17 years old	115	12%	146	13%
Yes, children 18 or older	100	11%	197	17%

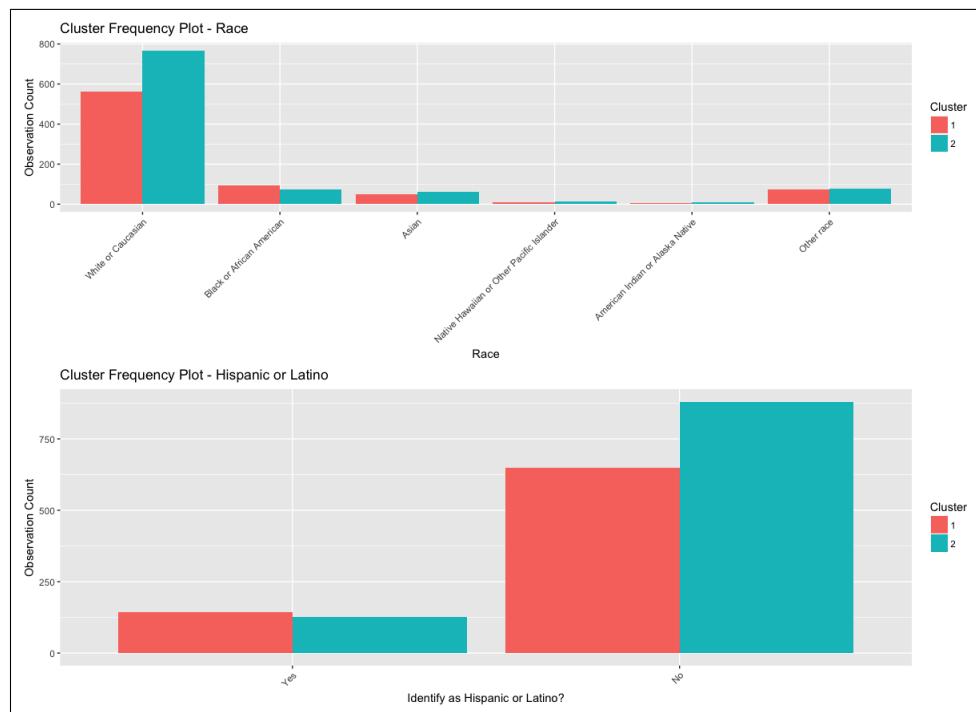


Figure 9.8: Cluster Breakout by Race

Table 9.8: Race Proportion Table by Cluster

Race	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
White or Caucasian	562	71%	768	76%
Black or African American	93	12%	75	7%
Asian	50	6%	61	6%
Native Hawaiian or Other Pacific Islander	10	1%	14	1%
American Indian or Alaska Native	6	1%	10	1%
Other race	72	9%	79	8%

Table 9.9: Latino Identification Proportion Table by Cluster

Identify as Latino	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Yes	144	18%	127	13%
No	649	82%	880	87%

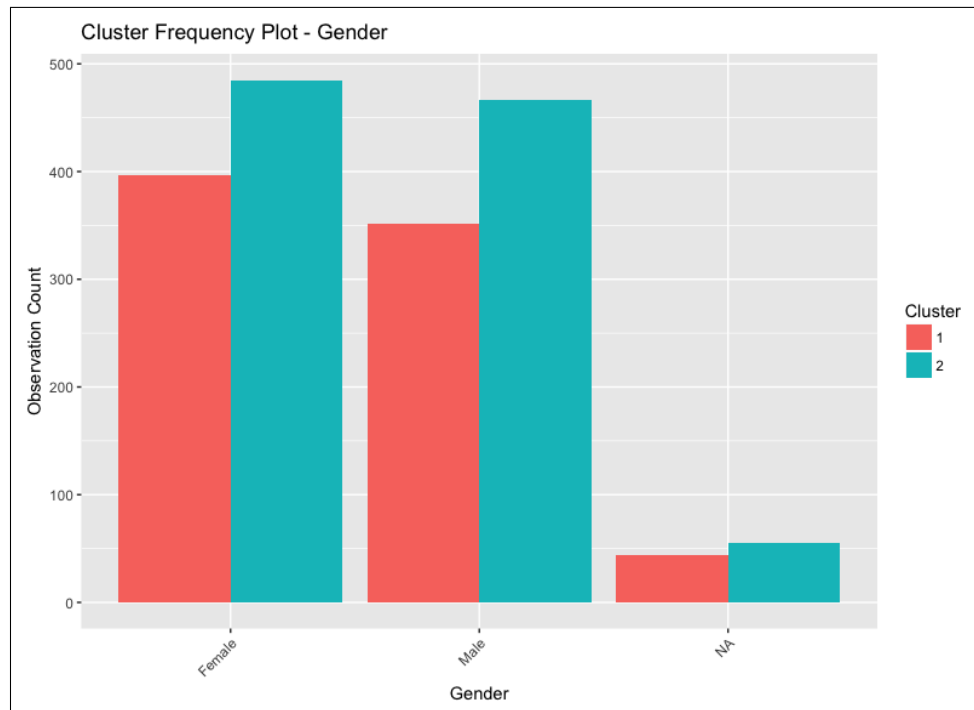


Figure 9.9: Cluster Breakout by Gender

Table 9.10: Gender Proportion Table by Cluster

Gender	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Female	397	50%	485	48%
Male	352	44%	467	46%
NA	44	6%	55	5%

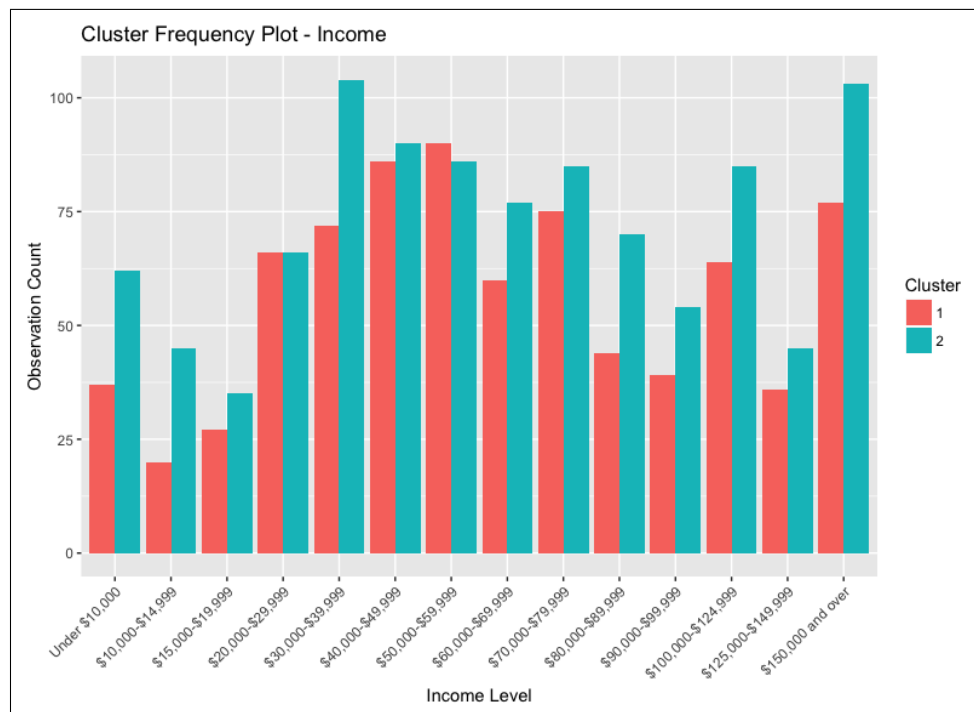


Figure 9.10: Cluster Breakout by Income

Table 9.11: Income Proportion Table by Cluster

Income	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Under \$10,000	37	5%	62	6%
\$10,000-\$14,999	20	3%	45	4%
\$15,000-\$19,999	27	3%	35	3%
\$20,000-\$29,999	66	8%	66	7%
\$30,000-\$39,999	72	9%	104	10%
\$40,000-\$49,999	86	11%	90	9%
\$50,000-\$59,999	90	11%	86	9%
\$60,000-\$69,999	60	8%	77	8%
\$70,000-\$79,999	75	9%	85	8%
\$80,000-\$89,999	44	6%	70	7%
\$90,000-\$99,999	39	5%	54	5%
\$100,000-\$124,999	64	8%	85	8%
\$125,000-\$149,999	36	5%	45	4%
\$150,000 and over	77	10%	103	10%

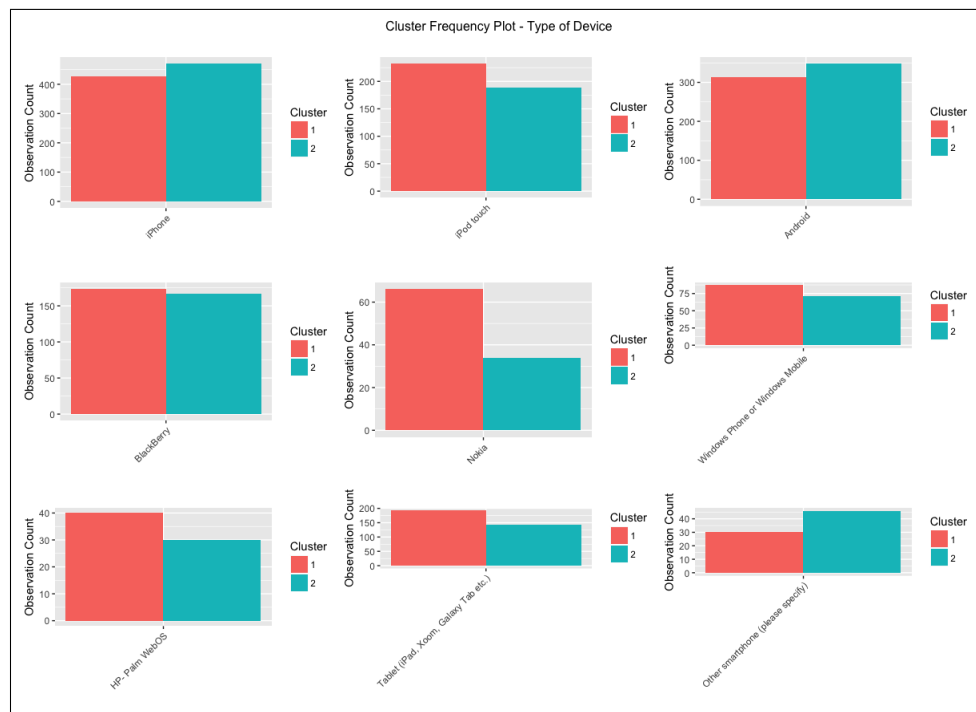


Figure 9.11: Cluster Breakout by Type of Device Owned

Table 9.12: Type of Device Proportion Table

Type of Device	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
iPhone	426	27%	470	31%
iPod Touch	233	15%	189	13%
Android	314	20%	348	23%
BlackBerry	174	11%	167	11%
Nokia	66	4%	34	2%
Windows Phone/Mobile	87	6%	71	5%
HP/Palm WebOS	40	3%	30	2%
Tablet	193	12%	144	10%
Other	30	2%	46	3%

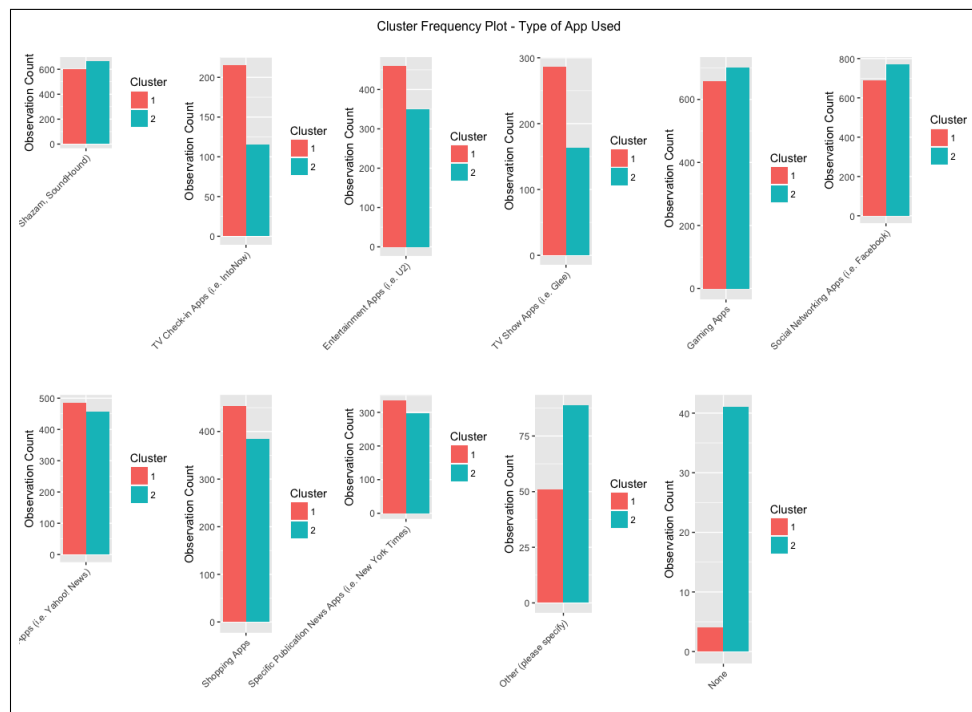


Figure 9.12: Cluster Breakout by Type of App Used

Table 9.13: Type of App Proportion Table by Cluster

Type of Apps Used	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
Music Identification	601	14%	666	17%
TV Check-In	215	5%	115	3%
Entertainment	460	11%	350	9%
TV Show	287	7%	163	4%
Gaming	658	16%	700	17%
Social Networking	688	16%	770	19%
General News	486	11%	458	11%
Shopping	454	11%	384	10%
Specific Publication News	335	8%	297	7%
Other	51	1%	89	2%
None	4	0%	41	1%

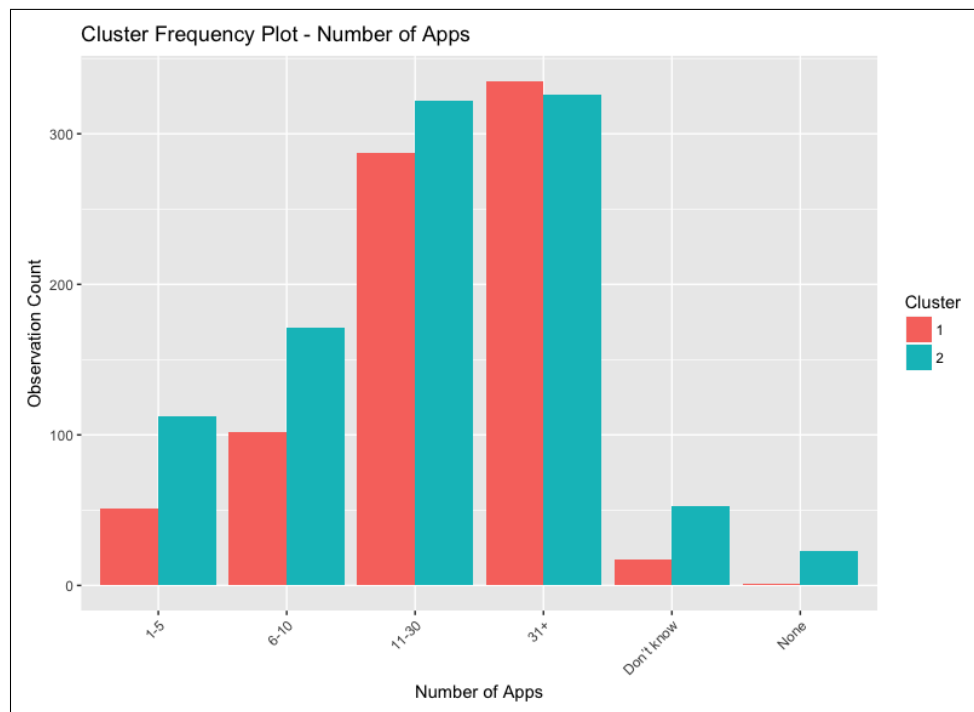


Figure 9.13: Cluster Breakout by Number of Apps

Table 9.14: Number of Apps Proportion Table by Cluster

Number of Apps	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
1-5	51	6%	112	11%
6-10	102	13%	171	17%
11-30	287	36%	322	32%
31+	335	42%	326	32%
Dont know	17	2%	53	5%
None	1	0%	23	2%

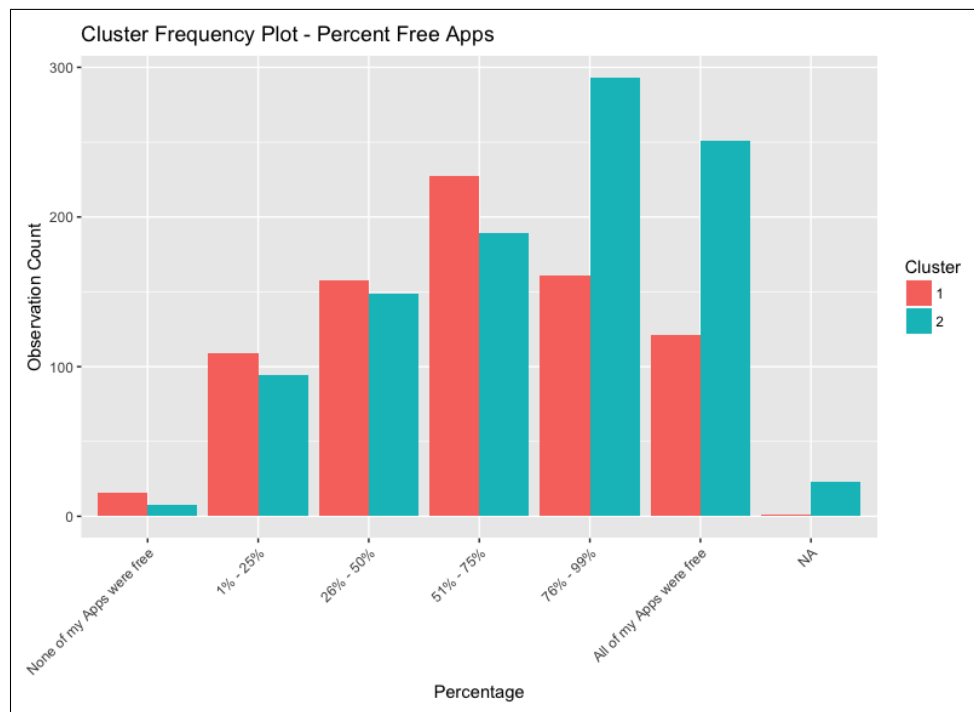


Figure 9.14: Cluster Breakout by Percentage of Free Apps

Table 9.15: Percentage of Free Apps Proportion Table by Cluster

Percentage of Free Apps	Cluster 1		Cluster 2	
	Number of Observations	Percentage	Number of Observations	Percentage
None of my Apps were free	16	2%	8	1%
1% - 25%	109	14%	94	9%
26% - 50%	158	20%	149	15%
51% - 75%	227	29%	189	19%
76% - 99%	161	20%	293	29%
All of my Apps were free	121	15%	251	25%
NA	1	0%	23	2%

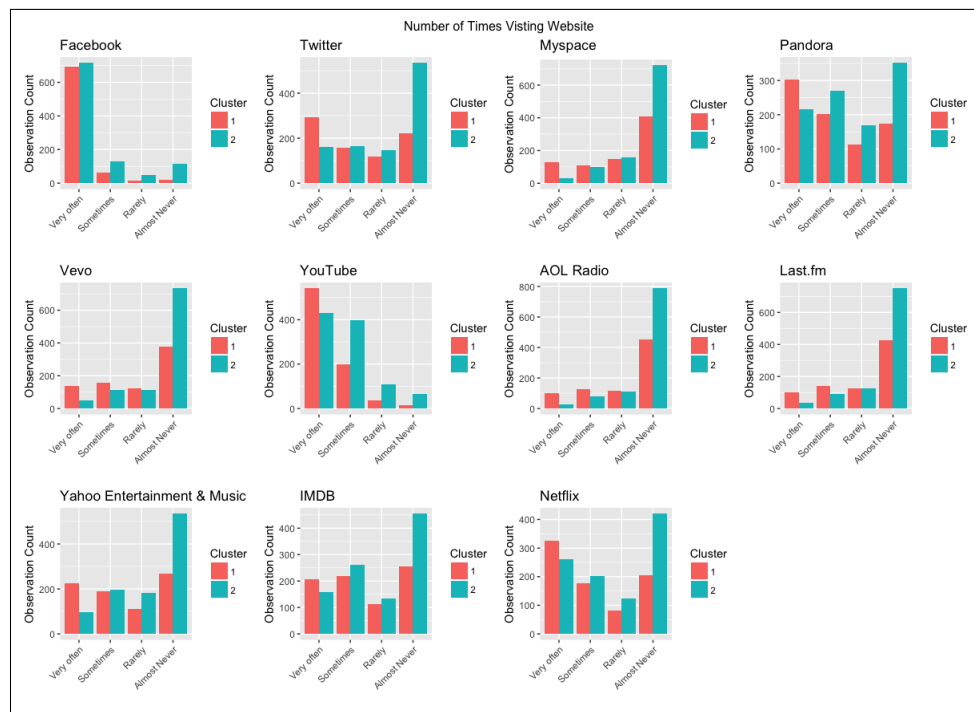


Figure 9.15: Cluster Breakout of Visiting Websites

R Code

```
# load libraries ----
library(tidyverse)
library(skimr) # disables from dplyr: contains, ends_with, everything, matches, num_range,
               one_of, starts_with
library(corrplot)
library(factoextra)
library(cluster)
library(NbClust)
library(mclust)
library(fpc)
library(gridExtra)

# load Rdata file ----

s1 <- load('/Users/nikhil/Documents/Northwestern/PREDICT450/Assignments/Solo1/data/
  apphappyData.RData')

#tblPath <- '/Users/nikhil/Documents/Northwestern/PREDICT450/Assignments/Solo1/paper/tbls/'

# preview data frames ----

glimpse(apphappy.3.labs.frame)
glimpse(apphappy.3.num.frame)

# summary on the data frames ----
summary(apphappy.3.num.frame)
```



```
summary(apphappy.3.labs.frame)

skim(apphappy.3.num.frame)

# https://ropensci.org/blog/2017/07/11/skimr/
apphappy.3.num.frame %>%
  skim() %>%
  filter(stat == 'missing') %>%
  arrange(desc(value)) %>%
  print(n=2000)

# missing data check ----
apphappy.3.num.frame %>%
  map_df(function(x) sum(is.na(x))) %>%
  gather(key = 'variable', value = 'missing') %>%
  arrange(desc(missing))

apphappy.3.labs.frame %>%
  map_df(function(x) sum(is.na(x))) %>%
  gather(key = 'variable', value = 'missing') %>%
  arrange(desc(missing))

# Quality Checks ----

## quality check Q1 (numerical)

apphappy.3.num.frame %>%
  select(caseID, q1) %>%
  summary

apphappy.3.labs.frame %>%
  select(caseID, q1) %>%
  filter(str_detect(q1, '65'))

# create a data frame for attitudinal questions ----
# attitudinal questions seem to be q24, q25, and q26 as they pertain to a person's
# perspective/attitude

att.num.df <- apphappy.3.num.frame %>%
  select(
    caseID,
    starts_with('q24'),
    starts_with('q25'),
    starts_with('q26')
  )

att.lbl.df <- apphappy.3.labs.frame %>%
  select(
    caseID,
    starts_with('q24'),
    starts_with('q25'),
    starts_with('q26')
  )

# custom summary for the attitudinal stuff
att.num.df %>%
  select(-caseID) %>%
  skim %>%
```

```

filter(stat != 'hist') %>%
select(-type, -level, -formatted) %>%
spread(key = stat, value = value) %>%
select(-complete, -n) %>%
transmute(
  variable,
  missing,
  avg = round(mean,3),
  sd = round(sd,3),
  median,
  min = p0,
  max = p100,
  p25,
  p75
) %>%
write_csv(path = '/Users/nikhil/Documents/Northwestern/PREDICT450/Assignments/Solo1/paper/
  tbls/summary_att.csv')

# EDA - all data ----
apphappy.3.labs.frame %>%
ggplot(aes(x = q1)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
scale_fill_discrete(name = 'Gender') +
ggtitle('Age Breakdown by Gender') +
labs(x = 'Age Group')

apphappy.3.labs.frame %>%
ggplot(aes(x = q56)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Household Income') +
scale_fill_discrete(name = "Gender")+
ggtitle('Household Income Frequency Count by Gender')

# breakdown of male vs. female respondents
apphappy.3.labs.frame %>%
ggplot(aes(x = q57)) +
geom_bar(stat = 'count') +
labs(x = 'Gender') +
ggtitle('Histogram of Gender')

# breakdown by race
apphappy.3.labs.frame %>%
ggplot(aes(x = q54)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Race') +
scale_fill_discrete(name = "Gender")+
ggtitle('Race Breakdown by Gender')

apphappy.3.labs.frame %>%
ggplot(aes(x = q55)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Identify as Latino?') +
scale_fill_discrete(name = "Gender")+
ggtitle('Identify as Latino Breakdown by Gender')

apphappy.3.labs.frame %>%

```

```

ggplot(aes(x = q54)) +
  geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Race') +
  scale_fill_discrete(name = "Gender")+
  ggtitle('Race Breakdown by Latino Identification & Gender') +
  facet_grid(q55 ~ .)

# age breakdown by race
apphappy.3.labs.frame %>%
ggplot(aes(x = q1)) +
  geom_bar(stat = 'count', aes(fill = q54), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Race') +
  scale_fill_discrete(name = "Race")+
  ggtitle('Age Breakdown by Race')

# household income breakdown by race
apphappy.3.labs.frame %>%
ggplot(aes(x = q56)) +
  geom_bar(stat = 'count', aes(fill = q54), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Income') +
  scale_fill_discrete(name = "Race")+
  ggtitle('Household Income Breakdown by Race')

# household income breakdown by marital status
apphappy.3.labs.frame %>%
ggplot(aes(x = q56)) +
  geom_bar(stat = 'count', aes(fill = q49), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Income') +
  scale_fill_discrete(name = "Marital Status")+
  ggtitle('Household Income Breakdown by Marital Status')

# household income breakdown by marital status & gender
apphappy.3.labs.frame %>%
ggplot(aes(x = q56)) +
  geom_bar(stat = 'count', aes(fill = q49), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Income') +
  scale_fill_discrete(name = "Marital Status")+
  ggtitle('Household Income Breakdown by Marital Status & Gender') +
  facet_grid(q57 ~ .)

# household income breakdown by education level
apphappy.3.labs.frame %>%
ggplot(aes(x = q56)) +
  geom_bar(stat = 'count', aes(fill = q48), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Income') +
  scale_fill_discrete(name = "Education Level")+
  ggtitle('Household Income Breakdown by Education Level')

# look at the type of apps
apphappy.3.num.frame %>%
select(starts_with('q4r')) %>%
map_df(function(x) sum(x)) %>%
gather(key = 'q4', value = 'count') %>%

```

```

ggplot(aes(x = reorder(as.factor(q4), -count), y = count)) +
  geom_col() +
  labs(x = 'Question 4 Response Types')
#scale_x_discrete(limits = c('q4r1', 'q4r2', 'q4r3', 'q4r4', 'q4r5', 'q4r6', 'q4r7', 'q4r8', 'q4r9',
  ', 'q4r10', 'q4r11'))

# look at the type of platforms
apphappy.3.num.frame %>%
  select(starts_with('q2r')) %>%
  map_df(function(x) sum(x)) %>%
  gather(key = 'q2', value = 'count') %>%
  ggplot(aes(x = q2, y = count)) +
  geom_col() +
  scale_x_discrete(limits = c('q2r1', 'q2r2', 'q2r3', 'q2r4', 'q2r5', 'q2r6', 'q2r7', 'q2r8', 'q2r9', 'q2r10'))

# look at the social sites

apphappy.3.labs.frame %>%
  select(q13r1) %>%
  group_by(q13r1) %>%
  summarize(
    obs.count = n()
  ) %>%
  ungroup

apphappy.3.labs.frame %>%
  select(q13r1) %>%
  group_by(q13r1) %>%
  summarize(
    obs.count = n()
  ) %>%
  ungroup %>%
  ggplot(aes(x = q13r1, y = obs.count)) +
  geom_col() +
  labs(x = 'Visit Frequency')

getSiteFrequency <- function(df, site, sitename) {
  site <- enquo(site)
  df %>%
  select(!site) %>%
  group_by(!site) %>%
  ggplot(aes_string(x = rlang::quo_text(site))) +
  geom_bar(stat = 'count') +
  labs(x = 'Visit Frequency') +
  ggtitle(paste0(sitename, ' Visit Frequency Plot'))
}

getSiteFrequency(apphappy.3.labs.frame, q13r1, 'Facebook')
getSiteFrequency(apphappy.3.labs.frame, q13r2, 'Twitter')
getSiteFrequency(apphappy.3.labs.frame, q13r3, 'Myspace')
getSiteFrequency(apphappy.3.labs.frame, q13r4, 'Pandora')
getSiteFrequency(apphappy.3.labs.frame, q13r5, 'Vevo')
getSiteFrequency(apphappy.3.labs.frame, q13r6, 'YouTube')
getSiteFrequency(apphappy.3.labs.frame, q13r7, 'AOL Radio')
getSiteFrequency(apphappy.3.labs.frame, q13r8, 'Last.fm')

```

```

getSiteFrequency(apphappy.3.labs.frame, q13r9, 'Yahoo Music')
getSiteFrequency(apphappy.3.labs.frame, q13r10, 'IMDB')
getSiteFrequency(apphappy.3.labs.frame, q13r11, 'LinkedIn')
getSiteFrequency(apphappy.3.labs.frame, q13r12, 'Netflix')

apphappy.3.labs.frame %>%
select(q13r1, q57) %>%
ggplot(aes(x = q13r1)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
labs(x = 'Visit Frequency') +
scale_fill_discrete(name = 'Gender') +
ggtitle('Facebook Visit Frequency Breakdown by Gender')

plotSite <- function(df, site, col2, sitename, col2name) {
  site <- enquo(site)
  col2 <- enquo(col2)
  df %>%
  select(!!site, !!col2) %>%
  ggplot(aes_string(x = rlang::quo_text(site))) +
  geom_bar(stat = 'count', aes_string(fill = rlang::quo_text(col2)), position = 'dodge') +
  labs(x = 'Visit Frequency') +
  ggtitle(paste0(sitename, ' Visit Frequency Breakdown by ', col2name)) +
  scale_fill_discrete(name = col2name)
}

plotSite(apphappy.3.labs.frame, q13r1, q57, 'Facebook', 'Gender')
plotSite(apphappy.3.labs.frame, q13r2, q57, 'Twitter', 'Gender')
plotSite(apphappy.3.labs.frame, q13r3, q57, 'Myspace', 'Gender')
plotSite(apphappy.3.labs.frame, q13r4, q57, 'Pandora', 'Gender')
plotSite(apphappy.3.labs.frame, q13r5, q57, 'Vevo', 'Gender')
plotSite(apphappy.3.labs.frame, q13r6, q57, 'YouTube', 'Gender')
plotSite(apphappy.3.labs.frame, q13r7, q57, 'AOL Radio', 'Gender')
plotSite(apphappy.3.labs.frame, q13r8, q57, 'Last.fm', 'Gender')
plotSite(apphappy.3.labs.frame, q13r9, q57, 'Yahoo Music', 'Gender')
plotSite(apphappy.3.labs.frame, q13r10, q57, 'IMDB', 'Gender')
plotSite(apphappy.3.labs.frame, q13r11, q57, 'LinkedIn', 'Gender')
plotSite(apphappy.3.labs.frame, q13r12, q57, 'Netflix', 'Gender')

plotSite(apphappy.3.labs.frame, q13r1, q1, 'Facebook', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r2, q1, 'Twitter', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r3, q1, 'Myspace', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r4, q1, 'Pandora', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r5, q1, 'Vevo', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r6, q1, 'YouTube', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r7, q1, 'AOL Radio', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r8, q1, 'Last.fm', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r9, q1, 'Yahoo Music', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r10, q1, 'IMDB', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r11, q1, 'LinkedIn', 'Age Group')
plotSite(apphappy.3.labs.frame, q13r12, q1, 'Netflix', 'Age Group')

# EDA - attitudinal data
apphappy.3.num.frame %>%
select(q1) %>%
group_by(q1) %>%

```

```

tally %>%
  ggplot(aes(x = q1, y = n)) +
  geom_bar(stat = 'identity') +
  ggtitle('Frequency Histogram for Q1') +
  labs(x = 'Q1', y = 'Frequency')

apphappy.3.num.frame %>%
  select(q1) %>%
  ggplot(aes(x = q1)) +
  geom_histogram() +
  ggtitle('Frequency Histogram for Q1 Numerical Responses') +
  labs(x = 'Q1 Response', y = 'Frequency')

getHistogram <- function(cname) {
  cname <- enquo(cname)
  apphappy.3.num.frame %>%
  select(!!cname) %>%
  ggplot(aes_string(x = rlang::quo_text(cname))) +
  geom_histogram(stat = 'count') +
  ggtitle('Frequency Histogram') +
  labs(x = 'Q1 Response', y = 'Frequency')
}

getHistogram(q1)
getHistogram(q5r1)

# EDA on numerical frame only

getGridFreqCounts <- function(cname) {
  cname <- enquo(cname)
  att.num.df %>%
  select(starts_with(!!cname)) %>%
  gather(key = 'variable', value = 'observation') %>%
  group_by(variable, observation) %>%
  summarize(cnt = n()) %>%
  ungroup %>%
  spread(key = variable, value = cnt)
}

getGridFreqCounts('q24')
getGridFreqCounts('q25')
getGridFreqCounts('q26')

att.num.df %>%
  select(q24r1, q25r1) %>%
  group_by(q24r1, q25r1) %>%
  summarize(
    cnt.obs = n()
  ) %>%
  ungroup %>%
  spread(key = q25r1, value = cnt.obs, fill = 0)

getGroupGrid <- function(df, cname1, cname2) {
  cname1 <- enquo(cname1)
  cname2 <- enquo(cname2)
  df %>%
  select(!!cname1, !!cname2) %>%
  rename(c1 = !!cname1, c2 = !!cname2) %>%

```

```

group_by(c1,c2) %>%
  summarize(
    cnt.obs = n()
  ) %>%
  ungroup %>%
  spread(key = c2, value = cnt.obs, fill = 0)
}
getGroupGrid(att.num.df, 'q24r1','q25r1')
getGroupGrid(att.num.df, 'q24r2','q25r2')

att.num.df %>%
  select(q24r1, q25r1) %>%
  group_by(q24r1, q25r1) %>%
  summarize(
    cnt.obs = n()
  ) %>%
  ungroup %>%
  ggplot(aes(x = q24r1, y = cnt.obs, fill= q25r1)) +
  geom_bar(stat = 'identity')

att.num.df %>%
  select(starts_with('q24')) %>%
  gather(key = 'variable', value = 'observation') %>%
  group_by(variable, observation) %>%
  summarize(cnt2 = n()) %>%
  ungroup %>%
  ggplot(aes(x = observation, y = cnt2, fill = variable)) +
  geom_bar(stat = 'identity') +
  labs(x = 'Numerical Response Value', y = 'Frequency') +
  ggtitle('Stacked Bar Chart for Numerical Response for Q24')

att.num.df %>%
  select(starts_with('q25')) %>%
  gather(key = 'variable', value = 'observation') %>%
  group_by(variable, observation) %>%
  summarize(cnt2 = n()) %>%
  ungroup %>%
  ggplot(aes(x = observation, y = cnt2, fill = variable)) +
  geom_bar(stat = 'identity') +
  labs(x = 'Numerical Response Value', y = 'Frequency') +
  ggtitle('Stacked Bar Chart for Numerical Response for Q25')

att.num.df %>%
  select(starts_with('q26')) %>%
  gather(key = 'variable', value = 'observation') %>%
  group_by(variable, observation) %>%
  summarize(cnt2 = n()) %>%
  ungroup %>%
  ggplot(aes(x = observation, y = cnt2, fill = variable)) +
  geom_bar(stat = 'identity') +
  labs(x = 'Numerical Response Value', y = 'Frequency') +
  ggtitle('Stacked Bar Chart for Numerical Response for Q26')

getStackedBarChart <- function(cname) {
  cname2 <- enquo(cname)

```

```

att.num.df %>%
select(starts_with(!cname2)) %>%
gather(key = 'variable', value = 'observation') %>%
group_by(variable, observation) %>%
summarize(cnt2 = n()) %>%
ungroup %>%
ggplot(aes(x = observation, y = cnt2, fill = variable)) +
geom_bar(stat = 'identity') +
labs(x = 'Numerical Response Value', y = 'Frequency') +
ggtitle(paste0('Stacked Bar Chart for Numerical Response for ',toupper(cname)))
}

getStackedBarChart('q24')
getStackedBarChart('q25')
getStackedBarChart('q26')

# lets do a correlation study ----

numcor <- cor(att.num.df)# %>% select(starts_with('q24'), starts_with('q25'))
corrplot(
numcor,
method = 'color',
order = 'hclust',
addCoef.col = 'red',
tl.col = 'black',
addrect = 5,
tl.cex = 0.6,
number.cex = 0.6
)

# PCA ----
# PCA on the original numerical dataset (cluster size does not matter)
pca <- prcomp(att.num.df %>% select(-caseID))
summary(pca)

tidy(pca, matrix = 'd') %>%
transmute(
principle_component = PC,
std_dev = round(std.dev,3),
percent = round(percent,3),
cumulative_pct = round(cumulative,3)
) %>%
top_n(n=15, wt = percent) %>%
write_csv(path = '/Users/nikhil/Documents/Northwestern/PREDICT450/Assignments/Solo1/paper/
tbls/pca.csv')
# note how 14 principle components are needed to get about 70% of the variation to be
explained.

#plot pca
fviz_nbclust(att.num.df %>% select(-caseID), method = 'wss', k.max = 15, FUNcluster = kmeans
) #elbow
fviz_eig(pca)
fviz_pca_ind(pca)

```



```

# note the big blob with very little distinction

# next bet is to combine the attitudinal questions
att.num.df2 <- att.num.df %>%
transmute(
  caseID,
  trendsetter = (q25r2 + q26r18 + q26r7 + q26r8 + q26r10) / 5,
  early.adopter = (q24r1 + q24r3 + q25r5 + q25r8 + q25r9 + q25r10 + q26r9) / 7,
  ease.of.life = (q24r5 + q24r6 + q25r12 + q26r5) / 4,
  entertainment = (q24r7 + q24r8) / 2,
  late.adopter = (q24r4 + q24r9 + q24r12 + q25r11) / 4,
  leader = (q25r1 + q25r4 + q25r7) / 3,
  shopper = (q26r3 + q26r4 + q26r6 + q26r12 + q26r13) / 5,
  social = (q24r2 + q24r10 + q24r11 + q25r3 + q25r6 + q25r11) / 6
)

# PCA on the new data frame
pca2 <- prcomp(att.num.df2 %>% select(-caseID))
summary(pca2)

tidy(pca2, matrix = 'd') %>%
transmute(
  principle_component = PC,
  std_dev = round(std.dev,3),
  percent = round(percent,3),
  cumulative_pct = round(cumulative,3)
) %>%
top_n(n=5, wt = percent) %>%
write_csv(path = '/Users/nikhil/Documents/Northwestern/PREDICT450/Assignments/Solo1/paper/
  tbls/pca2.csv')

fviz_nbclust(att.num.df2 %>% select(-caseID), method = 'wss', k.max = 15, FUNcluster =
  kmeans) #elbow
fviz_nbclust(att.num.df2 %>% select(-caseID), method = 'silhouette', k.max = 15, FUNcluster
  = kmeans)

#plot pca
fviz_eig(pca2)
fviz_pca_ind(pca2)
# note how 3 principle components can now explain almost 70% of the variance
# still not very clear on the image tho

# k means ----
#### scaling is not necessary since all of the questions analyzed for partitioning are on
  the same scale
# att.num.df <- att.num.df %>%
#   select(-caseID) %>%
#   map_df(function(x) scale(x, center = T, scale = T)) %>%
#   add_column(caseID = att.num.df$caseID)

# kmeans on full dataset ----
## method 1: use scree plot to determine correct (code has been provided by prof)
wssplot <- function(att.num.df, nc=15, seed=1234) {
wss <- (nrow(att.num.df)-1)*sum(apply(att.num.df,2,var))
for (i in 2:nc) {

```

```

set.seed(seed)
wss[i] <- sum(kmeans(att.num.df, centers=i)$withinss)}
plot(1:nc, wss, type="b", xlab="Number of Clusters",
ylab="Within groups sum of squares", main = 'Scree Plot')

wssplot(att.num.df %>% select(-caseID))

## method 2: custom function that loops through to find best cluster using R2
for (i in 1:15) {
  csize = i + 1
  set.seed(2018)
  kc <- kmeans(att.num.df %>% select(-caseID), centers = csize)
  r2 <- kc$betweenss / kc$totss
  print(paste0('cluster size: ', csize, ' r2: ',round(r2,3)))
}

## method 3: use avg. silhouette width to determine best cluster size
df <- tibble(x=1,y=0)
for (i in 2:15) {
  set.seed(2018)
  kc <- kmeans(att.num.df %>% select(-caseID), centers = i)
  dissE <- daisy(att.num.df %>% select(-caseID))
  dE2 <- dissE^2
  sk2 <- silhouette(kc$cluster, dE2)
  df <- add_row(df, x=i, y=round(summary(sk2)$avg.width,3))
  print(paste0('cluster size: ', i, ' avg. silhouette width: ', round(summary(sk2)$avg.width
,3)))
  if(i == 15) {
    print(
      df %>%
      ggplot(aes(x = x, y = y)) +
      geom_line(col='blue') +
      geom_point(col='red') +
      ggtitle('Silhouette Average Widths') +
      xlab('Cluster Size') +
      ylab('Average Width')
    )
  }
}

## method 4: use NbClust to go through 26 indices to determine which cluster size is best
#### I use Ratkowsky due to the white-paper I found
best.cluster.size <- NbClust(att.num.df %>% select(-caseID), distance = 'euclidean', method
= 'kmeans', min.nc = 2, max.nc = 10)
fviz_nbclust(best.cluster.size)

# method 5: visual graphs
fviz_nbclust(att.num.df %>% select(-caseID), method = 'wss', k.max = 15, FUNcluster = kmeans
) #elbow
fviz_nbclust(att.num.df %>% select(-caseID), method = 'silhouette', k.max = 15, FUNcluster =
kmeans)
fviz_nbclust(att.num.df %>% select(-caseID), method = 'silhouette', k.max = 15, FUNcluster =
pam)

# method 6: GAP statistic
clus.gap <- clusGap(att.num.df %>% select(-caseID), FUNcluster = kmeans, K.max = 15, B =
100)
print(clus.gap, method = 'firstmax')
fviz_gap_stat(clus.gap)

```

```

print(clus.gap, method = 'Tibs2001SEmax')
fviz_gap_stat(clus.gap, maxSE = list(method = "Tibs2001SEmax"))

# choose 2 clusters
set.seed(2018)
kclust.2c <- kmeans(att.num.df %>% select(-caseID), centers = 2)
kclust.2c
kclust.2c$centers
kclust.2c$cluster

fviz_cluster(kclust.2c, data = att.num.df %>% select(-caseID))

plot(silhouette(kclust.2c$cluster, (daisy(att.num.df %>% select(-caseID)))^2), col = 1:2,
     border = NA)
abline(v = summary(silhouette(kclust.2c$cluster, (daisy(att.num.df %>% select(-caseID)))^2))
       $avg.width, col = 'gray', lty = 2, lwd = 4)
#https://stackoverflow.com/questions/32570693/make-silhouette-plot-legible-for-k-means

# choose 4 clusters
set.seed(2018)
kclust.4c <- kmeans(att.num.df %>% select(-caseID), centers = 4)
fviz_cluster(kclust.4c, data = att.num.df %>% select(-caseID), geom = 'point', ellipse = F)

# kmeans on grouped dataset ----

fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = kmeans, method = "wss")
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = kmeans, method = "silhouette")

a <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = kmeans, method = "wss") +
  ggtitle('Scree Plot')
b <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = kmeans, method = "silhouette") +
  ggtitle('Avg. Silhouette Width Plot')
grid.arrange(a,b, ncol=2)

# choose 2 clusters
set.seed(2018)
kclust.2c <- kmeans(att.num.df2 %>% select(-caseID), centers = 2)
fviz_cluster(kclust.2c, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse = F)
  + ggtitle('2 Cluster Plot')
plot(silhouette(kclust.2c$cluster, (daisy(att.num.df2 %>% select(-caseID)))^2), col = 1:2,
     border = NA, main = 'Silhouette Plot - 2 Clusters')

# choose 3 clusters
set.seed(2018)
kclust.3c <- kmeans(att.num.df2 %>% select(-caseID), centers = 3)
fviz_cluster(kclust.3c, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse = T)

# compare clusters
cluster.stats(dist(att.num.df2 %>% select(-caseID)), kclust.2c$cluster, kclust.3c$cluster)
  $corrected.rand

# combine the data with the cluster (K-MEANS 2 cluster) ----
#att.num.df.kcluster <- bind_cols(att.num.df, tibble(kmeans.2cluster = kclust.2c$cluster))
#att.num.df.kcluster <- bind_cols(att.num.df.kcluster, tibble(kmeans.3cluster = kclust.3
  c$cluster))
apphappy.num.kcluster <- bind_cols(apphappy.3.num.frame, tibble(kmeans.2cluster = kclust.2
  c$cluster))
apphappy.num.kcluster <- bind_cols(apphappy.num.kcluster, tibble(kmeans.3cluster = kclust.3
  c$cluster))

```

```

apphappy.num.kcluster <- apphappy.num.kcluster %>%
  dplyr::select(
    -starts_with('q24'),
    -starts_with('q25'),
    -starts_with('q26')
  )

apphappy.labs.kcluster <- bind_cols(apphappy.3.labs.frame, tibble(kmeans.2cluster = kclust.2
  c$cluster))
apphappy.labs.kcluster <- bind_cols(apphappy.labs.kcluster, tibble(kmeans.3cluster = kclust
  .3c$cluster))
apphappy.labs.kcluster <- apphappy.labs.kcluster %>%
  dplyr::select(
    -starts_with('q24'),
    -starts_with('q25'),
    -starts_with('q26')
  )

# apphappy.lab.kcluster <- bind_cols(apphappy.3.labs.frame, tibble(cluster = kclust$cluster)
  )

# kmeans profiling with grouped data ----
# deep profiling using kcluster
apphappy.labs.kcluster %>%
  select(
    q48,
    kmeans.2cluster
  ) %>%
  group_by(q48, kmeans.2cluster) %>%
  summarize(
    obs.count = n()
  ) %>%
  ungroup() %>%
  ggplot(aes(x = q48, y = obs.count)) +
  geom_bar(stat = 'identity', aes(fill = as.factor(kmeans.2cluster)), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_discrete(name = 'cluster')

apphappy.labs.kcluster %>%
  select(
    q48,
    kmeans.2cluster
  ) %>%
  group_by(q48, kmeans.2cluster) %>%
  summarize(
    obs.count = n()
  ) %>%
  ungroup() %>%
  spread(
    key = kmeans.2cluster,
    value = obs.count,
    fill = 0
  )

apphappy.labs.kcluster %>%
  select(
    q48,
    kmeans.2cluster
  )

```

```

) %>%
mutate(
  kmeans.2cluster = as.factor(kmeans.2cluster)
) %>%
group_by(
  q48,
  kmeans.2cluster
) %>%
summarize(
  obs.pct = n()/1800
) %>%
ungroup() %>%
ggplot(aes(x = as.factor(kmeans.2cluster), y = obs.pct)) +
  geom_bar(stat = 'identity', aes(fill = q48)) +
  scale_fill_brewer(palette = 'YlGnBu')

# to mix aes and aes_string
'+.uneval' <- function(a,b) {
  'class<-'(modifyList(a,b), "uneval")
}
DeepCheck <- function(df, col1, cluster, title, xaxis) {
  col1 <- enquo(col1)
  cluster <- enquo(cluster)
  df %>%
  select(
    !!col1,
    !!cluster
  ) %>%
  group_by(!!col1, !!cluster) %>%
  mutate(
    obs.count = n(),
    cluster = as.factor(!!cluster)
  ) %>%
  ungroup() %>%
  ggplot(aes_string(x=rlang::quo_text(col1))+aes(y = obs.count)) +
  #geom_bar(stat = 'identity')
  geom_bar(stat = 'identity', aes_string(fill = 'cluster'), position = 'dodge') +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_discrete(name = 'Cluster') +
  ggtitle(title) +
  labs(x = xaxis, y = 'Observation Count')
}

DeepCheck(apphappy.labs.kcluster, q1, kmeans.2cluster, title = 'Cluster Frequency Plot - Age
  Group', xaxis = 'Age Groups')

# type of device
a <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r1)), q2r1, kmeans.2
  cluster, title = '', xaxis = '') # iphone
b <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r2)), q2r2, kmeans.2
  cluster, title = '', xaxis = '') # ipod touch
c <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r3)), q2r3, kmeans.2
  cluster, title = '', xaxis = '') # android
d <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r4)), q2r4, kmeans.2
  cluster, title = '', xaxis = '') # blackberry
e <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r5)), q2r5, kmeans.2
  cluster, title = '', xaxis = '') # nokia

```

```

f <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r6)), q2r6, kmeans.2
  cluster, title = '', xaxis = '') # windows phone
g <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r7)), q2r7, kmeans.2
  cluster, title = '', xaxis = '') # Palm OS
h <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r8)), q2r8, kmeans.2
  cluster, title = '', xaxis = '') # Tablet
i <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q2r9)), q2r9, kmeans.2
  cluster, title = '', xaxis = '') # other

grid.arrange(a, b, c, d, e, f, g, h, i, ncol = 3, top = 'Cluster Frequency Plot - Type of
  Device')

# type of app
a <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r1)), q4r1, kmeans.2
  cluster, title = '', xaxis = '') # music id app
b <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r2)), q4r2, kmeans.2
  cluster, title = '', xaxis = '') # tv check in app
c <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r3)), q4r3, kmeans.2
  cluster, title = '', xaxis = '') # entertainment app
d <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r4)), q4r4, kmeans.2
  cluster, title = '', xaxis = '') # tv show app
e <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r5)), q4r5, kmeans.2
  cluster, title = '', xaxis = '') # gaming app
f <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r6)), q4r6, kmeans.2
  cluster, title = '', xaxis = '') # social app
g <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r7)), q4r7, kmeans.2
  cluster, title = '', xaxis = '') # general news app
h <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r8)), q4r8, kmeans.2
  cluster, title = '', xaxis = '') # shopping app
i <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r9)), q4r9, kmeans.2
  cluster, title = '', xaxis = '') # specific news pub app
j <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r10)), q4r10, kmeans.2
  cluster, title = '', xaxis = '') # other
k <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q4r11)), q4r11, kmeans.2
  cluster, title = '', xaxis = '') # none

grid.arrange(a, b, c, d, e, f, g, h, i, j, k, ncol = 6, top = 'Cluster Frequency Plot - Type
  of App Used')

DeepCheck(apphappy.labs.kcluster, q11, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Number of Apps', xaxis = 'Number of Apps')
DeepCheck(apphappy.labs.kcluster, q12, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Percent Free Apps', xaxis = 'Percentage')

# frequency of visiting website
a <- DeepCheck(apphappy.labs.kcluster, q13r1, kmeans.2cluster, title = 'Facebook', xaxis =
  '')
b <- DeepCheck(apphappy.labs.kcluster, q13r2, kmeans.2cluster, title = 'Twitter', xaxis =
  '')
c <- DeepCheck(apphappy.labs.kcluster, q13r3, kmeans.2cluster, title = 'Myspace', xaxis =
  '')
d <- DeepCheck(apphappy.labs.kcluster, q13r4, kmeans.2cluster, title = 'Pandora', xaxis =
  '')
e <- DeepCheck(apphappy.labs.kcluster, q13r5, kmeans.2cluster, title = 'Vevo', xaxis = '')
f <- DeepCheck(apphappy.labs.kcluster, q13r6, kmeans.2cluster, title = 'YouTube', xaxis =
  '')
g <- DeepCheck(apphappy.labs.kcluster, q13r7, kmeans.2cluster, title = 'AOL Radio', xaxis =
  '')
h <- DeepCheck(apphappy.labs.kcluster, q13r8, kmeans.2cluster, title = 'Last.fm', xaxis =

```

```

    '')
i <- DeepCheck(apphappy.labs.kcluster, q13r9, kmeans.2cluster, title = 'Yahoo Entertainment
  & Music', xaxis = '')
j <- DeepCheck(apphappy.labs.kcluster, q13r10, kmeans.2cluster, title = 'IMDB', xaxis = '')
k <- DeepCheck(apphappy.labs.kcluster, q13r11, kmeans.2cluster, title = 'LinkedIn', xaxis =
  '')
l <- DeepCheck(apphappy.labs.kcluster, q13r12, kmeans.2cluster, title = 'Netflix', xaxis =
  '')

grid.arrange(a, b, c, d, e, f, g, h, i, j, l, ncol = 4, top = 'Number of Times Visting
  Website')

DeepCheck(apphappy.labs.kcluster, q48, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Education', xaxis = 'Education Level')
DeepCheck(apphappy.labs.kcluster, q49, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Marital Status', xaxis = 'Marital Status')

a <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r1)), q50r1, kmeans.2
  cluster, title = '', xaxis = '')
b <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r2)), q50r2, kmeans.2
  cluster, title = '', xaxis = '')
c <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r3)), q50r3, kmeans.2
  cluster, title = '', xaxis = '')
d <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r4)), q50r4, kmeans.2
  cluster, title = '', xaxis = '')
e <- DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r5)), q50r5, kmeans.2
  cluster, title = '', xaxis = '')

grid.arrange(a,b,c,d,e, ncol = 3, top = 'Cluster Frequency Plot - Children')

a <- DeepCheck(apphappy.labs.kcluster, q54, kmeans.2cluster, title = 'Cluster Frequency Plot
  - Race', xaxis = 'Race')
b <- DeepCheck(apphappy.labs.kcluster, q55, kmeans.2cluster, title = 'Cluster Frequency Plot
  - Hispanic or Latino', xaxis = 'Identify as Hispanic or Latino?')

grid.arrange(a,b)

DeepCheck(apphappy.labs.kcluster, q56, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Income', xaxis = 'Income Level')
DeepCheck(apphappy.labs.kcluster, q57, kmeans.2cluster, title = 'Cluster Frequency Plot -
  Gender', xaxis = 'Gender')

DeepCheck(apphappy.labs.kcluster, q1, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q11, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q12, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q48, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q49, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r1)), q50r1, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r2)), q50r2, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r3)), q50r3, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r4)), q50r4, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster %>% filter(!grepl('NO T0', q50r5)), q50r5, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q54, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q56, kmeans.3cluster)
DeepCheck(apphappy.labs.kcluster, q57, kmeans.3cluster)

apphappy.labs.kcluster %>%
select(

```

```

q48,
kmeans.2cluster
) %>%
mutate(
kmeans.2cluster = as.factor(kmeans.2cluster)
) %>%
group_by(
q48,
kmeans.2cluster
) %>%
summarize(
obs.pct = n()/1800
) %>%
ungroup() %>%
ggplot(aes(x = as.factor(kmeans.2cluster), y = obs.pct)) +
geom_bar(stat = 'identity', aes(fill = q48))

DeepCheck2 <- function(df, col1, cluster) {
col1 <- enquo(col1)
cluster <- enquo(cluster)
df %>%
select(
!!col1,
!!cluster
) %>%
group_by(
!!col1,
!!cluster
) %>%
summarize(
obs.pct = n()/1800
) %>%
ggplot(aes_string(x = rlang::quo_text(cluster)) + aes(y = obs.pct)) +
geom_bar(stat = 'identity', aes_string(fill = rlang::quo_text(col1))) +
scale_x_continuous(breaks = c(0,1,2,3), name = 'Cluster')
}

DeepCheck2(apphappy.labs.kcluster, q1, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q11, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q12, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q48, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q49, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r1)), q50r1, kmeans.3cluster
)
DeepCheck2(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r2)), q50r2, kmeans.3cluster
)
DeepCheck2(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r3)), q50r3, kmeans.3cluster
)
DeepCheck2(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r4)), q50r4, kmeans.3cluster
)
DeepCheck2(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r5)), q50r5, kmeans.3cluster
)
DeepCheck2(apphappy.labs.kcluster, q54, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q55, kmeans.2cluster)
DeepCheck2(apphappy.labs.kcluster, q56, kmeans.2cluster)

DeepCheck2(apphappy.labs.kcluster, q1, kmeans.3cluster)

```



```

deepTbl <- function(df, col1, cluster) {
  col1 <- enquo(col1)
  cluster <- enquo(cluster)
  df %>%
  select(
    !!col1,
    !!cluster
  ) %>%
  group_by(!!col1, !!cluster) %>%
  summarize(
    obs.count = n()
  ) %>%
  ungroup() %>%
  spread(
    key = !!cluster,
    value = obs.count,
    fill = 0
  )
}

# using 2 clusters
deepTbl(apphappy.labs.kcluster, q1, kmeans.2cluster) #age
# type of device
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r1)), q2r1, kmeans.2cluster) #
  iphone
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r2)), q2r2, kmeans.2cluster) #
  ipod touch
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r3)), q2r3, kmeans.2cluster) #
  android
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r4)), q2r4, kmeans.2cluster) #
  blackberry
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r5)), q2r5, kmeans.2cluster) #
  nokia
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r6)), q2r6, kmeans.2cluster) #
  windows phone
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r7)), q2r7, kmeans.2cluster) #
  Palm OS
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r8)), q2r8, kmeans.2cluster) #
  Tablet
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r9)), q2r9, kmeans.2cluster) #
  other
# type of app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r1)), q4r1, kmeans.2cluster) #
  music id app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r2)), q4r2, kmeans.2cluster) #
  tv check in app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r3)), q4r3, kmeans.2cluster) #
  entertainment app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r4)), q4r4, kmeans.2cluster) #
  tv show app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r5)), q4r5, kmeans.2cluster) #
  gaming app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r6)), q4r6, kmeans.2cluster) #
  social app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r7)), q4r7, kmeans.2cluster) #

```

```

    general news app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r8)), q4r8, kmeans.2cluster) #
    shopping app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r9)), q4r9, kmeans.2cluster) #
    specific news pub app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r10)), q4r10, kmeans.2cluster) #
    other
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r11)), q4r11, kmeans.2cluster) #
    none
deepTbl(apphappy.labs.kcluster, q11, kmeans.2cluster) #number of apps
# website visit frequency
deepTbl(apphappy.labs.kcluster, q13r1, kmeans.2cluster) # facebook
deepTbl(apphappy.labs.kcluster, q13r2, kmeans.2cluster) # twitter
deepTbl(apphappy.labs.kcluster, q13r3, kmeans.2cluster) # myspace
deepTbl(apphappy.labs.kcluster, q13r4, kmeans.2cluster) # Pandora
deepTbl(apphappy.labs.kcluster, q13r5, kmeans.2cluster) # Vevo
deepTbl(apphappy.labs.kcluster, q13r6, kmeans.2cluster) # YouTube
deepTbl(apphappy.labs.kcluster, q13r7, kmeans.2cluster) # AOL Radio
deepTbl(apphappy.labs.kcluster, q13r8, kmeans.2cluster) # Last.fm
deepTbl(apphappy.labs.kcluster, q13r9, kmeans.2cluster) # Yahoo Entertainment
deepTbl(apphappy.labs.kcluster, q13r10, kmeans.2cluster) # IMDB
deepTbl(apphappy.labs.kcluster, q13r11, kmeans.2cluster) # LinkedIn
deepTbl(apphappy.labs.kcluster, q13r12, kmeans.2cluster) # Netflix

deepTbl(apphappy.labs.kcluster, q12, kmeans.2cluster) # % free apps
deepTbl(apphappy.labs.kcluster, q48, kmeans.2cluster) #education level
deepTbl(apphappy.labs.kcluster, q49, kmeans.2cluster) # marital status
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r1)), q50r1, kmeans.2cluster) #
    no children
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r2)), q50r2, kmeans.2cluster) #
    children under 6
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r3)), q50r3, kmeans.2cluster) #
    kids 6-12
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r4)), q50r4, kmeans.2cluster) #
    kids 13-17
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r5)), q50r5, kmeans.2cluster) #
    kids 18+
deepTbl(apphappy.labs.kcluster, q54, kmeans.2cluster) #race
deepTbl(apphappy.labs.kcluster, q55, kmeans.2cluster) #latino?
deepTbl(apphappy.labs.kcluster, q56, kmeans.2cluster) #income
deepTbl(apphappy.labs.kcluster, q57, kmeans.2cluster) #gender

# using 3 clusters
deepTbl(apphappy.labs.kcluster, q1, kmeans.3cluster) #age
# type of device
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r1)), q2r1, kmeans.3cluster) #
    iphone
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r2)), q2r2, kmeans.3cluster) #
    ipod touch
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r3)), q2r3, kmeans.3cluster) #
    android
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r4)), q2r4, kmeans.3cluster) #
    blackberry
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r5)), q2r5, kmeans.3cluster) #
    nokia
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r6)), q2r6, kmeans.3cluster) #
    windows phone
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r7)), q2r7, kmeans.3cluster) #
    Palm OS

```

```

deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r8)), q2r8, kmeans.3cluster) #
  Tablet
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q2r9)), q2r9, kmeans.3cluster) #
  other
# type of app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r1)), q4r1, kmeans.3cluster) #
  music id app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r2)), q4r2, kmeans.3cluster) #
  tv check in app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r3)), q4r3, kmeans.3cluster) #
  entertainment app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r4)), q4r4, kmeans.3cluster) #
  tv show app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r5)), q4r5, kmeans.3cluster) #
  gaming app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r6)), q4r6, kmeans.3cluster) #
  social app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r7)), q4r7, kmeans.3cluster) #
  general news app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r8)), q4r8, kmeans.3cluster) #
  shopping app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r9)), q4r9, kmeans.3cluster) #
  specific news pub app
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r10)), q4r10, kmeans.3cluster) #
  other
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q4r11)), q4r11, kmeans.3cluster) #
  none
deepTbl(apphappy.labs.kcluster, q11, kmeans.3cluster) #number of apps
# visit frequency
deepTbl(apphappy.labs.kcluster, q13r1, kmeans.3cluster) # facebook
deepTbl(apphappy.labs.kcluster, q13r2, kmeans.3cluster) # twitter
deepTbl(apphappy.labs.kcluster, q13r3, kmeans.3cluster) # myspace
deepTbl(apphappy.labs.kcluster, q13r4, kmeans.3cluster) # Pandora
deepTbl(apphappy.labs.kcluster, q13r5, kmeans.3cluster) # Vevo
deepTbl(apphappy.labs.kcluster, q13r6, kmeans.3cluster) # YouTube
deepTbl(apphappy.labs.kcluster, q13r7, kmeans.3cluster) # AOL Radio
deepTbl(apphappy.labs.kcluster, q13r8, kmeans.3cluster) # Last.fm
deepTbl(apphappy.labs.kcluster, q13r9, kmeans.3cluster) # Yahoo Entertainment
deepTbl(apphappy.labs.kcluster, q13r10, kmeans.3cluster) # IMDB
deepTbl(apphappy.labs.kcluster, q13r11, kmeans.3cluster) # LinkedIn
deepTbl(apphappy.labs.kcluster, q13r12, kmeans.3cluster) # Netflix

deepTbl(apphappy.labs.kcluster, q12, kmeans.3cluster) # % free apps
deepTbl(apphappy.labs.kcluster, q48, kmeans.3cluster) #education level
deepTbl(apphappy.labs.kcluster, q49, kmeans.3cluster) # marital status
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r1)), q50r1, kmeans.3cluster) #
  no children
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r2)), q50r2, kmeans.3cluster) #
  children under 6
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r3)), q50r3, kmeans.3cluster) #
  kids 6-12
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r4)), q50r4, kmeans.3cluster) #
  kids 13-17
deepTbl(apphappy.labs.kcluster %>% filter(!grepl('NO TO', q50r5)), q50r5, kmeans.3cluster) #
  kids 18+
deepTbl(apphappy.labs.kcluster, q54, kmeans.3cluster) #race
deepTbl(apphappy.labs.kcluster, q55, kmeans.3cluster) #latino?
deepTbl(apphappy.labs.kcluster, q56, kmeans.3cluster) #income
deepTbl(apphappy.labs.kcluster, q57, kmeans.3cluster) #gender

```

```

apphappy.labs.kcluster %>%
ggplot(aes(x = q48)) +
geom_bar(stat = 'count', aes(fill = as.factor(kmeans.2cluster)), position = 'dodge') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
facet_grid(q57 ~ .) +
scale_fill_discrete(name = 'cluster')

apphappy.labs.kcluster %>%
ggplot(aes(x = q49)) +
geom_bar(stat = 'count', aes(fill = q57), position = 'dodge') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
facet_grid(kmeans.3cluster ~ .) +
scale_fill_discrete(name = 'gender')

# Heirarchical Clustering ----

fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "wss")
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "silhouette")

set.seed
a <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "wss") +
ggtitle('Scree Plot')
b <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "silhouette") +
ggtitle('Avg. Silhouette Width Plot')
grid.arrange(a,b, ncol=2)

hclust1 <- hclust(dist(att.num.df2 %>% select(-caseID)), method = 'ward.D2')
plot(hclust1, xlab = 'Observations', main = 'Dendrogram for Grouped Responses', cex = 0.6)
rect.hclust(hclust1, k=2, border=2:5)

cor(cophenetic(hclust1), dist(att.num.df2 %>% select(-caseID)))
hclust1.cut3 <- cutree(hclust1, k = 3)
table(hclust1.cut3)
plot(silhouette(hclust1.cut3, dist(att.num.df2 %>% select(-caseID))), border = NA)
summary(silhouette(hclust1.cut3, dist(att.num.df2 %>% select(-caseID))))

hclust1.cut2 <- cutree(hclust1, k=2)
table(hclust1.cut2)
plot(silhouette(hclust1.cut2, dist(att.num.df2 %>% select(-caseID))), border = NA)
summary(silhouette(hclust1.cut2, dist(att.num.df2 %>% select(-caseID))))

fviz_cluster(list(data = att.num.df2 %>% select(-caseID), cluster = hclust1.cut2), ellipse =
F)
fviz_cluster(list(data = att.num.df2 %>% select(-caseID), cluster = hclust1.cut3), ellipse.
type = 't')

#http://uc-r.github.io/hc_clustering
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "wss")
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = hcut, method = "silhouette")
#set.seed(2018)

```

```

#gap_stat <- clusGap(att.num.df %>% select(-caseID), FUN = hcut, nstart = 25, K.max = 10, B
= 50)
#fviz_gap_stat(gap_stat)

# methods to assess
m <- c( "average", "single", "complete", "ward")
names(m) <- c( "average", "single", "complete", "ward")
#the ward above is actually Ward.D2 in method

# function to compute coefficient
ac <- function(x) {
  agnes(att.num.df2 %>% select(-caseID), method = x)$ac
}

map_dbl(m, ac)
hc3 <- agnes(att.num.df2 %>% select(-caseID), method = "ward")
pltree(hc3, cex = 0.6, hang = -1, main = "Dendrogram of agnes")
#cutree(as.hclust(hc3), k = 3)
fviz_cluster(list(data = att.num.df2 %>% select(-caseID), cluster = cutree(as.hclust(hc3), k
= 3)), ellipse = F)
fviz_cluster(list(data = att.num.df2 %>% select(-caseID), cluster = cutree(as.hclust(hc3), k
= 2)), ellipse = F, geom = 'point', main = 'Cluster Plot using Hierarchical Clustering
with Ward Linkage')

# loop to determine best # of cuts
hdf <- tibble(x=1,y=0)
hclust1 <- hclust(dist(att.num.df2 %>% select(-caseID)), method = 'complete')
plot(hclust1)
for (i in 2:15) {
  hc <- cutree(hclust1, k = i)
  sk2 <- silhouette(hc, dist(att.num.df2 %>% select(-caseID)))
  hdf <- add_row(hdf, x=i, y=round(summary(sk2)$avg.width,3))
  print(paste0('cluster size: ', i, ' avg. silhouette width: ', round(summary(sk2)$avg.width
,3)))
  if(i == 15) {
    print(
      hdf %>%
      ggplot(aes(x = x, y = y)) +
      geom_line(col='blue') +
      geom_point(col='red') +
      ggtitle('Silhouette Average Widths') +
      xlab('Cluster Size') +
      ylab('Average Width')
    )
  }
}

plot(hclust1)
rect.hclust(hclust1, k=2, border="red")

# find WSS, TSS and BSS for heirarchical clustering
{
  library(proxy)
  hclust1.cut2 <- cutree(hclust1, k=2)
  hclust1.combo.df <- bind_cols(att.num.df, tibble(cluster = hclust1.cut2))

  # TSS
  TSS <- sum(
    dist(
      hclust1.combo.df %>%

```

```

select(-caseID, -cluster) %>%
map_df(function(x) mean(x))
,
hclust1.combo.df %>%
select(-caseID, -cluster)
)^2
)

# WSS
# cluster 1
dis1 <- sum(
dist(
hclust1.combo.df %>%
dplyr::filter(cluster == 1) %>%
select(-caseID, -cluster) %>%
map_df(function(x) mean(x))
,
hclust1.combo.df %>%
dplyr::filter(cluster == 1) %>%
select(-caseID, -cluster)
)^2
)

#cluster 2
dis2 <- sum(
dist(
hclust1.combo.df %>%
dplyr::filter(cluster == 2) %>%
select(-caseID, -cluster) %>%
map_df(function(x) mean(x))
,
hclust1.combo.df %>%
dplyr::filter(cluster == 2) %>%
select(-caseID, -cluster)
)^2
)

WSS <- sum(dis1, dis2)
BSS <- TSS - WSS
rsquare <- BSS/TSS
rsquare
}

# PAM Clustering ----
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = pam, method = "wss")
fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = pam, method = "silhouette")

a <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = pam, method = "wss") + ggtitle('
  Scree Plot - PAM')
b <- fviz_nbclust(att.num.df2 %>% select(-caseID), FUN = pam, method = "silhouette") +
  ggtitle('Avg. Silhouette Width - PAM')
grid.arrange(a,b,ncol=2)

#pam.2clust
set.seed(2018)
pam.2clust <- pam(att.num.df2 %>% select(-caseID), k=2)
fviz_cluster(pam.2clust, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse.
  type = 't', repel = T)

```

```

fviz_cluster(pam.2clust, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse = F
) + ggtitle('2 Cluster PAM')

#pam.3clust
set.seed(2018)
pam.3clust <- pam(att.num.df2 %>% select(-caseID), k=3)
fviz_cluster(pam.3clust, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse.
  type = 't', repel = T)
fviz_cluster(pam.3clust, data = att.num.df2 %>% select(-caseID), geom = 'point', ellipse = F
)

# compare clusters
cluster.stats(dist(att.num.df2 %>% select(-caseID)), pam.2clust$clustering, pam.3
  clust$clustering)$corrected.rand
plot(silhouette(klust.2c$cluster, (dist(att.num.df2 %>% select(-caseID)))^2), col = 1:2,
  border = NA)
plot(silhouette(pam.2clust$clustering, (dist(att.num.df2 %>% select(-caseID)))^2), col =
  1:2, border = NA)
plot(silhouette(klust.3c$cluster, (dist(att.num.df2 %>% select(-caseID)))^2), col = 1:3,
  border = NA)
plot(silhouette(pam.3clust$clustering, (dist(att.num.df2 %>% select(-caseID)))^2), col =
  1:3, border = NA)

# Model Based Clustering ----
set.seed(2018)
mclust.fit <- Mclust(att.num.df2 %>% select(-caseID), 2)
plot(mclust.fit, data=numsub, what="density") # plot results (does not work)
summary(mclust.fit) # display the best model

mclustBIC(att.num.df2 %>% select(-caseID))

mclust.fit1 <- Mclust(att.num.df2 %>% select(-caseID))
summary(mclust.fit1)
mclust.fit2 <- Mclust(att.num.df2 %>% select(-caseID), G = 3)
summary(mclust.fit2)

BIC(mclust.fit1, mclust.fit2)

seg.summ <- function(data, groups) {
  aggregate(data, list(groups), function(x) mean(as.numeric(x)))
}

seg.summ(att.num.df2 %>% select(-caseID), mclust.fit1$classification)

clusplot(att.num.df2 %>% select(-caseID), mclust.fit1$classification, color=TRUE, shade=TRUE
  , labels=4, lines=0, main="Model-based cluster plot")

fviz_mclust(mclust.fit1, what = 'classification')
fviz_mclust(mclust.fit2, what = 'classification')

fviz_mclust_bic(mclust.fit1)

```

Introduction

In this assignment, a cleansed dataset consisting of 424 respondents' information was used to construct different Hierarchical Bayes Multinomial Logit (HB-MNL) models. The intent of this assignment is to help Obee Juan, a product development manager at Star Technologies Company (STC), to estimate preference shares for two different choice scenarios that are different from the initial choice scenarios studied. The conclusion of this analysis will also contain a recommended tablet configuration. For the extra scenarios, a developed model was used to predict the alternative that would be picked by the respondents. Since STC is not in the business of the computer tablet market, it is essential that the company identify a winning combination of tablet specs and price to ensure a successful market debut.

Data Overview

Two key datasets were provided by the Neverending Marketing Insights (NMI) company. One was a choice based conjoint task plan (this consisted of 108 different combinations) and a dataset consisting of 424 respondents' responses for their chosen preference (based on the alternatives presented) as well as the respondents' demographic information. The task plan (i.e., choice set) consisted of six different attributes and a total of 36 choice sets were provided:

- brand (STC, Somesong, Pear, Gaggle)
- price (\$199, \$299, \$399)
- screen (5", 7", 10")
- RAM (8 GB, 16 GB, 32 GB)
- processor (1.5 GHz, 2 GHz, 2.5 GHz)
- interaction between brand and price

As for the survey, the respondents were given three alternatives to pick from within each choice set. In addition, they were also asked to provide demographic information such as age, gender, income level, children, parental status, resident state, and their interest level on the following:

- purchasing a new tablet
- purchasing a new mobile phone
- using cloud storage
- taking an online course

Exploratory Data Analysis

A comprehensive exploratory data analysis was also completed (A small sample of the charts constructed can be found in the Appendix). Recall that the dataset provided for analysis was cleansed. The original dataset consisted of 1024¹ observations, however, the analyzed dataset consisted of only 424 observations (i.e., respondents). It was found that the respondents were equally distributed in terms of gender (see Table 8.1). In terms of age, the majority of the respondents were aged 25 and older (see Figure 8.1). Approximately 44% of the respondents were between the ages of 25 and 44. In terms of income (see Figure 8.2), over 55% of the respondents had an income of \$75,000 or less. It was also found that approximately 65% of the respondents had at least one or more child (see Table 8.2).

¹This value was provided in a sync session to the author of this report and cannot be found in the supporting documentation provided for this assignment

In terms of the interest in purchasing a tablet, it was found that more females were inclined² to purchase a tablet (see Figure 8.3). It was also discovered that respondents between the ages of 25 and 44 were more interested to purchase a tablet than older respondents (aged 55 and older). A similar conclusion was found in terms of interest in purchasing a smartphone, interest in using cloud storage. Interestingly, it was found that both genders did not have a strong interest in taking online courses (see Figure 8.4). Furthermore, older respondents were less interested in online courses than younger respondents.

Data Preprocessing

A number of steps were undertaken to prepare the datasets for analysis. This included recoding the responses with the appropriate 'dummy' variables for use in a Multinomial Logit model. Furthermore, a new set of variables were constructed to document the interaction between brand and price. Ultimately, the resultant matrices were incorporated into a nested list so that each respondent's responses could be paired up with the appropriate choice set matrix (see the section R Code in the Appendix). Table 8.3 in the Appendix documents the variables that were constructed. Note that since this is a multinomial logit problem, the following equation syntax was used:

$$E(Y) = \log\left(\frac{p}{1-p}\right) = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_{14} X_{14} \quad (3.1)$$

Note that β_i is the coefficient and X_i is the dummy variable and i is the variable number. For this analysis, the β coefficient values will be derived.

Since there is a need to create dummy (i.e., indicator) variables, the baseline variables will be screen5 (referring to the 5" screen), ram8 (referring to the 8 GB RAM), proc1.5 (referring to the 1.5 GHz processor), brand.stc (referring to the STC brand), and brand.price.stc (referring to the interaction between price and the STC brand). These variables have been coded as -1 in the appropriate choice set matrix (these values are also known as the preferences). This will enable the ability to derive β coefficients for these baseline variables.

Model Development

Two models were constructed: one without any covariates such as prior ownership of an STC product and another with the covariate (prior ownership of an STC product). Both of these models are HB-MNL³ models. Furthermore, both models made use of Markov Chain Monte Carlo (MCMC) simulations leveraging 150,000 iterations and keeping every 5th simulated sample. In order to construct these models, the package "bayesm" was used and the function within that package was "rhierMnIDP()". Each model was assessed with a confusion matrix (illustrating the predicted responses versus the actual observed responses) and an area under the curve (AUC) value (assessing the accuracy in terms of sensitivity vs specificity). This discussion can be found in the Model Results & Conclusion section below.

Model 1: No covariate

The intent of the simulated values (using MCMC) is to derive the posterior β values for each predictor and each respondent. The mean of the simulated values is then used to derive the posterior β means. This in turn is then matrix-multiplied with the original choice set matrix, the product is then exponentiated, and then finally divided by the row sums to obtain the most likely (i.e., predicted) choice.

²The interest was based on a rating of 1 - 10 where 10 meant most interested and 1 meant least interested

³Hierarchical Bayes Multinomial Logistic

One of the requirements during the analysis of MCMC values is to "make sure that the algorithm produces a Markov chain that converges to the appropriate density (the posterior density) and that mixes well throughout the support of the density" (Lynch, 2007). One way to assess mixing and convergence is to analyze the trace plot for the simulation. Due to the number of retained samples (30,000), a trace plot using the cumulative means was constructed. Note that respondent 23 was arbitrarily chosen to assess the mixing and convergence.

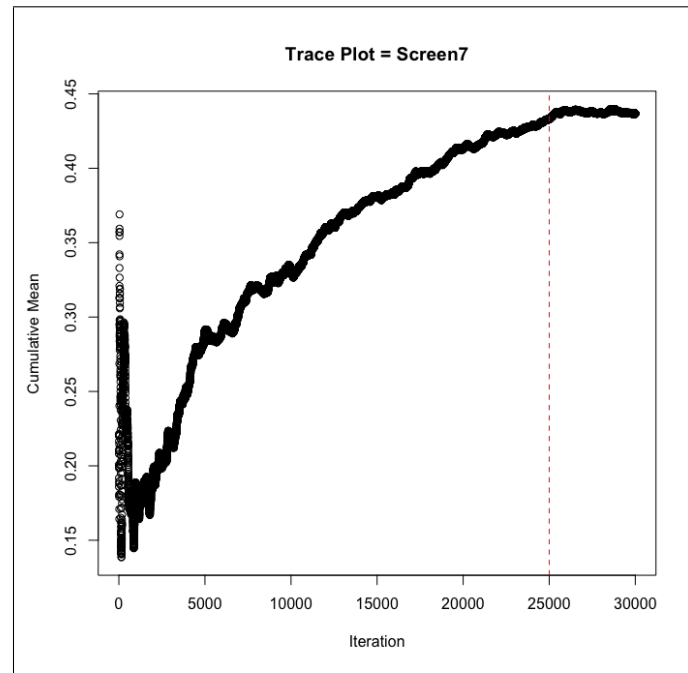


Figure 4.1: Cumulative Mean Trace Plot - Respondent 23, Screen7

In Figure 4.1, the red dotted line suggests that convergence was found around 25,000 iterations. Note how the plot enters a steady-state around the same point. Similar plots were constructed for all predictors for respondent 23. It is essential to note that not every variable converged at the same iteration (see Figure 8.5 - which seems to converge at around iteration 15,000). For the sake of brevity, the burn-in period was assumed to be 25,000 and the remaining 5,000 samples were preserved for analysis.

Typically, the average β values for each respondent was calculated from the simulated β values. Thus, each of the 424 respondents had their own respective HB-MNL model. For simplicity's sake, it was assumed that all respondents are homogeneous. Thus, the overall average β values were calculated. Table 4.1 summarizes the average β coefficient values along with the respective odds ratio.

Table 4.1: Model 1 - Overall Beta Coefficient Summary

Variable	Beta Coefficient	Odds Ratio
screen5	-0.265	0.767
screen7	-0.185	0.831
screen10	0.45	1.57
ram8	-0.731	0.482
ram16	0.0864	1.09
ram32	0.644	1.9
proc1.5	-2.35	0.0951
proc2	1.02	2.78
proc2.5	1.33	3.78
price199	2.62	13.7
price299	0.314	1.37
price399	-2.93	0.0535
brand.stc	0.424	1.53
brand.somesong	-0.202	0.817
brand.pear	0.0822	1.09
brand.gaggle	-0.304	0.738
brand.price.stc	-0.114	0.893
brand.price.somesong	0.0465	1.05
brand.price.pear	0.055	1.06
brand.price.gaggle	0.0122	1.01

The Beta Coefficients in Table 4.1 can be written out as follows (recall that this is the log odds equation):

$$\begin{aligned}
 E(Y) &= \log\left(\frac{p}{1-p}\right) = \beta_1 * X_1 + \beta_2 * X_2 + \dots + \beta_n * X_n \\
 E(Y) &= screen5 * X_1 + screen7 * X_2 + \dots + brand.price.gaggle * X_{20} \\
 E(Y) &= -0.265 * X_1 + -0.185 * X_2 + \dots + 0.0122 * X_{20}
 \end{aligned}
 \tag{4.1}$$

Results in Table 4.1 can be interpreted fairly easily. For instance, screen 10 is preferred 1.57 times versus no preference. In other words, the probability of preferring a 10" screen is 157% higher than no preference. Note how dominant the price of \$199 is at 13.7 times versus no preference.

Another approach is to compare two similarly categorized options together. For instance, the following equation can be used to help understand how much more (or less) a 10" screen is compared to a 5" screen.

$$\begin{aligned}
 Screen10 - Screen7 &= 0.45 - (-0.185) = 0.635 \\
 e^{0.635} &= oddsratio = 1.89
 \end{aligned}
 \tag{4.2}$$

This means that a 10" screen is preferred 1.89 times more than a 7" screen. A similar approach could be used to compare other options with respective categories.

The variables screen5, ram8, proc1.5, price199, brand.stc, brand.price.stc were not derived directly since the effects coded version of the attributes was developed in the pre-processing stage. The β values for these variables was derived by subtracting the β values for the other respective indicator variables from zero. For instance, the screen5 β coefficient value was derived using the following equation:

$$screen5 = 0 - screen7 - screen10 = 0 - (-0.185) - (0.45) \tag{4.3}$$

The interaction between brand and price is interpreted slightly differently. Essentially, the interaction between price and brand is dependent on the preference. Recall from earlier that the preference can be coded as -1 (preferring the baseline), 0 (preferring the next best choice above the baseline), and 1 (preferring the best choice). For example, if the preference is the baseline and the comparison is between the brands STC and Somesong, then the log odds can be calculated as follows:

$$\begin{aligned} \log odds &= \text{brand.stc} - \text{brand.somesong} + (\text{brand.price.stc} + \text{brand.price.somesong}) * \text{preference} \\ \log odds &= 0.424 - (-0.202) + (-0.114 - 0.0465) * -1 \quad (4.4) \\ \log odds &= 0.6935 \end{aligned}$$

Taking the exponential of the logodds ($\exp(0.7865)$) results in an odd ratio of 2.20. In other words, the STC brand is preferred 2.2 times to Somesong if the price is at \$199. Table 8.4 summarizes the odds ratio for the different brands and prices (interaction effects). Note how the brand STC is preferred at all price points to other brands. However, STC is strongly preferred at the lowest price point as compared to other brands at different pricing levels.

Model 1 predicted values can be found in the Appendix under Table 8.6. Note how the predicted number of respondents picking each alternative within each choice set is quite similar to the actual values.

Model 2: Covariate (prior ownership)

For this model, the focus was on understanding how consumer preference may be affected given that particular respondents have previously owned an STC product. Following a similar approach to what was done in Model 1, the only addition was a matrix indicating whether a respondent had owned an STC product or not. In order to maintain consistency with Model 1, this model was also run with 150,000 iterations and keeping every five samples. Furthermore, the last 5,000 samples were kept.

One interesting output from this new model is the "Delta Draw". This dataset essentially describes the posterior distributions of the regression coefficients mean-centered on prior ownership of an STC product. Table 4.2 illustrates the average delta-draw values for each of the variables.

Table 4.2: Average Delta Draw Values

Variable	Average
screen7	-0.0462831
screen10	-0.04571877
ram16	0.08536959
ram32	0.02684507
proc2	0.25485435
proc2.5	0.39755898
price299	-0.01887764
price399	-0.68340868
brand.somesong	-0.23349499
brand.pear	0.95924565
brand.gaggle	-0.05984522
brand.price.somesong	-0.17979772
brand.price.pear	0.05361792
brand.price.gaggle	0.16931045

Loosely put, the average values could be construed as the correlations between prior ownership of an STC product and the appropriate variables. Note how the average values for both a 7" and a 10" screen are

around -0.04. This suggests that there is not much correlation between screen size and prior ownership of an STC product. On the other hand, note the stronger positive relationship between the two different types of processor and prior ownership of an STC product. Another interesting note is the strong positive relationship (0.96) of the Pear brand with prior ownership of an STC product.

Similar to Table 4.1, Table 4.3 illustrates the average coefficient values for each variable. The interpretation for this table is made in a similar way as was done for Table 4.1. For instance, the 10" screen is preferred 1.51 times versus no preference. In other words, the probability of preferring a 10" screen is 151 times higher than no preference. Furthermore (leveraging equation 4.2), a 10" screen is preferred 1.78 times more than a 7" screen. Note that the syntax for Equation 4.1 can be used for Table 4.3.

Table 4.3: Model 2 - Overall Beta Coefficient Summary

Variable	Beta Coefficient	Odds Ratio
screen5	-0.248	0.78
screen7	-0.164	0.848
screen10	0.412	1.51
ram8	-0.671	0.511
ram16	0.0934	1.1
ram32	0.578	1.78
proc1.5	-2.33	0.0971
proc2	1.08	2.95
proc2.5	1.25	3.49
price199	2.62	13.8
price299	0.292	1.34
price399	-2.91	0.0543
brand.stc	0.413	1.51
brand.somesong	-0.23	0.795
brand.pear	0.109	1.11
brand.gaggle	-0.292	0.747
brand.price.stc	-0.0836	0.92
brand.price.somesong	0.0973	1.1
brand.price.pear	0.0147	1.01
brand.price.gaggle	-0.0284	0.972

Finally, Table 8.5 summarizes the odds ratio of the different brands and prices (interaction effects) for model 2. Note how the STC brand is preferred 2.28 times to Somesong if the price is at \$199. Comparing the odds ratio of Tables 8.4 and 8.5, it is clear to see that both models are very similar in their results.

Model 2 predicted values can be found in the Appendix under Table 8.6. Note how the predicted number of respondents picking each alternative within each choice set is quite similar to the actual values as well as the predicted results from Model 1.

Model Results & Conclusion

Recall that Table 8.6 in the Appendix provides a summarization of the predicted count of respondents choosing a particular alternative for each choice set. In order to determine how well each model's fit is, two key assessments were used to assess each model: confusion matrix and an area under the curve (AUC) metric. Table 4.4 is the confusion matrix for model 1. A perfect model (which is also a good indicator of an overfitted model) will have non-zero values on diagonal and zero values everywhere else. In reality, the diagonal should have the highest numbers while the non-diagonal parts should have the least number of observations. As an example, 3,679 observations were correctly predicted to be alternative 1 given that they were actually alternative 1. However, the model also predicted that 258 were to be alternative 2 and

315 were to be alternative 3. In reality, both were actually alternative 1. The overall accuracy for the model 1 (using the confusion matrix) is 0.883 or 88.3%. The AUC for alternative 1 is about 0.866 (which is the area under the curve found in the upper left corner of figure 8.6). Overall, the multi-class AUC is at 0.9074 for model 1.

Table 4.4: Model 1: Confusion Matrix

Modeled Response	Actual Response		
	1	2	3
1	3679	419	215
2	234	3754	204
3	291	416	6052

Similarly, Table 4.5 is the confusion matrix for model 2 (which utilizes the covariate of prior STC device ownership). The overall accuracy of the confusion matrix is 0.872 or about 87.2% The individual class AUC's can be found in figure 8.7 in the Appendix.

Table 4.5: Model 2: Confusion Matrix

Modeled Response	Actual Response		
	1	2	3
1	3631	477	217
2	258	3659	226
3	315	453	6028

The multi-class AUC for model 2 is at 0.9001. The key takeaway is that both models are quite similar to each other. Model 2 (with the covariate), however, performs slightly lower (in terms of accuracy) as compared to Model 1.

Another conclusive chart is the parts worth plot (see Figure 4.2). This plot essentially illustrates the average coefficient values (seen in Table 4.1) and plots them. From this plot, it is much more evident to see which coefficients have strong preferences.

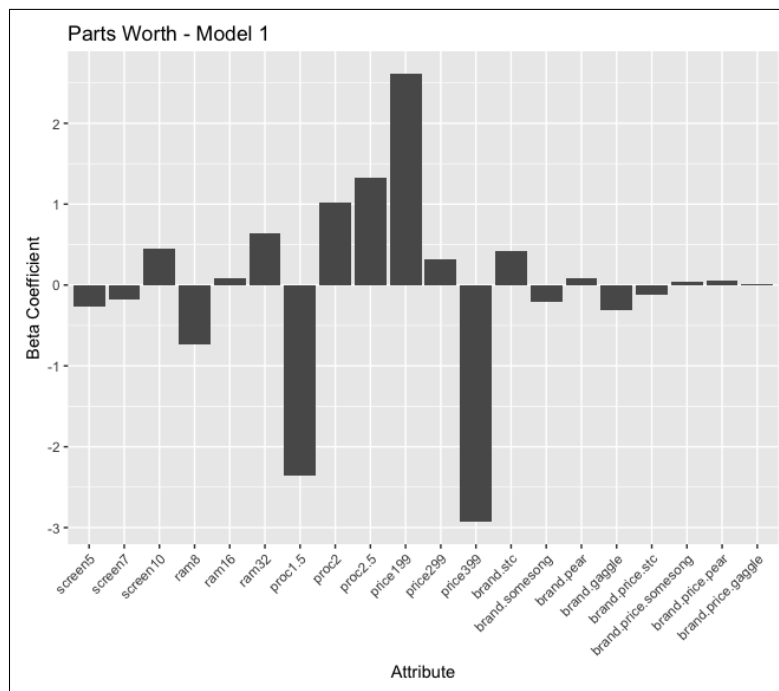


Figure 4.2: Parts Worth - Model 1

In the figure above, it is clear to see that a 10" screen is preferred along with 32 GB RAM, with either a 2 GHz or 2.5 GHz processor and a preferred price point of \$199 or \$299 as well as a preference for the brand STC. The aforementioned attributes could also serve as potential designs for the new tablet. The parts worth plot for model 2 can be found in the Appendix - see Figure 8.8. Note the similarity of this plot and Figure 4.2. It is also important to note how the interaction between brand and price has very little preference. In other words, price sensitivity does not necessarily vary over the brands in a meaningful way.

In order to calculate the importance of each attribute, a programmer can simply take the absolute value for each coefficient found in Table 4.1 or Table 4.3 and divide it by the sum of the absolute values of all the coefficients within the respective table. This step was not completed for this analysis, but is described here as a potential next step opportunity.

Extra Scenarios

Two additional choice sets were also provided for this analysis. Information about the additional choice sets can be found within the dataset that was provided for this analysis (and is not included in this paper for brevity's sake). Model 1 was used to help predict which alternative most respondents would pick for both choice set. Recall that Model 1 does not make use of the covariate related to the prior ownership of an STC product.

A voting technique was used to determine the most likely alternative. In other words, the Model 1 approach was used - where 424 models were dynamically leveraged to understand how each of the individual respondents would respond. Then the most preferred alternative for each respondent was leveraged and then tallied to understand how many respondents would pick a particular alternative. This is in contrast to using the overall coefficient means that were documented in Table 4.1. Results of this voting approach can be found in Table 5.1 below.

Table 5.1: Extra Scenario Voting Results

Choice Set	Alternative 1	Alternative 2	Alternative 3
1	92	232	100
2	99	236	89

Note how the majority of the respondents would likely choose alternative 2. Alternative 2 for choice set 1 corresponds to a tablet with a 10" screen, 2 GHz processor, 32 GB RAM, \$199 price, and the STC brand. On the other hand, alternative 2 for choice set 2 corresponds to a tablet with a 5" screen, 16 GB RAM, 1.5 GHz processor, \$199 price, and the Gaggie brand.

Limitations & Assumptions

For this analysis, some key assumptions had to be made. One key assumption is that the population of respondents is considered to be homogeneous. We also assume that the population of respondents are like-minded and that they have a linear thought process. In reality, there is a very good likelihood that some of the thought processes are not as linear. Furthermore, during the model development phase, not every variable converged after 150,000 iterations. There is also no awareness in this dataset if the respondent was in a hurry or after a certain point was disinterested in the survey.

Another limitation of this study is the lack of training, validation, and test datasets. Typically, the respondents would be randomly split into the three different aforementioned datasets. This would help to improve the model design as well as reduce the likelihood of having an overfitted model. Another key point is the feasibility of certain configurations. For instance, manufacturing costs, research & development costs, etc. are not known and it is difficult to recommend a configuration with full confidence since its feasibility may seem impossible.

In today's competitive tablet market, STC may want to consider updating the configuration options and redoing the assessment. For instance, many modern smartphones sport a display that is 5" or more. STC may want to consider investigating tablets with a 7", 10" or 12" screen. Furthermore, STC may want to explore the operating system (OS) as well. Although Android is a popular option for 3rd party OEM vendors, STC could consider⁴ going with its own OS or implementing Windows. Finally, STC may also want to explore refining their terminology regarding memory and expanding the memory size. For instance, STC referred to the disk space as RAM. As of right now, it is technically not feasible to have 32 GB RAM in a portable tablet form. Rather, STC may want to rename the term to disk space such as 32 GB SSD. Furthermore, today's tablets are sporting up to 256 GB or more. STC may want to consider upping the maximum and minimum disk spaces.

Conclusion & Next Steps

Two Hierarchical Bayes Multinomial Logit (HB-MNL) models were developed for this analysis: one utilizing the prior ownership of an STC product and one not. Both models were explored to understand price sensitivity on brand and each attribute assessed to understand the respondents' preference.

Evaluating the different beta coefficient values, it was found (in Model 1) that the interaction between brand and price was virtually negligible. Furthermore, respondents were inclined to prefer a tablet that had a 10" screen, 32 GB RAM, a 2.5 GHz processor, and a \$199 price point. From the original 36 choice sets, the most popular configuration chosen consists of a 7" screen, 16 GB RAM, 2.5 GHz processor, price point of \$199, and made by STC. If the two extra scenarios are also considered, the preferred configuration stays the same.

⁴Apple's iOS is not an option as it is a proprietary OS that is limited to devices engineered and produced by Apple only.

When utilizing the prior ownership of an STC product (i.e., the covariate), similar results were found. However, when looking at the average delta draw values, it was interesting to note that the respondents (who had owned an STC product previously) preferred the brand Pear. This is a good indicator that STC may wish to invest in advertising and marketing strategies to help convince consumers on the values of owning and using an STC branded tablet versus one from Pear.

As a next step, STC may want to look at gender and household income as covariates. This would enable a deeper understanding whether different genders prefer different types of configurations. Furthermore, it will also help STC understand how different household income levels could affect respondent proclivities. On the other hand, STC could also investigate the development of two different types of models: one that focuses on respondents that have had a prior STC product and another model that focuses on respondents that do not and have never owned an STC product.

References

Lynch, S. M. (2007). *Introduction to applied bayesian statistics and estimation for social scientists*. New York: Springer.

Appendix

EDA

Table 8.1: Frequency Count - Gender

	Count	Percent
Male	214	0.505
Female	210	0.495

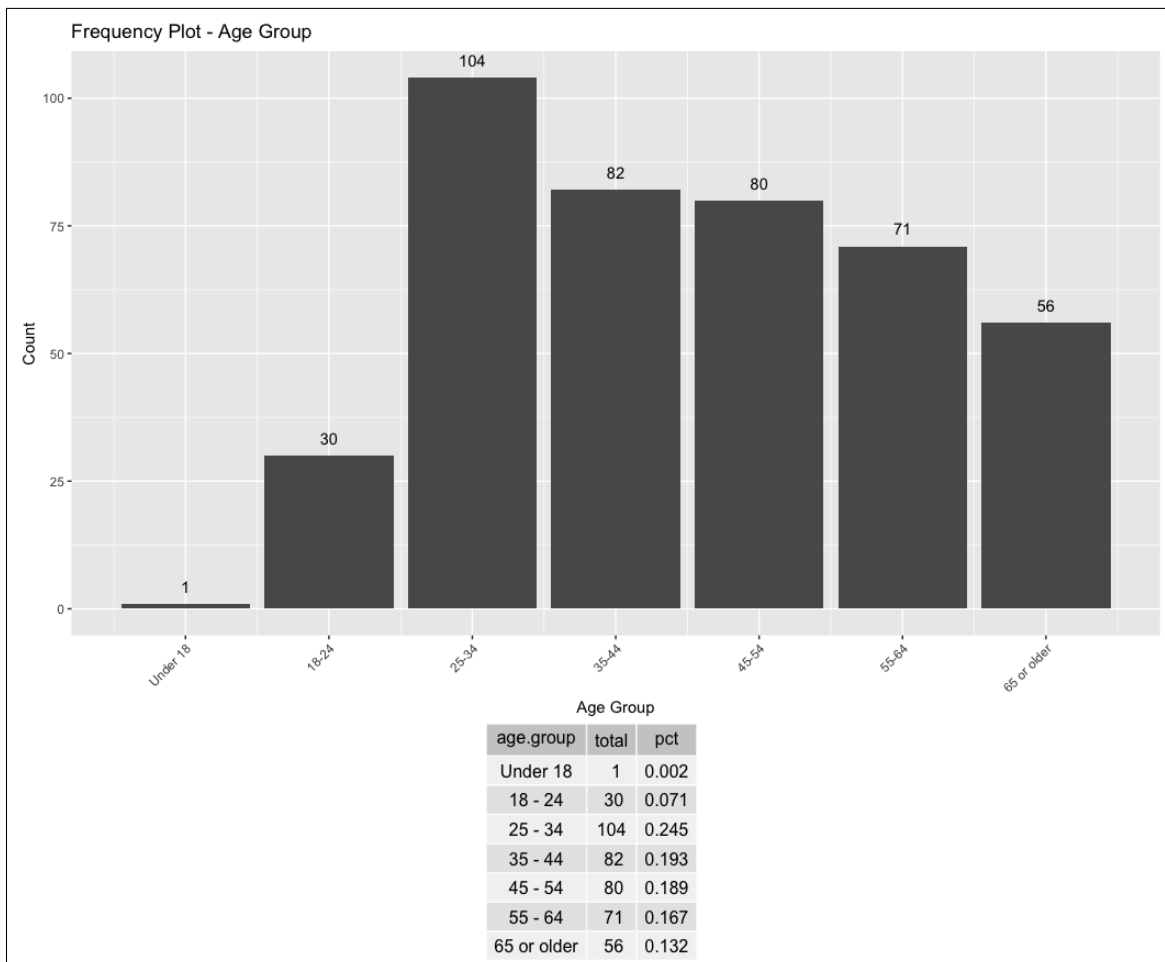


Figure 8.1: Frequency Plot & Table: Age Group

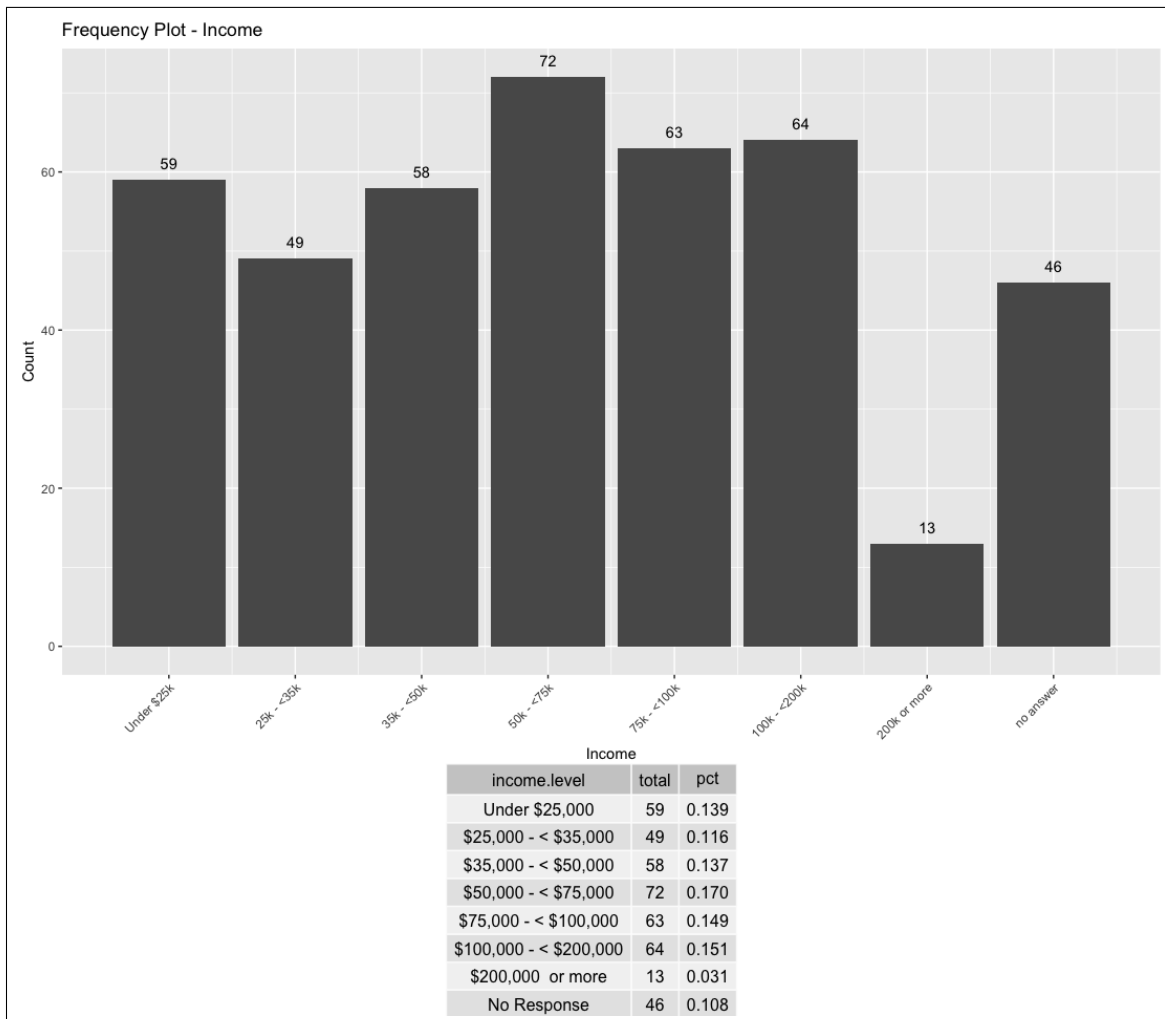


Figure 8.2: Frequency Plot & Table: Income Level

Table 8.2: Frequency Count - Have Children

Have Kids?	Count	Percent
Yes	277	0.653
No	147	0.347

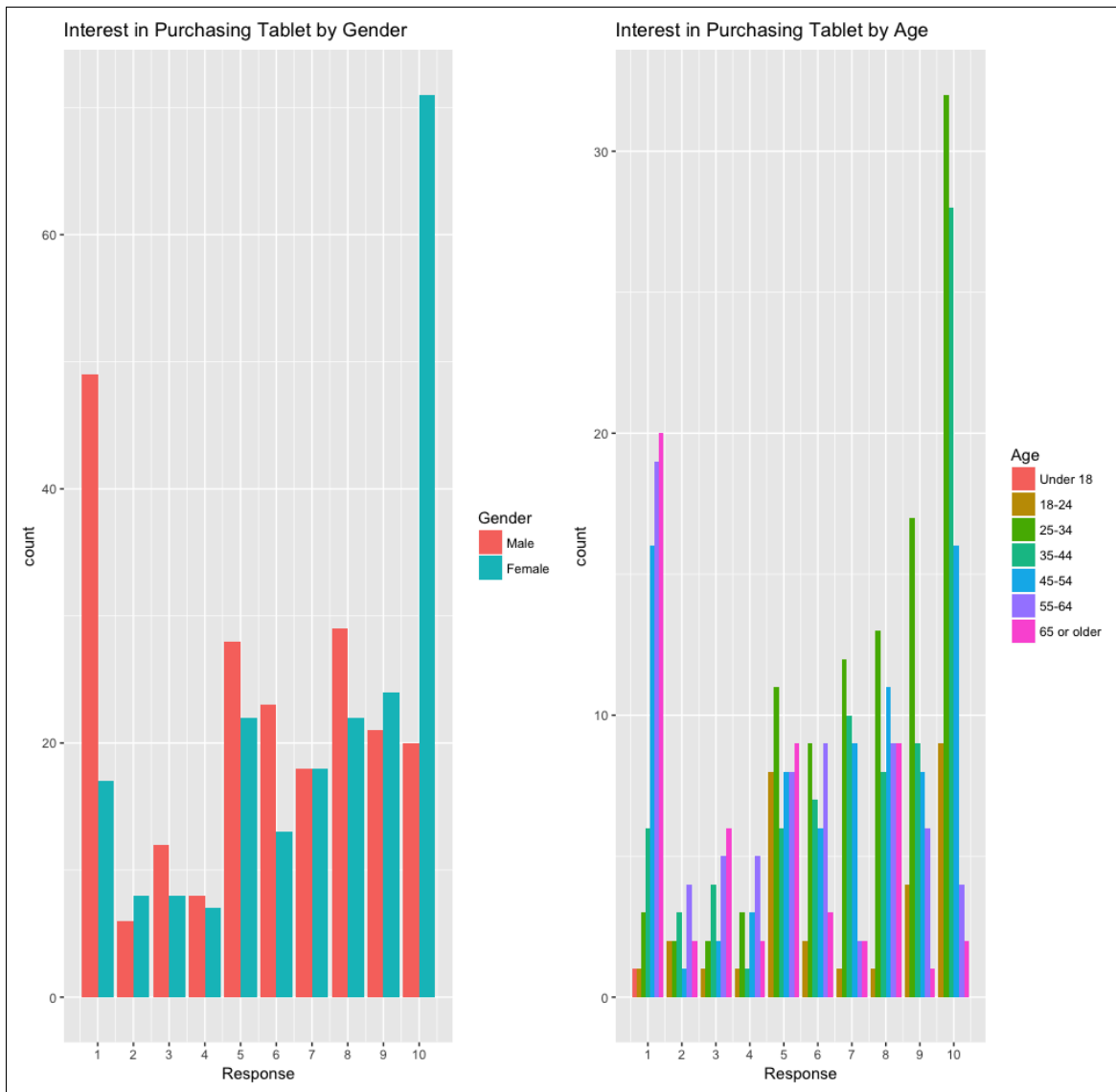


Figure 8.3: Likelihood of Purchasing Tablet by Gender & Age

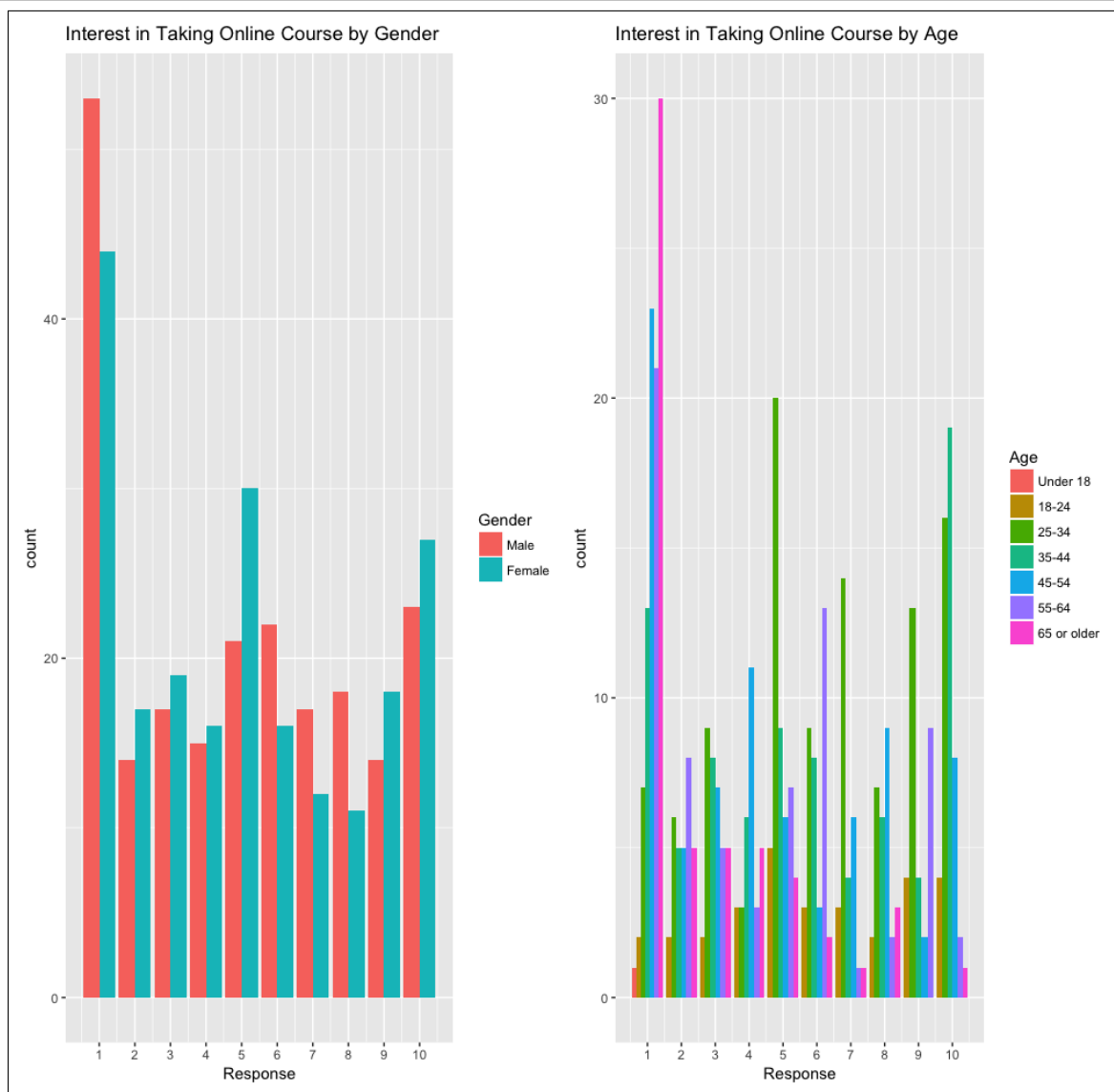


Figure 8.4: Interest in Taking Online Course by Gender & Age

Data Preprocessing

Table 8.3: List of Variables

Variable Name	Variable
screen7	X_1
screen10	X_2
ram16	X_3
ram32	X_4
proc2	X_5
proc2.5	X_6
price299	X_7
price399	X_8
brand.somesong	X_9
brand.pear	X_{10}
brand.gaggle	X_{11}
brand.price.somesong	X_{12}
brand.price.pear	X_{13}
brand.price.gaggle	X_{14}

Model 1

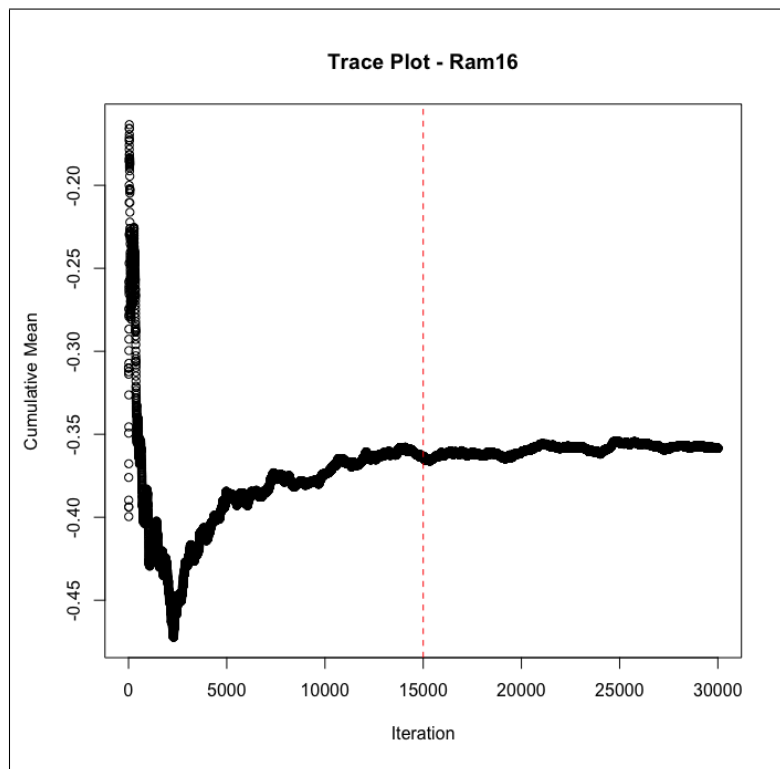


Figure 8.5: Cumulative Mean Trace Plot - Respondent 23, RAM16

Table 8.4: Odds Ratio Table for Interaction Variables in Model 1

Price	Preference	brand.stc	brand.somesong	brand.price.stc	brand.price.somesong	Log Odds	Odds Ratio
199	-1	0.424	-0.202	-0.114	0.0465	0.7865	2.20
299	0	0.424	-0.202	-0.114	0.0465	0.626	1.87
399	1	0.424	-0.202	-0.114	0.0465	0.4655	1.59

Price	Preference	brand.stc	brand.pear	brand.price.stc	brand.price.pear	Log Odds	Odds Ratio
199	-1	0.424	0.0822	-0.114	0.055	0.5108	1.67
299	0	0.424	0.0822	-0.114	0.055	0.3418	1.41
399	1	0.424	0.0822	-0.114	0.055	0.1728	1.19

Price	Preference	brand.stc	brand.gaggle	brand.price.stc	brand.price.gaggle	Log Odds	Odds Ratio
199	-1	0.424	-0.304	-0.114	0.0122	0.8542	2.35
299	0	0.424	-0.304	-0.114	0.0122	0.728	2.07
399	1	0.424	-0.304	-0.114	0.0122	0.6018	1.83

Model 2

Table 8.5: Odds Ratio Table for Interaction Variables in Model 2

Price	Preference	brand.stc	brand.somesong	brand.price.stc	brand.price.somesong	Log Odds	Odds Ratio
199	-1	0.413	-0.23	-0.0836	0.0973	0.8239	2.28
299	0	0.413	-0.23	-0.0836	0.0973	0.643	1.90
399	1	0.413	-0.23	-0.0836	0.0973	0.4621	1.59

Price	Preference	brand.stc	brand.pear	brand.price.stc	brand.price.pear	Log Odds	Odds Ratio
199	-1	0.413	0.109	-0.0836	0.0147	0.4023	1.50
299	0	0.413	0.109	-0.0836	0.0147	0.304	1.36
399	1	0.413	0.109	-0.0836	0.0147	0.2057	1.23

Price	Preference	brand.stc	brand.gaggle	brand.price.stc	brand.price.gaggle	Log Odds	Odds Ratio
199	-1	0.413	-0.292	-0.0836	-0.0284	0.7602	2.14
299	0	0.413	-0.292	-0.0836	-0.0284	0.705	2.02
399	1	0.413	-0.292	-0.0836	-0.0284	0.6498	1.92

Model Results

Table 8.6: Predicted Results for Models 1 and 2

Choice Set	Actual Results			Model 1 Predicted Results			Model 2 Predicted Results		
	Alternative 1	Alternative 2	Alternative 3	Alternative 1	Alternative 2	Alternative 3	Alternative 1	Alternative 2	Alternative 3
1	93	242	89	86	247	91	87	250	87
2	316	45	63	342	31	51	345	30	49
3	218	180	26	238	166	20	242	163	19
4	125	28	271	111	15	298	110	13	301
5	157	132	135	183	111	130	190	107	127
6	70	83	271	69	61	294	70	53	301
7	81	182	161	73	191	160	77	189	158
8	65	85	274	59	70	295	60	65	299
9	100	21	303	106	12	306	105	7	312
10	25	307	92	32	302	90	32	305	87
11	271	113	40	301	95	28	306	91	27
12	192	217	15	192	219	13	199	214	11
13	77	75	272	61	68	295	54	71	299
14	151	148	125	174	128	122	176	128	120
15	18	140	266	20	122	282	16	115	293
16	65	207	152	47	217	160	46	220	158
17	22	130	272	17	119	288	16	115	293
18	64	68	292	58	62	304	55	64	305
19	41	276	107	35	279	110	34	286	104
20	303	42	79	330	27	67	330	28	66
21	225	147	52	262	114	48	275	99	50
22	114	25	285	105	13	306	109	11	304
23	183	88	153	211	56	157	211	53	160
24	37	88	299	29	77	318	30	75	319
25	71	177	176	53	185	186	51	190	183
26	32	103	289	36	79	309	33	84	307
27	109	19	296	96	8	320	95	8	321
28	34	308	82	36	309	79	32	313	79
29	312	67	45	327	54	43	331	49	44
30	189	209	26	214	188	22	222	181	21
31	88	43	293	78	31	315	71	30	323
32	164	121	139	177	103	144	181	97	146
33	27	114	283	22	90	312	20	87	317
34	44	207	173	33	218	173	26	225	173
35	35	116	273	25	99	300	24	101	299
36	86	36	302	75	26	323	64	26	334

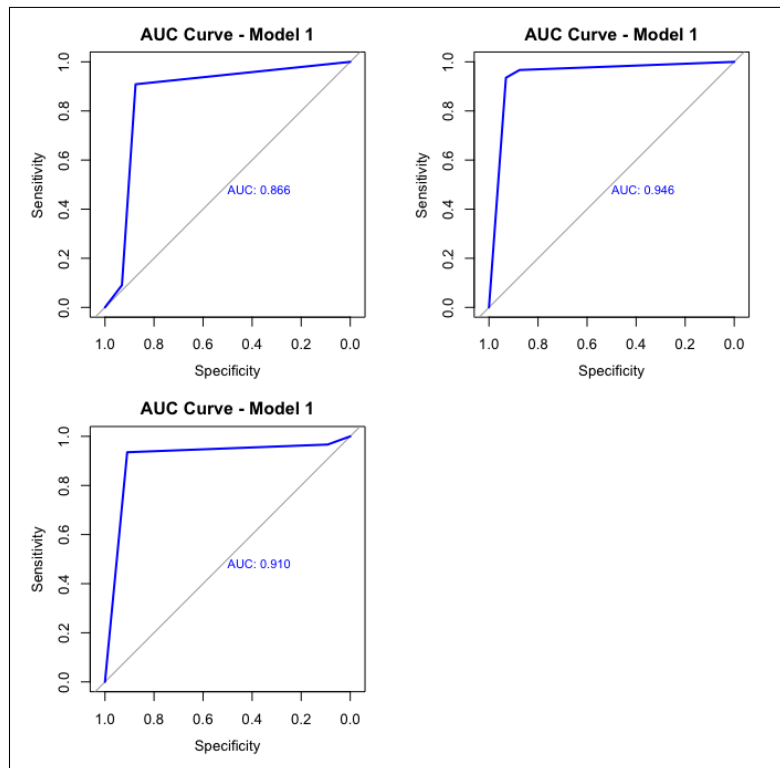


Figure 8.6: AUC Curves - Model 1

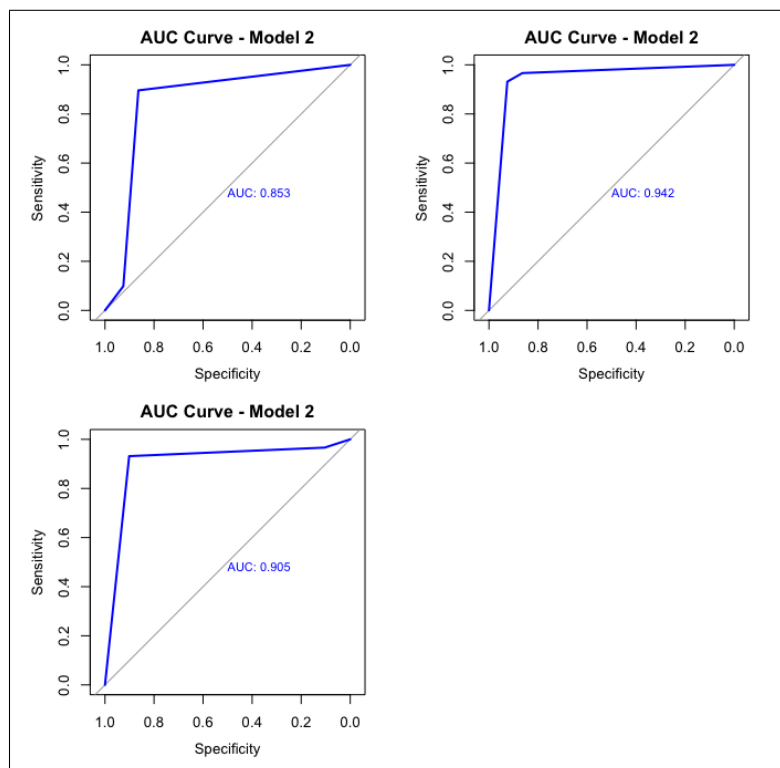


Figure 8.7: AUC Curves - Model 2

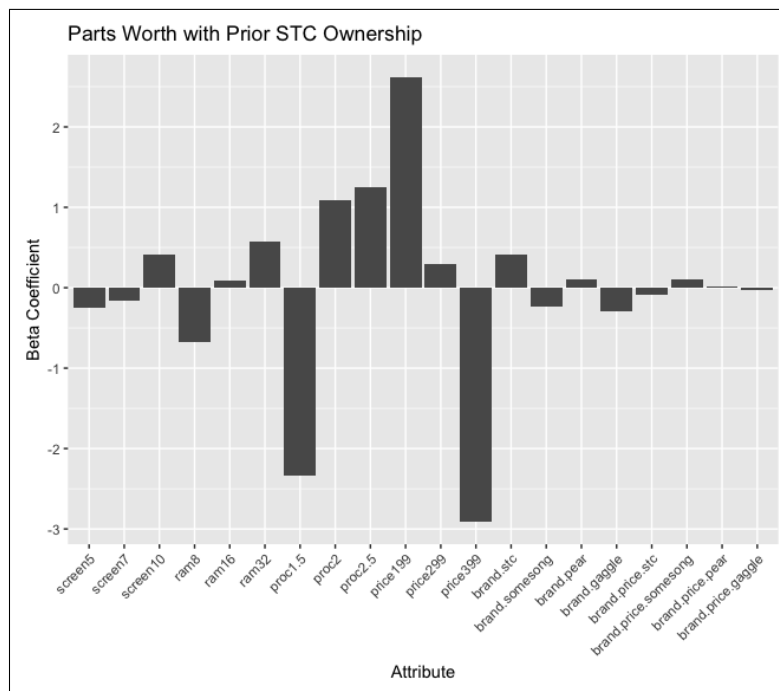


Figure 8.8: Parts Worth - Model 2

R Code

```
# Nikhil Agarwal
# PREDICT 450
# Northwestern University

# load libraries ----
library(tidyverse)
library(bayesm)
library(pROC)
library(gridExtra)

# load input files ----

load('data/stc-cbc-respondents-v3(1).RData')
load('data/efCode.RData')
cbc <- read_delim(file = 'data/stc-dc-task-cbc-v3.csv', delim = '\t')

# load respondent file in a new df
response.data <- resp.data.v3

# preview files ----
glimpse(cbc)
glimpse(response.data)

# get NA count
cbc %>%
```

```

map_df(function(x) sum(is.na(x))) %>%
gather() %>%
arrange(desc(value))

response.data %>%
map_df(function(x) sum(is.na(x))) %>%
gather() %>%
arrange(desc(value))

# EDA ----

# get total counts for each preference within each choice set
response.data %>%
select(starts_with('DCM'), -dcm1_timer) %>%
gather() %>%
group_by(key, value) %>%
summarize(
  cnt = n()
) %>%
spread(key = value, value = cnt, fill = 0) %>%
head()

response.data %>%
select(starts_with('DCM'), -dcm1_timer) %>%
gather() %>%
group_by(value) %>%
summarize(
  total.count = n()
) %>%
ungroup()

# Likelihood of purchasing a new tablet
response.data %>%
select(uuid, STT2r1) %>%
ggplot(aes(x = STT2r1)) +
geom_histogram() +
scale_x_continuous(breaks = seq(1,10,1))

response.data %>%
select(D1, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Tablet by Gender') +
scale_fill_discrete(
  name = 'Gender',
  breaks = c(1,2),
  labels = c('Male','Female'))
)

response.data %>%
select(D2, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +

```

```

ggtitle('Likelihood of Purchasing Tablet by Age') +
scale_fill_discrete(
  name = 'Age',
  breaks = c(1,2,3,4,5,6,7),
  labels = c('Under 18','18-24','25-34','35-44','45-54','55-64','65 or older')
)

a <- response.data %>%
select(D1, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Purchasing Tablet by Gender') +
scale_fill_discrete(
  name = 'Gender',
  breaks = c(1,2),
  labels = c('Male','Female')
)

b <- response.data %>%
select(D2, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Purchasing Tablet by Age') +
scale_fill_discrete(
  name = 'Age',
  breaks = c(1,2,3,4,5,6,7),
  labels = c('Under 18','18-24','25-34','35-44','45-54','55-64','65 or older')
)

grid.arrange(a,b,ncol = 2)

response.data %>%
select(D3, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D3))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Tablet if Parent') +
scale_fill_discrete(
  name = 'Parental Status',
  breaks = c(1,2),
  labels = c('Parent','Not Parent')
)

response.data %>%
select(D5, STT2r1) %>%
ggplot(aes(x = STT2r1, fill = as.factor(D5))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Tablet by Income') +
scale_fill_discrete(
  name = 'Income',
  breaks = c(1,2,3,4,5,6,7,8),
  labels = c('Under $25k','25k - <35k','35k - <50k','50k - <75k','75k - <100k','100k - <200k')
)

```

```

    ', '200k or more', 'no answer')
  )

response.data %>%
select(uuid, STT2r1) %>%
group_by(
STT2r1
) %>%
summarize(
total.respondents = n(),
pct.of.total = total.respondents / 424
) %>%
ungroup()

# Likelihood of purchasing a new smartphone
response.data %>%
select(uuid, STT2r2) %>%
ggplot(aes(x = STT2r2)) +
geom_histogram() +
scale_x_continuous(breaks = seq(1,10,1))

response.data %>%
select(D1, STT2r2) %>%
ggplot(aes(x = STT2r2, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Smartphone by Gender') +
scale_fill_discrete(
name = 'Gender',
breaks = c(1,2),
labels = c('Male','Female'))
)

response.data %>%
select(D2, STT2r2) %>%
ggplot(aes(x = STT2r2, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Smartphone by Age') +
scale_fill_discrete(
name = 'Age',
breaks = c(1,2,3,4,5,6,7),
labels = c('Under 18', '18-24', '25-34', '35-44', '45-54', '55-64', '65 or older'))
)

response.data %>%
select(D3, STT2r2) %>%
ggplot(aes(x = STT2r2, fill = as.factor(D3))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Smartphone if Parent') +
scale_fill_discrete(
name = 'Parental Status',
breaks = c(1,2),
labels = c('Parent', 'Not Parent'))

```

```

)

response.data %>%
select(D5, STT2r2) %>%
ggplot(aes(x = STT2r2, fill = as.factor(D5))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Purchasing Smartphone by Income') +
scale_fill_discrete(
name = 'Income',
breaks = c(1,2,3,4,5,6,7,8),
labels = c('Under $25k', '25k - <35k', '35k - <50k', '50k - <75k', '75k - <100k', '100k - <200k', '200k or more', 'no answer')
)

```

```

response.data %>%
select(uuid, STT2r2) %>%
group_by(
STT2r2
) %>%
summarize(
total.respondents = n(),
pct.of.total = total.respondents / 424
) %>%
ungroup()

```

```

# using cloud storage
response.data %>%
select(uuid, STT2r3) %>%
ggplot(aes(x = STT2r3)) +
geom_histogram() +
scale_x_continuous(breaks = seq(1,10,1))

```

```

response.data %>%
select(D1, STT2r3) %>%
ggplot(aes(x = STT2r3, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Using Cloud Storage by Gender') +
scale_fill_discrete(
name = 'Gender',
breaks = c(1,2),
labels = c('Male', 'Female')
)

```

```

response.data %>%
select(D2, STT2r3) %>%
ggplot(aes(x = STT2r3, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Using Cloud Storage by Age') +
scale_fill_discrete(
name = 'Age',

```

```

breaks = c(1,2,3,4,5,6,7),
labels = c('Under 18', '18-24', '25-34', '35-44', '45-54', '55-64', '65 or older')
)

response.data %>%
select(D3, STT2r3) %>%
ggplot(aes(x = STT2r3, fill = as.factor(D3))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Using Cloud Storage if Parent') +
scale_fill_discrete(
name = 'Parental Status',
breaks = c(1,2),
labels = c('Parent', 'Not Parent')
)

response.data %>%
select(D5, STT2r3) %>%
ggplot(aes(x = STT2r3, fill = as.factor(D5))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Using Cloud Storage by Income') +
scale_fill_discrete(
name = 'Income',
breaks = c(1,2,3,4,5,6,7,8),
labels = c('Under $25k', '25k - <35k', '35k - <50k', '50k - <75k', '75k - <100k', '100k - <200k', '200k or more', 'no answer')
)

response.data %>%
select(uuid, STT2r3) %>%
group_by(
STT2r3
) %>%
summarize(
total.respondents = n(),
pct.of.total = total.respondents / 424
) %>%
ungroup()

# taking online course
response.data %>%
select(uuid, STT2r4) %>%
ggplot(aes(x = STT2r4)) +
geom_histogram() +
scale_x_continuous(breaks = seq(1,10,1))

response.data %>%
select(D1, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Taking Online Course by Gender') +
scale_fill_discrete(

```



```

name = 'Gender',
breaks = c(1,2),
labels = c('Male','Female')
)

response.data %>%
select(D2, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Taking Online Course by Age') +
scale_fill_discrete(
name = 'Age',
breaks = c(1,2,3,4,5,6,7),
labels = c('Under 18','18-24','25-34','35-44','45-54','55-64','65 or older')
)

a <- response.data %>%
select(D1, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Taking Online Course by Gender') +
scale_fill_discrete(
name = 'Gender',
breaks = c(1,2),
labels = c('Male','Female')
)

b <- response.data %>%
select(D2, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D2))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Interest in Taking Online Course by Age') +
scale_fill_discrete(
name = 'Age',
breaks = c(1,2,3,4,5,6,7),
labels = c('Under 18','18-24','25-34','35-44','45-54','55-64','65 or older')
)

grid.arrange(a, b, ncol=2)

response.data %>%
select(D3, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D3))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Taking Online Course if Parent') +
scale_fill_discrete(
name = 'Parental Status',
breaks = c(1,2),
labels = c('Parent','Not Parent')
)

```

```

response.data %>%
select(D5, STT2r4) %>%
ggplot(aes(x = STT2r4, fill = as.factor(D5))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,10,1)) +
labs(x = 'Response') +
ggtitle('Likelihood of Taking Online Course by Income') +
scale_fill_discrete(
name = 'Income',
breaks = c(1,2,3,4,5,6,7,8),
labels = c('Under $25k', '25k - <35k', '35k - <50k', '50k - <75k', '75k - <100k', '100k - <200k', '200k or more', 'no answer')
)

response.data %>%
select(uuid, STT2r4) %>%
group_by(
STT2r4
) %>%
summarize(
total.respondents = n(),
pct.of.total = total.respondents / 424
) %>%
ungroup()

# gender breakout
response.data %>%
select(D1) %>%
group_by(D1) %>%
summarize(
total = n(),
pct = n() / 424
) %>%
ungroup()

# age breakout
response.data %>%
select(D2) %>%
group_by(D2) %>%
summarize(
total = n(),
pct = n() / 424
) %>%
ungroup()

response.data %>%
select(D2) %>%
ggplot(aes(x = D2)) +
geom_bar(stat = 'count') +
geom_text(stat = 'count', aes(label = ..count..), vjust = -1) +
scale_x_continuous(
breaks = c(1, 2, 3, 4, 5, 6, 7),
labels = c('Under 18', '18-24', '25-34', '35-44', '45-54', '55-64', '65 or older')
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Age Group', y = 'Count') +
ggtitle("Frequency Plot - Age Group")

```

```

a <- response.data %>%
  select(D2) %>%
  group_by(D2) %>%
  summarize(
    total = n(),
    pct = round(n() / 424,3)
  ) %>%
  ungroup() %>%
  transmute(
    age.group = case_when(
      D2 == 1 ~ 'Under 18',
      D2 == 2 ~ '18 - 24',
      D2 == 3 ~ '25 - 34',
      D2 == 4 ~ '35 - 44',
      D2 == 5 ~ '45 - 54',
      D2 == 6 ~ '55 - 64',
      D2 == 7 ~ '65 or older'
    ),
    total,
    pct
  )

b <- response.data %>%
  select(D2) %>%
  ggplot(aes(x = D2)) +
  geom_bar(stat = 'count') +
  geom_text(stat = 'count', aes(label = ..count..), vjust = -1) +
  scale_x_continuous(
    breaks = c(1, 2, 3, 4, 5, 6, 7),
    labels = c('Under 18', '18-24', '25-34', '35-44', '45-54', '55-64', '65 or older')
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = 'Age Group', y = 'Count') +
  ggtitle("Frequency Plot - Age Group")

grid.arrange(b, tableGrob(a, theme = ttheme_default(colhead=list(fg_params = list(parse=TRUE
))), rows = NULL), nrow = 2, as.table = T, heights = c(3,1))

# parent?
response.data %>%
  select(D3) %>%
  group_by(D3) %>%
  summarize(
    total = n(),
    pct = n() / 424
  ) %>%
  ungroup()

# children
response.data %>%
  select(starts_with('D4r')) %>%
  head

response.data %>%
  select(starts_with('D4r')) %>%
  gather(key = 'question', value = 'response') %>%
  group_by(question, response) %>%
  summarize(
    count = n()
  )

```

```

) %>%
ungroup() %>%
spread(key = response, value = count) %>%
mutate(
  pct_0 = '0' / 424,
  pct_1 = '1' / 424,
  pct_na = '<NA>' / 424
)

response.data %>%
select(starts_with('D4r')) %>%
mutate(
  have.kids = case_when(
    D4r1 + D4r2 + D4r3 + D4r4 + D4r5 + D4r6 >= 1 ~ 'Yes',
    TRUE ~ 'No'
  )
) %>%
group_by(have.kids) %>%
summarize(
  obs.count = n(),
  obs.pct = n() / 424
) %>%
head

# income

response.data %>%
select(D5) %>%
ggplot(aes(x = D5)) +
geom_histogram()

response.data %>%
select(D5) %>%
group_by(D5) %>%
summarize(
  total = n(),
  pct = n() / 424
) %>%
ungroup()

response.data %>%
ggplot(aes(x = D5)) +
geom_bar(stat = 'count') +
geom_text(stat = 'count', aes(label = ..count..), vjust = -1) +
scale_x_continuous(
  breaks = c(1, 2, 3, 4, 5, 6, 7, 8),
  labels = c('Under $25k', '25k - <35k', '35k - <50k', '50k - <75k', '75k - <100k', '100k - <200k', '200k or more', 'no answer')
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Income', y = 'Count') +
ggtitle("Frequency Plot - Income")

a <- response.data %>%
select(D5) %>%
group_by(D5) %>%
summarize(
  total = n(),

```

```

pct = round(n() / 424,3)
) %>%
ungroup() %>%
transmute(
  income.level = case_when(
    D5 == 1 ~ 'Under $25,000',
    D5 == 2 ~ '$25,000 - < $35,000',
    D5 == 3 ~ '$35,000 - < $50,000',
    D5 == 4 ~ '$50,000 - < $75,000',
    D5 == 5 ~ '$75,000 - < $100,000',
    D5 == 6 ~ '$100,000 - < $200,000',
    D5 == 7 ~ '$200,000 or more',
    D5 == 8 ~ 'No Response'
  ),
  total,
  pct
)

b <- response.data %>%
ggplot(aes(x = D5)) +
geom_bar(stat = 'count') +
geom_text(stat = 'count', aes(label = ..count..), vjust = -1) +
scale_x_continuous(
  breaks = c(1, 2, 3, 4, 5, 6, 7, 8),
  labels = c('Under $25k', '25k - <35k', '35k - <50k', '50k - <75k', '75k - <100k', '100k - <200k',
    '200k or more', 'no answer')
) +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
labs(x = 'Income', y = 'Count') +
ggtitle("Frequency Plot - Income")

grid.arrange(b, tableGrob(a, theme = ttheme_default(colhead=list(fg_params = list(parse=TRUE
))), rows = NULL), nrow = 2, as.table = T, heights = c(3,1))

# state of residence

response.data %>%
select(D6) %>%
ggplot(aes(x = D6)) +
geom_bar(stat = 'count') +
scale_x_continuous(breaks = seq(1,53,1))

response.data %>%
select(D1, D6) %>%
ggplot(aes(x = D6, fill = as.factor(D1))) +
geom_bar(stat = 'count', position = 'dodge') +
scale_x_continuous(breaks = seq(1,53,1))

response.data %>%
select(D6) %>%
group_by(D6) %>%
summarize(
  total = n(),
  pct = n() / 424
) %>%
ungroup()

```

```
# create the proper dummies ----

cbc2 <- cbc %>%
  transmute(
    choice.set,
    choice.ID,
    # screen7 = ifelse(screen == 0, -1, screen),
    # screen10 = ifelse(screen == 0, -1, screen)
    screen7 = case_when(
      screen == 0 ~ -1,
      screen == 1 ~ 1,
      TRUE ~ 0
    ),
    screen10 = case_when(
      screen == 0 ~ -1,
      screen == 2 ~ 1,
      TRUE ~ 0
    ),
    ram16 = case_when(
      RAM == 0 ~ -1,
      RAM == 1 ~ 1,
      TRUE ~ 0
    ),
    ram32 = case_when(
      RAM == 0 ~ -1,
      RAM == 2 ~ 1,
      TRUE ~ 0
    ),
    proc2 = case_when(
      processor == 0 ~ -1,
      processor == 1 ~ 1,
      TRUE ~ 0
    ),
    proc2.5 = case_when(
      processor == 0 ~ -1,
      processor == 2 ~ 1,
      TRUE ~ 0
    ),
    price299 = case_when(
      price == 0 ~ -1,
      price == 1 ~ 1,
      TRUE ~ 0
    ),
    price399 = case_when(
      price == 0 ~ -1,
      price == 2 ~ 1,
      TRUE ~ 0
    ),
    brand.somesong = case_when(
      brand == 0 ~ -1,
      brand == 1 ~ 1,
      TRUE ~ 0
    ),
    brand.pear = case_when(
      brand == 0 ~ -1,
      brand == 2 ~ 1,
      TRUE ~ 0
    )
  )
```

```

),
brand.gaggle = case_when(
  brand == 0 ~ -1,
  brand == 3 ~ 1,
  TRUE ~ 0
),
scaled.price = scale(price, scale = F),
brand.price.somesong = brand.somesong * scaled.price,
brand.price.pear = brand.pear * scaled.price,
brand.price.gaggle = brand.gaggle * scaled.price
)

y.data <- as.matrix(response.data %>%
  select(starts_with('DCM'), -dcm1_timer))

z.owner <- response.data %>%
  transmute(
    zowner = 1 * (!is.na(vList3))
  )

# create a list for bayesm pkg ----

cbc2 <- as.matrix(cbc2 %>% select(-choice.ID, -choice.set, -scaled.price))
#det(t(cbc2) %*% cbc2) # check to make sure my data frame method is working as expected

lgtdata2 = NULL # a starter placeholder for your list
for (i in 1:424) {
  lgtdata2[[i]]=list(y=y.data[i,],X=cbc2)
}
length(lgtdata2)
str(lgtdata2)

#lgtdata2[[3]] %>% View

# trial run ----
lgt100 <- lgtdata2[1:100]
mcmctest <- list(R = 5000, keep = 5)
data.list <- list(lgtdata = lgt100, p = 3)

set.seed(1234)
dummy.test <- rhierMnlDP(Data = data.list, Mcmc = mcmctest)

names(dummy.test)
dim(dummy.test$betadraw)

plot(1:length(dummy.test$betadraw[1,1,]), dummy.test$betadraw[1,1,])
plot(density(dummy.test$betadraw[1,1,701:1000], width = 2))
abline(v = mean(dummy.test$betadraw[1,1,701:1000]), col = 'red') # mean of beta 1 using the
  last 300 obs.
abline(v = 0, col = 'blue', lty = 2) # no preference
# now that we know that the mean is greater than 0, we know that beta1 (see the second
  number within the brackets) is more preferred than not liking it (which happens to be
  the 7" screen)
pnorm(0, mean = mean(dummy.test$betadraw[1,1,701:1000]), sd = sd(dummy.test$betadraw
  [1,1,701:1000]), lower.tail = F)
# area to the right of 0 in the plot happens to be 0.979. in other words, 98% like, 2% don't

```

```

quantile(dummy.test$betadraw[1,1,701:1000], probs = seq(0, 1, by= 0.1))
summary(dummy.test$betadraw[1,1,701:1000])

plot.bayesm.hcoef(dummy.test$betadraw)

apply(dummy.test$betadraw[, ,701:1000], c(2), mean)
apply(dummy.test$betadraw[, ,701:1000], c(1,2), mean)

betameans.overall <- apply(dummy.test$betadraw[, ,701:1000], c(2), mean)
descrip.quantiles <- apply(dummy.test$betadraw[, ,701:1000], 2, quantile, probs=c
  (0.05,0.10,0.25,0.5 ,0.75,0.90,0.95))

# Model 1 - no covariate ----
mcmctest.final <- list(R = 150000, keep = 5)
data.list.final <- list(lgtdata = lgtdata2, p = 3)

set.seed(1234)
logit.model.1 <- rhierMnlDP(Data = data.list.final, Mcmc = mcmctest.final)

# let's plot the hcoef betadraws
plot.bayesm.hcoef(logit.model.1$betadraw)
plot.bayesm.mat(logit.model.1$loglike)
plot.bayesm.nmix(logit.model.1$nmix)

# choose person 23
# plot person 23's beta values for var 1
logit.model.1$betadraw[23,1,]

p23.betadraws <- tibble(
  screen7 = logit.model.1$betadraw[23,1,],
  screen10 = logit.model.1$betadraw[23,2,],
  ram16 = logit.model.1$betadraw[23,3,],
  ram32 = logit.model.1$betadraw[23,4,],
  proc2 = logit.model.1$betadraw[23,5,],
  proc2.5 = logit.model.1$betadraw[23,6,],
  price299 = logit.model.1$betadraw[23,7,],
  price399 = logit.model.1$betadraw[23,8,],
  brand.somesong = logit.model.1$betadraw[23,9,],
  brand.pear = logit.model.1$betadraw[23,10,],
  brand.gaggle = logit.model.1$betadraw[23,11,],
  brand.price.somesong = logit.model.1$betadraw[23,12,],
  brand.price.pear = logit.model.1$betadraw[23,13,],
  brand.price.gaggle = logit.model.1$betadraw[23,14,]
)

p23.betadraws %>%
  ggplot(aes(x=screen7)) +
  geom_density()

p23.betadraws %>%
  ggplot(aes(x = seq(1,nrow(.),1), y = screen7)) +
  geom_line() +
  labs(x = "Iteration", y = 'Beta Values - Screen7')

p23.betadraws %>%
  ggplot(aes(x = seq(1,nrow(.),1), y = screen7)) +
  geom_point() +

```



```

labs(x = "Iteration", y = 'Beta Values - Screen7')

plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$screen7) / seq_along(p23.betadraws$screen7),
     xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot = Screen7')
abline(v = 25000, col = 'red', lty=2)
# seems like at 25000, it goes steady state
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$screen10) / seq_along(p23.
     betadraws$screen10), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot -
     Screen10')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$ram16) / seq_along(p23.betadraws$ram16),
     xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Ram16')
abline(v = 15000, col = 'red', lty = 2)
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$ram32) / seq_along(p23.betadraws$ram32),
     xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Ram32')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$proc2) / seq_along(p23.betadraws$proc2),
     xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Processor2')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$proc2.5) / seq_along(p23.betadraws$proc2
     .5), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Processor2.5')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$price299) / seq_along(p23.
     betadraws$price299), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot -
     Price299')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$price399) / seq_along(p23.
     betadraws$price399), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot -
     Price399')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.somesong) / seq_along(p23.
     betadraws$brand.somesong), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
     Plot - Somesong')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.pear) / seq_along(p23.
     betadraws$brand.pear), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot
     - Pear')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.gaggle) / seq_along(p23.
     betadraws$brand.gaggle), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
     Plot - Gaggle')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.price.somesong) / seq_along(p23.
     betadraws$brand.price.somesong), xlab = 'Iteration', ylab = 'Cumulative Mean', main = '
     Trace Plot - Price & Somesong')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.price.pear) / seq_along(p23.
     betadraws$brand.price.pear), xlab = 'Iteration', ylab = 'Cumulative Mean', main = '
     Trace Plot - Price & Pear')
plot(x = seq(1,30000,1), y = cumsum(p23.betadraws$brand.price.gaggle) / seq_along(p23.
     betadraws$brand.price.gaggle), xlab = 'Iteration', ylab = 'Cumulative Mean', main = '
     Trace Plot - Price & Gaggle')

minVal = 25001
maxVal = 30000

p23.betadraws.ss <- tibble(
  screen7 = logit.model.1$betadraw[23,1,minVal:maxVal],
  screen10 = logit.model.1$betadraw[23,2,minVal:maxVal],
  ram16 = logit.model.1$betadraw[23,3,minVal:maxVal],
  ram32 = logit.model.1$betadraw[23,4,minVal:maxVal],
  proc2 = logit.model.1$betadraw[23,5,minVal:maxVal],
  proc2.5 = logit.model.1$betadraw[23,6,minVal:maxVal],
  price299 = logit.model.1$betadraw[23,7,minVal:maxVal],
  price399 = logit.model.1$betadraw[23,8,minVal:maxVal],
  brand.somesong = logit.model.1$betadraw[23,9,minVal:maxVal],
  brand.pear = logit.model.1$betadraw[23,10,minVal:maxVal],

```

```

brand.gaggle = logit.model.1$betadraw[23,11,minVal:maxVal],
brand.price.somesong = logit.model.1$betadraw[23,12,minVal:maxVal],
brand.price.pear = logit.model.1$betadraw[23,13,minVal:maxVal],
brand.price.gaggle = logit.model.1$betadraw[23,14,minVal:maxVal]
)

p23.betadraws.ss %>%
ggplot(aes(x=screen7)) +
geom_density() +
geom_vline(data = p23.betadraws.ss, aes(xintercept = mean(screen7)), col = 'red') +
geom_vline(xintercept = 0, col = 'blue', linetype = 2)

# the following gives area under the curve for the simulated beta1 samples for each
# predictor for candidate 23 in relation to 0 (which implies no preference). The first
# one comes out to be 0.847. This means that the area under the curve is 0.847. This can
# be interpreted as 84% of the simulated beta1 values prefer a screen size of 7" versus
# no preference
pnorm(0, mean = mean(p23.betadraws.ss$screen7), sd = sd(p23.betadraws.ss$screen7), lower.
tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$screen10), sd = sd(p23.betadraws.ss$screen10), lower.
tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$ram16), sd = sd(p23.betadraws.ss$ram16), lower.tail =
F)
pnorm(0, mean = mean(p23.betadraws.ss$ram32), sd = sd(p23.betadraws.ss$ram32), lower.tail =
F)
pnorm(0, mean = mean(p23.betadraws.ss$proc2), sd = sd(p23.betadraws.ss$proc2), lower.tail =
F)
pnorm(0, mean = mean(p23.betadraws.ss$proc2.5), sd = sd(p23.betadraws.ss$proc2.5), lower.
tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$price299), sd = sd(p23.betadraws.ss$price299), lower.
tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$price399), sd = sd(p23.betadraws.ss$price399), lower.
tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.somesong), sd = sd(p23.betadraws.ss$brand.
somesong), lower.tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.pear), sd = sd(p23.betadraws.ss$brand.pear),
lower.tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.gaggle), sd = sd(p23.betadraws.ss$brand.gaggle),
lower.tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.price.somesong), sd = sd(p23.betadraws.ss$brand.
price.somesong), lower.tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.price.pear), sd = sd(p23.betadraws.ss$brand.
price.pear), lower.tail = F)
pnorm(0, mean = mean(p23.betadraws.ss$brand.price.gaggle), sd = sd(p23.betadraws.ss$brand.
price.gaggle), lower.tail = F)

# get the mean beta values for all respondents and the 14 predictors
model1.avgBetaVal <- as.data.frame(apply(logit.model.1$betadraw[,minVal:maxVal],c(1,2),mean
))

model1.avgBetaVal <- model1.avgBetaVal %>%
rename(
screen7 = V1,
screen10 = V2,
ram16 = V3,
ram32 = V4,
proc2 = V5,
proc2.5 = V6,

```

```

price299 = V7,
price399 = V8,
brand.somesong = V9,
brand.pear = V10,
brand.gaggle = V11,
brand.price.somesong = V12,
brand.price.pear = V13,
brand.price.gaggle =V14
)

summary(model1.avgBetaVal)

min.values <- model1.avgBetaVal %>%
summarise_all(min) %>%
gather(key = variable, value = minimum.value)
max.values <- model1.avgBetaVal %>%
summarize_all(max) %>%
gather(key = variable, value = maximum.value)
q05.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.05) %>%
gather(key = variable, value = p05.value)
q10.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.10) %>%
gather(key = variable, value = p10.value)
q25.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.25) %>%
gather(key = variable, value = p25.value)
median.values <- model1.avgBetaVal %>%
summarize_all(median) %>%
gather(key = variable, value = median.value)
q75.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.75) %>%
gather(key = variable, value = p75.value)
q90.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.90) %>%
gather(key = variable, value = p90.value)
q95.values <- model1.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.95) %>%
gather(key = variable, value = p95.value)

descriptive.stats.model1.avgBetaVals <- min.values %>%
inner_join(max.values, by = 'variable') %>%
inner_join(median.values, by = 'variable') %>%
inner_join(q05.values, by = 'variable') %>%
inner_join(q10.values, by = 'variable') %>%
inner_join(q25.values, by = 'variable') %>%
inner_join(q75.values, by = 'variable') %>%
inner_join(q90.values, by = 'variable') %>%
inner_join(q95.values, by = 'variable')

dim(model1.avgBetaVal)

# Model 1 Fit Statistics (no covariate) ----
dim(model1.avgBetaVal)
dim(cbc2)

model1.xbetas <- cbc2 %*% t(as.matrix(model1.avgBetaVal))

```

```

# convert to 3 columns across
model1.xbetas <- matrix(model1.xbetas, ncol = 3, byrow = T)

# take the exponential (remember, it's a log-odds model) of the matrix
model1.xbetas <- exp(model1.xbetas)

# convert to dataframe
model1.xbetas <- as.data.frame(model1.xbetas)
model1.xbetas <- model1.xbetas %>%
  rename(
    'choice1' = V1,
    'choice2' = V2,
    'choice3' = V3
  ) %>%
  mutate(
    sum.of.row = rowSums(.)
  ) %>%
  transmute(
    choice1 = choice1 / sum.of.row,
    choice2 = choice2 / sum.of.row,
    choice3 = choice3 / sum.of.row
  )

model1.choice <- max.col(model1.xbetas)

responses <- as.vector(t(y.data))

table(model1.choice, responses) # confusion matrix

par(mfrow=c(2,2))
auc(multiclass.roc(responses, model1.choice, print.auc = T, levels = c(1,2,3), col = 'blue',
  percent = F, plot = T, main = 'AUC Curve - Model 1')) # AUC
par(mfrow=c(1,1))

mean(logit.model.1$loglike) # -2loglike test

model1.predicted.values <- as.data.frame(apply(t(matrix(model1.choice, nrow = 36, ncol =
  424)), 2, function(x){tabulate(na.omit(x))}))

model1.predicted.values %>%
  mutate(
    alternative = c(1,2,3)
  ) %>%
  gather(key = choice.set, value, starts_with('V')) %>%
  spread(key = alternative, value = value) %>%
  mutate(
    choice.set = case_when(
      choice.set == 'V1' ~ 1,
      choice.set == 'V2' ~ 2,
      choice.set == 'V3' ~ 3,
      choice.set == 'V4' ~ 4,
      choice.set == 'V5' ~ 5,
      choice.set == 'V6' ~ 6,
      choice.set == 'V7' ~ 7,
      choice.set == 'V8' ~ 8,
      choice.set == 'V9' ~ 9,
      choice.set == 'V10' ~ 10,
      choice.set == 'V11' ~ 11,
      choice.set == 'V12' ~ 12,

```

```

choice.set == 'V13' ~ 13,
choice.set == 'V14' ~ 14,
choice.set == 'V15' ~ 15,
choice.set == 'V16' ~ 16,
choice.set == 'V17' ~ 17,
choice.set == 'V18' ~ 18,
choice.set == 'V19' ~ 19,
choice.set == 'V20' ~ 20,
choice.set == 'V21' ~ 21,
choice.set == 'V22' ~ 22,
choice.set == 'V23' ~ 23,
choice.set == 'V24' ~ 24,
choice.set == 'V25' ~ 25,
choice.set == 'V26' ~ 26,
choice.set == 'V27' ~ 27,
choice.set == 'V28' ~ 28,
choice.set == 'V29' ~ 29,
choice.set == 'V30' ~ 30,
choice.set == 'V31' ~ 31,
choice.set == 'V32' ~ 32,
choice.set == 'V33' ~ 33,
choice.set == 'V34' ~ 34,
choice.set == 'V35' ~ 35,
choice.set == 'V36' ~ 36
)
)%>%
rename(
  alt.1 = '1',
  alt.2 = '2',
  alt.3 = '3'
)%>%
arrange(choice.set) %>%
mutate(
  alt.1.pct = round(alt.1 / rowSums(.,dims = 1),3),
  alt.2.pct = round(alt.2 / rowSums(.,dims = 1),3),
  alt.3.pct = round(alt.3 / rowSums(.,dims = 1),3)
)%>%
head(n=36)

# actual data
response.data %>%
select(starts_with('DCM'), -dcm1_timer) %>%
gather() %>%
group_by(key, value) %>%
summarize(
  cnt = n()
)%>%
ungroup() %>%
spread(key = value, value = cnt, fill = 0) %>%
rename(
  choice.set = key,
  alt.1 = '1',
  alt.2 = '2',
  alt.3 = '3'
)%>%
mutate(
  choice.set = case_when(
    choice.set == 'DCM1_1' ~ 1,
    choice.set == 'DCM1_2' ~ 2,

```

```

choice.set == 'DCM1_3' ~ 3,
choice.set == 'DCM1_4' ~ 4,
choice.set == 'DCM1_5' ~ 5,
choice.set == 'DCM1_6' ~ 6,
choice.set == 'DCM1_7' ~ 7,
choice.set == 'DCM1_8' ~ 8,
choice.set == 'DCM1_9' ~ 9,
choice.set == 'DCM1_10' ~ 10,
choice.set == 'DCM1_11' ~ 11,
choice.set == 'DCM1_12' ~ 12,
choice.set == 'DCM1_13' ~ 13,
choice.set == 'DCM1_14' ~ 14,
choice.set == 'DCM1_15' ~ 15,
choice.set == 'DCM1_16' ~ 16,
choice.set == 'DCM1_17' ~ 17,
choice.set == 'DCM1_18' ~ 18,
choice.set == 'DCM1_19' ~ 19,
choice.set == 'DCM1_20' ~ 20,
choice.set == 'DCM1_21' ~ 21,
choice.set == 'DCM1_22' ~ 22,
choice.set == 'DCM1_23' ~ 23,
choice.set == 'DCM1_24' ~ 24,
choice.set == 'DCM1_25' ~ 25,
choice.set == 'DCM1_26' ~ 26,
choice.set == 'DCM1_27' ~ 27,
choice.set == 'DCM1_28' ~ 28,
choice.set == 'DCM1_29' ~ 29,
choice.set == 'DCM1_30' ~ 30,
choice.set == 'DCM1_31' ~ 31,
choice.set == 'DCM1_32' ~ 32,
choice.set == 'DCM1_33' ~ 33,
choice.set == 'DCM1_34' ~ 34,
choice.set == 'DCM1_35' ~ 35,
choice.set == 'DCM1_36' ~ 36
)
) %>%
arrange(choice.set) %>%
mutate(choice.set = as.character(choice.set)) %>%
View

```

```
# Model 1 Overall Summary ----
```

```

# get the overall beta values
model1.avgBetaValOverall <- tibble(variable = c('screen7', 'screen10', 'ram16', 'ram32', 'proc2',
'proc2.5', 'price299', 'price399', 'brand.somesong', 'brand.pear', 'brand.gaggle', 'brand.
price.somesong', 'brand.price.pear', 'brand.price.gaggle'), coefficient = apply(logit.
model.1$betadraw[,minVal:maxVal],c(2),mean))

```

```

model1.coefficient.table <- model1.avgBetaValOverall %>%
spread(key = variable, value = coefficient) %>%
transmute(
screen5 = 0 - screen7 - screen10,
screen7,
screen10,
ram8 = 0 - ram16 - ram32,
ram16,
ram32,

```

```

proc1.5 = 0 - proc2 - proc2.5,
proc2,
proc2.5,
price199 = 0 - price299 - price399,
price299,
price399,
brand.stc = 0 - brand.somesong - brand.pear - brand.gaggle,
brand.somesong,
brand.pear,
brand.gaggle,
brand.price.stc = 0 - brand.price.somesong - brand.price.pear - brand.price.gaggle,
brand.price.somesong,
brand.price.pear,
brand.price.gaggle
) %>%
gather(key = variable, value = coefficient) %>%
mutate(
odds.ratio = exp(coefficient)
)

model1.coefficient.table

model1.coefficient.table %>%
select(variable, coefficient) %>%
ggplot(aes(x = variable, y = coefficient)) +
geom_bar(stat = 'identity') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
scale_x_discrete(limits = c('screen5', 'screen7', 'screen10', 'ram8', 'ram16', 'ram32', 'proc1
.5', 'proc2', 'proc2.5', 'price199', 'price299', 'price399', 'brand.stc', 'brand.somesong', '
brand.pear', 'brand.gaggle', 'brand.price.stc', 'brand.price.somesong', 'brand.price.pear
', 'brand.price.gaggle')) +
labs(x = 'Attribute', y = 'Beta Coefficient') +
ggtitle('Parts Worth - Model 1')

# Model 2 - with covariate ----
owner.status = as.matrix(scale(z.owner$zowner, scale = F))
mcmcetest.model2 <- list(R = 150000, keep = 5)
data.list.model2 <- list(lgtdata = lgtdata2, p = 3, Z = owner.status)

set.seed(1234)
logit.model.2 <- rhierMnlDP(Data = data.list.model2, Mcmc = mcmcetest.model2)

table(logit.model.2$Istardraw)

dim(logit.model.2$Deltadraw)

model2.deltadraw <- as.data.frame(logit.model.2$Deltadraw[minVal:maxVal,])

model2.deltadraw <- model2.deltadraw %>%
rename(
screen7 = V1,
screen10 = V2,

```

```

ram16 = V3,
ram32 = V4,
proc2 = V5,
proc2.5 = V6,
price299 = V7,
price399 = V8,
brand.somesong = V9,
brand.pear = V10,
brand.gaggle = V11,
brand.price.somesong = V12,
brand.price.pear = V13,
brand.price.gaggle = V14
)

mean.values <- model2.deltadraw %>%
summarize_all(mean) %>%
gather(key = variable, value = mean.value)
min.values <- model2.deltadraw %>%
summarize_all(min) %>%
gather(key = variable, value = minimum.value)
max.values <- model2.deltadraw %>%
summarize_all(max) %>%
gather(key = variable, value = maximum.value)
q05.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.05) %>%
gather(key = variable, value = p05.value)
q10.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.10) %>%
gather(key = variable, value = p10.value)
q25.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.25) %>%
gather(key = variable, value = p25.value)
median.values <- model2.deltadraw %>%
summarize_all(median) %>%
gather(key = variable, value = median.value)
q75.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.75) %>%
gather(key = variable, value = p75.value)
q90.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.90) %>%
gather(key = variable, value = p90.value)
q95.values <- model2.deltadraw %>%
summarize_all(funs(quantile), probs = 0.95) %>%
gather(key = variable, value = p95.value)

descriptive.stats.model2.DeltaDraw <- mean.values %>%
inner_join(min.values, by = 'variable') %>%
inner_join(max.values, by = 'variable') %>%
inner_join(median.values, by = 'variable') %>%
inner_join(q05.values, by = 'variable') %>%
inner_join(q10.values, by = 'variable') %>%
inner_join(q25.values, by = 'variable') %>%
inner_join(q75.values, by = 'variable') %>%
inner_join(q90.values, by = 'variable') %>%
inner_join(q95.values, by = 'variable')

# p23.betadraws.model2 <- tibble(

```



```

# screen7 = logit.model.2$betadraw[23,1,],
# screen10 = logit.model.2$betadraw[23,2,],
# ram16 = logit.model.2$betadraw[23,3,],
# ram32 = logit.model.2$betadraw[23,4,],
# proc2 = logit.model.2$betadraw[23,5,],
# proc2.5 = logit.model.2$betadraw[23,6,],
# price299 = logit.model.2$betadraw[23,7,],
# price399 = logit.model.2$betadraw[23,8,],
# brand.somesong = logit.model.2$betadraw[23,9,],
# brand.pear = logit.model.2$betadraw[23,10,],
# brand.gaggle = logit.model.2$betadraw[23,11,],
# brand.price.somesong = logit.model.2$betadraw[23,12,],
# brand.price.pear = logit.model.2$betadraw[23,13,],
# brand.price.gaggle = logit.model.2$betadraw[23,14,]
# )
#
#
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$screen7) / seq_along(p23.
  betadraws.model2$screen7), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
  Plot = Screen7')
# # seems like at 25000, it goes steady state
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$screen10) / seq_along(p23.
  betadraws.model2$screen10), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
  Plot - Screen10')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$ram16) / seq_along(p23.betadraws.
  model2$ram16), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Ram16
  ')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$ram32) / seq_along(p23.betadraws.
  model2$ram32), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot - Ram32
  ')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$proc2) / seq_along(p23.betadraws.
  model2$proc2), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace Plot -
  Processor2')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$proc2.5) / seq_along(p23.
  betadraws.model2$proc2.5), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
  Plot - Processor2.5')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$price299) / seq_along(p23.
  betadraws.model2$price299), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
  Plot - Price299')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$price399) / seq_along(p23.
  betadraws.model2$price399), xlab = 'Iteration', ylab = 'Cumulative Mean', main = 'Trace
  Plot - Price399')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.somesong) / seq_along(p23.
  betadraws.model2$brand.somesong), xlab = 'Iteration', ylab = 'Cumulative Mean', main =
  'Trace Plot - Somesong')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.pear) / seq_along(p23.
  betadraws.model2$brand.pear), xlab = 'Iteration', ylab = 'Cumulative Mean', main = '
  Trace Plot - Pear')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.gaggle) / seq_along(p23.
  betadraws.model2$brand.gaggle), xlab = 'Iteration', ylab = 'Cumulative Mean', main = '
  Trace Plot - Gaggle')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.price.somesong) / seq_along
  (p23.betadraws.model2$brand.price.somesong), xlab = 'Iteration', ylab = 'Cumulative
  Mean', main = 'Trace Plot - Price & Somesong')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.price.pear) / seq_along(p23
  .betadraws.model2$brand.price.pear), xlab = 'Iteration', ylab = 'Cumulative Mean', main
  = 'Trace Plot - Price & Pear')
# plot(x = seq(1,30000,1), y = cumsum(p23.betadraws.model2$brand.price.gaggle) / seq_along(
  p23.betadraws.model2$brand.price.gaggle), xlab = 'Iteration', ylab = 'Cumulative Mean',

```

```

main = 'Trace Plot - Price & Gaggle')

model2.avgBetaVal <- as.data.frame(apply(logit.model.2$betadraw[, ,minVal:maxVal], c(1,2), mean
))

model2.avgBetaVal <- model2.avgBetaVal %>%
rename(
screen7 = V1,
screen10 = V2,
ram16 = V3,
ram32 = V4,
proc2 = V5,
proc2.5 = V6,
price299 = V7,
price399 = V8,
brand.somesong = V9,
brand.pear = V10,
brand.gaggle = V11,
brand.price.somesong = V12,
brand.price.pear = V13,
brand.price.gaggle =V14
)

summary(model2.avgBetaVal)

min.values <- model2.avgBetaVal %>%
summarise_all(min) %>%
gather(key = variable, value = minimum.value)
max.values <- model2.avgBetaVal %>%
summarize_all(max) %>%
gather(key = variable, value = maximum.value)
q05.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.05) %>%
gather(key = variable, value = p05.value)
q10.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.10) %>%
gather(key = variable, value = p10.value)
q25.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.25) %>%
gather(key = variable, value = p25.value)
median.values <- model2.avgBetaVal %>%
summarize_all(median) %>%
gather(key = variable, value = median.value)
q75.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.75) %>%
gather(key = variable, value = p75.value)
q90.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.90) %>%
gather(key = variable, value = p90.value)
q95.values <- model2.avgBetaVal %>%
summarize_all(funs(quantile), probs = 0.95) %>%
gather(key = variable, value = p95.value)

descriptive.stats.model2.avgBetaVals <- min.values %>%
inner_join(max.values, by = 'variable') %>%
inner_join(median.values, by = 'variable') %>%
inner_join(q05.values, by = 'variable') %>%
inner_join(q10.values, by = 'variable') %>%

```

```

inner_join(q25.values, by = 'variable') %>%
inner_join(q75.values, by = 'variable') %>%
inner_join(q90.values, by = 'variable') %>%
inner_join(q95.values, by = 'variable')

# Model 2 - Fit Statistics (with covariate) ----

model2.xbetas <- cbc2 %*% t(as.matrix(model2.avgBetaVal))

# convert to 3 columns across
model2.xbetas <- matrix(model2.xbetas, ncol = 3, byrow = T)

# take the exponential (remember, it's a log-odds model) of the matrix
model2.xbetas <- exp(model2.xbetas)

# convert to dataframe
model2.xbetas <- as.data.frame(model2.xbetas)
model2.xbetas <- model2.xbetas %>%
  rename(
    'choice1' = V1,
    'choice2' = V2,
    'choice3' = V3
  ) %>%
  mutate(
    sum.of.row = rowSums(.)
  ) %>%
  transmute(
    choice1 = choice1 / sum.of.row,
    choice2 = choice2 / sum.of.row,
    choice3 = choice3 / sum.of.row
  )

model2.choice <- max.col(model2.xbetas)

responses <- as.vector(t(y.data))

table(model2.choice, responses) # confusion matrix

par(mfrow=c(2,2))
auc(multiclass.roc(responses, model2.choice, print.auc = T, levels = c(1,2,3), col = 'blue',
  percent = F, plot = T, main = 'AUC Curve - Model 2')) # AUC
par(mfrow=c(1,1))
auc(multiclass.roc(responses, model2.choice, plot = T, main = 'AUC Curve - Model 2')) # AUC

mean(logit.model.2$loglike) # -2loglike test

model2.predicted.values <- as.data.frame(apply(t(matrix(model2.choice, nrow = 36, ncol =
  424)), 2, function(x){tabulate(na.omit(x))}))

model2.predicted.values %>%
  mutate(

```

```
alternative = c(1,2,3)
) %>%
gather(key = choice.set, value, starts_with('V')) %>%
spread(key = alternative, value = value) %>%
mutate(
  choice.set = case_when(
    choice.set == 'V1' ~ 1,
    choice.set == 'V2' ~ 2,
    choice.set == 'V3' ~ 3,
    choice.set == 'V4' ~ 4,
    choice.set == 'V5' ~ 5,
    choice.set == 'V6' ~ 6,
    choice.set == 'V7' ~ 7,
    choice.set == 'V8' ~ 8,
    choice.set == 'V9' ~ 9,
    choice.set == 'V10' ~ 10,
    choice.set == 'V11' ~ 11,
    choice.set == 'V12' ~ 12,
    choice.set == 'V13' ~ 13,
    choice.set == 'V14' ~ 14,
    choice.set == 'V15' ~ 15,
    choice.set == 'V16' ~ 16,
    choice.set == 'V17' ~ 17,
    choice.set == 'V18' ~ 18,
    choice.set == 'V19' ~ 19,
    choice.set == 'V20' ~ 20,
    choice.set == 'V21' ~ 21,
    choice.set == 'V22' ~ 22,
    choice.set == 'V23' ~ 23,
    choice.set == 'V24' ~ 24,
    choice.set == 'V25' ~ 25,
    choice.set == 'V26' ~ 26,
    choice.set == 'V27' ~ 27,
    choice.set == 'V28' ~ 28,
    choice.set == 'V29' ~ 29,
    choice.set == 'V30' ~ 30,
    choice.set == 'V31' ~ 31,
    choice.set == 'V32' ~ 32,
    choice.set == 'V33' ~ 33,
    choice.set == 'V34' ~ 34,
    choice.set == 'V35' ~ 35,
    choice.set == 'V36' ~ 36
  )
) %>%
rename(
  alt.1 = '1',
  alt.2 = '2',
  alt.3 = '3'
) %>%
arrange(choice.set) %>%
mutate(
  alt.1.pct = round(alt.1 / rowSums(.,dims = 1),3),
  alt.2.pct = round(alt.2 / rowSums(.,dims = 1),3),
  alt.3.pct = round(alt.3 / rowSums(.,dims = 1),3)
) %>%
head(n=36)
```

```

# Model 2 Overall Summary (with covariate) ----

# get the overall beta values
model2.avgBetaValOverall <- tibble(variable = c('screen7', 'screen10', 'ram16', 'ram32', 'proc2',
  'proc2.5', 'price299', 'price399', 'brand.somesong', 'brand.pear', 'brand.gaggle', 'brand.
  price.somesong', 'brand.price.pear', 'brand.price.gaggle'), coefficient = apply(logit.
  model.2$betadraw[,minVal:maxVal],c(2),mean))

model2.coefficient.table <- model2.avgBetaValOverall %>%
spread(key = variable, value = coefficient) %>%
transmute(
screen5 = 0 - screen7 - screen10,
screen7,
screen10,
ram8 = 0 - ram16 - ram32,
ram16,
ram32,
proc1.5 = 0 - proc2 - proc2.5,
proc2,
proc2.5,
price199 = 0 - price299 - price399,
price299,
price399,
brand.stc = 0 - brand.somesong - brand.pear - brand.gaggle,
brand.somesong,
brand.pear,
brand.gaggle,
brand.price.stc = 0 - brand.price.somesong - brand.price.pear - brand.price.gaggle,
brand.price.somesong,
brand.price.pear,
brand.price.gaggle
) %>%
gather(key = variable, value = coefficient) %>%
mutate(
odds.ratio = exp(coefficient)
)

model2.coefficient.table

model2.coefficient.table %>%
select(variable, coefficient) %>%
ggplot(aes(x = variable, y = coefficient)) +
geom_bar(stat = 'identity') +
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
scale_x_discrete(limits = c('screen5', 'screen7', 'screen10', 'ram8', 'ram16', 'ram32', 'proc1
.5', 'proc2', 'proc2.5', 'price199', 'price299', 'price399', 'brand.stc', 'brand.somesong',
brand.pear', 'brand.gaggle', 'brand.price.stc', 'brand.price.somesong', 'brand.price.pear
', 'brand.price.gaggle')) +
labs(x = 'Attribute', y = 'Beta Coefficient') +
ggtitle('Parts Worth with Prior STC Ownership')

# Extra Scenarios ----

```

```

# Load scenario data

extra.scenario.raw <- read_csv('data/extra-scenarios-v3.csv')

extra.scenario.processed <- read_csv('data/extra-scenarios(1).csv')

# rename the columns in processed
extra.scenario.processed <- extra.scenario.processed %>%
  rename(
    screen7 = V1,
    screen10 = V2,
    ram16 = V3,
    ram32 = V4,
    proc2 = V5,
    proc2.5 = V6,
    price299 = V7,
    price399 = V8,
    brand.somesong = V9,
    brand.pear = V10,
    brand.gaggle = V11,
    brand.price.somesong = V12,
    brand.price.pear = V13,
    brand.price.gaggle = V14
  )

# preview dataframes
extra.scenario.raw %>%
  head

extra.scenario.processed %>%
  head

# convert the extra scenarios to a matrix format for prediction purposes

extra.scenario.matrix <- as.matrix(extra.scenario.processed)

# use model 1 for predictions (a pooled approach)

extra.scenario.coefficients <- extra.scenario.matrix %*% as.matrix(model1.avgBetaValOverall
  %>% select(coefficient), ncol = 1, byrow = T)

extra.scenario.xbetas <- matrix(extra.scenario.coefficients, ncol=3, byrow = T)

extra.scenario.xbetas <- exp(extra.scenario.xbetas)

extra.scenario.xbetas <- as.data.frame(extra.scenario.xbetas)

extra.scenario.xbetas <- extra.scenario.xbetas %>%
  rename(
    'choice1' = V1,
    'choice2' = V2,
    'choice3' = V3
  ) %>%
  mutate(
    sum.of.row = rowSums(.)
  ) %>%
  transmute(
    choice1 = choice1 / sum.of.row,
    choice2 = choice2 / sum.of.row,

```

```

choice3 = choice3 / sum.of.row
)

# use model 1 for predictions (individualized)

extra.scenario.ind.xbetas <- extra.scenario.matrix %*% t(as.matrix(model1.avgBetaVal))

extra.scenario.ind.xbetas <- matrix(extra.scenario.ind.xbetas, ncol = 3, byrow = T)

extra.scenario.ind.xbetas <- exp(extra.scenario.ind.xbetas)

extra.scenario.ind.xbetas <- as.data.frame(extra.scenario.ind.xbetas)

extra.scenario.ind.xbetas <- extra.scenario.ind.xbetas %>%
rename(
'choice1' = V1,
'choice2' = V2,
'choice3' = V3
) %>%
mutate(
sum.of.row = rowSums(.)
) %>%
transmute(
choice1 = choice1 / sum.of.row,
choice2 = choice2 / sum.of.row,
choice3 = choice3 / sum.of.row
)

extra.scenario.ind.xbetas

extra.scenario.ind.choice <- max.col(extra.scenario.ind.xbetas)

table(extra.scenario.ind.choice[1:424]) # for the 1st choiceset
table(extra.scenario.ind.choice[425:848]) # for the 2nd choiceset

# use model 2 for predictions (individualized)

extra.scenario.ind.xbetas2 <- extra.scenario.matrix %*% t(as.matrix(model2.avgBetaVal))

extra.scenario.ind.xbetas2 <- matrix(extra.scenario.ind.xbetas2, ncol = 3, byrow = T)

extra.scenario.ind.xbetas2 <- exp(extra.scenario.ind.xbetas2)

extra.scenario.ind.xbetas2 <- as.data.frame(extra.scenario.ind.xbetas2)

extra.scenario.ind.xbetas2 <- extra.scenario.ind.xbetas2 %>%
rename(
'choice1' = V1,
'choice2' = V2,
'choice3' = V3
) %>%
mutate(
sum.of.row = rowSums(.)
) %>%
transmute(
choice1 = choice1 / sum.of.row,
choice2 = choice2 / sum.of.row,
choice3 = choice3 / sum.of.row
)

```

```
extra.scenario.ind.xbetas2  
  
extra.scenario.ind.choice2 <- max.col(extra.scenario.ind.xbetas2)  
  
table(extra.scenario.ind.choice2[1:424]) # for the 1st choicest  
table(extra.scenario.ind.choice2[425:848]) # for the 2nd choicest
```


Introduction

In this assignment, a large dataset pertaining to the XYZ organization, was provided to help determine which customers to send mailers to for XYZ's 17th campaign. The intent of this assignment is to determine the approximate net profit XYZ can expect from those customers that XYZ sends the mailer to. Three different models were developed to derive a strong predictive model that could be leveraged. Furthermore, the best performing model was used to determine the potential net profit XYZ could realize.

Data Overview

The dataset used in this assignment was provided in a pre-processed format (herein referred to as the raw data). Dr. Miller had taken the raw data from XYZ's database and converted it into a 'tidy' format - where each row is an observation and each column is a variable (Wickham, 2014). The raw data consisted of 30,779 observations across 554 variables (i.e., predictors). Upon importing the dataset, there were 345 character variables, 48 integer variables, and 161 numeric variables.

In order to reduce the large number of potential predictors, a subjective analysis was done to determine if a variable should be kept or not. This analysis relied on the provided descriptions in the data dictionary, a quick overview of the unique values within each variable, and the amount of missing observations. If a description could not be found, and/or if the variable was determined to have excessively high number of unique values, and/or if the variable was missing more than 10% of the observations then it was removed from the variable list¹. This initial run-through resulted in 191 variables being selected. However, this number was further reduced through feature engineering (which resulted in the removal of some of the variables) and a construction of a naïve random forest model to select only the most important variables. These steps are discussed further in the next sections.

Data Pre-Processing

Pre-processing and missing data imputation was done in junction with the EDA. However, for this report, they are presented in a pseudo-sequential manner. For instance, all of the character variables were converted to factors. Furthermore, integer based variables were explored to determine if they needed to be factors or integers. As an example, the variable CHANNEL_ACQUISITION was initially a character variable that was converted to factors as it consisted of only three different types. On the other hand, the variable ZPETSP is an integer that was converted to a factor as it is ordinal.

Exploratory Data Analysis

A comprehensive exploratory data analysis was completed for this analysis (a small sample of the charts constructed can be found in the Appendix). Many of the values were found to have missing observations. In fact, of the numerous predictors, 300 of them had missing values. From this process, variables that had more than 10% missing observations were removed from the final dataset used to construct the training and validation datasets. Table 2.1 summarizes the overall response for campaign 16 given that a customer had received a mailer for the 16th campaign. Note the imbalance between customers who had responded to a mailer and customers who had not (approximately 10% of the customers responded to mailers).

¹ A separate document was kept for a deeper analysis and can be provided upon request.

Table 2.1: Summary Table on Response16

Response	Count	Percent
0	13482	0.9035
1	1440	0.0965

It was also found that multiple mailers were sent to customers in certain campaigns. It was not clear from the analysis and the data dictionaries as to why a customer may receive more than one mailer. One interesting observation was found in the variable related to the median travel time to work. In Figure 2.1, it is interesting to note that the customers who had a positive response were seeing 200 - 300 minutes of travel time. Intuitively, this could mean that customers who have a longer commute have more time to dwell on their buying decision and may be responsive to making a purchase. This enables the potential consumer to absorb the information from a mailer and engage in purchasing as it may pass the time.

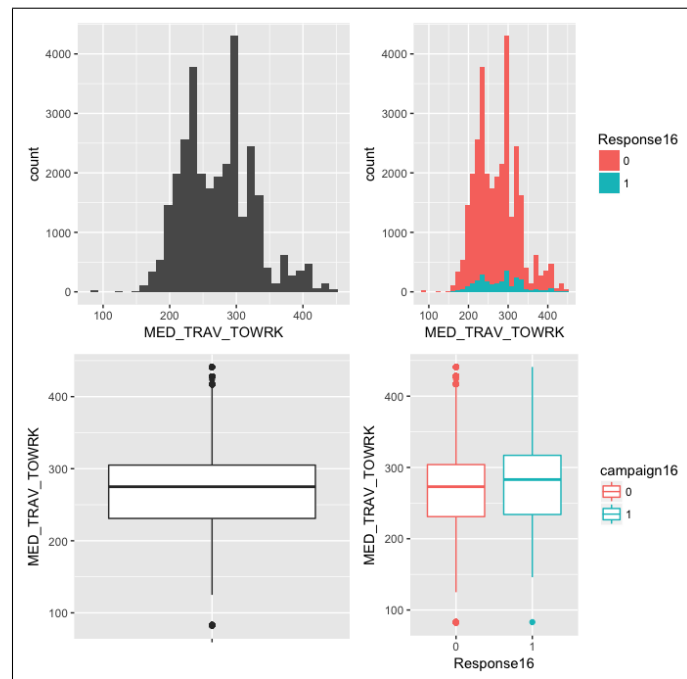


Figure 2.1: Histogram and Boxplot for Total Amount Prior to Campaign 16

Missing Data Imputation

Many of the variables had missing values or observations. To simplify matters, all factors that were either missing values or had the letter U², were classified as "Unknown". For any numeric variable, the missing values were imputed with the median value (the median value was calculated by removing the missing values first).

Feature Engineering

Nine feature variables were created for this analysis. Table 7.1 in the Appendix summarizes the new variables that were constructed. Of note is the variable *high.value.acct*. This variable makes use of the 90th

²the character U was construed as meaning "unknown" and this was used consistently in the provided data dictionary

percentile of total revenue amount prior to the 16th campaign. However, for simplicity's sake, the value was rounded down to \$500.

Naïve Random Forest Model

Since the initial variable reduction (completed subjectively) resulted in 163 potential predictors, a naïve random forest model was constructed to help highlight the top 25 predictors using the gini impurity index. Figure 2.2 lists the top 25 variables.

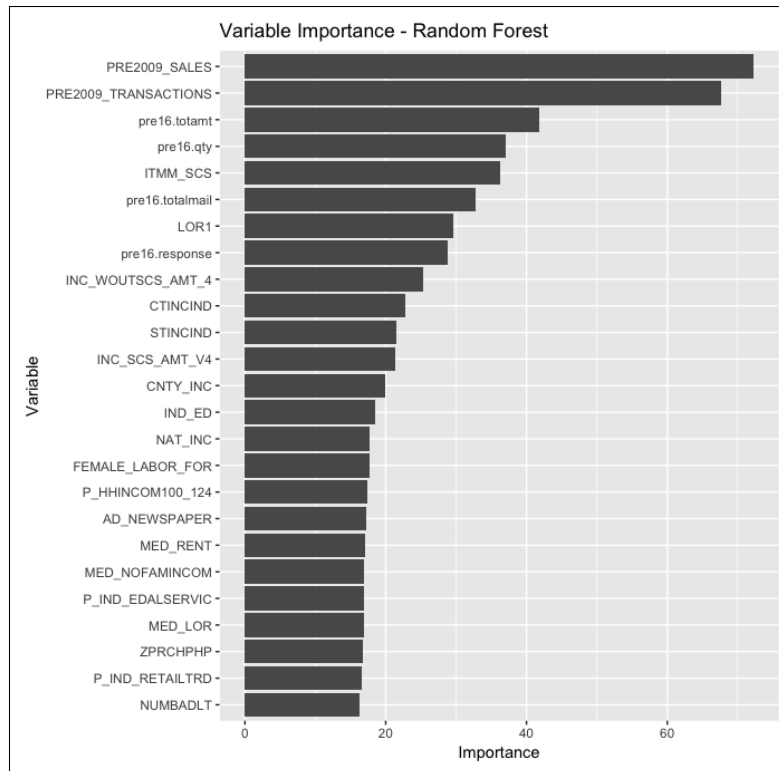


Figure 2.2: Variable Importance Plot

Note the steep drop off in variable importance after the first two variables. It was concluded that these top 25 variables would be used in the model development phase as it would offer suitable trade-off between accuracy and performance.

Model Development

Three different models were constructed for this analysis: Random Forest, Generalized Linear Model, and Naïve Bayes. Each model was developed using the 16th campaign response (RESPONSE16³) as the response variable. Two distinct datasets were constructed: one for use to modeling the responses to the 16th campaign and another to predict which accounts should get the 17th campaign mailers. In order to model the 16th campaign, a subset of the overall data was created by setting the ANY_MAIL_16 variable equal to 1. Once this was done, this dataset was split into a training partition (consisting of 80% of the observations) and a validation partition (consisting of the remaining 20% of the observations). Once a

³this variable has 2 classes: 0 for no mailer and 1 for if mailer was sent

preferred model was derived, a new dataset was created by setting the ANY_MAIL_16 variable equal to 0⁴. This dataset acted like the test dataset for which predictions would be made against.

All three models were used to classify the appropriate response as well as the probabilities for each class. Each model was assessed using several key attributes: the accuracy, sensitivity, specificity, and Kappa⁵ for the classification and AUC (area under the curve) for the probabilities. All metrics were compared holistically to determine the best performing model and are discussed more in Model Results & Conclusion.

Model 1: Random Forest

The package 'ranger' was used to develop the random forest model. 1000 trees were constructed and five random variables were considered at each split within each tree. The confusion matrices for both the in-sample (i.e., the training dataset) and out-of-sample (i.e., the validation dataset) are in Table 3.1.

Table 3.1: Random Forest: In-Sample & Out-of-Sample Confusion Matrix

In-Sample			Out-of-Sample		
	Observed: 0	Observed: 1		Observed: 0	Observed: 1
Predicted: 0	10786	0	Predicted: 1	2693	286
Predicted: 1	0	1152	Predicted: 1	3	2

Note how the random forest model performed exceptionally well in the in-sample dataset, but performed poorly in the out-of-sample dataset. Table 3.4 in the Model Results & Conclusion summarizes the model metrics for the random forest model (along with the others).

Figure 3.1 highlights the in-sample and out-of-sample ROC curves. The benefit of the AUC curve, especially in a binary outcome problem such as the one in this analysis, is that it is much more robust to class imbalances. Note how the in-sample ROC curve is a perfect 1.

⁴this implies that the account never received a mailer

⁵known as Cohen's Kappa

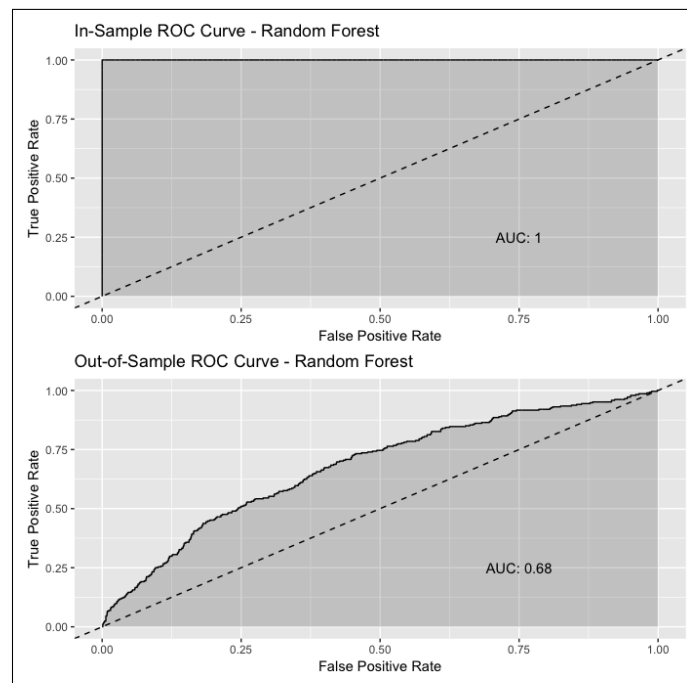


Figure 3.1: Random Forest AUC Curves

Model 2: GLMNet

The GLMNet model was actually an elastic-net model that was constructed. In this model, the regularization was a balance of the lasso and ridge regression techniques. Essentially, the model blended the ability of automated variable selection (lasso) and reducing the coefficients of unimportant variables to zero. This was first achieved by using cross-validation to determine the best α and then another cross-validation to determine the best λ value.

The confusion matrices for both the in-sample (i.e., the training dataset) and out-of-sample (i.e., the validation dataset) are in Table 3.2

Table 3.2: GLMNet: In-Sample & Out-of-Sample Confusion Matrix

	In-Sample			Out-of-Sample	
	Observed: 0	Observed: 1		Observed: 0	Observed: 1
Predicted: 0	10766	1138	Predicted: 0	2691	287
Predicted: 1	20	14	Predicted: 1	5	1

Note how the out-of-sample performance is not much better than the random forest model. Figure 3.2 highlights the in-sample and out-of-sample ROC curves.

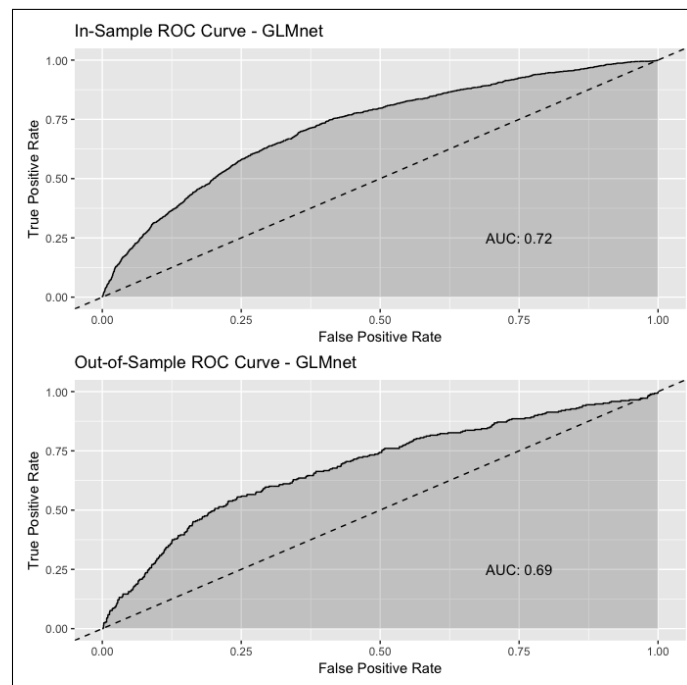


Figure 3.2: GLMNet AUC Curves

Similar to the Random Forest model, the out-of-sample AUC (Area Under the Curve) is similar, but the in-sample curve is not 'perfect'. Table 3.4 in the Model Results & Conclusion summarizes the model metrics for the glmnet model (along with the others).

Model 3: Naïve Bayes

For this model, a 5-fold cross validation was completed using the "naivebayes" method in the package "caret". The confusion matrices for both the in-sample (i.e., the training dataset) and out-of-sample (i.e., the validation dataset) are in Table 3.3.

Table 3.3: Naïve Bayes: In-Sample & Out-of-Sample Confusion Matrix

	In-Sample			Out-of-Sample	
	Observed: 0	Observed: 1		Observed: 0	Observed: 1
Predicted: 0	9999	856	Predicted: 0	2500	225
Predicted: 1	787	296	Predicted: 1	196	63

Note how the out-of-sample classifications are substantially better than both GLMNet and Random Forest. Figure 3.3 highlights the in-sample and out-of-sample ROC curves.

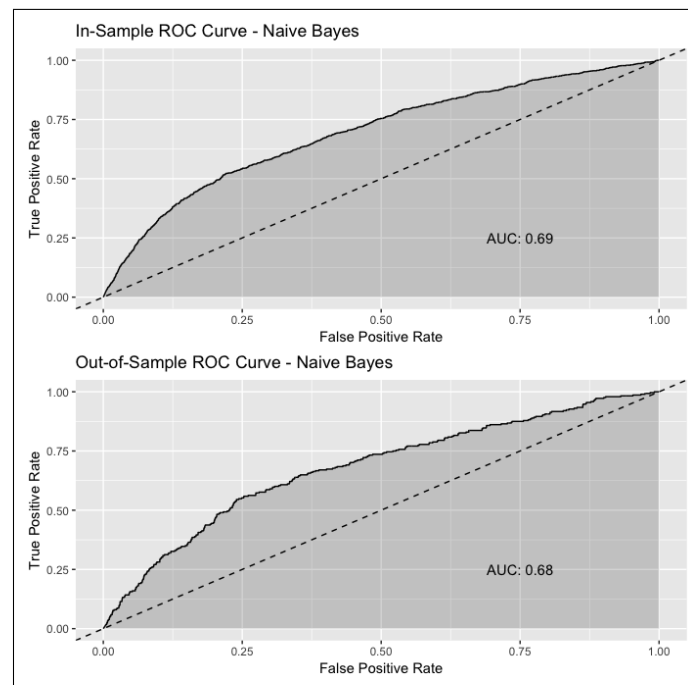


Figure 3.3: GLMNet AUC Curves

The in-sample ROC curves for the Naïve Bayes model is better than the GLMNet model, but nowhere near as well as the Random Forest. However, the out-of-sample AUC for the Naïve Bayes model is very slightly better than both the GLMnet and Random Forest models. Table 3.4 in the Model Results & Conclusion summarizes the model metrics for the Naïve Bayes model (along with the others).

Model Results & Conclusion

Table 3.4 summarizes each of the three models' statistics. All three models perform well in terms of accuracy on the out-of-sample dataset, but the same cannot be said about sensitivity. Recall that sensitivity is the true positive rate and that specificity is the rate of correctly predicted negatives. Note how the Naïve Bayes model had the best sensitivity and the random forest model had the best specificity.

Table 3.4: Model Statistics Summary

	Random Forest		GLMNet		Naïve Bayes	
	In-Sample	Out-of-Sample	In-Sample	Out-of-Sample	In-Sample	Out-of-Sample
accuracy	1	0.903	0.903	0.9025	0.8202	0.859
sensitivity	1	0.007	0.014	0.0104	0.39	0.219
specificity	1	0.999	0.998	0.998	0.866	0.927
Kappa	1	0.011	0.0209	0.0144	0.1989	0.1529
AUC	1	0.676	0.719	0.685	0.694	0.679

In terms of AUC, all three models used a probability of 0.5 as the threshold to determine if a classification should be 1 (mailer sent) or 0 (no mailer sent). Typically in an imbalanced dataset such as this, the probability threshold will have to be different. This threshold could be determined by using the derived ROC curves, and has a direct impact on accuracy and sensitivity of the confusion matrices. Hence, in the test dataset, different probability thresholds will be assessed to determine the optimal net revenue.

Looking at the various metrics holistically, it appears that the Naïve Bayes model performs well. For instance, it has a much more robust Kappa value than the other two models and its sensitivity is much stronger than the others. Therefore, the Naïve model will be used to assess the test dataset and project net revenue.

Net Revenue Summary

The average net profit of sending one mailer was assumed to be approximately \$269. This was a derived value based on the total amount of money spent by customers who had received a mailer for the 16th campaign. In other words, the training dataset was used to derive the aforementioned profit. Within this calculation, the cost of the mailer, assumed to be \$3 per mailer, was included.

Leveraging the Naïve Bayes model, the probabilities of a customer (in the test data set) being sent a mailer was derived. Different probability thresholds were tested to estimate the projected profit. The estimated (i.e., projected profit) was calculated by taking the probability (assuming it was equal to or greater than the threshold) and multiplying by the average profit. Table 4.1 summarizes the expected profit by different probability thresholds.

Table 4.1: Summary of Expected Profit by Probability Threshold

Probability Threshold	Projected Number of Mailers to Send	Expected Profit
0.0	15857	143421.16
0.1	782	115803.83
0.2	579	108262.98
0.3	484	102055.09
0.4	422	96303.79
0.5	388	92204.96
0.6	351	86736.64
0.7	317	80828.39
0.8	289	75134.06
0.9	252	66655.31
1.0	124	33323.7

From an initial perspective, it appears that by sending a mailer to every customer, the XYZ organization can realize maximum profit - given the assumptions made. However, by sending mailers to only 782 (less than 5% of the overall total of 15,857 accounts) accounts, the company only loses out on about \$27,000 in profit. This is suggestive of the fact that many of the accounts have a probability of less than 10% and may not be worthwhile pursuing.

Limitations & Assumptions

Several assumptions were made to achieve the results highlighted in this analysis. For instance, the initial variable reduction technique was highly subjective. This may have resulted in variables being discarded that would have been useful for the overall modeling process. Furthermore, with an inadequate data dictionary, it was not readily apparent on how to develop deep contextual understanding of many of the variables. This, in turn, prevented the ability to select variables that may prove to be more valuable. Another assumption was made on the type of variables chosen. It is possible that some variables may have been marked as factors (i.e., categorical) when in reality they should have been numeric. It is recommended that these

results be reviewed by a subject matter expert so that a deeper contextual understanding can be achieved prior to any key decision making process.

The key limitation of this analysis is that it may not be transferable to other regions or even different customer bases. All of the consumers in this analysis were concentrated in the Greater Chicago-land area and their purchasing habits may be similar. Furthermore, there is no clear context into the actual purchase that a customer made. For example, a customer may have purchased a single high ticket item or numerous items that were of low value but were equal. Another limitation is that the key time data had been removed and this removal may have resulted in the loss of seasonality. Depending on the actual products purchased and in what season, the predicted outcomes may vary. For instance, customers may be willing to purchase more clothing in spring and fall rather than in winter (as an example).

Conclusion & Next Steps

Three different models were constructed for this analysis: random forest, GLMnet, and Naïve Bayes. All three models were assessed using both classification oriented metrics (such as accuracy & sensitivity) in addition to the area under the curve (AUC). Due to the imbalanced dataset, it was essential to look at each metric holistically to ensure that the most impactful model would be chosen. In the end, the Naïve Bayes model was selected due to its overall performance on the training and validation datasets.

Many of the models constructed use a default probability of 0.5 to determine if the binary response variable should be 1 or 0. However, in this analysis different probability thresholds were explored to understand the impact on total profit. Unsurprisingly, the organization can realize a maximum profit by sending a mailer to every account. However, the amount of revenue gained this way versus using a minimal threshold of 10% (which essentially means the probability of a customer making a purchase given that they receive a mailer) is marginal at best.

For next steps, it is suggested that a more robust data dictionary be created. It is also important for the XYZ organization to work more towards data quality to ensure that the end result continue to be relevant. These methods will, undoubtedly, result in improved variable selection that could further improve the overall models.

References

Wickham, H. (2014, 8). Tidy data. *Journal of Statistical Software*, 59(10).

Appendix

EDA

The figure below illustrates the frequency chart for the variable (i.e., predictor) Channel Acquisition. Note how the values are heavily skewed for each type of channel acquisition. This is due to the imbalance in the dataset pertaining to the response of the accounts in the 16th campaign.

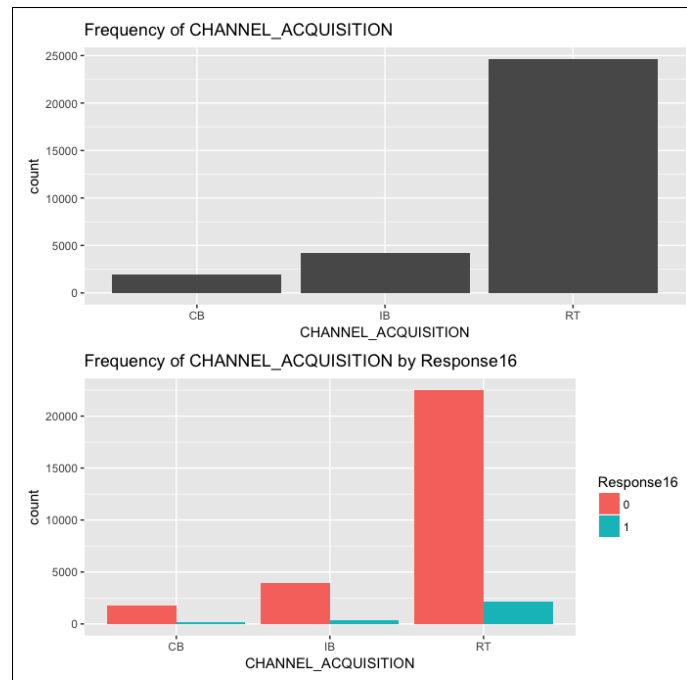


Figure 7.1: Frequency Chart for Channel Acquisition

In the figure below, note the difference in the distribution between a response of 0 or 1 in campaign 16. Unsurprisingly, the histograms are skewed heavily to the right due to no revenue for the large number of mailers sent.

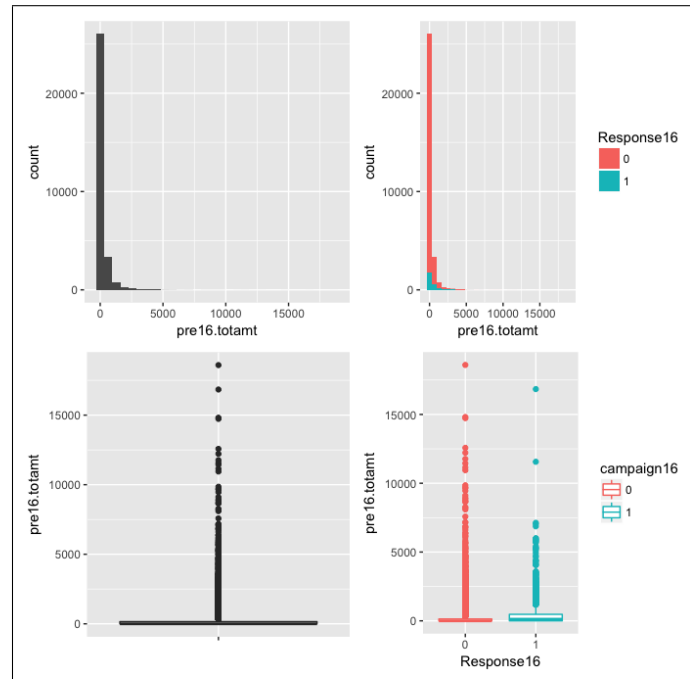


Figure 7.2: Histogram and Boxplot for Total Amount Prior to Campaign 16

The figure below highlights the county income percentile. Note how the median customers who had a positive response to the campaign seem to be from a slightly higher percentile than ones from a negative response. Furthermore, it is interesting to note that many of the respondents come from a high county income percentile.

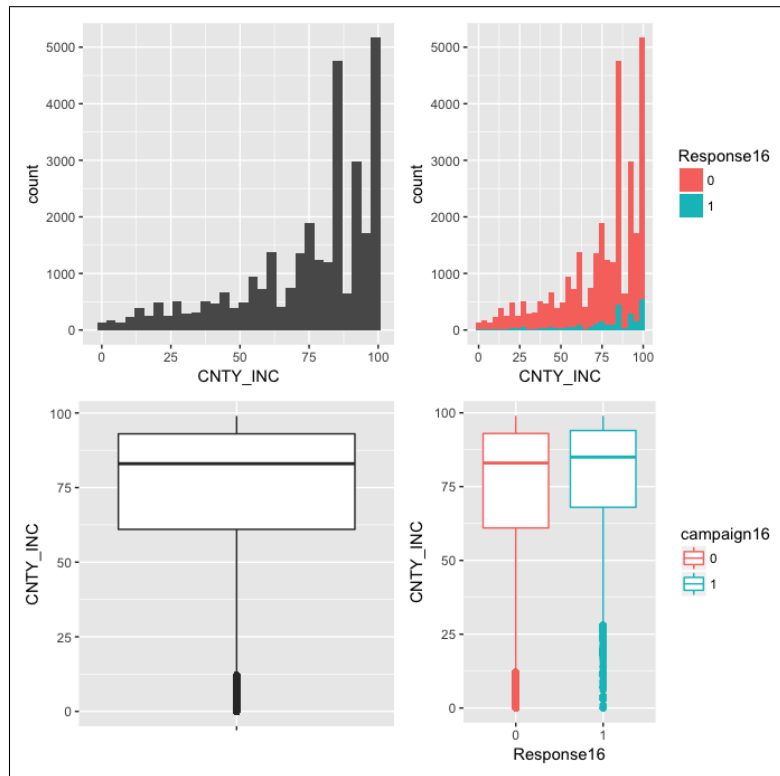


Figure 7.3: Histogram and Boxplot for Total Amount Prior to Campaign 16

Feature Engineering

Table 7.1 lists the features that were created for the analysis along with a brief description.

Table 7.1: Descriptions of Feature Variables

New Variable	Type	Description
pre16.qty	numeric	number of items ordered before the 16th campaign
pre16.totamt	numeric	total revenue before the 16th campaign
high.value.acct	factor	true if the account had \$500 or more in sales prior to the 16th campaign
pre16.response	numeric	total number of responses prior to the 16th campaign
pre16.totalmail	numeric	total number of mailers prior to the 16th campaign
pre16.buyer.status	factor	true if the account had purchased at least once prior to the 16th campaign
pre16.got.mailer	factor	true if the account had received any mailer prior to the 16th campaign
pre16.bought.with.mailer	factor	true if the account had purchased anything if sent a mailer
pre16.bought.without.mailer	factor	true if the account had purchased anything if no mailer was sent

R Code

The R code for this assignment was developed using a modular approach. Several files were created and the file called "01-executor.R" was used to load and process the dataset. Each file is documented below.

01-executor.R

```
# Nikhil Agarwal
# PREDICT 450
# Northwestern University

# THIS FILE IS USED TO LOAD THE APPROPRIATE FILES

# load libraries
library(tidyverse) # ez coding
library(caret)     # model development
library(gridExtra) # multiple grids

# Load the script that loads data frames ----

source('scripts/02-load-data.R')
source('scripts/03-condense.R')
source('scripts/03a-campaign17-df.R')
source('scripts/04-convert-data-type.R')
source('scripts/06-impute.R')
source('scripts/07-data-split.R')

# activate Multicore processing (macs only)
library(doParallel)
registerDoParallel(cores=2)
```

02-load-data.R

```
# Nikhil Agarwal

# Load the data files ----

load('data/XYZ_complete_customer_data_frame.RData')

# store dataframe in a new named dataframe

customer.data <- complete.customer.data.frame

# need to replace the "" and " " with NA's
# assume that all U's are unknowns and equal to NAs

customer.data[customer.data == "" | customer.data == " " | customer.data == 'U'] <- NA
```

03-condense.R

```
# data consolidation

# first consolidate the number of columns

keep.vars <- c(
  'ACCTNO',
  'ANY_MAIL_16',
  'CHANNEL_ACQUISITION',
  'ZIP',
  'INC_SCS_AMT_V4',
  'INC_WIOUTSCS_V4',
  'INC_WITHSCS_V4',
  'INC_WOUTSCS_AMT_4',
  'FIPSCNTY',
  'P_HHINCOLES10M',
  'P_HHINCOM10_14',
  'P_HHINCOM15_19',
  'P_HHINCOM20_24',
  'P_HHINCOM25_29',
  'P_HHINCOM30_34',
  'P_HHINCOM35_39',
  'P_HHINCOM40_44',
  'P_HHINCOM45_49',
  'P_HHINCOM50_59',
  'P_HHINCOM60_74',
  'P_HHINCOM75_99',
  'P_HHINCOM100_124',
  'P_HHINCOM125_149',
  'P_HHINCOM150_199',
  'P_HHINCOM20_UP',
  'MED_INC',
  'MED_FAMINCOM',
  'MED_NOFAMINCOM',
  'P_CAPITA_INCOM',
  'FEMALE_LABOR_FOR',
  'P_IND_AGRICULTUR',
  'P_IND_MINING',
  'P_IND_CONSTRUCT',
  'P_IND_MANUFACT',
  'P_IND_WHOLESALE',
  'P_IND_RETAILTRD',
  'P_IND_TRANSPORT',
  'P_IND_UTILITIE',
  'P_IND_INFORMAT',
  'P_IND_FINALNCE',
  'P_IND_REALESTAT',
  'P_IND_PROFFES',
  'P_IND_MANAGE',
  'P_IND_ADM_SUPPOT',
  'P_IND_EDALSERVIC',
  'P_IND_HEALTHCARE',
  'P_IND_ARTS',
  'P_IND_ACCOMMOD',
  'P_IND_OTHERSERV',
```


'P_IND_PUBADMIS',
 'PCARPOOL',
 'MED_TRAV_TOWRK',
 'MED_HOME',
 'MED_RENT',
 'MED_DWELL_AGE',
 'MED_LOR',
 'HOUSE_STABILITY',
 'CUR_EST_MED_INC',
 'CENSUS_SEG1',
 'EXAGE',
 'ESTAGE',
 'ADULT1_G',
 'MARRIED',
 'RELIGION',
 'ETHNIC_COUNTRY',
 'ADD_TYPE',
 'LOR1',
 'DUS',
 'NUM_CHILD',
 'C_00_03',
 'C_04_06',
 'C_07_09',
 'C_10_12',
 'C_13_18',
 'NUMBADLT',
 'IND_ED',
 'CNTY_INC',
 'NAT_INC',
 'MAILPREF',
 'ITMM',
 'ITMM_SCS',
 'NEWCAR',
 'USED CAR',
 'AD_WEB',
 'AD_MAGAZINE',
 'AD_NEWSPAPER',
 'AD_RADIO',
 'AD_TV',
 'STINCIND',
 'CTINCIND',
 'ESTHMVL',
 'ECHVINX',
 'ZGOLFERP',
 'ZDONORSP',
 'ZPETSP',
 'ZARTSP',
 'ZMOBP',
 'ZFITNESP',
 'ZOUTDOOP',
 'ZTRAVANP',
 'ZINVESTP',
 'ZAUTOOWP',
 'ZGARDENP',
 'ZCOLLECP',
 'ZCRUISEP',
 'ZSPORTSP',
 'ZSWEEPSP',
 'ZPOLITIP',

'ZMUSICP',
'ZREADP',
'ZCHLDPRP',
'ZDIYP',
'ZSELFIPP',
'ZRELIGOP',
'ZGRANDPP',
'ZCLOTHNP',
'ZDONENVP',
'ZMUTUALP',
'ZWGHTCOP',
'ZPRCHPHP',
'ZPRCHTVP',
'ZMOBMULP',
'ZDOGSP',
'ZCATSP',
'ZHEALTHP',
'ZAUTOINP',
'ZSKIINGP',
'ZASTRLGP',
'ZBOATSP',
'ZCELLP',
'ZCOMMCOP',
'ZHMDECOP',
'ZHOMEENP',
'ZKITCHEP',
'ZMOBAVP',
'ZMOBBOOP',
'ZMOBCLTP',
'ZMOBFINP',
'ZMOBGIFP',
'ZMOBGRDP',
'ZMOBJWLP',
'ZMUSCLAP',
'ZMUSCNTP',
'ZMUSCRSP',
'ZMUSOLDP',
'ZMUSROCP',
'ZPBCAREP',
'ZPHOTOP',
'ZPRCHONP',
'ZTENNISP',
'ZTRAVDOP',
'ZTRAVFOP',
'ZVOLUNTP',
'PRE2009_SALES',
'PRE2009_TRANSACTIONS',
'TOTAL_MAIL_1',
'TOTAL_MAIL_2',
'TOTAL_MAIL_3',
'TOTAL_MAIL_4',
'TOTAL_MAIL_5',
'TOTAL_MAIL_6',
'TOTAL_MAIL_7',
'TOTAL_MAIL_8',
'TOTAL_MAIL_9',
'TOTAL_MAIL_10',
'TOTAL_MAIL_11',
'TOTAL_MAIL_12',

```

'TOTAL_MAIL_13',
'TOTAL_MAIL_14',
'TOTAL_MAIL_15',
'QTY',
'TOTAMT',
'QTY16',
'TOTAMT16',
'RESPONSE0',
'RESPONSE1',
'RESPONSE2',
'RESPONSE3',
'RESPONSE4',
'RESPONSE5',
'RESPONSE6',
'RESPONSE7',
'RESPONSE8',
'RESPONSE9',
'RESPONSE10',
'RESPONSE11',
'RESPONSE12',
'RESPONSE13',
'RESPONSE14',
'RESPONSE15',
'RESPONSE16'
)

customer.data.clean <- customer.data %>%
select(keep.vars)

# condense the total mail, qty, totamt, response to *.pre2009

customer.data.clean <- customer.data.clean %>%
mutate(
pre16.qty = QTY - QTY16,
pre16.totamt = TOTAMT - TOTAMT16,
high.value.account = ifelse(pre16.totamt > 500, 1, 0), # roughly 90%
pre16.response = RESPONSE0 + RESPONSE1 + RESPONSE2 + RESPONSE3 + RESPONSE4 + RESPONSE5 +
  RESPONSE6 + RESPONSE7 + RESPONSE8 + RESPONSE9 + RESPONSE10 + RESPONSE11 + RESPONSE12 +
  RESPONSE13 + RESPONSE14 + RESPONSE15,
pre16.totalmail = TOTAL_MAIL_15,
pre16.buyer.status = ifelse(pre16.totamt > 0, 1, 0),
pre16.got.mailer = ifelse(pre16.totalmail > 0, 1, 0),
pre16.bought.with.mailer = ifelse(pre16.got.mailer == 1 & pre16.totamt > 0, 1, 0), # had
  mailer and bought
pre16.bought.withnomailer = ifelse(pre16.got.mailer == 0 & pre16.totamt > 0, 1, 0), # had no
  mailer and bought
campaign16 = RESPONSE16
) %>%
select(-starts_with('RESPONSE'), -starts_with('TOTAL_MAIL'), -starts_with('TOTAMT'), -QTY, -
  QTY16)

# combine the EXAGE and ESTAGE

customer.data.clean <- customer.data.clean %>%
mutate(derived.age = ifelse(is.na(EXAGE), ESTAGE, EXAGE)) %>%
select(-EXAGE, -ESTAGE)

```

```
# the MARRIED variable has 2 bytes. the 1st byte tells us the predicted likelihood (5 is
  extremely likely, 1 is likely, 0 unknown). See the Experian data dictionary provided by
  the professor.

customer.data.clean <- customer.data.clean %>%
mutate(
predicted.marital.status = case_when(
substring(MARRIED, 1,1) == '1' ~ 'LIKELY_MARRIED',
substring(MARRIED, 1, 1) == '5' ~ 'STRONGLY_MARRIED',
TRUE ~ 'UNKNOWN'
)
) %>%
select(-MARRIED)
```

04-convert-data-type.R

```
# impute missing values before converting to appropriate types

customer.data.clean <- customer.data.clean %>%
transmute(
ACCTNO = as.character(ACCTNO),
ANY_MAIL_16,
CHANNEL_ACQUISITION = as.factor(CHANNEL_ACQUISITION),
ZIP = as.factor(ZIP),
INC_SCS_AMT_V4 = as.numeric(INC_SCS_AMT_V4),
INC_WIOUTSCS_V4 = as.factor(INC_WIOUTSCS_V4),
INC_WITHSCS_V4 = as.factor(INC_WITHSCS_V4),
INC_WOUTSCS_AMT_4 = as.numeric(INC_WOUTSCS_AMT_4),
FIPSCNTY = as.factor(FIPSCNTY),
P_HHINCOLES10M = as.numeric(P_HHINCOLES10M),
P_HHINCOM10_14 = as.numeric(P_HHINCOM10_14),
P_HHINCOM15_19 = as.numeric(P_HHINCOM15_19),
P_HHINCOM20_24 = as.numeric(P_HHINCOM20_24),
P_HHINCOM25_29 = as.numeric(P_HHINCOM25_29),
P_HHINCOM30_34 = as.numeric(P_HHINCOM30_34),
P_HHINCOM35_39 = as.numeric(P_HHINCOM35_39),
P_HHINCOM40_44 = as.numeric(P_HHINCOM40_44),
P_HHINCOM45_49 = as.numeric(P_HHINCOM45_49),
P_HHINCOM50_59 = as.numeric(P_HHINCOM50_59),
P_HHINCOM60_74 = as.numeric(P_HHINCOM60_74),
P_HHINCOM75_99 = as.numeric(P_HHINCOM75_99),
P_HHINCOM100_124 = as.numeric(P_HHINCOM100_124),
P_HHINCOM125_149 = as.numeric(P_HHINCOM125_149),
P_HHINCOM150_199 = as.numeric(P_HHINCOM150_199),
P_HHINCOM20_UP = as.numeric(P_HHINCOM20_UP),
MED_INC = as.numeric(MED_INC),
MED_FAMINCOM = as.numeric(MED_FAMINCOM),
MED_NOFAMINCOM = as.numeric(MED_NOFAMINCOM),
P_CAPITA_INCOM = as.numeric(P_CAPITA_INCOM),
FEMALE_LABOR_FOR = as.numeric(FEMALE_LABOR_FOR),
P_IND_AGRICULTUR = as.numeric(P_IND_AGRICULTUR),
P_IND_MINING = as.numeric(P_IND_MINING),
P_IND_CONSTRUCT = as.numeric(P_IND_CONSTRUCT),
P_IND_MANUFACT = as.numeric(P_IND_MANUFACT),
P_IND_WHOLESALE = as.numeric(P_IND_WHOLESALE),
P_IND_RETAILTRD = as.numeric(P_IND_RETAILTRD),
P_IND_TRANSPORT = as.numeric(P_IND_TRANSPORT),
```

```

P_IND_UTILITIE = as.numeric(P_IND_UTILITIE),
P_IND_INFORMAT = as.numeric(P_IND_INFORMAT),
P_IND_FINALNCE = as.numeric(P_IND_FINALNCE),
P_IND_REALESTAT = as.numeric(P_IND_REALESTAT),
P_IND_PROFFES = as.numeric(P_IND_PROFFES),
P_IND_MANAGE = as.numeric(P_IND_MANAGE),
P_IND_ADM_SUPPOT = as.numeric(P_IND_ADM_SUPPOT),
P_IND_EDALSERVIC = as.numeric(P_IND_EDALSERVIC),
P_IND_HEALTHCARE = as.numeric(P_IND_HEALTHCARE),
P_IND_ARTS = as.numeric(P_IND_ARTS),
P_IND_ACCOMMOD = as.numeric(P_IND_ACCOMMOD),
P_IND_OTHERSERV = as.numeric(P_IND_OTHERSERV),
P_IND_PUBADMIS = as.numeric(P_IND_PUBADMIS),
PCARPOOL = as.numeric(PCARPOOL),
MED_TRAV_TOWRK = as.numeric(MED_TRAV_TOWRK),
MED_HOME = as.numeric(MED_HOME),
MED_RENT = as.numeric(MED_RENT),
MED_DWELL_AGE = as.numeric(MED_DWELL_AGE),
MED_LOR = as.numeric(MED_LOR),
HOUSE_STABILITY = as.numeric(HOUSE_STABILITY),
CUR_EST_MED_INC = as.numeric(CUR_EST_MED_INC),
CENSUS_SEG1 = as.factor(CENSUS_SEG1),
ADULT1_G = as.factor(ADULT1_G),
RELIGION = as.factor(RELIGION),
ETHNIC_COUNTRY = as.factor(ETHNIC_COUNTRY),
ADD_TYPE = as.factor(ADD_TYPE),
LOR1 = as.numeric(LOR1),
DUS = as.factor(DUS),
NUM_CHILD = as.numeric(NUM_CHILD),
C_00_03 = as.factor(C_00_03),
C_04_06 = as.factor(C_04_06),
C_07_09 = as.factor(C_07_09),
C_10_12 = as.factor(C_10_12),
C_13_18 = as.factor(C_13_18),
NUMBADLT = as.numeric(NUMBADLT),
IND_ED = as.factor(IND_ED),
CNTY_INC = as.numeric(CNTY_INC),
NAT_INC = as.numeric(NAT_INC),
MAILPREF = as.factor(MAILPREF),
ITMM = as.factor(ITMM),
ITMM_SCS = as.numeric(ITMM_SCS),
NEWCAR = as.factor(NEWCAR),
USED CAR = as.factor(USED CAR),
AD_WEB = as.factor(AD_WEB),
AD_MAGAZINE = as.factor(AD_MAGAZINE),
AD_NEWSPAPER = as.factor(AD_NEWSPAPER),
AD_RADIO = as.factor(AD_RADIO),
AD_TV = as.factor(AD_TV),
STINCIND = as.numeric(STINCIND),
CTINCIND = as.numeric(CTINCIND),
ESTHVL = as.numeric(ESTHVL),
ECHVINX = as.numeric(ECHVINX),
ZGOLFERP = as.factor(ZGOLFERP),
ZDONORSP = as.factor(ZDONORSP),
ZPETSP = as.factor(ZPETSP),
ZARTSP = as.factor(ZARTSP),
ZMOBP = as.factor(ZMOBP),
ZFITNESP = as.factor(ZFITNESP),
ZOUTDOOP = as.factor(ZOUTDOOP),

```

```

ZTRAVANP = as.factor(ZTRAVANP),
ZINVESTP = as.factor(ZINVESTP),
ZAUTOOWP = as.factor(ZAUTOOWP),
ZGARDENP = as.factor(ZGARDENP),
ZCOLLECP = as.factor(ZCOLLECP),
ZCRUISEP = as.factor(ZCRUISEP),
ZSPORTSP = as.factor(ZSPORTSP),
ZSWEEPSP = as.factor(ZSWEEPSP),
ZPOLITIP = as.factor(ZPOLITIP),
ZMUSICP = as.factor(ZMUSICP),
ZREADP = as.factor(ZREADP),
ZCHLDPRP = as.factor(ZCHLDPRP),
ZDIYP = as.factor(ZDIYP),
ZSELFIPP = as.factor(ZSELFIPP),
ZRELIGOP = as.factor(ZRELIGOP),
ZGRANDPP = as.factor(ZGRANDPP),
ZCLOTHNP = as.factor(ZCLOTHNP),
ZDONENVP = as.factor(ZDONENVP),
ZMUTUALP = as.factor(ZMUTUALP),
ZWGHTCOP = as.factor(ZWGHTCOP),
ZPRCHPHP = as.factor(ZPRCHPHP),
ZPRCHTVP = as.factor(ZPRCHTVP),
ZMOBMULP = as.factor(ZMOBMULP),
ZDOGSP = as.factor(ZDOGSP),
ZCATSP = as.factor(ZCATSP),
ZHEALTHP = as.factor(ZHEALTHP),
ZAUTOINP = as.factor(ZAUTOINP),
ZSKIINGP = as.factor(ZSKIINGP),
ZASTRLGP = as.factor(ZASTRLGP),
ZBOATSP = as.factor(ZBOATSP),
ZCELLP = as.factor(ZCELLP),
ZCOMMCOPI = as.factor(ZCOMMCOPI),
ZHMDECOP = as.factor(ZHMDECOP),
ZHOMEENP = as.factor(ZHOMEENP),
ZKITCHEP = as.factor(ZKITCHEP),
ZMOBAVP = as.factor(ZMOBAVP),
ZMOBBOOP = as.factor(ZMOBBOOP),
ZMOBCLTP = as.factor(ZMOBCLTP),
ZMOBFINP = as.factor(ZMOBFINP),
ZMOBGIFP = as.factor(ZMOBGIFP),
ZMOBGRDP = as.factor(ZMOBGRDP),
ZMOBJWLP = as.factor(ZMOBJWLP),
ZMUSCLAP = as.factor(ZMUSCLAP),
ZMUSCNTP = as.factor(ZMUSCNTP),
ZMUSCRSP = as.factor(ZMUSCRSP),
ZMUSOLDP = as.factor(ZMUSOLDP),
ZMUSROCP = as.factor(ZMUSROCP),
ZPBCAREP = as.factor(ZPBCAREP),
ZPHOTOP = as.factor(ZPHOTOP),
ZPRCHONP = as.factor(ZPRCHONP),
ZTENNISP = as.factor(ZTENNISP),
ZTRAVDOP = as.factor(ZTRAVDOP),
ZTRAVFOP = as.factor(ZTRAVFOP),
ZVOLUNTP = as.factor(ZVOLUNTP),
PRE2009_SALES = as.numeric(PRE2009_SALES),
PRE2009_TRANSACTIONS = as.numeric(PRE2009_TRANSACTIONS),

pre16.qty = as.numeric(pre16.qty),
pre16.totamt = as.numeric(pre16.totamt),

```

```

high.value.account = as.factor(high.value.account),
pre16.response = as.numeric(pre16.response),
pre16.totalmail = as.numeric(pre16.totalmail),
pre16.buyer.status = as.factor(pre16.buyer.status),
pre16.got.mailer = as.factor(pre16.got.mailer),
pre16.bought.with.mailer = as.factor(pre16.bought.with.mailer),
pre16.bought.withnomailer = as.factor(pre16.bought.withnomailer),

campaign16 = as.factor(campaign16),
derived.age = as.numeric(derived.age),
predicted.marital.status = as.factor(predicted.marital.status)
)

```

05-eda.R

```

# Nikhil Agarwal

# THIS FILE IS USED FOR EDA

# check for missing values ----

customer.data %>%
  map_df(function(x) sum(is.na(x))) %>%
  gather(key = 'variable', value = 'missing.count') %>%
  arrange(desc(missing.count)) %>%
  mutate(
    missing.pct = round(missing.count / nrow(customer.data),3)
  ) %>%
  filter(missing.pct > 0.49) %>%
  View

customer.data %>%
  map_df(function(x) sum(is.na(x))) %>%
  gather(key = 'variable', value = 'missing.count') %>%
  arrange(desc(missing.count)) %>%
  mutate(
    missing.pct = round(missing.count / nrow(customer.data),3)
  ) %>%
  filter(missing.count > 0) %>%
  View

customer.data.clean %>%
  map_df(function(x) sum(is.na(x))) %>%
  gather(key = 'variable', value = 'missing.count') %>%
  # arrange(desc(missing.count)) %>%
  mutate(
    missing.pct = round(missing.count / nrow(customer.data.clean),3)
  ) %>%
  # filter(missing.pct > 0.49) %>%
  View

## quite a few missing values

customer.data %>%

```

```
filter(ANY_MAIL_16 > 0) %>%
select(RESPONSE16) %>%
group_by(RESPONSE16) %>%
summarize(
  cnt = n()
)

1# determine the number of numeric & character columns ----

customer.data %>%
select_if(is.numeric) %>%
summarize(
  numeric.fields = length(colnames(.))
)

customer.data %>%
select_if(is.character) %>%
summarize(
  character.fields = length(colnames(.))
)

# both add up to 554, so we know we only have two types of fields thus far: numeric &
  character

# do a quick glimpse ----

glimpse(customer.data)

# Verify that account number column (ACCTNO) is unique ----

length(unique(customer.data$ACCTNO))

# ACCTNO has all unique values (30,779) w/ no repeating values

# random data exploration ----

customer.data %>%
select(ACCTNO, starts_with('TOTAMT'), starts_with('ANY_MAIL')) %>%
transmute(
  pre16.totamt = TOTAMT - TOTAMT16
) %>%
summarize_all(funs(quantile), probs = 0.9)
head

# summarized table values ----

getNumericSummary <- function(df){
  df <- df %>%
  select_if(is.numeric)

  mean.values <- df %>%
  summarize_all(mean, na.rm = T) %>%
  gather(key = variable, value = avg.value) %>%
```



```

mutate(avg.value = round(avg.value, 3))
min.values <- df %>%
summarise_all(min, na.rm = T) %>%
gather(key = variable, value = minimum.value)
max.values <- df %>%
summarize_all(max, na.rm = T) %>%
gather(key = variable, value = maximum.value)
q05.values <- df %>%
summarize_all(funs(quantile), probs = 0.05, na.rm = T) %>%
gather(key = variable, value = p05.value)
q10.values <- df %>%
summarize_all(funs(quantile), probs = 0.10, na.rm = T) %>%
gather(key = variable, value = p10.value)
q25.values <- df %>%
summarize_all(funs(quantile), probs = 0.25, na.rm = T) %>%
gather(key = variable, value = p25.value)
median.values <- df %>%
summarize_all(median, na.rm = T) %>%
gather(key = variable, value = median.value)
q75.values <- df %>%
summarize_all(funs(quantile), probs = 0.75, na.rm = T) %>%
gather(key = variable, value = p75.value)
q90.values <- df %>%
summarize_all(funs(quantile), probs = 0.90, na.rm = T) %>%
gather(key = variable, value = p90.value)
q95.values <- df %>%
summarize_all(funs(quantile), probs = 0.95, na.rm = T) %>%
gather(key = variable, value = p95.value)
missing.values <- df %>%
summarize_all(funs(sum(is.na(.)))) %>%
gather(key = variable, value = missing)

sd.values <- df %>%
summarize_all(sd, na.rm = T) %>%
gather(key = variable, value = sd) %>%
mutate(sd = round(sd,3))
var.values <- df %>%
summarize_all(var, na.rm = T) %>%
gather(key = variable, value = var) %>%
mutate(var = round(var,3))

descriptive.stats <- min.values %>%
inner_join(median.values, by = 'variable') %>%
inner_join(mean.values, by = 'variable') %>%
inner_join(max.values, by = 'variable') %>%
inner_join(q05.values, by = 'variable') %>%
inner_join(q10.values, by = 'variable') %>%
inner_join(q25.values, by = 'variable') %>%
inner_join(q75.values, by = 'variable') %>%
inner_join(q90.values, by = 'variable') %>%
inner_join(q95.values, by = 'variable') %>%
inner_join(sd.values, by = 'variable') %>%
inner_join(var.values, by = 'variable') %>%
inner_join(missing.values, by = 'variable')

return(descriptive.stats)
}

numericValues.stats <- getNumericSummary(customer.data.clean)

```

```

characterValue.uniqueCount <- customer.data.clean %>%
select_if(is.factor) %>%
summarize_all(funs(length(unique(.)))) %>%
gather(key = variable, value = unique.value.count)

characterValue.missingSummary <- customer.data.clean %>%
select_if(is.factor) %>%
map_df(function(x) sum(is.na(x))) %>%
gather(key = variable, value = missing.cnt) %>%
mutate(
missing.pct = round(missing.cnt / nrow(customer.data.clean),3)
)

characterValue.Summary <- inner_join(characterValue.uniqueCount, characterValue.
missingSummary, by = c('variable' = 'variable'))

# correlation matrix

cor.matrix <- as.data.frame(cor(x = customer.data %>% select_if(is.numeric) %>% select(-
TOTAMT16), y = customer.data$TOTAMT16, method = 'pearson'))

cor.matrix <- rownames_to_column(cor.matrix)

cor.matrix %>%
rename(
variable = rowname,
correlation = V1
) %>%
mutate(
correlation = round(correlation, 3)
) %>%
arrange(desc(correlation)) %>%
head(n = 10)

# graphs ----

customer.data.clean %>%
ggplot(aes(x = ZARTSP)) +
geom_bar(stat = 'count') +
ggtitle('Frequency of ZARTSP')

customer.data.clean %>%
ggplot(aes(x = ZARTSP, fill = campaign16)) +
geom_bar(stat = 'count', position = 'dodge') +
scale_fill_discrete(name = 'Response16') +
ggtitle('Frequency of ZARTSP by Response16')

```

```

library(gridExtra)
# factor graphs ----
getFactorGraphs <- function(df, var) {
  var <- enquo(var)
  a <- df %>%
    ggplot(aes_string(x = rlang::quo_text(var))) +
    geom_bar(stat = 'count') +
    ggtitle(paste('Frequency of ', rlang::quo_text(var), sep = ''))

  b <- df %>%
    ggplot(aes_string(x = rlang::quo_text(var), fill = 'campaign16')) +
    geom_bar(stat = 'count', position = 'dodge') +
    scale_fill_discrete(name = 'Response16') +
    ggtitle(paste('Frequency of ', rlang::quo_text(var), ' by Response16', sep = ''))

  grid.arrange(a, b, nrow = 2)
}

getFactorGraphs(customer.data.clean, CHANNEL_ACQUISITION)
getFactorGraphs(customer.data.clean, ZIP)
getFactorGraphs(customer.data.clean, INC_WIOUTSCS_V4)
getFactorGraphs(customer.data.clean, INC_WITHSCS_V4)
getFactorGraphs(customer.data.clean, FIPSSCTD)
getFactorGraphs(customer.data.clean, FIPSCNTY)
getFactorGraphs(customer.data.clean, CENSUS_SEG1)
getFactorGraphs(customer.data.clean, ADULT1_G)
getFactorGraphs(customer.data.clean, RELIGION)
getFactorGraphs(customer.data.clean, ETHNIC_COUNTRY)
getFactorGraphs(customer.data.clean, ADD_TYPE)
getFactorGraphs(customer.data.clean, DUS)
getFactorGraphs(customer.data.clean, C_00_03)
getFactorGraphs(customer.data.clean, C_04_06)
getFactorGraphs(customer.data.clean, C_07_09)
getFactorGraphs(customer.data.clean, C_10_12)
getFactorGraphs(customer.data.clean, C_13_18)
getFactorGraphs(customer.data.clean, IND_ED)
getFactorGraphs(customer.data.clean, MAILPREF)
getFactorGraphs(customer.data.clean, ITMM)
getFactorGraphs(customer.data.clean, NEWCAR)
getFactorGraphs(customer.data.clean, USED CAR)
getFactorGraphs(customer.data.clean, AD_WEB)
getFactorGraphs(customer.data.clean, AD_MAGAZINE)
getFactorGraphs(customer.data.clean, AD_NEWSPAPER)
getFactorGraphs(customer.data.clean, AD_RADIO)
getFactorGraphs(customer.data.clean, AD_TV)
getFactorGraphs(customer.data.clean, ZGOLFERP)
getFactorGraphs(customer.data.clean, ZDONORSP)
getFactorGraphs(customer.data.clean, ZPETSP)
getFactorGraphs(customer.data.clean, ZARTSP)
getFactorGraphs(customer.data.clean, ZMOBP)
getFactorGraphs(customer.data.clean, ZFITNESP)
getFactorGraphs(customer.data.clean, ZOUTDOOP)
getFactorGraphs(customer.data.clean, ZTRAVANP)
getFactorGraphs(customer.data.clean, ZINVESTP)
getFactorGraphs(customer.data.clean, ZAUTOOWP)
getFactorGraphs(customer.data.clean, ZGARDENP)
getFactorGraphs(customer.data.clean, ZCOLLECP)
getFactorGraphs(customer.data.clean, ZCRUISEP)
getFactorGraphs(customer.data.clean, ZSPORTSP)

```

```

getFactorGraphs(customer.data.clean, ZSWEEPSP)
getFactorGraphs(customer.data.clean, ZPOLITIP)
getFactorGraphs(customer.data.clean, ZMUSICP)
getFactorGraphs(customer.data.clean, ZREADP)
getFactorGraphs(customer.data.clean, ZCHLDPRP)
getFactorGraphs(customer.data.clean, ZDIYP)
getFactorGraphs(customer.data.clean, ZSELFIPP)
getFactorGraphs(customer.data.clean, ZRELIGOP)
getFactorGraphs(customer.data.clean, ZGRANDPP)
getFactorGraphs(customer.data.clean, ZCLOTHNP)
getFactorGraphs(customer.data.clean, ZDONENVP)
getFactorGraphs(customer.data.clean, ZMUTUALP)
getFactorGraphs(customer.data.clean, ZWGHTCOP)
getFactorGraphs(customer.data.clean, ZPRCHPHP)
getFactorGraphs(customer.data.clean, ZPRCHTVP)
getFactorGraphs(customer.data.clean, ZMOBMULP)
getFactorGraphs(customer.data.clean, ZDOGSP)
getFactorGraphs(customer.data.clean, ZCATSP)
getFactorGraphs(customer.data.clean, ZHEALTHP)
getFactorGraphs(customer.data.clean, ZAUTOINP)
getFactorGraphs(customer.data.clean, ZSKIINGP)
getFactorGraphs(customer.data.clean, ZASTRLGP)
getFactorGraphs(customer.data.clean, ZBOATSP)
getFactorGraphs(customer.data.clean, ZCELLP)
getFactorGraphs(customer.data.clean, ZCOMMCP)
getFactorGraphs(customer.data.clean, ZHMDECOP)
getFactorGraphs(customer.data.clean, ZHOMEENP)
getFactorGraphs(customer.data.clean, ZKITCHEP)
getFactorGraphs(customer.data.clean, ZMOBAVP)
getFactorGraphs(customer.data.clean, ZMOBBOOP)
getFactorGraphs(customer.data.clean, ZMOBCLTP)
getFactorGraphs(customer.data.clean, ZMOBFINP)
getFactorGraphs(customer.data.clean, ZMOBGIFP)
getFactorGraphs(customer.data.clean, ZMOBGRDP)
getFactorGraphs(customer.data.clean, ZMOBJWLP)
getFactorGraphs(customer.data.clean, ZMUSCLAP)
getFactorGraphs(customer.data.clean, ZMUSCNTP)
getFactorGraphs(customer.data.clean, ZMUSCRSP)
getFactorGraphs(customer.data.clean, ZMUSOLDP)
getFactorGraphs(customer.data.clean, ZMUSROCP)
getFactorGraphs(customer.data.clean, ZPBCAREP)
getFactorGraphs(customer.data.clean, ZPHOTOP)
getFactorGraphs(customer.data.clean, ZPRCHONP)
getFactorGraphs(customer.data.clean, ZTENNISP)
getFactorGraphs(customer.data.clean, ZTRAVDOP)
getFactorGraphs(customer.data.clean, ZTRAVFOP)
getFactorGraphs(customer.data.clean, ZVOLUNTP)
getFactorGraphs(customer.data.clean, campaign16)
getFactorGraphs(customer.data.clean, predicted.marital.status)
getFactorGraphs(customer.data.clean, pre16.buyer.status)

```

```
# numeric graphs
```

```

customer.data.clean %>%
select(derived.age) %>%
na.omit() %>%
ggplot(aes(x = derived.age)) +
geom_histogram(bins = 30)

```

```

customer.data.clean %>%
select(derived.age, campaign16) %>%
na.omit() %>%
ggplot(aes(x = derived.age, fill = campaign16)) +
geom_histogram(bins = 30)

customer.data.clean %>%
select(derived.age) %>%
na.omit() %>%
ggplot(aes(x = '', y = derived.age)) +
geom_boxplot()

customer.data.clean %>%
select(derived.age, campaign16) %>%
na.omit() %>%
ggplot(aes(x = campaign16, y = derived.age, color = campaign16)) +
geom_boxplot()

#https://stackoverflow.com/questions/28777626/how-do-i-combine-aes-and-aes-string-options
'+.uneval' <- function(a,b) {
  'class<-'(modifyList(a,b), "uneval")
}

getNumericGraphs <- function(df, var) {
  var <- enquo(var)

  a <- df %>%
  select(!var) %>%
  na.omit() %>%
  ggplot(aes_string(x = rlang::quo_text(var))) +
  geom_histogram(bins = 30)

  b <- df %>%
  select(!var, campaign16) %>%
  na.omit() %>%
  ggplot(aes_string(x = rlang::quo_text(var), fill = 'campaign16')) +
  geom_histogram(bins = 30) +
  scale_fill_discrete(name = 'Response16')

  c <- df %>%
  select(!var) %>%
  na.omit() %>%
  ggplot(aes_string(y = rlang::quo_text(var)) + aes(x = '')) +
  geom_boxplot() +
  labs(x = '')

  d <- df %>%
  select(!var, campaign16) %>%
  na.omit() %>%
  ggplot(aes(x = campaign16) + aes_string(y = rlang::quo_text(var)) + aes(color = campaign16))
  +
  geom_boxplot() +
  labs(x = 'Response16') +
  scale_fill_discrete('Response16')

  grid.arrange(a,b,c,d, ncol = 2, nrow = 2)

```

```
}

```

```

getNumericGraphs(customer.data.clean, INC_SCS_AMT_V4)
getNumericGraphs(customer.data.clean, INC_WOUTSCS_AMT_4)
getNumericGraphs(customer.data.clean, P_HHINCOLES10M)
getNumericGraphs(customer.data.clean, P_HHINCOM10_14)
getNumericGraphs(customer.data.clean, P_HHINCOM15_19)
getNumericGraphs(customer.data.clean, P_HHINCOM20_24)
getNumericGraphs(customer.data.clean, P_HHINCOM25_29)
getNumericGraphs(customer.data.clean, P_HHINCOM30_34)
getNumericGraphs(customer.data.clean, P_HHINCOM35_39)
getNumericGraphs(customer.data.clean, P_HHINCOM40_44)
getNumericGraphs(customer.data.clean, P_HHINCOM45_49)
getNumericGraphs(customer.data.clean, P_HHINCOM50_59)
getNumericGraphs(customer.data.clean, P_HHINCOM60_74)
getNumericGraphs(customer.data.clean, P_HHINCOM75_99)
getNumericGraphs(customer.data.clean, P_HHINCOM100_124)
getNumericGraphs(customer.data.clean, P_HHINCOM125_149)
getNumericGraphs(customer.data.clean, P_HHINCOM150_199)
getNumericGraphs(customer.data.clean, P_HHINCOM20_UP)
getNumericGraphs(customer.data.clean, MED_INC)
getNumericGraphs(customer.data.clean, MED_FAMINCOM)
getNumericGraphs(customer.data.clean, MED_NOFAMINCOM)
getNumericGraphs(customer.data.clean, P_CAPITA_INCOM)
getNumericGraphs(customer.data.clean, FEMALE_LABOR_FOR)
getNumericGraphs(customer.data.clean, P_IND_AGRICULTUR)
getNumericGraphs(customer.data.clean, P_IND_MINING)
getNumericGraphs(customer.data.clean, P_IND_CONSTRUCT)
getNumericGraphs(customer.data.clean, P_IND_MANUFACT)
getNumericGraphs(customer.data.clean, P_IND_WHOLESALE)
getNumericGraphs(customer.data.clean, P_IND_RETAILTRD)
getNumericGraphs(customer.data.clean, P_IND_TRANSPORT)
getNumericGraphs(customer.data.clean, P_IND_UTILITIE)
getNumericGraphs(customer.data.clean, P_IND_INFORMAT)
getNumericGraphs(customer.data.clean, P_IND_FINALNCE)
getNumericGraphs(customer.data.clean, P_IND_REALESTAT)
getNumericGraphs(customer.data.clean, P_IND_PROFFES)
getNumericGraphs(customer.data.clean, P_IND_MANAGE)
getNumericGraphs(customer.data.clean, P_IND_ADM_SUPPOT)
getNumericGraphs(customer.data.clean, P_IND_EDALSERVIC)
getNumericGraphs(customer.data.clean, P_IND_HEALTHCARE)
getNumericGraphs(customer.data.clean, P_IND_ARTS)
getNumericGraphs(customer.data.clean, P_IND_ACCOMMOD)
getNumericGraphs(customer.data.clean, P_IND_OTHERSERV)
getNumericGraphs(customer.data.clean, P_IND_PUBADMIS)
getNumericGraphs(customer.data.clean, PCARPOOL)
getNumericGraphs(customer.data.clean, MED_TRAV_TOWRK)
getNumericGraphs(customer.data.clean, MED_HOME)
getNumericGraphs(customer.data.clean, MED_RENT)
getNumericGraphs(customer.data.clean, MED_DWELL_AGE)
getNumericGraphs(customer.data.clean, MED_LOR)
getNumericGraphs(customer.data.clean, HOUSE_STABILITY)
getNumericGraphs(customer.data.clean, CUR_EST_MED_INC)
getNumericGraphs(customer.data.clean, LOR1)
getNumericGraphs(customer.data.clean, NUM_CHILD)
getNumericGraphs(customer.data.clean, NUMBADLT)
getNumericGraphs(customer.data.clean, CNTY_INC)
getNumericGraphs(customer.data.clean, NAT_INC)
getNumericGraphs(customer.data.clean, ITMM_SCS)

```

```

getNumericGraphs(customer.data.clean, STINCIND)
getNumericGraphs(customer.data.clean, CTINCIND)
getNumericGraphs(customer.data.clean, ESTHML)
getNumericGraphs(customer.data.clean, ECHVINX)
getNumericGraphs(customer.data.clean, PRE2009_SALES)
getNumericGraphs(customer.data.clean, PRE2009_TRANSACTIONS)
getNumericGraphs(customer.data.clean, pre2009.qty)
getNumericGraphs(customer.data.clean, pre2009.totamt)
getNumericGraphs(customer.data.clean, pre2009.response)
getNumericGraphs(customer.data.clean, pre2009.totalmail)
getNumericGraphs(customer.data.clean, derived.age)
getNumericGraphs(customer.data.clean, pre16.totamt)

```

06-impute.R

```

# need to impute the missing values in each one

customer.data.clean <- customer.data.clean %>%
  transmute(
    ACCTNO,
    ANY_MAIL_16,
    CHANNEL_ACQUISITION = fct_explicit_na(CHANNEL_ACQUISITION, na_level = 'Unknown'),
    ZIP = fct_explicit_na(ZIP, na_level = 'Unknown'),
    INC_SCS_AMT_V4,
    INC_WIOUTSCS_V4 = fct_explicit_na(INC_WIOUTSCS_V4, na_level = 'Unknown'),
    INC_WITHSCS_V4 = fct_explicit_na(INC_WITHSCS_V4, na_level = 'Unknown'),
    INC_WOUTSCS_AMT_4,
    FIPSCNTY = fct_explicit_na(FIPSCNTY, na_level = 'Unknown'),
    P_HHINCOLES10M,
    P_HHINCOM10_14,
    P_HHINCOM15_19,
    P_HHINCOM20_24,
    P_HHINCOM25_29,
    P_HHINCOM30_34,
    P_HHINCOM35_39,
    P_HHINCOM40_44,
    P_HHINCOM45_49,
    P_HHINCOM50_59,
    P_HHINCOM60_74,
    P_HHINCOM75_99,
    P_HHINCOM100_124,
    P_HHINCOM125_149,
    P_HHINCOM150_199,
    P_HHINCOM20_UP,
    MED_INC,
    MED_FAMINCOM,
    MED_NOFAMINCOM,
    P_CAPITA_INCOM,
    FEMALE_LABOR_FOR,
    P_IND_AGRICULTUR,
    P_IND_MINING,
    P_IND_CONSTRUCT,
    P_IND_MANUFACT,
    P_IND_WHOLESALE,
    P_IND_RETAILTRD,
    P_IND_TRANSPORT,
    P_IND_UTILITIE,

```

```

P_IND_INFORMAT,
P_IND_FINALNCE,
P_IND_REALESTAT,
P_IND_PROFFES,
P_IND_MANAGE,
P_IND_ADM_SUPPOT,
P_IND_EDALSERVIC,
P_IND_HEALTHCARE,
P_IND_ARTS,
P_IND_ACCOMMOD,
P_IND_OTHERSERV,
P_IND_PUBADMIS,
PCARPOOL,
MED_TRAV_TOWRK,
MED_HOME,
MED_RENT,
MED_DWELL_AGE,
MED_LOR,
HOUSE_STABILITY,
CUR_EST_MED_INC,
CENSUS_SEG1 = fct_explicit_na(CENSUS_SEG1, na_level = 'Unknown'),
ADULT1_G = fct_explicit_na(ADULT1_G, na_level = 'Unknown'),
RELIGION = fct_explicit_na(RELIGION, na_level = 'Unknown'),
ETHNIC_COUNTRY = fct_explicit_na(ETHNIC_COUNTRY, na_level = 'Unknown'),
ADD_TYPE = fct_explicit_na(ADD_TYPE, na_level = 'Unknown'),
LOR1,
DUS = fct_explicit_na(DUS, na_level = 'Unknown'),
NUM_CHILD = median(NUM_CHILD, na.rm = T),
C_00_03 = fct_explicit_na(C_00_03, na_level = 'Unknown'),
C_04_06 = fct_explicit_na(C_04_06, na_level = 'Unknown'),
C_07_09 = fct_explicit_na(C_07_09, na_level = 'Unknown'),
C_10_12 = fct_explicit_na(C_10_12, na_level = 'Unknown'),
C_13_18 = fct_explicit_na(C_13_18, na_level = 'Unknown'),
NUMBADLT,
IND_ED = fct_explicit_na(IND_ED, na_level = 'Unknown'),
CNTY_INC,
NAT_INC,
MAILPREF = fct_explicit_na(MAILPREF, na_level = 'Unknown'),
ITMM = fct_explicit_na(ITMM, na_level = 'Unknown'),
ITMM_SCS,
NEWCAR = fct_explicit_na(NEWCAR, na_level = 'Unknown'),
USED CAR = fct_explicit_na(USED CAR, na_level = 'Unknown'),
AD_WEB = fct_explicit_na(AD_WEB, na_level = 'Unknown'),
AD_MAGAZINE = fct_explicit_na(AD_MAGAZINE, na_level = 'Unknown'),
AD_NEWSPAPER = fct_explicit_na(AD_NEWSPAPER, na_level = 'Unknown'),
AD_RADIO = fct_explicit_na(AD_RADIO, na_level = 'Unknown'),
AD_TV = fct_explicit_na(AD_TV, na_level = 'Unknown'),
STINCIND,
CTINCIND,
ESTHML = median(ESTHML, na.rm = T),
ECHVINX = median(ECHVINX, na.rm = T),
ZGOLFERP = fct_explicit_na(ZGOLFERP, na_level = 'Unknown'),
ZDONORSP = fct_explicit_na(ZDONORSP, na_level = 'Unknown'),
ZPETSP = fct_explicit_na(ZPETSP, na_level = 'Unknown'),
ZARTSP = fct_explicit_na(ZARTSP, na_level = 'Unknown'),
ZMOBP = fct_explicit_na(ZMOBP, na_level = 'Unknown'),
ZFITNESP = fct_explicit_na(ZFITNESP, na_level = 'Unknown'),
ZOUTDOOP = fct_explicit_na(ZOUTDOOP, na_level = 'Unknown'),
ZTRAVANP = fct_explicit_na(ZTRAVANP, na_level = 'Unknown'),

```



```

ZINVESTP = fct_explicit_na(ZINVESTP, na_level = 'Unknown'),
ZAUTOOWP = fct_explicit_na(ZAUTOOWP, na_level = 'Unknown'),
ZGARDENP = fct_explicit_na(ZGARDENP, na_level = 'Unknown'),
ZCOLLECP = fct_explicit_na(ZCOLLECP, na_level = 'Unknown'),
ZCRUISEP = fct_explicit_na(ZCRUISEP, na_level = 'Unknown'),
ZSPORTSP = fct_explicit_na(ZSPORTSP, na_level = 'Unknown'),
ZSWEEPSP = fct_explicit_na(ZSWEEPSP, na_level = 'Unknown'),
ZPOLITIP = fct_explicit_na(ZPOLITIP, na_level = 'Unknown'),
ZMUSICP = fct_explicit_na(ZMUSICP, na_level = 'Unknown'),
ZREADP = fct_explicit_na(ZREADP, na_level = 'Unknown'),
ZCHLDPRP = fct_explicit_na(ZCHLDPRP, na_level = 'Unknown'),
ZDIYP = fct_explicit_na(ZDIYP, na_level = 'Unknown'),
ZSELFIPP = fct_explicit_na(ZSELFIPP, na_level = 'Unknown'),
ZRELIGOP = fct_explicit_na(ZRELIGOP, na_level = 'Unknown'),
ZGRANDPP = fct_explicit_na(ZGRANDPP, na_level = 'Unknown'),
ZCLOTHNP = fct_explicit_na(ZCLOTHNP, na_level = 'Unknown'),
ZDONENVP = fct_explicit_na(ZDONENVP, na_level = 'Unknown'),
ZMUTUALP = fct_explicit_na(ZMUTUALP, na_level = 'Unknown'),
ZWGHTCOP = fct_explicit_na(ZWGHTCOP, na_level = 'Unknown'),
ZPRCHPHP = fct_explicit_na(ZPRCHPHP, na_level = 'Unknown'),
ZPRCHTVP = fct_explicit_na(ZPRCHTVP, na_level = 'Unknown'),
ZMOBMULP = fct_explicit_na(ZMOBMULP, na_level = 'Unknown'),
ZDOGSP = fct_explicit_na(ZDOGSP, na_level = 'Unknown'),
ZCATSP = fct_explicit_na(ZCATSP, na_level = 'Unknown'),
ZHEALTHP = fct_explicit_na(ZHEALTHP, na_level = 'Unknown'),
ZAUTOINP = fct_explicit_na(ZAUTOINP, na_level = 'Unknown'),
ZSKIINGP = fct_explicit_na(ZSKIINGP, na_level = 'Unknown'),
ZASTRLGP = fct_explicit_na(ZASTRLGP, na_level = 'Unknown'),
ZBOATSP = fct_explicit_na(ZBOATSP, na_level = 'Unknown'),
ZCELLP = fct_explicit_na(ZCELLP, na_level = 'Unknown'),
ZCOMMCOPI = fct_explicit_na(ZCOMMCOPI, na_level = 'Unknown'),
ZHMDECOP = fct_explicit_na(ZHMDECOP, na_level = 'Unknown'),
ZHOMEENP = fct_explicit_na(ZHOMEENP, na_level = 'Unknown'),
ZKITCHEP = fct_explicit_na(ZKITCHEP, na_level = 'Unknown'),
ZMOBAVP = fct_explicit_na(ZMOBAVP, na_level = 'Unknown'),
ZMOBBOOP = fct_explicit_na(ZMOBBOOP, na_level = 'Unknown'),
ZMOBCLTP = fct_explicit_na(ZMOBCLTP, na_level = 'Unknown'),
ZMOBFINP = fct_explicit_na(ZMOBFINP, na_level = 'Unknown'),
ZMOBGIFP = fct_explicit_na(ZMOBGIFP, na_level = 'Unknown'),
ZMOBGRDP = fct_explicit_na(ZMOBGRDP, na_level = 'Unknown'),
ZMOBJWLP = fct_explicit_na(ZMOBJWLP, na_level = 'Unknown'),
ZMUSCLAP = fct_explicit_na(ZMUSCLAP, na_level = 'Unknown'),
ZMUSCNTP = fct_explicit_na(ZMUSCNTP, na_level = 'Unknown'),
ZMUSCRSP = fct_explicit_na(ZMUSCRSP, na_level = 'Unknown'),
ZMUSOLDP = fct_explicit_na(ZMUSOLDP, na_level = 'Unknown'),
ZMUSROCP = fct_explicit_na(ZMUSROCP, na_level = 'Unknown'),
ZPBCAREP = fct_explicit_na(ZPBCAREP, na_level = 'Unknown'),
ZPHOTOP = fct_explicit_na(ZPHOTOP, na_level = 'Unknown'),
ZPRCHONP = fct_explicit_na(ZPRCHONP, na_level = 'Unknown'),
ZTENNISP = fct_explicit_na(ZTENNISP, na_level = 'Unknown'),
ZTRAVDOP = fct_explicit_na(ZTRAVDOP, na_level = 'Unknown'),
ZTRAVFOP = fct_explicit_na(ZTRAVFOP, na_level = 'Unknown'),
ZVOLUNTP = fct_explicit_na(ZVOLUNTP, na_level = 'Unknown'),
PRE2009_SALES,
PRE2009_TRANSACTIONS,
pre16.qty,
pre16.totamt,
high.value.account,
pre16.response,

```

```

pre16.totalmail,
pre16.buyer.status,
pre16.got.mailer,
pre16.bought.with.mailer,
pre16.bought.withnomailer,
campaign16 = fct_explicit_na(campaign16, na_level = 'Unknown'),
derived.age = median(derived.age, na.rm = T),
predicted.marital.status = fct_explicit_na(predicted.marital.status, na_level = 'Unknown')
)

```

07-data-split.R

```

# create 2 distinct datasets. One that will be used to model campaign16 (ANY_MAIL_16 = 1).
  This is why high accuracy is super important, we want as minimal false positives. The 2
  nd dataset (referring to campaign17) will use ANY_MAIL_16 = 0. The model constructed
  will then be applied to the campaign17 dataset.

```

```

# used for training & validation
campaign16.data <- customer.data.clean %>%
filter(ANY_MAIL_16 > 0) %>%
select(-ANY_MAIL_16)

```

```

# used for test dataset
campaign17.data <- customer.data.clean %>%
filter(ANY_MAIL_16 == 0) %>%
select(-ANY_MAIL_16)

```

```

# split the data

```

```

set.seed(360)
trainIndex <- createDataPartition(campaign16.data$campaign16, p = 0.80, list = F, times = 1)

```

```

customer.train.initial <- campaign16.data[trainIndex,]
customer.valid.initial <- campaign16.data[-trainIndex,]

```

```

# after running the initial random forest model, only the top 25 variables are kept

```

```

customer.train <- customer.train.initial %>%
select(
ACCTNO,
PRE2009_SALES,
PRE2009_TRANSACTIONS,
pre16.totamt,
pre16.qty,
ITMM_SCS,
pre16.totalmail,
LOR1,
pre16.response,
INC_WOUTSCS_AMT_4,

```

```

CTINCIND,
STINCIND,
INC_SCS_AMT_V4,
CNTY_INC,
IND_ED,
NAT_INC,
FEMALE_LABOR_FOR,
P_HHINCOM100_124,
AD_NEWSPAPER,
MED_RENT,
MED_NOFAMINCOM,
P_IND_EDALSERVIC,
MED_LOR,
ZPRCHPHP,
P_IND_RETAILTRD,
NUMBADLT,
campaign16
)

```

```

customer.valid <- customer.valid.initial %>%
select(
  ACCTNO,
  PRE2009_SALES,
  PRE2009_TRANSACTIONS,
  pre16.totamt,
  pre16.qty,
  ITMM_SCS,
  pre16.totalmail,
  LOR1,
  pre16.response,
  INC_WOUTSCS_AMT_4,
  CTINCIND,
  STINCIND,
  INC_SCS_AMT_V4,
  CNTY_INC,
  IND_ED,
  NAT_INC,
  FEMALE_LABOR_FOR,
  P_HHINCOM100_124,
  AD_NEWSPAPER,
  MED_RENT,
  MED_NOFAMINCOM,
  P_IND_EDALSERVIC,
  MED_LOR,
  ZPRCHPHP,
  P_IND_RETAILTRD,
  NUMBADLT,
  campaign16
)

```

08a-ranger.R

```

# ranger model

# ranger ----
library(ranger)

```

```

# create ranger model for classification
ranger.model1.class <- ranger(campaign16 ~ .,
data = customer.train %>% select(-ACCTNO),
num.trees = 1000,
mtry = 5,
classification = T,
importance = 'impurity',
seed = 2018
)

# create ranger model for probability
ranger.model1.prob <- ranger(campaign16 ~ .,
data = customer.train %>% select(-ACCTNO),
num.trees = 1000,
mtry = 5,
classification = T,
importance = 'impurity',
seed = 2018,
probability = T
)

ranger.model1.class$confusion.matrix
ranger.model1.class$variable.importance

enframe(ranger.model1.class$variable.importance) %>%
rename(
variable = name,
importance = value
) %>%
arrange(desc(importance)) %>%
top_n(25) %>%
ggplot(aes(x = reorder(variable, importance), y = importance)) +
geom_bar(stat = 'identity') +
coord_flip() +
ggtitle('Variable Importance - Random Forest') +
labs(y = 'Importance', x = 'Variable')

#in sample predictions
pred1.is.class <- predict(ranger.model1.class, data = customer.train, type = 'response')
pred1.is.prob <- predict(ranger.model1.prob, data = customer.train, type = 'response')

confusionMatrix(data = pred1.is.class$predictions, reference = customer.train$campaign16,
positive = '1')
postResample(pred1.is.class$predictions, customer.train$campaign16)

library(ROCR)
rocr_pred <- prediction(pred1.is.prob$predictions[,2], customer.train$campaign16)
rocr_perf <- performance(rocr_pred, measure="tpr", x.measure="fpr")
auc <- performance(rocr_pred, measure='auc')
auc <- auc@y.values[[1]]

rocr.data <- data.frame(fpr=unlist(rocr_perf@x.values),
tpr=unlist(rocr_perf@y.values))

insample.plot <- ggplot(rocr.data, aes(x=fpr, ymin=0, ymax=tpr)) +
geom_ribbon(alpha=0.2) +
geom_line(aes(y=tpr)) +

```

```

geom_abline(linetype="dashed") +
#ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
ggtitle("In-Sample ROC Curve - Random Forest") +
labs(x="False Positive Rate", y="True Positive Rate") +
annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

# out of sample predictions
pred1.oos.class <- predict(ranger.model1.class, data = customer.valid, type = 'response')
pred1.oos.prob <- predict(ranger.model1.prob, data = customer.valid, type = 'response')

confusionMatrix(pred1.oos.class$predictions, customer.valid$campaign16, positive = '1')
postResample(pred1.oos.class$predictions, customer.valid$campaign16)

library(ROCR)
rocr_pred <- prediction(pred1.oos.prob$predictions[,2], customer.valid$campaign16)
rocr_perf <- performance(rocr_pred, measure="tpr", x.measure="fpr")
auc <- performance(rocr_pred, measure='auc')
auc <- auc@y.values[[1]]

rocr.data <- data.frame(fpr=unlist(rocr_perf@x.values),
tpr=unlist(rocr_perf@y.values))

outofsample.plot <- ggplot(rocr.data, aes(x=fpr, ymin=0, ymax=tpr)) +
geom_ribbon(alpha=0.2) +
geom_line(aes(y=tpr)) +
geom_abline(linetype="dashed") +
#ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
ggtitle("Out-of-Sample ROC Curve - Random Forest") +
labs(x="False Positive Rate", y="True Positive Rate") +
annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

grid.arrange(insample.plot, outofsample.plot, nrow = 2)

```

08c-glmnet.R

```

# glmnet model

library(glmnet)

# need to create a model matrix of the xvariables

glm.train <- customer.train %>%
select(-ACCTNO)

glm.train.x <- model.matrix(campaign16 ~ ., glm.train)[,-1]
glm.train.y <- customer.train$campaign16

glm.valid <- customer.valid %>%
select(-ACCTNO)

glm.valid.x <- model.matrix(campaign16 ~ ., glm.valid)[,-1]
glm.valid.y <- customer.valid$campaign16

```

```

# find the best alpha (hyper parameter tuning for the elasticnet)
# https://www.r-bloggers.com/variable-selection-with-elastic-net/
a <- seq(0.10, 0.9, 0.05)
search <- foreach(i = a, .combine = rbind) %dopar% {
  cv <- cv.glmnet(glm.train.x, glm.train.y, family = "binomial", nfold = 10, type.measure = "
    deviance", parallel = TRUE, alpha = i)
  data.frame(cvm = cv$cvm[cv$lambda == cv$lambda.1se], lambda.1se = cv$lambda.1se, alpha = i)
}

#search <- readRDS(file = 'data/glmnet-alpha-hyperparameter-results.RDS')
cv3 <- search[search$cvm == min(search$cvm), ]
cv3

fit.elnet <- glmnet(glm.train.x, glm.train.y, family="binomial", alpha=cv3$alpha, lambda =
  cv3$lambda.1se)
# fit.elnet <- glmnet(glm.train.x, glm.train.y, family = 'binomial', alpha = cv3$alpha)
#plot(fit.elnet, xvar="lambda")

# cross validation: https://stats.stackexchange.com/questions/72251/an-example-lasso-
  regression-using-glmnet-for-binary-outcome
cv.elnet1 <- cv.glmnet(glm.train.x, y=glm.train.y, alpha=cv3$alpha, family = 'binomial',
  parallel = T)
plot(cv.elnet1)
cv.elnet1$lambda.min
cv.elnet1$lambda.1se

coef(fit.elnet)

# in sample prediction
pred.insample.elnet <- predict(fit.elnet, newx = glm.train.x, type = 'response', s = cv.
  elnet1$lambda.1se)
insample.predictions <- rep(0,nrow(customer.train))
insample.predictions[pred.insample.elnet > 0.5] <-1
table(pred = insample.predictions, true = glm.train.y)

confusionMatrix(data = insample.predictions, reference = glm.train.y, positive = '1')

rocr_pred <- prediction(pred.insample.elnet, customer.train$campaign16)
rocr_perf <- performance(rocr_pred, measure="tpr", x.measure="fpr")
auc <- performance(rocr_pred, measure='auc')
auc <- auc@y.values[[1]]

rocr.data <- data.frame(fpr=unlist(rocr_perf@x.values),
  tpr=unlist(rocr_perf@y.values))

insample.plot <- ggplot(rocr.data, aes(x=fpr, ymin=0, ymax=tpr)) +
  geom_ribbon(alpha=0.2) +
  geom_line(aes(y=tpr)) +
  geom_abline(linetype="dashed") +
  #ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
  ggtitle("In-Sample ROC Curve - GLMnet") +
  labs(x="False Positive Rate", y="True Positive Rate") +
  annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

```

```

# out of sample predictions
pred.oos.elnet <- predict(fit.elnet, newx = glm.valid.x, type = 'response', s = cv.
  elnet1$lambda.1se)
oos.predictions <- rep(0, nrow(customer.valid))
oos.predictions[pred.oos.elnet > 0.5] <- 1
table(pred=oos.predictions, true = glm.valid.y)

confusionMatrix(data = oos.predictions, reference = glm.valid.y, positive = '1')

rocr_pred <- prediction(pred.oos.elnet, customer.valid$campaign16)
rocr_perf <- performance(rocr_pred, measure="tpr", x.measure="fpr")
auc <- performance(rocr_pred, measure='auc')
auc <- auc@y.values[[1]]

rocr.data <- data.frame(fpr=unlist(rocr_perf@x.values),
  tpr=unlist(rocr_perf@y.values))

outofsample.plot <- ggplot(rocr.data, aes(x=fpr, ymin=0, ymax=tpr)) +
  geom_ribbon(alpha=0.2) +
  geom_line(aes(y=tpr)) +
  geom_abline(linetype="dashed") +
  #ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
  ggtitle("Out-of-Sample ROC Curve - GLMnet") +
  labs(x="False Positive Rate", y="True Positive Rate") +
  annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

grid.arrange(insample.plot, outofsample.plot, nrow = 2)

pred.elnet <- predict(fit.elnet, newx = glm.valid.x, type = 'response', s = cv.elnet1$lambda
  .1se)
elasticnet.predictions <- rep(0,nrow(customer.valid))
elasticnet.predictions[pred.elnet > 0.5] <- 1
table(pred = elasticnet.predictions, true = glm.valid.y)
postResample(elasticnet.predictions, glm.valid.y)[[1]]

i = 0
for(i in seq(0.1, 0.9, 0.1)) {
  ep <- rep(0, nrow(customer.valid))
  ep[pred.insample.elnet > i] <- 1
  print(postResample(ep, glm.valid.y)[[1]])
}

confusionMatrix(elasticnet.predictions, customer.valid$campaign16, positive = '0')

library(pROC)
roc(glm.valid.y, elasticnet.predictions)
plot(roc(glm.valid.y, elasticnet.predictions))

# verify that all coded factors have at least 2 levels
# https://stackoverflow.com/questions/18171246/error-in-contrasts-when-defining-a-linear-
# model-in-r
l <- sapply(glm.train, function(x) is.factor(x))

```

```

1
m <- glm.train[, 1]
ifelse(n <- sapply(m, function(x) length(levels(x))) == 1, "DROP", "NODROP")

```

08d-naivebayes.R

```

# Naive Bayes Classifier

# library(naivebayes)
#
# nb.model <- naive_bayes(
#   x = customer.train %>% select(-ACCTNO, -campaign16),
#   y = customer.train$campaign16
# )
#
# nb.insample.class <- predict(nb.model, newdata = customer.train, type = 'class')
# postResample(nb.insample.class, customer.train$campaign16)
#
# nb.predict.class <- predict(nb.model, newdata = customer.valid, type = 'class')
#
# confusionMatrix(nb.predict.class, customer.valid$campaign16)
# postResample(nb.predict.class, customer.valid$campaign16)
#
#
# plot(roc(response = customer.valid$campaign16, predictor = as.numeric(nb.predict.class)))

# using caret
tControl <- trainControl(
  method = 'cv',
  number = 5,
  allowParallel = T,
  # classProbs = T,
  # summaryFunction = twoClassSummary,
  # savePredictions = 'final',
  # sampling = 'down'
)

fitNB <- train(
  y = customer.train$campaign16,
  x = customer.train %>% select(-campaign16, -ACCTNO),
  method = 'naive_bayes',
  trControl = tControl,
  metric = 'Kappa',
  preProc = c("center", "scale")
)

# in sample
fitNB.insample.pred <- predict(object = fitNB, newdata = customer.train)
confusionMatrix(data = fitNB.insample.pred, reference = customer.train$campaign16, positive
  = '1')
fitNB.insample.prob <- predict(object = fitNB, newdata = customer.train, type = 'prob')

library(ROCR)
rocr_pred <- prediction(fitNB.insample.prob[,2], customer.train$campaign16)
rocr_perf <- performance(rocr_pred, measure="tpr", x.measure="fpr")

```



```

auc <- performance(roc_pred, measure='auc')
auc <- auc@y.values[[1]]

roc.data <- data.frame(fpr=unlist(roc_perf@x.values),
  tpr=unlist(roc_perf@y.values))

insample.plot <- ggplot(roc.data, aes(x=fpr, ymin=0, ymax=tpr)) +
  geom_ribbon(alpha=0.2) +
  geom_line(aes(y=tpr)) +
  geom_abline(linetype="dashed") +
  #ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
  ggtitle("In-Sample ROC Curve - Naive Bayes") +
  labs(x="False Positive Rate", y="True Positive Rate") +
  annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

# out of sample
fitNB.oos.pred <- predict(object = fitNB, newdata = customer.valid)
fitNB.oos.prob <- predict(object = fitNB, newdata = customer.valid, type = 'prob')
confusionMatrix(data = fitNB.oos.pred, reference = customer.valid$campaign16, positive =
  '1')

roc_pred <- prediction(fitNB.oos.prob[,2], customer.valid$campaign16)
roc_perf <- performance(roc_pred, measure="tpr", x.measure="fpr")
auc <- performance(roc_pred, measure='auc')
auc <- auc@y.values[[1]]

roc.data <- data.frame(fpr=unlist(roc_perf@x.values),
  tpr=unlist(roc_perf@y.values))

outofsample.plot <- ggplot(roc.data, aes(x=fpr, ymin=0, ymax=tpr)) +
  geom_ribbon(alpha=0.2) +
  geom_line(aes(y=tpr)) +
  geom_abline(linetype="dashed") +
  #ggtitle(expression(atop("Naive Bayes", atop("In-Sample ROC Curve")))) +
  ggtitle("Out-of-Sample ROC Curve - Naive Bayes") +
  labs(x="False Positive Rate", y="True Positive Rate") +
  annotate("text", x=0.75, y=0.25, label=paste("AUC:", round(auc,2)))

grid.arrange(insample.plot, outofsample.plot, nrow = 2)

```

value.R

```

catalog.cost <- 3

#campaign 16 revenue
customer.data %>%
  filter(ANY_MAIL_16 == 0) %>%
  select(ACCTNO, starts_with('TOTAMT'), TOTAL_MAIL_16) %>%
  transmute(
    ACCTNO,
    total.sales = TOTAMT1 + TOTAMT2 + TOTAMT3 + TOTAMT4 + TOTAMT5 + TOTAMT6 + TOTAMT7 + TOTAMT8
      + TOTAMT9 + TOTAMT10 + TOTAMT11 + TOTAMT12 + TOTAMT13 + TOTAMT14 + TOTAMT15 + TOTAMT16,
    TOTAL_MAIL_16
  ) %>%
  filter(TOTAL_MAIL_16 > 0) %>%

```

```

mutate(
  sales.per.mail = total.sales / TOTAL_MAIL_16
) %>%
summarize(
  total.mailers = sum(TOTAL_MAIL_16),
  all.sales = sum(total.sales),
  total_accounts = length(ACCTNO),
  total.costs = catalog.cost * total.mailers,
  revenue.per.customer = all.sales / total_accounts,
  profit.per.customer = (all.sales - total.costs) / total_accounts
)

test.data.summary <- customer.data %>%
filter(ANY_MAIL_16 == 1) %>%
select(ACCTNO, starts_with('TOTAMT'), TOTAL_MAIL_16) %>%
transmute(
  ACCTNO,
  total.sales = TOTAMT1 + TOTAMT2 + TOTAMT3 + TOTAMT4 + TOTAMT5 + TOTAMT6 + TOTAMT7 + TOTAMT8
    + TOTAMT9 + TOTAMT10 + TOTAMT11 + TOTAMT12 + TOTAMT13 + TOTAMT14 + TOTAMT15 + TOTAMT16,
  TOTAL_MAIL_16
) %>%
filter(TOTAL_MAIL_16 > 0) %>%
mutate(
  sales.per.mail = total.sales / TOTAL_MAIL_16
) %>%
summarize(
  total.mailers = sum(TOTAL_MAIL_16),
  all.sales = sum(total.sales),
  total_accounts = length(ACCTNO),
  total.costs = catalog.cost * total.mailers,
  revenue.per.customer = all.sales / total_accounts,
  profit.per.customer = (all.sales - total.costs) / total_accounts
)

profit <- round(test.data.summary$profit.per.customer,2)
profit

# profit per customer is 268.74

campaign17.data %>%
summarize(
  num.accounts = n()
) %>%
mutate(
  total.profit = num.accounts * profit
)

fitNB.test.prob <- predict(object = fitNB, newdata = campaign17.data, type = 'prob')

test.data <- tibble(success.prob = round(fitNB.test.prob[,2],3))

# now let's pick a probability threshold and create a classification

threshold <- 0

```

```
predicted.values <- test.data %>%
mutate(
predicted.class = ifelse(success.prob >= threshold, 1, 0),
expected.profit = ifelse(predicted.class == 1, success.prob * profit, 0)
)

predicted.values %>%
group_by(predicted.class) %>%
summarize(
obs.count = n(),
total.revenue = sum(expected.profit)
)

i = 0
tmp <- tibble(threshold = as.numeric(), obs.count = as.numeric(), total.revenue = as.numeric
())
for(i in seq(0,1,0.1)) {
# tmp <- tibble(threshold = as.numeric(), obs.count = as.numeric(), total.revenue = as.
numeric())
threshold <- i
a <- test.data %>%
mutate(
predicted.class = ifelse(success.prob >= threshold, 1, 0),
expected.profit = ifelse(predicted.class == 1, success.prob * profit, 0)
)

b <- a %>%
group_by(predicted.class) %>%
filter(predicted.class == 1) %>%
summarize(
obs.count = n(),
total.revenue = sum(expected.profit)
)

tmp <- add_row(tmp, threshold = i, obs.count = b$obs.count, total.revenue = b$total.revenue)
}
```

Assignment 4: Wine Revisited

**Nikhil Agarwal
PREDICT 454 | Sec.55
Northwestern University**

Table of Contents

Introduction.....	1
Data Quality Check.....	1
Data Overview.....	1
Outlier Detection.....	5
Missing Data.....	7
Exploratory Data Analysis.....	7
Quantitative Exploration.....	7
Qualitative Exploration.....	8
Outlier Detection – Visual.....	9
Naïve Tree Model.....	10
Modeling Suite.....	12
Random Forest.....	12
In Sample Performance.....	14
Conclusion.....	15
Support Vector Machine.....	15
In Sample Performance.....	15
Conclusion.....	16
Neural Network.....	16
In Sample Performance.....	17
A personal comment on Neural Network.....	17
Conclusion.....	17
Model Performance Summary.....	17
Conclusion & Next Steps.....	18
Works Cited.....	20
Appendix A – R Code.....	i
Table Code.....	i
Table 2.....	i
Table 3, Table 4, Table 6.....	i
Table 5.....	iii
Figure Code.....	iii
Figure 1 Code.....	iii
Figure 2 Code.....	iii
Figure 3 Code.....	iv
Figure 4 Code.....	iv
Figure 5 Code.....	iv
Figure 6 Code.....	iv
Model Code.....	iv

Libraries Used in Script.....	v
Naïve Tree Model.....	v
Random Forest Model.....	v
Support Vector Machines (SVM) Model.....	vi
Neural Network Model.....	vi
Appendix B – Figures and Tables.....	vii
Random Forest Hypertuning.....	vii
Support Vector Machine Hypertuning.....	vii

Introduction

The purpose of this analysis is to perform a data quality check, conduct an exploratory data analysis, and build three specific model types: random forest, support vector machine (SVM), and neural network. Due to the limited number of observations, all models were built and tested against in-sample data. Therefore, the models constructed are not necessarily models that would be implemented in a production environment. Both the dataset and the corresponding data dictionary were provided. The dataset consisted of 177 observations across 13 predictors. The goal of the models is to determine the type of wine (multi-classification) based on the different attributes. This report is divided into five key sections: Data Quality Check, Exploratory Data Analysis, Model Development, Model Comparison, and Next Steps.

Data Quality Check

Data Overview

The dataset consisted of 177 observations spanning 13 predictors and the response was a categorical variable with three different classifications: W1 (denoting wine type 1), W2 (denoting wine type 2), and W3 (denoting wine type 3). Upon importing the data, the variables *magnesium* and *proline* were converted to numerical values. Furthermore, the response variable was renamed to *WineType* and coded as a categorical (i.e., factor) variable with three levels (W1, W2, and W3). Table 1 describes the different variables in the dataset.

Table 1

Attribute	Type	Category
WineType	categorical	response
Alcohol	numerical	predictor
MalicAcid	numerical	predictor
Ash	numerical	predictor
AlcalinityOfAsh	numerical	predictor
Magnesium	numerical	predictor
TotalPhenols	numerical	predictor
Flavanoids	numerical	predictor
NonflavanoidPhenols	numerical	predictor
Proanthocyanins	numerical	predictor
ColorIntensity	numerical	predictor
Hue	numerical	predictor
OD280_OD315_of_DilutedWines	numerical	predictor
Proline	numerical	predictor

Table 2 provides a breakdown of the observation count by the different factors in the variable *WineType*.

Table 2

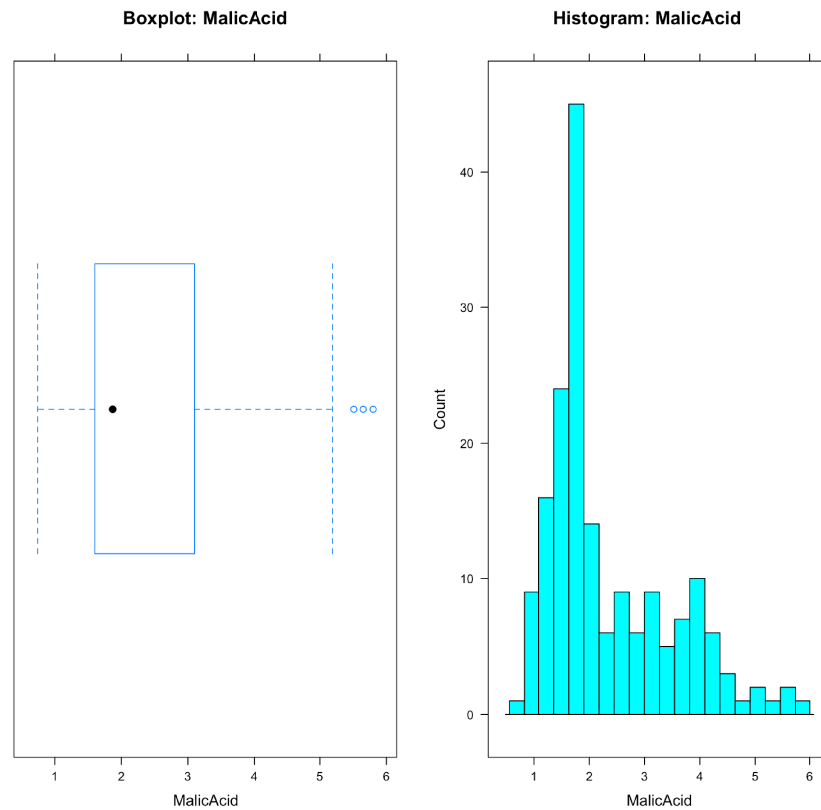
Response Type	Observation Count	Observation Percentage
W1	58	32.8%
W2	71	40.1%
W3	48	27.1%

Table 3 provides the descriptive statistics on the predictor variables.

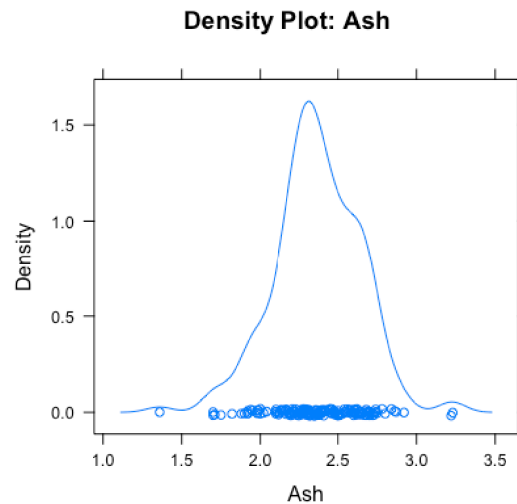
Table 3

Variable	Q 0.01	Q 0.05	Q 0.25	Q 0.50	Q 0.75	Q 0.95	Q 0.99	Mean	Variance	Min	Max
Alcohol	11.4404	11.658	12.36	13.05	13.67	14.22	14.4764	12.994	0.654	11.03	14.83
MalicAcid	0.8976	1.058	1.6	1.87	3.1	4.464	5.5436	2.34	1.253	0.74	5.8
Ash	1.7	1.92	2.21	2.36	2.56	2.742	2.992	2.366	0.076	1.36	3.23
AlcalinityOfAsh	11.352	14.76	17.2	19.5	21.5	25	28.5	19.517	11.129	10.6	30
Magnesium	78	80.8	88	98	107	123.2	141.88	99.588	200.903	70	162
TotalPhenols	1.138	1.38	1.74	2.35	2.8	3.276	3.5992	2.292	0.392	0.98	3.88
Flavanoids	0.47	0.544	1.2	2.13	2.86	3.5	3.7932	2.023	0.997	0.34	5.08
NonflavanoidPhenols	0.14	0.19	0.27	0.34	0.44	0.6	0.63	0.362	0.016	0.13	0.66
Proanthocyanins	0.42	0.73	1.25	1.55	1.95	2.712	3.0368	1.587	0.327	0.41	3.58
ColorIntensity	1.8616	2.112	3.21	4.68	6.2	9.604	11.028	5.055	5.403	1.28	13
Hue	0.5476	0.57	0.78	0.96	1.12	1.286	1.4272	0.957	0.053	0.48	1.71
OD280_OD315_of_DilutedWines	1.29	1.46	1.93	2.78	3.17	3.572	3.7364	2.604	0.497	1.27	4
Proline	306.72	354.4	500	672	985	1298	1522.68	745.096	99151.962	278	1680

Note the abnormally high variances for the variables *Magnesium* and *Proline*. Furthermore, if the mean and median (see column Q0.50) values are equal, that would suggest a normal distribution. In other words, the larger the difference between the mean and median, the stronger the evidence for skewness. The variable *MalicAcid*, for instance, has a difference of 0.47 units between the mean and median. Although this difference is not very large, there is some slight skewness. Figure 1 shows the boxplot and histogram for this variable.

*Figure 1*

Note how the variable, *MalicAcid*, is slightly skewed to the right. In contrast, Figure 2 illustrates the density plot (which is similar to a histogram) of the variable *Ash*.

*Figure 2*

This plot shows that the variable is close to normal distribution. However, looking closely at the row of points, it is clear to see that a few points are the extreme ends. This could suggest the presence of outliers. Visual checks for outliers (such as boxplots and histograms) are discussed in the Exploratory Data Analysis section. Multiple plots were constructed, but not all plots are documented in this report for relevancy and brevity.

Outlier Detection

Detecting outliers can be a subjective task and can also be done numerically or visually.

Table 4

Variable	Q 0.01	Q 0.99	Min	Max	Range
Alcohol	11.4404	14.4764	11.03	14.83	3.8
MalicAcid	0.8976	5.5436	0.74	5.8	5.06
Ash	1.7	2.992	1.36	3.23	1.87
AlcalinityOfAsh	11.352	28.5	10.6	30	19.4
Magnesium	78	141.88	70	162	92
TotalPhenols	1.138	3.5992	0.98	3.88	2.9
Flavanoids	0.47	3.7932	0.34	5.08	4.74
NonflavanoidPhenols	0.14	0.63	0.13	0.66	0.53
Proanthocyanins	0.42	3.0368	0.41	3.58	3.17
ColorIntensity	1.8616	11.028	1.28	13	11.72
Hue	0.5476	1.4272	0.48	1.71	1.23
OD280_OD315_of_DilutedWines	1.29	3.7364	1.27	4	2.73
Proline	306.72	1522.68	278	1680	1402

Table 4 is very similar to Table 3, but it also has the range for each variable. As mentioned earlier, the variable *Proline* has a very large range. The large range of values also contributes to the large variance which in turn can be used to help understand the skewness. A large difference, for example, between Q0.99 (the 99th percentile) and the maximum value could be indicative of an outlier. For example, the difference between the 99th percentile and the maximum value for the variable, *Magnesium*, is about 20 units. Figure 3 illustrates the boxplot for this variable.

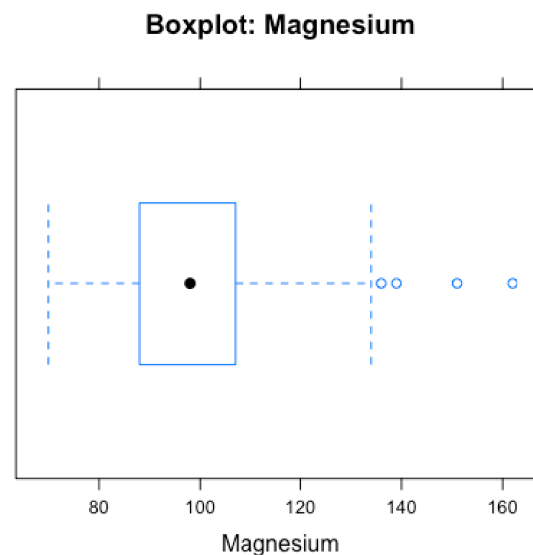


Figure 3

Note the four observations that are beyond the last whisker to the right. This is not necessarily a clear indicator of outliers, but suggestive that these points could be influential or even outliers.

Missing Data

No missing data were found in this dataset.

Exploratory Data Analysis

Quantitative Exploration

A correlation matrix with 13 predictors is not easy to present visually in this report. However, Table 5 lists the combination of predictors that were found to have an absolute value correlation of 0.6 or higher between themselves. In this instance, all of these combinations meeting the threshold of 0.6 or higher were positive correlations.

Table 5

Predictor 1	Predictor 2	Correlation	Absolute Value of Correlation
Flavanoids	TotalPhenols	0.864	0.864
OD280_OD315_of_DilutedWines	Flavanoids	0.786	0.786
OD280_OD315_of_DilutedWines	TotalPhenols	0.7	0.7
Proanthocyanins	Flavanoids	0.65	0.65
Proline	Alcohol	0.641	0.641
Proanthocyanins	TotalPhenols	0.611	0.611

Generally, the stronger a correlation between the two variables, the higher the likelihood that collinearity may be an issue. Note how the variables *Flavanoids* and *TotalPhenols* have a very strong positive correlation of 0.864. Figure 4 illustrates this relationship in a scatterplot form.

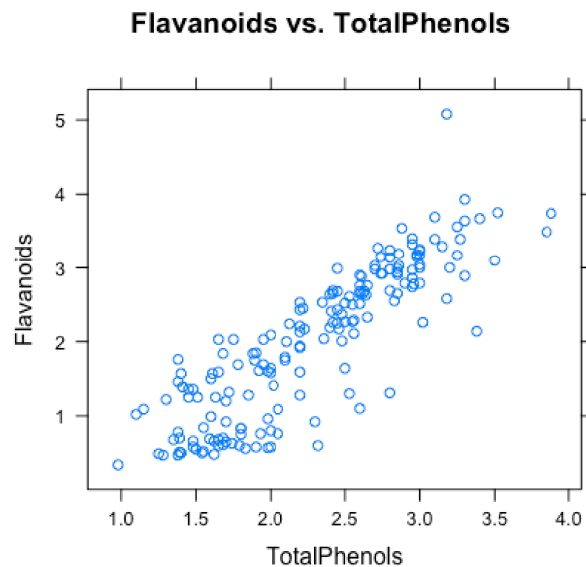


Figure 4

Note how there is distinctive linear relationship between these two variables. As the value of *TotalPhenols* increases, it can be expected that the *Flavanoids* value would also increase.

Qualitative Exploration

The average values for each predictor was calculated and then appropriate dotplots (illustrating the average values by the type of wine) were created. One interesting conclusion drawn was that wine type 2 (W2) has a low average alcohol content as well as a low average color intensity. Figure 5 illustrates the average alcohol content by type of wine versus the average color intensity by the type of wine.

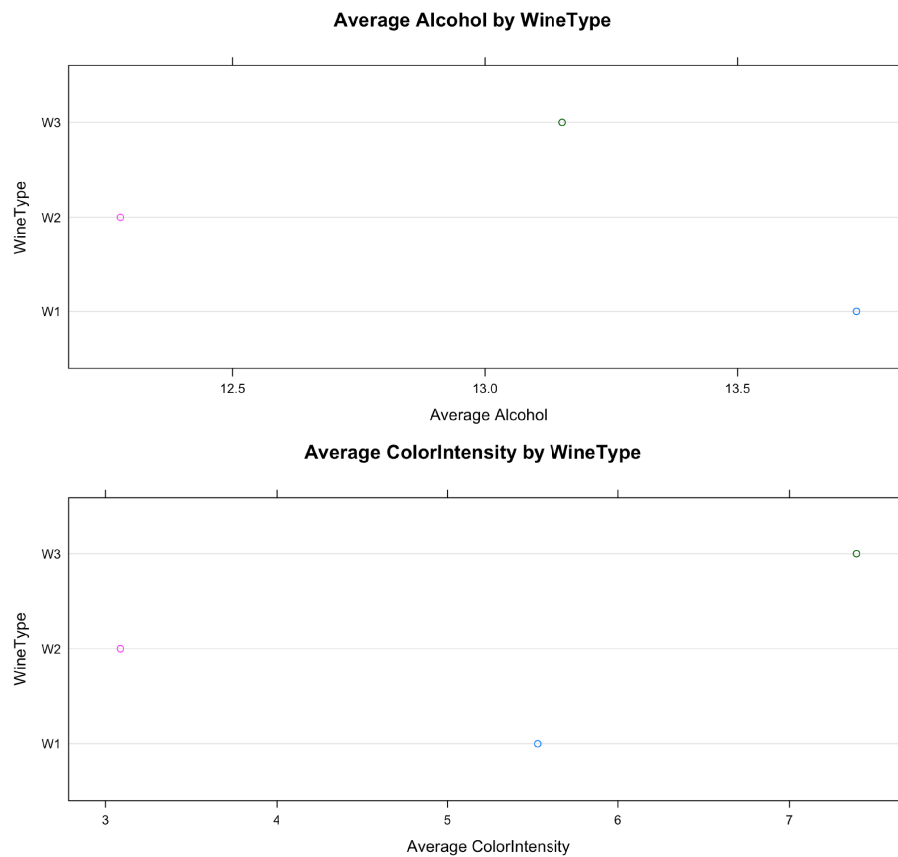


Figure 5

From the background documentation provided, it is not known if what type of wine is wine type 2 (e.g, rosé, white, red, etc.). However, generally, white wines have reduced color intensity and lower alcohol content than red wines. However, this cannot be a definitive conclusion. Regardless, this does provide some subjective insight into the relationship between the types of wine.

Outlier Detection – Visual

Since the type of wine is a categorical variable, boxplots and histograms were constructed for each predictor variable conditional to the three different types of wine. Figure 6 is an example of the predictor Magnesium. Note the presence of outliers¹ of

¹ The individual circles in a boxplot are strong indicators of the presence of outliers

wine types 1 and 2 (note that *Magnesium* – as a whole – has outliers as seen in Figure 3). Furthermore, the histogram shows that there is some skewness for this predictor by wine type.

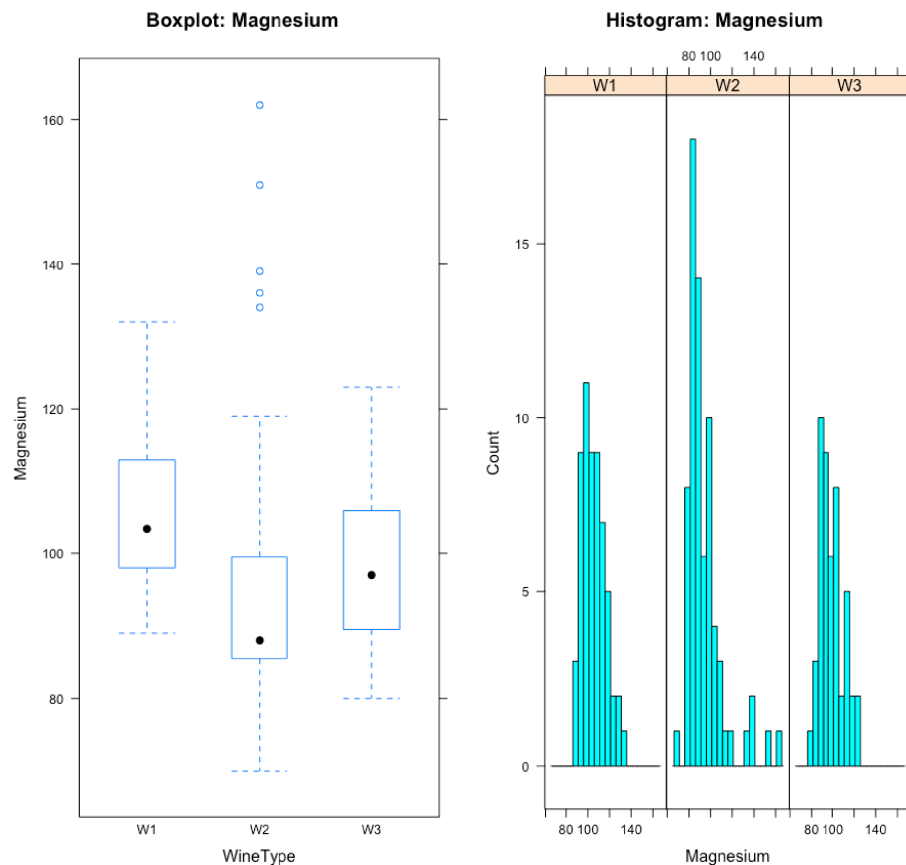


Figure 6

Typically, transformations would be undertaken to stabilize the variances and imputations would be made to manage the outliers. However, for this assignment, neither variable transformations or imputations are in scope.

Naïve Tree Model

A naïve tree model² was created in which the response variable, price, was fitted against the other predictors (see Figure 7). The first node is at the top and is

² The rendering system on a Mac was not able to clearly draw the lines in this figure properly

considered the root. The value listed within each rectangle (on the first line) is the class (response). The second line (in the rectangle) lists the percentage of observations by class within that node. The third line is the percentage of observations at that node. Note how the first split is on the predictor *Proline* and if the observed value is greater than or equal to 755, then it will go to the left and proceed to the next node *Flavanoids*. The color (green for W1, orange for W3, blue for W2) denotes the prevailing class at that node.

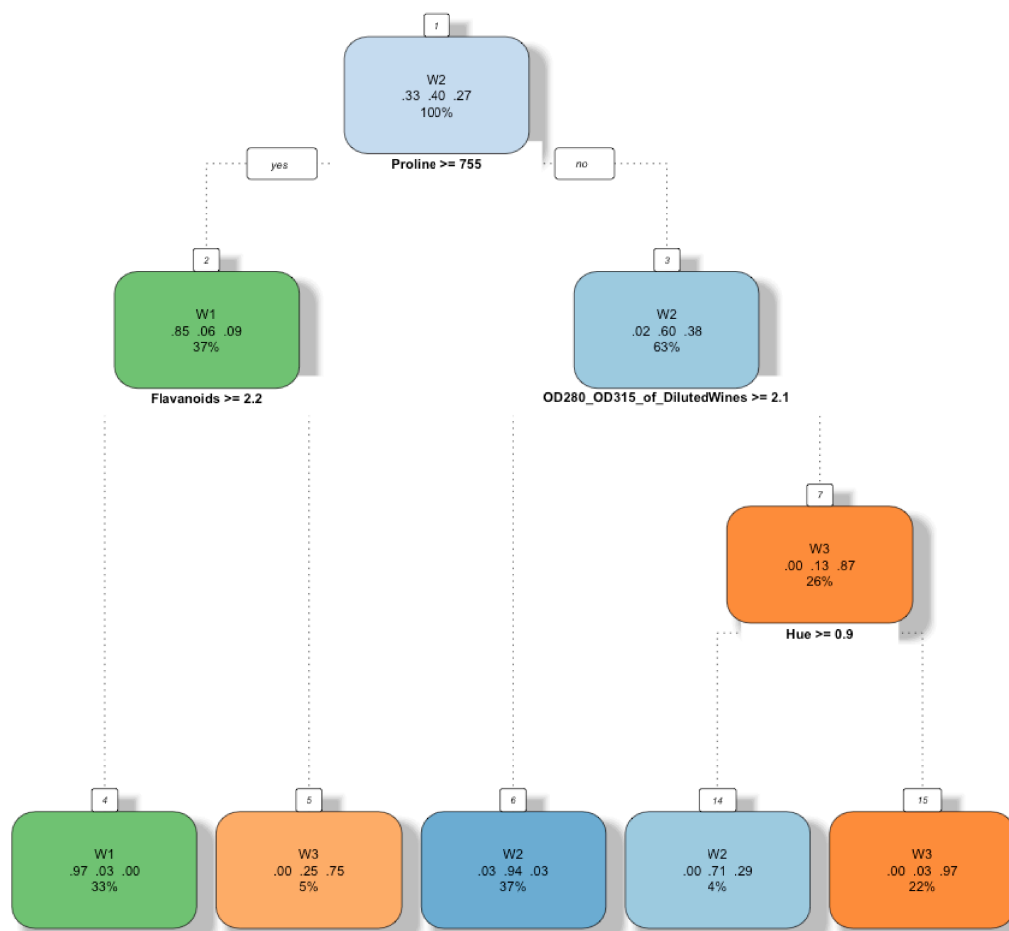


Figure 7

Note how this naïve tree model leverages only four distinct predictors: *Proline*, *Flavanoids*, *OD280_OD315_of_DilutedWines*, and *Hue*. This naïve model provides a

quick glance at the important variables that were considered for this model. This tree model clearly suggests that not all 13 predictor variables may be necessary for the overall model.

Modeling Suite

Three different models were constructed: random forest, support vector machines (SVM), and neural network. Typically, the dataset would be divided into a training and test sets, however, the assignment instructions specifically state that all models should use the original dataset for training and for in-sample testing due to the limited number of observations.

Random Forest

For the random forest model, cross validation was completed using out of bag samples. Implicitly, the random forest model conducts a repeated cross validation. , random forest is a complex tree-based model that does not yield a simple equation. Hypertuning of the 'mtry' parameter (see Random Forest Model code in Appendix) was executed to determine the optimal number of random variables to sample at each split (see Table 11). In this instance, the parameter value of four was chosen as it yielded the highest kappa value.

Table 6 summarizes the variable importance. These values represent the mean decrease in the Gini Index. This table can be leveraged to identify key variables that are impactful to the model. Note how the importance is separated by each potential categorical value for the response variable *WineType*.

Table 6

Predictor	WineType		
	W1	W2	W3
Alcohol	100.000	61.279	26.010
Flavanoids	86.290	41.488	95.350
ColorIntensity	67.890	67.109	68.980
Hue	48.950	31.494	54.310
Magnesium	52.020	30.096	19.550
TotalPhenols	50.380	9.024	37.170
MalicAcid	37.600	26.790	26.130
AlcalinityOfAsh	28.530	15.027	29.050
Proanthocyanins	24.520	7.576	27.850
Ash	19.030	18.577	24.340
NonflavanoidPhenols	22.690	0.000	16.310

Note how the predictor *Alcohol* is extremely important for the class W1 (wine type 1), but is not very important for class W3. However, the variable *Flavanoids* is quite important for both classes W1 and W3. This implies that although a particular variable may not be as essential for a particular response, it may be essential for another.

500 trees were constructed for this model. Figure 8 illustrates the error based on the number of trees. Note that after about 75 trees, the error stays fairly constant. The three lines correspond to the different repeats that were done during the model development.

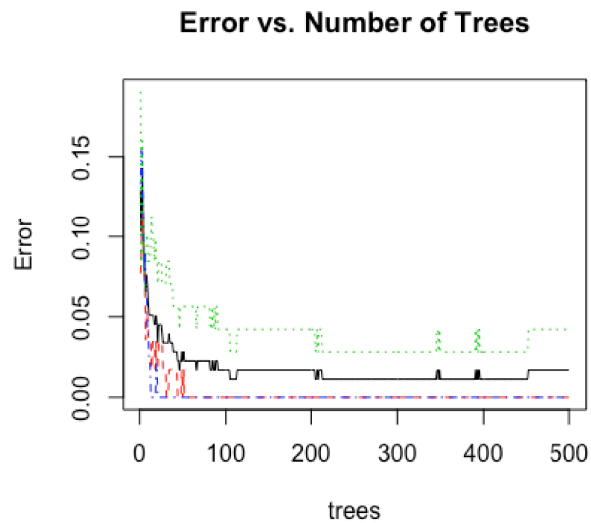


Figure 8

In Sample Performance

Table 7 summarizes the proportion of observations (using in-sample data) that are predicted to be W1, W2, or W3 (the three different types of wine). Unsurprisingly, the model has perfect fit. This perfect fit means perfect accuracy and kappa values.

Table 7

		Observed Values		
		W1	W2	W3
Predicted Values	W1	0.328	0	0
	W2	0	0.401	0
	W3	0	0	0.271

Conclusion

Evaluating predictive accuracy in a multi-classification problem with random forest is fairly simple with a confusion matrix. Unfortunately, random forest models do not have a simple tree diagram that can be leveraged to visually understand the decisions the multitude of trees are making.

Support Vector Machine

In general, SVMs are “a kernel-based classification approach where the kernels are represented in terms of a...subset of the training examples” (Zumel & Mount, 2014). SVMs have a tendency to have better performance than logistic regression when the classes are well separated as compared to logistic regression where it may perform well with not so well separated classes (James, Witten, Hastie, & Tibshirani, 2015). For this model, the cost parameter (which is helpful in reducing training errors) was hypertuned. Table 12 documents the results of the hypertuning. In this iteration, a cost of 0.01 was used. Furthermore, this iteration also used a linear kernel.

The challenge with SVMs are that they are designed to perform well with binary classifiers. In order to leverage SVMs for multi-classification problems, SVMs can be modified to use several approaches such as One-vs-All (OVA). Inherently, this presents several challenges as the interpretability is reduced and becomes more complex.

In Sample Performance

Table 8 summarizes the proportion of observations (using in-sample data) that are predicted to be W1, W2, or W2 (the three different types of wine). Somewhat surprisingly, the model is close to perfect, but not as perfect as was seen in the confusion matrix for the random forest model. The accuracy and kappa for this model are 0.9944 and 0.9914 respectively.

Table 8

		Observed Values		
		W1	W2	W3
Predicted Values	W1	0.328	0	0
	W2	0	0.395	0
	W3	0	0.006	0.271

Conclusion

SVMs could be used for multi-classification problems, however, their reduced interpretability and increased complexity does not necessarily make this approach a desirable one. Implicitly, the SVM approach is creating a binary model for each type of wine and then combining the results. This increases the likelihood of potential errors and is reflected in the model's inability to be perfect – as compared to the random forest model.

Neural Network

Neural network models are perhaps the most convoluted models to describe. At a fundamental level, neural networks are “two-stage...classification model[s]” (Hastie, Tibshirani, & Friedman, 2009). For this iteration, the package `nnet` was used in the package `caret`. There are two parameters for this approach: size and decay. Size is essentially the number of hidden layers whereas decay is the regularization parameter to reduce overfitting. In this instance, the size was kept constant at 5 and decay was set to 0.1. Furthermore, the predictors were preprocessed (prior to training the model) to be centered and scaled. This essentially made each predictor have a mean of 0 and a standard deviation of 1. Furthermore, the model was cross-validated using a 5-fold repeat three times methodology (repeated cross-validation).

In Sample Performance

Table 9 summarizes the proportion of observations (using in-sample data) that are predicted to be W1, W2, or W3 (the three different types of wine). Unsurprisingly, the model has perfect fit. This perfect fit means perfect accuracy and kappa values.

Table 9

		Observed Values		
		W1	W2	W3
Predicted Values	W1	0.328	0	0
	W2	0	0.401	0
	W3	0	0	0.271

A personal comment on Neural Network

The methodology that I used in this iteration does not have the capability of producing a visual plot. However, the visual plot created by the package 'neuralnet' is quite cumbersome to read and interpret. For that reason, the plot was excluded from this report.

Conclusion

The neural network model is by far extremely complex to interpret and explain. This is a clear example of a "black-box" model that works well (as can be seen from the confusion matrix) but is very difficult to explain. One of the key challenges with this model is the inability to clearly articulate the model in an easy to understand and verifiable method.

Model Performance Summary

TABLE XX summarizes the three models' performance. Recall that this performance is based on in-sample testing (i.e., using the same data that were used to train the model). Therefore, there is a strong likelihood that overfitting may have occurred.

Table 10

Model	Accuracy	Kappa
Random forest	1.0000	1.0000
Support Vector Machine (SVM)	0.9944	0.9914
Neural Network	1.0000	1.0000

It is not clear which model should be chosen. All three models have significant complexity and reduced interpretability. Furthermore, all three models are not suitable for operational use since they were never tested against out-of-sample data (data that were not used in any way for training the models).

Conclusion & Next Steps

First and foremost, the models constructed are nowhere near ready for operational use. Since they were 'tested' using in-sample data, there is no way to really know how well the models may perform on unseen data. In addition, both the random forest and neural network models performed perfectly, but explaining the exact 'decisions' or the math behind their algorithms are not simple to explain. For instance, 500 trees were constructed for the random forest model, yet there is no clear way to visualize each tree.

In a multi-classification problem, metrics such as accuracy or kappa could be used to assess a model's performance. ROC curves are typically for binary classification problems. Therefore, if each type wine was converted into a binary classification, three different ROC curves could be constructed. However, summarizing all three together would make for challenging interpretations. Therefore, it is not appropriate to leverage ROC curves in a multi-classification model. In this assignment, all three models in the modeling suite can be classified (no pun intended) as "black-box models". Technically, neural network models can have visualizations, but they are convoluted and extremely difficult to explain. At a fundamental level, confusion matrices can easily summarize the results of a fitted model and provide the end-user with a clear perspective. Furthermore, standard metrics such as accuracy and kappa can be used as KPI's. Perhaps the most challenging aspect of these models is the

communication. In a typical business setting, the inability to clearly and easily explain models could be construed as a liability. Another challenge is that there is a very limited way to replicate the math behind each of these models. This reduces the ability to audit the algorithms.

Next steps include further examination of neural network models. The author of this assignment spent quite some time learning about neural network models, but was unable to fully comprehend their abilities. Furthermore, hypertuning the parameters will also enable greater understanding of their impact on the final model.

Works Cited

Hastie, T., Tibshirani, R., Friedman, J. (2009). *The Elements of Statistical Learning*. New York, New York: Springer Science & Business Media.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2015). *An Introduction to Statistical Learning with Applications in R*. New York, New York: Springer Science & Business Media.

Zumel, N., & Mount, J. (2014). *Practical Data Science with R*. Shelter Island, New York: Manning Publications Co.

Appendix A – R Code

Table Code

Table 2

```
wine %>%
  group_by(WineType) %>%
  summarize(
    TotalObservations = n(),
    ObservationPCT = n()/nrow(wine)
  ) %>%
  ungroup()
```

Table 3, Table 4, Table 6

```
custom.summary <- function(df) {
  # create the quantile values
  q01 <- df %>%
    summarise_all(funs(quantile), probs = 0.01)
  q05 <- df %>%
    summarise_all(funs(quantile), probs = 0.05)
  q25 <- df %>%
    summarise_all(funs(quantile), probs = 0.25)
  q50 <- df %>%
    summarise_all(funs(quantile), probs = 0.50)
  q75 <- df %>%
    summarise_all(funs(quantile), probs = 0.75)
  q95 <- df %>%
    summarise_all(funs(quantile), probs = 0.95)
  q99 <- df %>%
    summarise_all(funs(quantile), probs = 0.99)
  missingPCT <- df %>%
    summarize_all(funs(sum(is.na(.)) / length(.)))

  # create the mean, variance, min, max
  avg <- round(df %>%
    dplyr::summarise_all(mean), 3)
  variance <- round(df %>%
    dplyr::summarise_all(var), 3)
  minimum <- df %>%
    dplyr::summarise_all(min)
  maximum <- df %>%
```

```
dplyr::summarize_all(max)

# flatten & widen the dataframe
q01 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.01' = value)
q05 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.05' = value)
q25 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.25' = value)
q50 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.50' = value)
q75 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.75' = value)
q95 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.95' = value)
q99 %<>%
  reshape2::melt() %>%
  dplyr::rename('Q 0.99' = value)
avg %<>%
  reshape2::melt() %>%
  dplyr::rename('Mean' = value)
variance %<>%
  reshape2::melt() %>%
  dplyr::rename('Variance' = value)
minimum %<>%
  reshape2::melt() %>%
  dplyr::rename('Min' = value)
maximum %<>%
  reshape2::melt() %>%
  dplyr::rename('Max' = value)
missingPCT %<>%
  reshape2::melt() %>%
  dplyr::rename('Missing PCT' = value)

# combine the flat files
tblGrande <- inner_join(q01, q05, by='variable')
tblGrande <- inner_join(tblGrande, q25, by = 'variable')
tblGrande <- inner_join(tblGrande, q50, by = 'variable')
tblGrande <- inner_join(tblGrande, q75, by = 'variable')
tblGrande <- inner_join(tblGrande, q95, by = 'variable')
tblGrande <- inner_join(tblGrande, q99, by = 'variable')
tblGrande <- inner_join(tblGrande, avg, by = 'variable')
tblGrande <- inner_join(tblGrande, variance, by = 'variable')
tblGrande <- inner_join(tblGrande, minimum, by = 'variable')
```

```
tblGrande <- inner_join(tblGrande, maximum, by = 'variable')
tblGrande <- inner_join(tblGrande, missingPCT, by = 'variable')

# return the final table
return(tblGrande)
}

CustomSummary <- custom.summary(wine[2:14])
CustomSummary %<>%
  mutate(
    range = Max - Min
  )
```

Table 5

```
library(corr)
x <- correlate(wine[,2:14])
x.m <- melt(x)
x.m %<>%
  mutate(
    corr = round(value, 3),
    absCorr = abs(corr)
  ) %>%
  # filter(absCorr >= 0.7) %>%
  arrange(desc(absCorr))
```

Figure Code

Figure 1 Code

```
duo2 <- function(y) {
  a <- bwplot(~ wine[[y]], data=wine, xlab=y, main=paste('Boxplot:',y))
  b <- histogram(~wine[[y]], data=wine, xlab=y, type='count', nint=20, layout
= c(1,1), main=paste('Histogram:',y))
  grid.arrange(a,b,ncol=2)
}
duo2('MalicAcid')
```

Figure 2 Code

```
dp <- function(y) {
  densityplot(~wine[[y]] | WineType, data = wine, as.table=T, xlab = y,
main=paste('Density Plot:',y))
}
for(i in colnames(wine)) {
  print(dp(i))
}
```

```
}
```

Figure 3 Code

```
bwp <- function(x) {
  bwplot(~wine[[x]], data = wine, xlab = x, main = paste('Boxplot:',x,sep='
'))
}
bwp('Magneisum')
```

Figure 4 Code

```
xyp <- function(x,y) {
  xyplot(wine[[y]] ~ wine[[x]] | WineType, data=wine, as.table = T,
  type=c('g','p'), xlab=x, ylab=y, main=paste(y,'vs',x,'by WineType'),
  layout=c(1,3))
}
xyp('TotalPhenols','Flavanoids')
```

Figure 5 Code

```
a1 <- dotplot(WineType ~ Alcohol, data = AvgVars, group = WineType, xlab =
'Average Alcohol', ylab = 'WineType', main = 'Average Alcohol by WineType')
a2 <- dotplot(WineType ~ ColorIntensity, data = AvgVars, group = WineType,
xlab = 'Average ColorIntensity', ylab = 'WineType', main = 'Average Hue by
WineType')

grid.arrange(a1, a2)
```

Figure 6 Code

```
duo <- function(x,y) {
  a <- bwplot(wine[[y]] ~ wine[[x]], data=wine, xlab=x, ylab=y,
main=paste('Boxplot:',y))
  b <- histogram(~wine[[y]] | wine[[x]], data=wine, xlab=y, type='count',
nint=20, layout = c(3,1), main=paste('Histogram:',y))
  grid.arrange(a,b,ncol=2)
}
duo('WineType', 'Magnesium')
```

Model Code

The code in this section was used to generate the appropriate tables and figures for each model.

Libraries Used in Script

```
library(tidyverse)      # easier data frame manipulation
library(lattice)        # for the graphs/charts
library(moments)        # for skewness & kurtosis
library(gridExtra)      # for multiple types of plots on one 'page'
library(magrittr)       # for easier storing of data frame changes
library(rpart)          # for tree making
library(rpart.plot)
library(rattle)
library(reshape2)
library(caret)
library(gridExtra)

library(doMC)
registerDoMC(4)
```

Naïve Tree Model

```
tree <- rpart(WineType~., data = wine)
fancyRpartPlot(tree, sub = '', cex = 0.7)
```

Random Forest Model

```
set.seed(33)
rforest_grid <- expand.grid(
  mtry=c(1:12)
)

fitControl_RF <- trainControl(
  method = 'oob',
  classProbs = F,
  allowParallel = T
)
set.seed(2017)
rFit1 <- train(x = wine[,2:12], y=wine[,1], data = wine, method = "rf",
trControl = fitControl_RF, verbose = F, tuneGrid = rforest_grid, num.trees =
500, metric='Kappa', importance = T)

rFit1
plot(rFit1$finalModel, main = 'Error vs. Number of Trees')
plot(rFit1$results$mtry, rFit1$results$Kappa, lty = 2, type = 'o', xlab =
'mtry value', ylab = 'Kappa',main='Kappa Results for mtry Parameter')
abline(col='red',v=4, h = max(rFit1$results$Kappa))
varImp(rFit1)
```

```
wineRF.fitted <- predict(rFit1, newdata = wine)
confusionMatrix(wineRF.fitted, wine$WineType)
```

Support Vector Machines (SVM) Model

```
svmControl <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 3,
  allowParallel = T
)

grid <- expand.grid(cost = c(0.001, 0.01, 0.05, 0.1, 0.25, 0.5, 0.75, 1, 1.25,
1.5, 1.75, 2, 5))
set.seed(2017)

svm_Lin2 <- train(WineType ~., data = wine, method = "svmLinear2",
trControl=svmControl,tuneLength = 10, tuneGrid = grid, metric = "Kappa")

svm_Lin2

wineSVM.fitted <- predict(svm_Lin2, newdata = wine)
confusionMatrix(wineSVM.fitted, wine$WineType)
```

Neural Network Model

```
nnControl <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 3,
  allowParallel = T
)
set.seed(2018)
nn1 <- train(WineType ~ ., data = wine, method = 'nnet', trControl = nnControl,
tuneGrid=expand.grid(size=c(5), decay=c(0.1)), preProcess = c('center',
'scale'))
nn1
nn1$finalModel
nn1$finalModel$wts
nn1.fitted <- predict(nn1, newdata = wine)
confusionMatrix(nn1.fitted, wine$WineType)
nn1$finalModel$wts
```


Appendix B – Figures and Tables

Random Forest Hypertuning

Table 11

mtry	Kappa
1	0.966
2	0.974
3	0.974
4	0.983
5	0.974
6	0.974
7	0.966
8	0.966
9	0.949
10	0.940
11	0.940
12	0.914

Support Vector Machine Hypertuning

Table 12

cost	Kappa
0.001	0.0173
0.01	0.9803
0.05	0.9715
0.1	0.9657
0.25	0.9485
0.5	0.9569
0.75	0.9599
1	0.9627
1.25	0.9600
1.5	0.9600
1.75	0.9600
2	0.9600
5	0.9600